



NETWORK ARCHITECTURES

Basic Networking Concepts

G. Kalfas

Notes on the slides

- The material (slides) of this course is a mixture of the following:
 1. slides provided freely by J.F Kurose and K.W. Ross accompanying the book “Computer Networking: A top-down approach” (this book is mentioned in the references of the Syllabus - slides can be found here: https://gaia.cs.umass.edu/kurose_ross/ppt.php)
 2. slides provided by CITY college and
 3. my own slides

NETWORK ARCHITECTURES

- ▶ **Lecturer**

Dr. George Kalfas

- ▶ **E-mail**

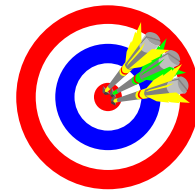
gkalfas@yorkeuropecampus.eu

- ▶ **Office**

L. Sofou Building / 6th Floor

MODULE AIMS

- examine the structure and architecture of data communication systems and computer networks
- discuss hardware components required to form computer networks
- describe challenges for achieving effective and efficient data communications and their solutions
- investigate various networking and internetworking protocols and standards
- explain the layered approach of the TCP/IP protocol stack



MODULE OUTLINE*

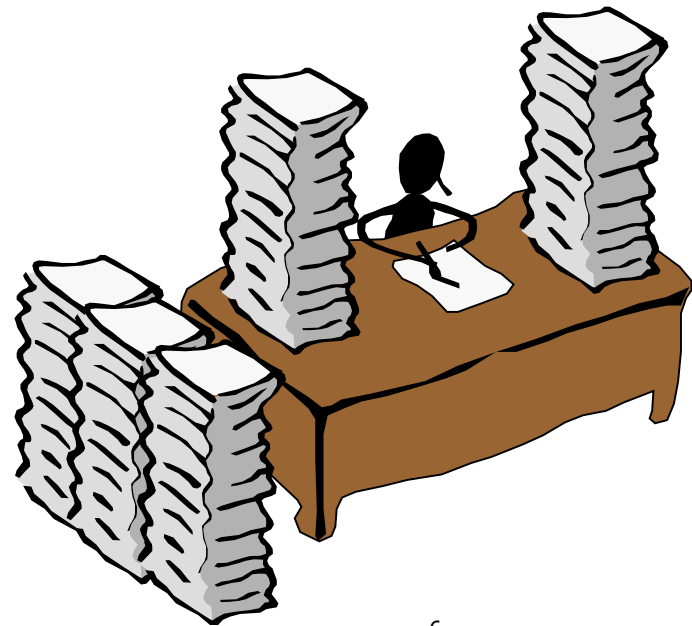
WEEK	TOPIC
1	Introduction – Basic Networking Concepts
2	Layered Approach of Network Models
3	Network Applications Fundamentals
4	Modern Network Applications
5	Local Area Networks- Fundamentals/Devices
6	Consolidation Week
7	Error Detection - Flow and Error Control
8	Wide Area Networking
9-10-11	Internetworking I,II, III
12	Revision Week



*I plan to make some improvements compared to last year so some minor changes might happen throughout the semester

ASSESSMENT

- Quiz (35%)
 - W8*
- Final Exam (65%)
 - RA13



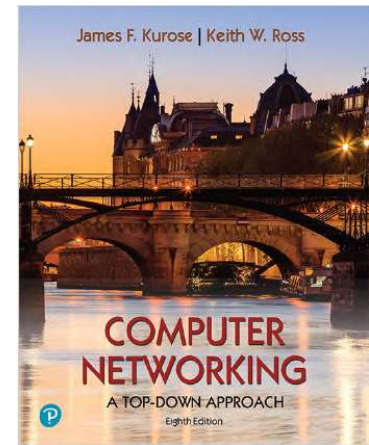
TEXTBOOK and REFERENCES

■ Textbook

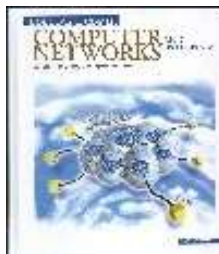
Kurose/Ross

Computer Networking: A top down approach

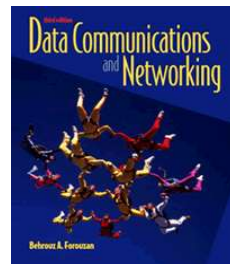
Latest Edition, Pearson



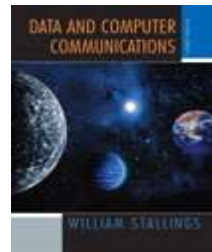
➤ References



Douglas



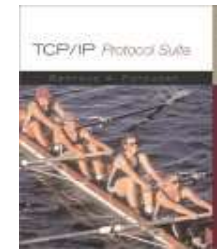
Forouzan



Stallings



Tanenbaum



Forouzan

Google Classroom

- All material will be posted in **Google Classroom**
- <https://classroom.google.com/>
- Enrollment key: **5rflkkmu**

**DO NOT HESITATE TO COME AND FIND ME OR SEND ME
EMAILS IF YOU HAVE ANY QUESTIONS**

Lecture 1: introduction

Chapter goal:

- Get “feel,” “big picture,” introduction to terminology
 - more depth, detail *later* in course



Overview/roadmap:

- What *is* the Internet? What *is* a protocol?
- **Network edge:** hosts, access network, physical media
- **Network core:** packet/circuit switching, internet structure
- **Performance:** loss, delay, throughput
- Protocol layers, service models
- Security

The Internet: a “nuts and bolts” view



Billions of connected computing *devices*:

- *hosts* = end systems
- running *network apps* at Internet's “edge”



Packet switches: forward packets (chunks of data)

- *routers, switches*

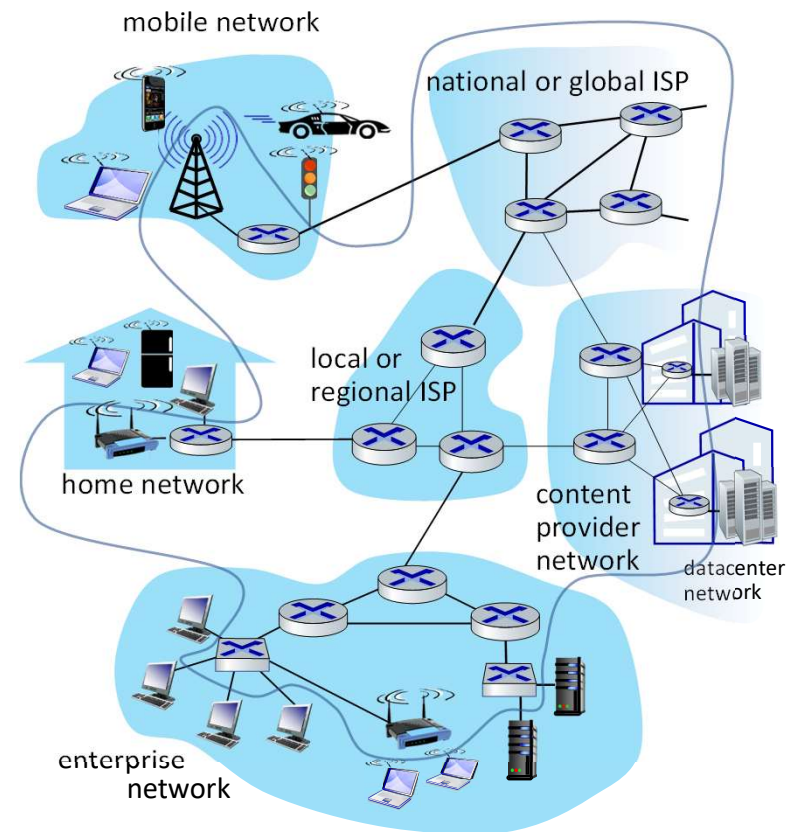


Communication links

- fiber, copper, radio, satellite
- transmission rate: *bandwidth*

Networks

- collection of devices, routers, links: managed by an organization



“Fun” Internet-connected devices



Amazon Echo



Internet refrigerator



IP picture frame



Pacemaker & Monitor



Tweet-a-watt:
monitor energy use



Security Camera



Slingbox: remote
control cable TV



Web-enabled toaster +
weather forecaster



AR devices



Internet phones



Gaming devices



sensorized,
bed
mattress



Fitbit



diapers



bikes



cars



scooters

Others?

The Internet: a “nuts and bolts” view

- *Internet: “network of networks”*

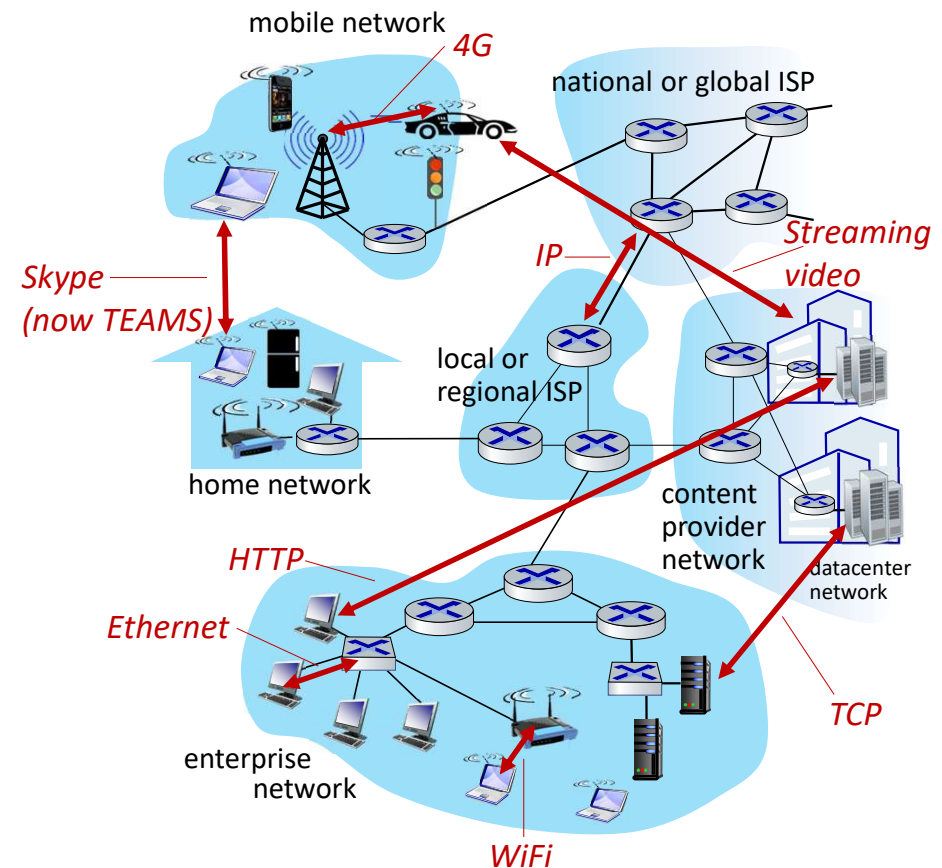
- Interconnected Internet Service Providers (ISPs)

- *protocols are everywhere*

- control sending, receiving of messages
- e.g., HTTP (Web), streaming video, Skype, TCP, IP, WiFi, 4/5G, Ethernet

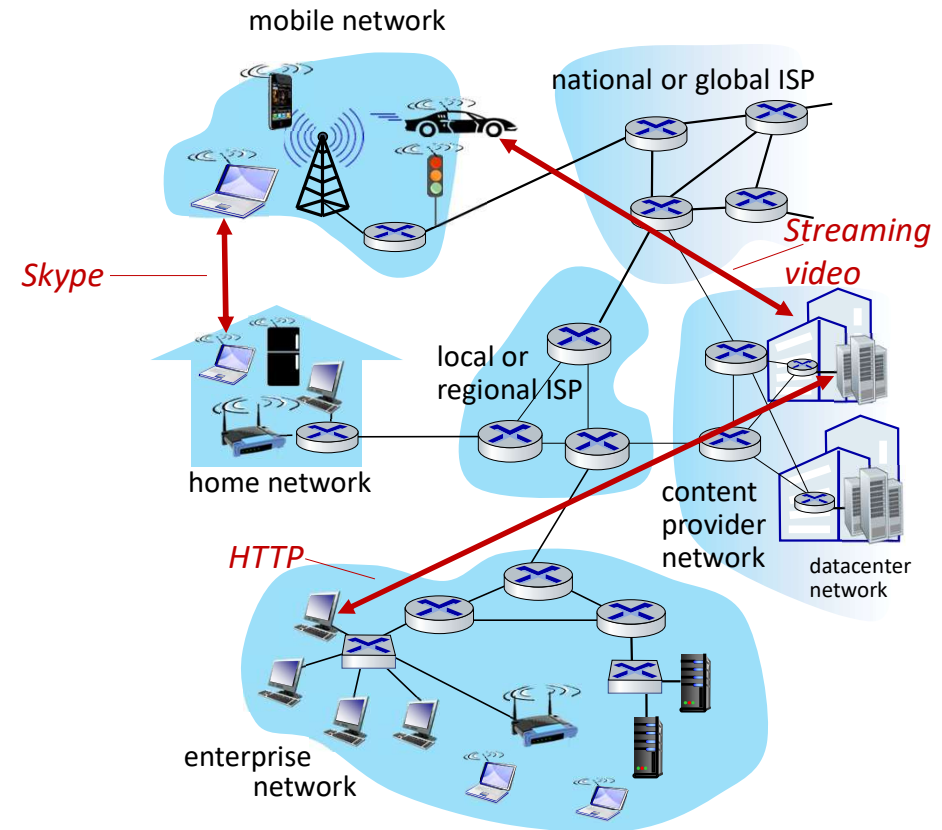
- *Internet standards*

- RFC: Request for Comments
- IETF: Internet Engineering Task Force



The Internet: a “services” view

- *Infrastructure* that provides services to applications:
 - Web, streaming video, multimedia teleconferencing, email, games, e-commerce, social media, inter-connected appliances, ...
- provides *programming interface* to distributed applications:
 - “hooks” allowing sending/receiving apps to “connect” to, use Internet transport service
 - provides service options, analogous to postal service



What's a protocol?

Human protocols:

- “what’s the time?”
- “I have a question”
- introductions

Rules for:

... specific messages sent
... specific actions taken
when message received,
or other events

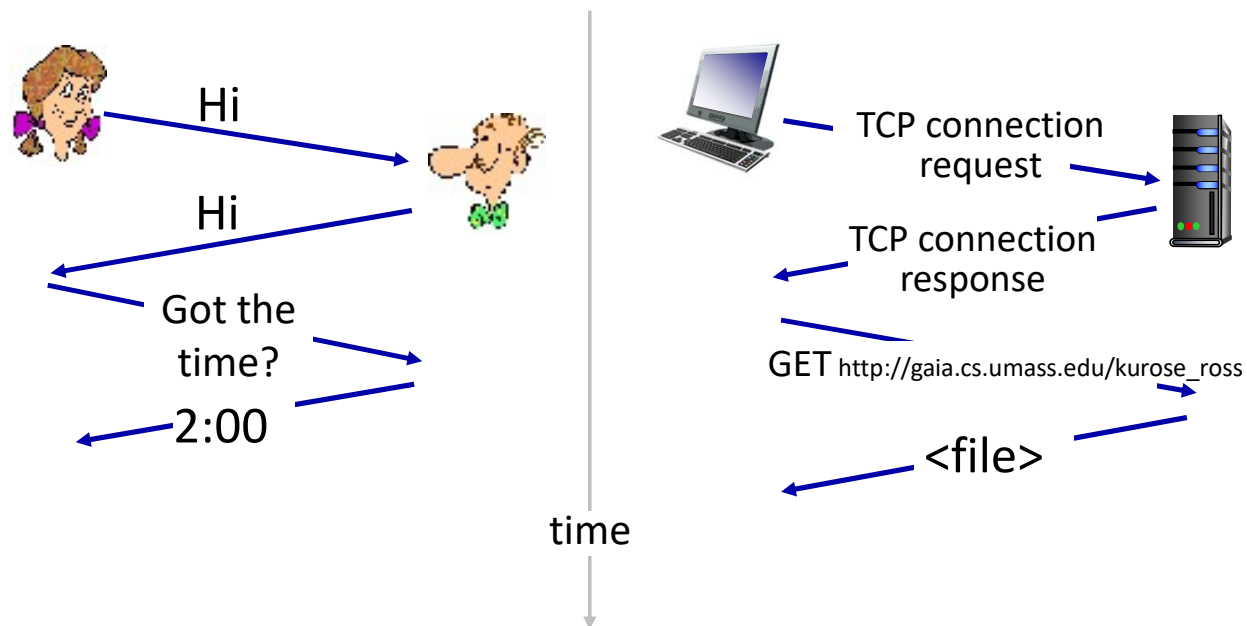
Network protocols:

- computers (devices) rather than humans
- all communication activity in Internet governed by protocols

*Protocols define the **format, order** of **messages sent and received** among network entities, and **actions taken** on message transmission, receipt*

What's a protocol?

A human protocol and a computer network protocol:



Q: other human protocols?

Lecture 1: roadmap

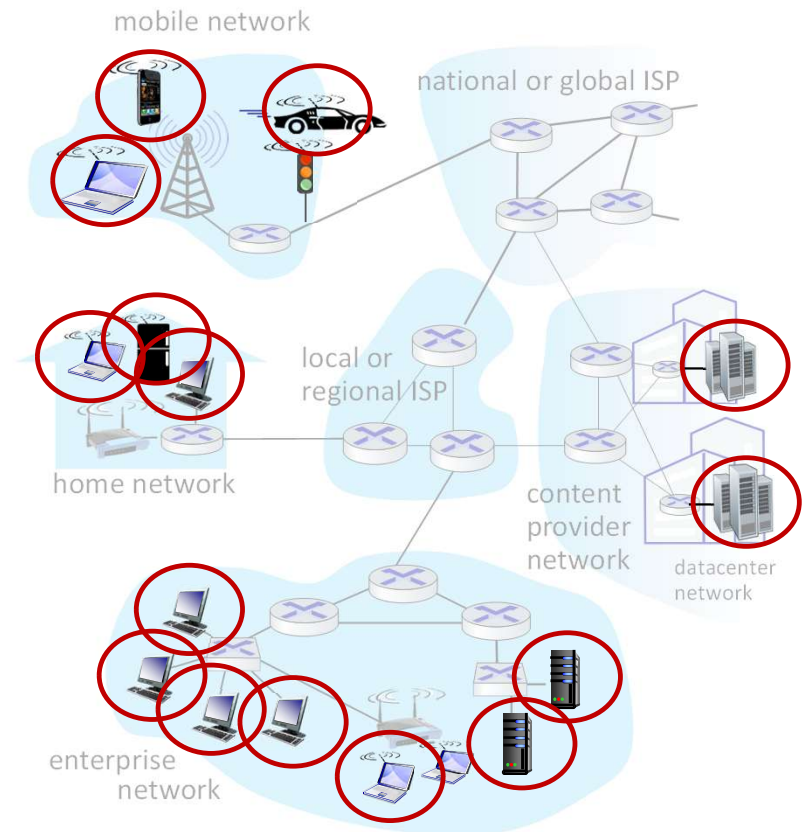
- What *is* the Internet?
- What *is* a protocol?
- **Network edge:** hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Security
- Protocol layers, service models



A closer look at Internet structure

Network edge:

- hosts: clients and servers
- servers often in data centers



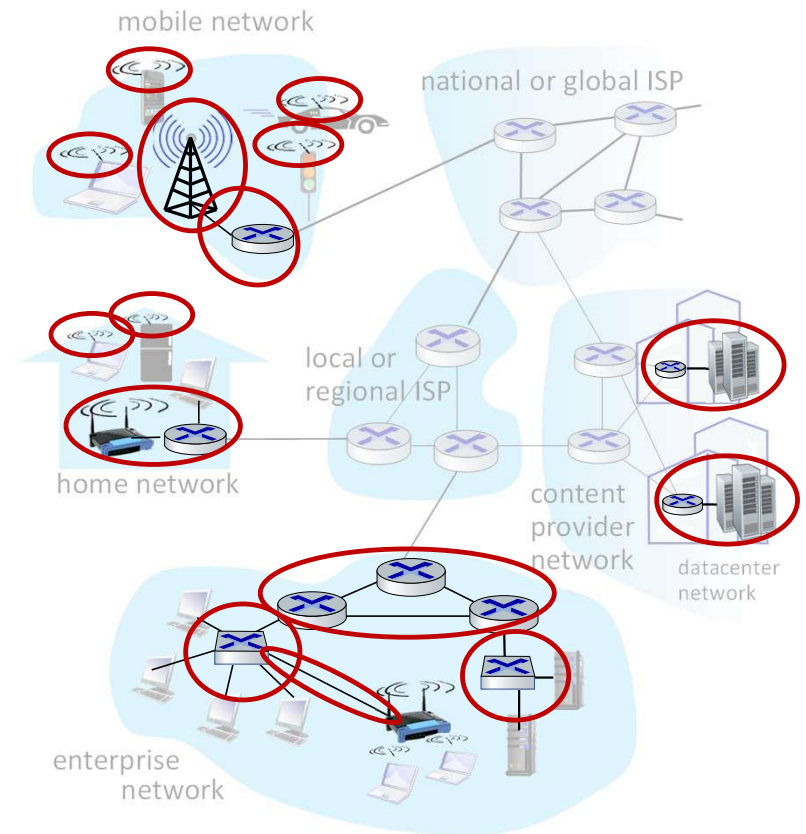
A closer look at Internet structure

Network edge:

- hosts: clients and servers
- servers often in data centers

Access networks, physical media:

- wired, wireless communication links



A closer look at Internet structure

Network edge:

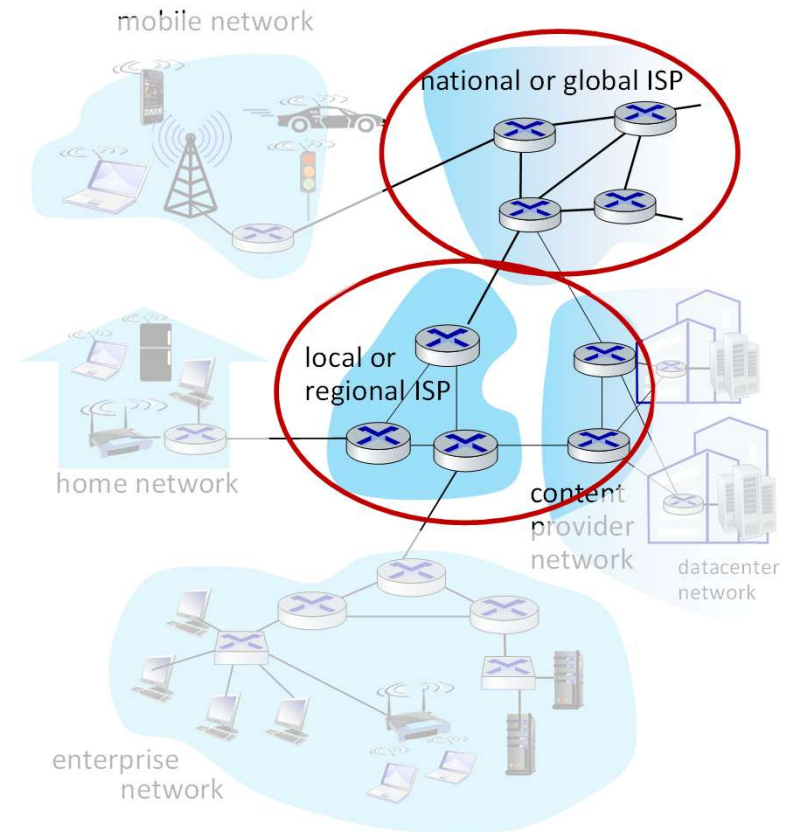
- hosts: clients and servers
- servers often in data centers

Access networks, physical media:

- wired, wireless communication links

Network core:

- interconnected routers
- network of networks



Data Communications

- **Telecommunication**

- Communication of signals to remote places (e.g. telephony, telegraphy)

- **Data Communication**

- The exchange of data between two devices via some form of communication medium

Effectiveness of data communications depends on:

Delivery

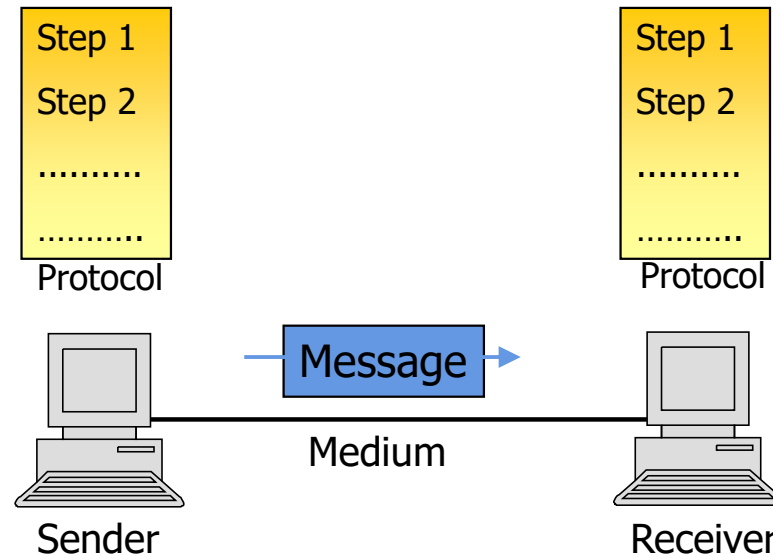
Accuracy

Timeliness

Communication Components

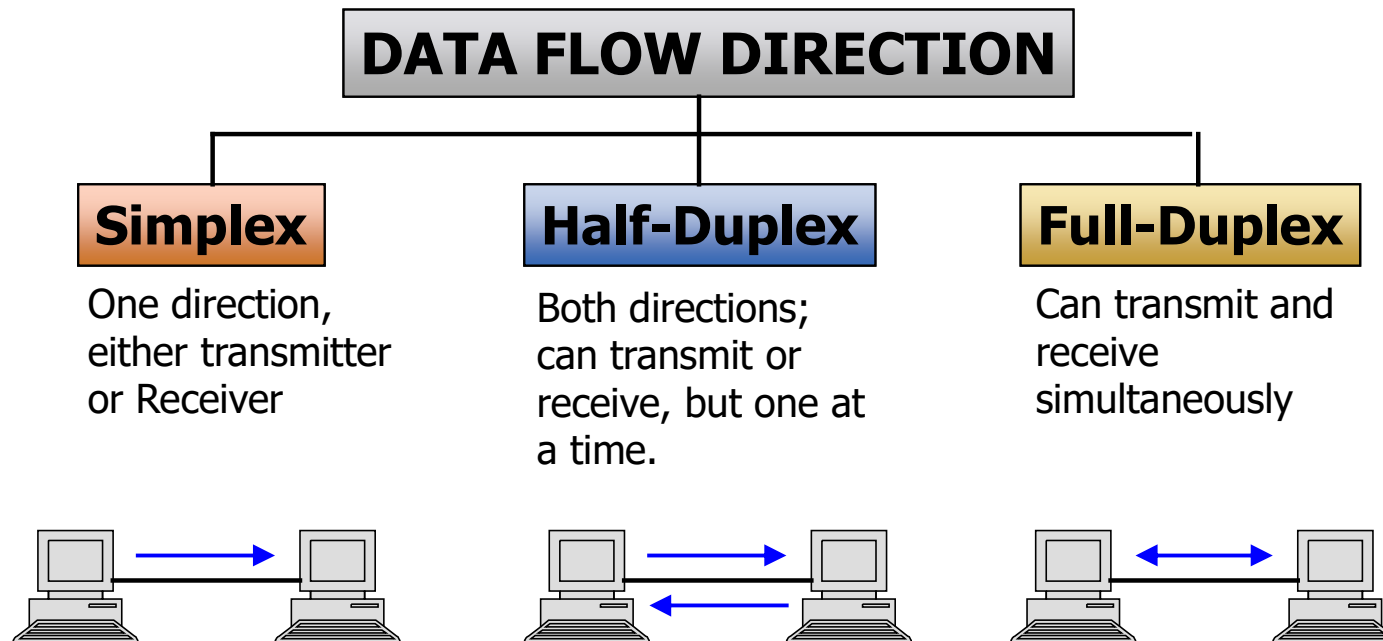
➤ A Data Communication system is made up of:

1. Sender
2. Receiver
3. Message
4. Medium
5. Protocol



Direction of Data Flow

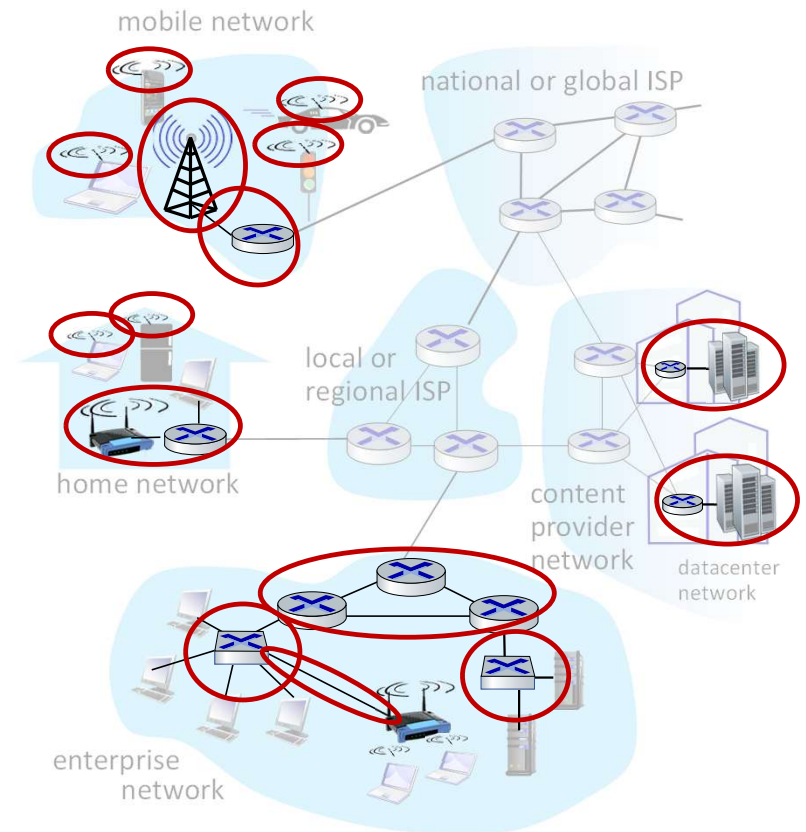
The direction of signal flow between two linked devices.



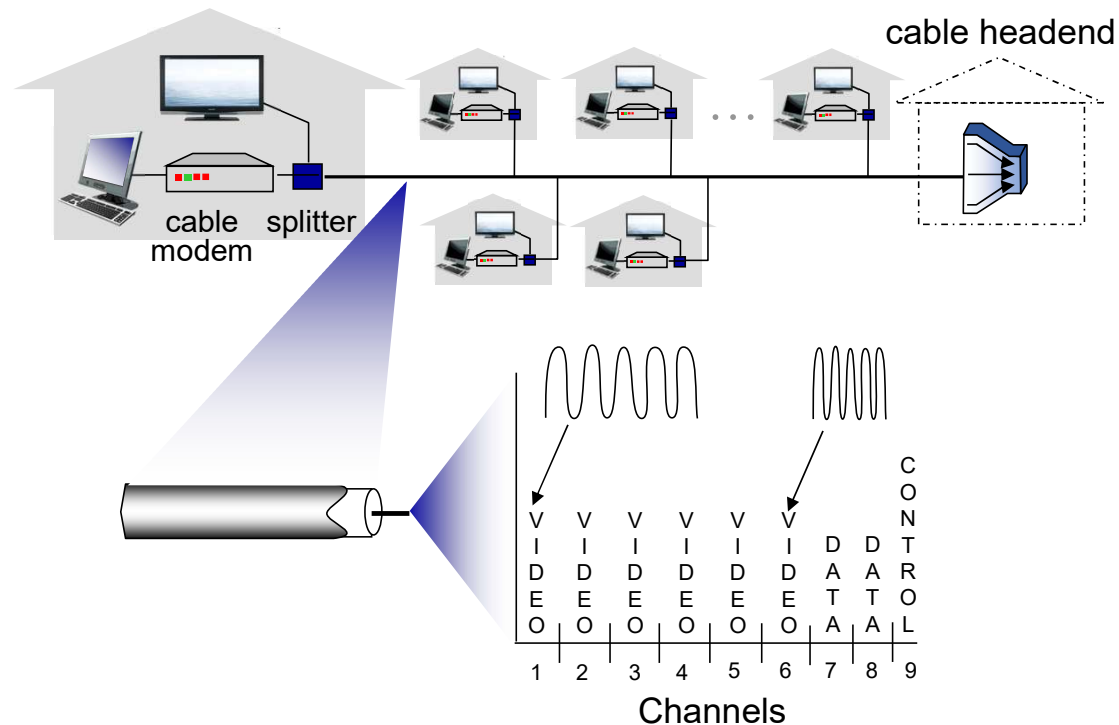
Access networks and physical media

Q: How to connect end systems to edge router?

- residential access nets
- institutional access networks (school, company)
- mobile access networks (WiFi, 4G/5G)

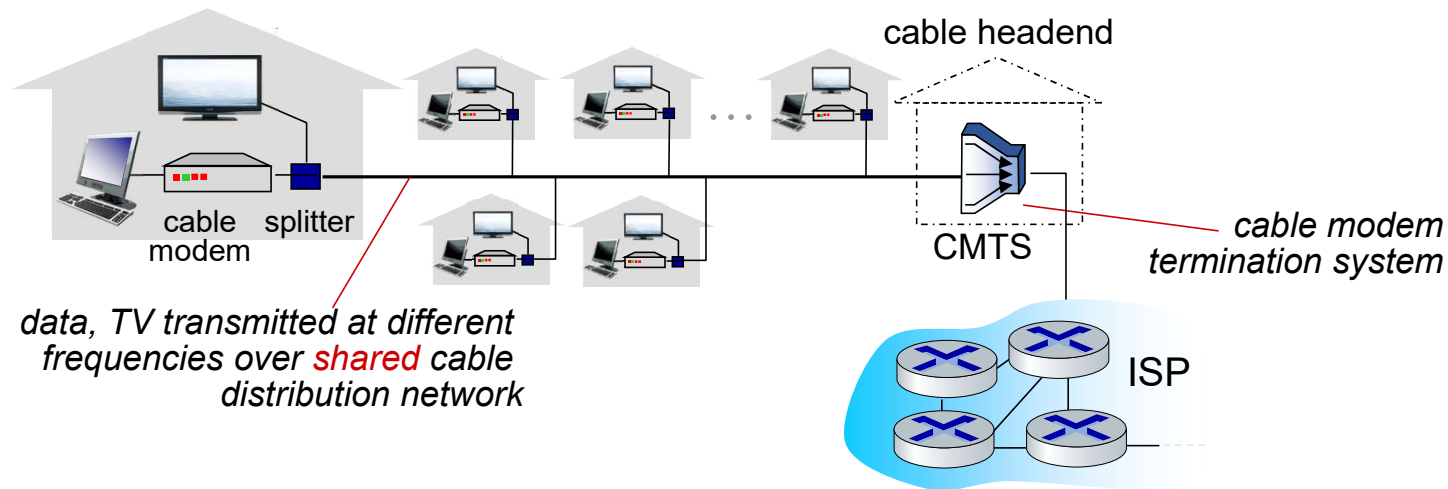


Access networks: cable-based access



frequency division multiplexing (FDM): different channels transmitted in different frequency bands

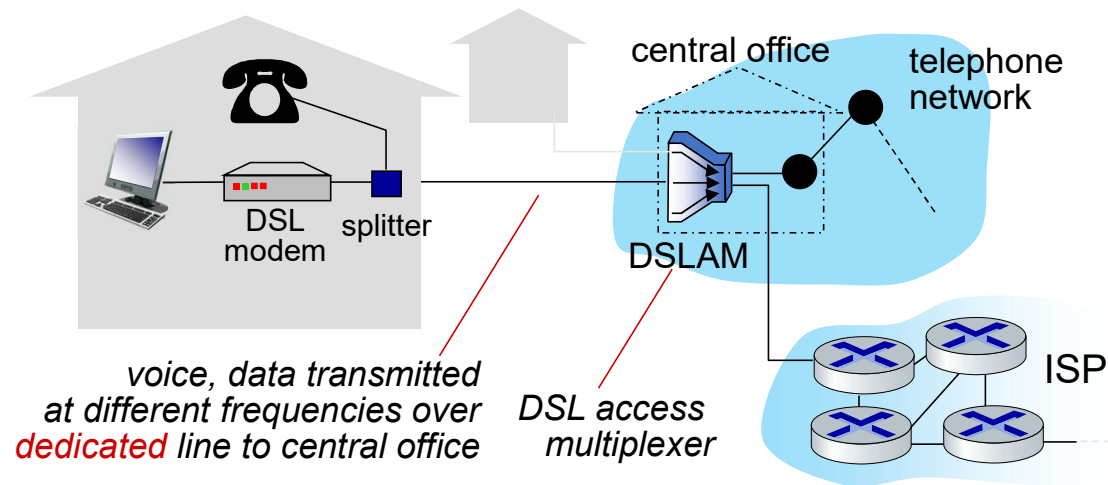
Access networks: cable-based access



■ HFC: hybrid fiber coax

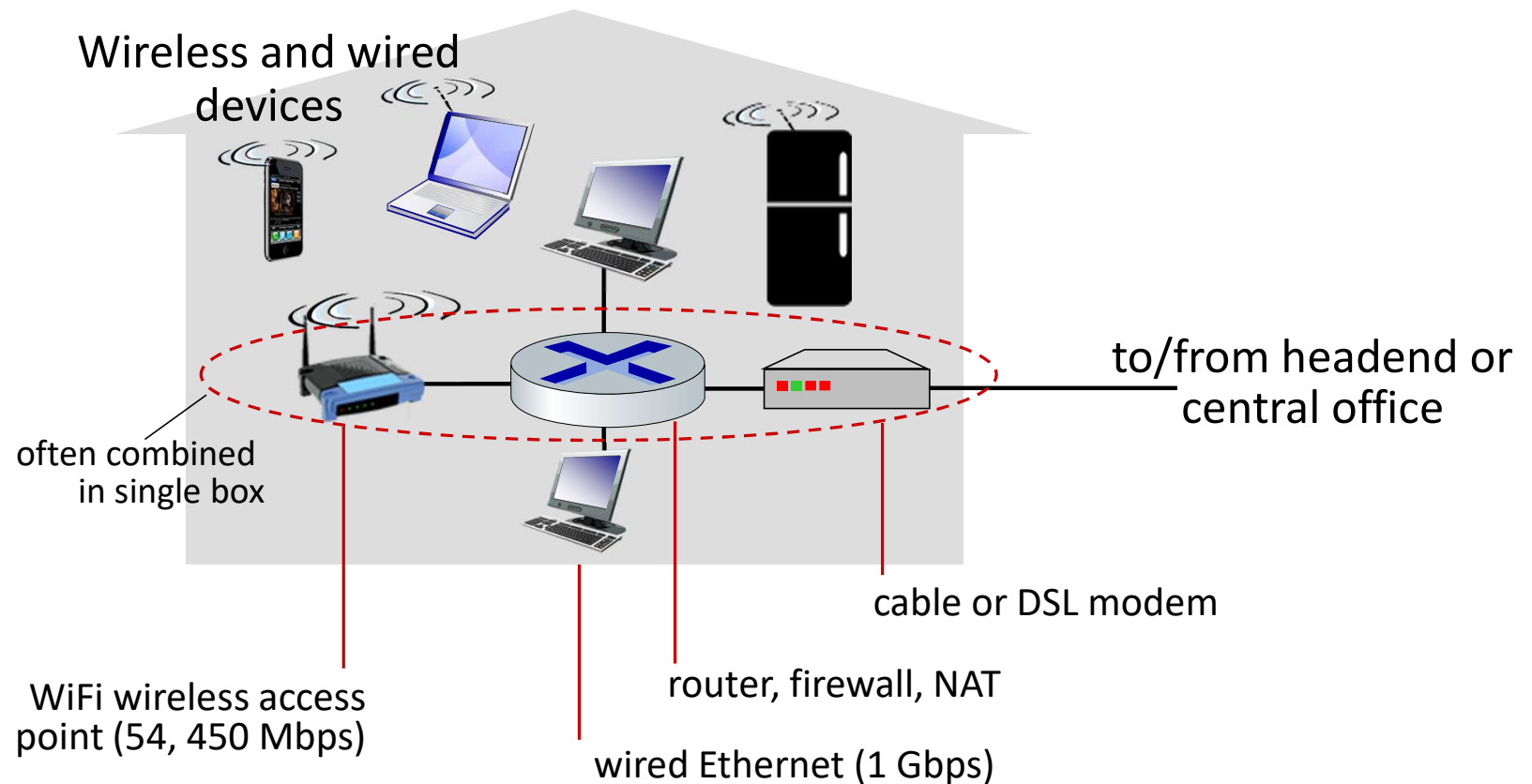
- asymmetric: up to 40 Mbps – 1.2 Gbps downstream transmission rate, 30-100 Mbps upstream transmission rate
- network of cable, fiber attaches homes to ISP router
 - homes *share access network* to cable headend

Access networks: digital subscriber line (DSL)



- use *existing* telephone line to central office DSLAM
 - data over DSL phone line goes to Internet
 - voice over DSL phone line goes to telephone net
- 24-52 Mbps dedicated downstream transmission rate
- 3.5-16 Mbps dedicated upstream transmission rate

Access networks: home networks



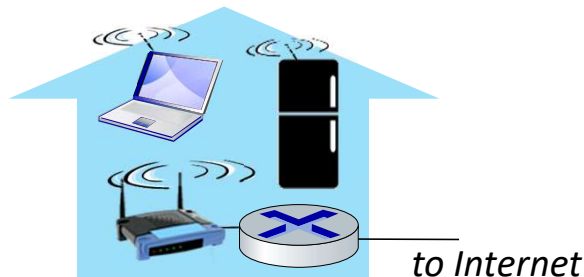
Wireless access networks

Shared *wireless* access network connects end system to router

- via base station aka “access point”

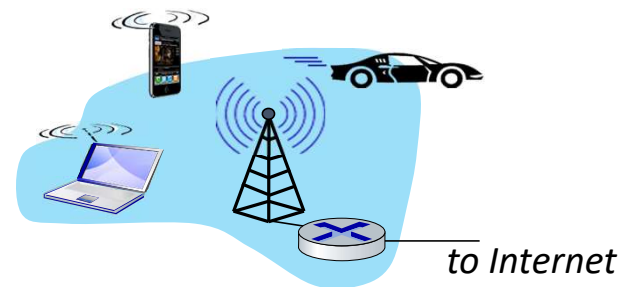
Wireless local area networks (WLANs)

- typically within or around building (~30m)
- 802.11b/g/n (WiFi): 11, 54, 450 Mbps transmission rate

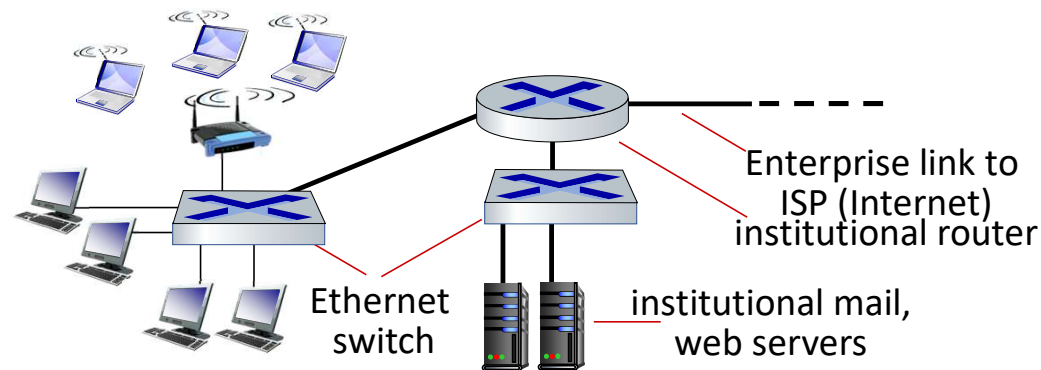


Wide-area cellular access networks

- provided by mobile, cellular network operator (10's km)
- 10's Mbps
- 4G/5G cellular networks



Access networks: enterprise networks



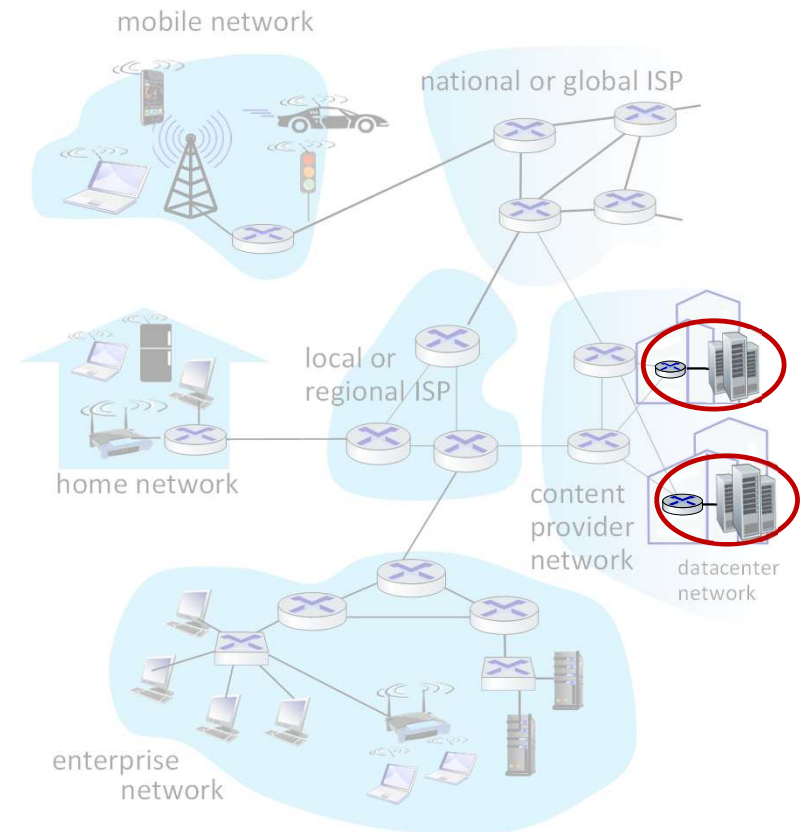
- companies, universities, etc.
- mix of wired, wireless link technologies, connecting a mix of switches and routers (we'll cover differences shortly)
 - Ethernet: wired access at 100Mbps, 1Gbps, 10Gbps
 - WiFi: wireless access points at 11, 54, 450 Mbps

Access networks: data center networks

- high-bandwidth links (10s to 100s Gbps) connect hundreds to thousands of servers together, and to Internet



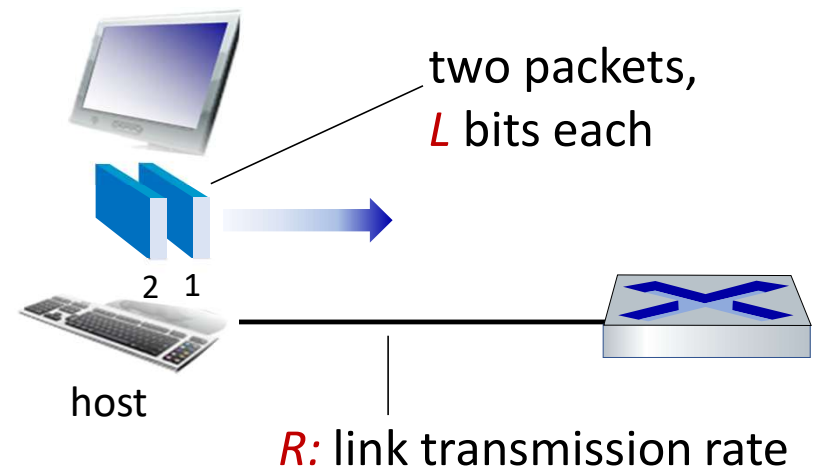
Courtesy: Massachusetts Green High Performance Computing Center (mghpcc.org)



Host: sends *packets* of data

host sending function:

- takes application message
- breaks into smaller chunks, known as *packets*, of length L bits
- transmits packet into access network at *transmission rate R*
 - link transmission rate, aka link *capacity, aka link bandwidth*



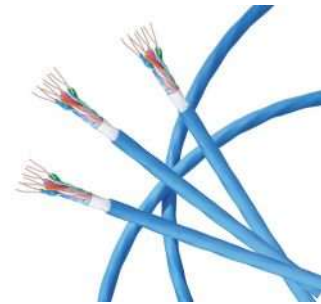
$$\begin{array}{l} \text{packet} \\ \text{transmission} \\ \text{delay} \end{array} = \begin{array}{l} \text{time needed to} \\ \text{transmit } L\text{-bit} \\ \text{packet into link} \end{array} = \frac{L \text{ (bits)}}{R \text{ (bits/sec)}}$$

Links: physical media

- **bit**: propagates between transmitter/receiver pairs
- **physical link**: what lies between transmitter & receiver
- **guided media**:
 - signals propagate in solid media: copper, fiber, coax
- **unguided media**:
 - signals propagate freely, e.g., radio

Twisted pair (TP)

- two insulated copper wires
 - Category 5: 100 Mbps, 1 Gbps Ethernet
 - Category 6: 10Gbps Ethernet



Links: physical media

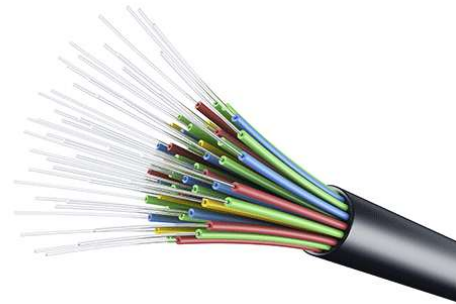
Coaxial cable:

- two concentric copper conductors
- bidirectional
- broadband:
 - multiple frequency channels on cable
 - 100's Mbps per channel



Fiber optic cable:

- glass fiber carrying light pulses, each pulse a bit
- high-speed operation:
 - high-speed point-to-point transmission (10's-100's Gbps)
- low error rate:
 - repeaters spaced far apart
 - immune to electromagnetic noise



Links: physical media

Wireless radio

- signal carried in various “bands” in electromagnetic spectrum
- no physical “wire”
- broadcast, “half-duplex” (sender to receiver)
- propagation environment effects:
 - reflection
 - obstruction by objects
 - Interference/noise

Radio link types:

- **Wireless LAN (WiFi)**
 - 10-100's Mbps; 10's of meters
- **wide-area** (e.g., 4G/5G cellular)
 - 10's Mbps (4G) over ~10 Km
- **Bluetooth:** cable replacement
 - short distances, limited rates
- **terrestrial microwave**
 - point-to-point; 45 Mbps channels
- **satellite**
 - up to < 100 Mbps (Starlink) downlink
 - 270 msec end-end delay (geostationary)

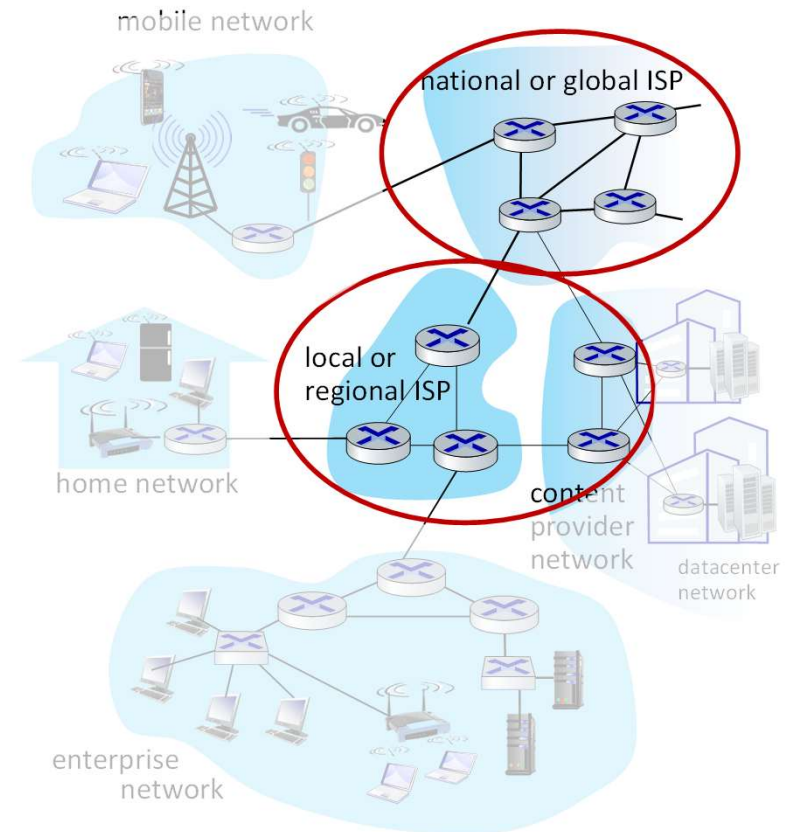
Lecture 1: roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- **Network core:** packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Security
- Protocol layers, service models



The network core

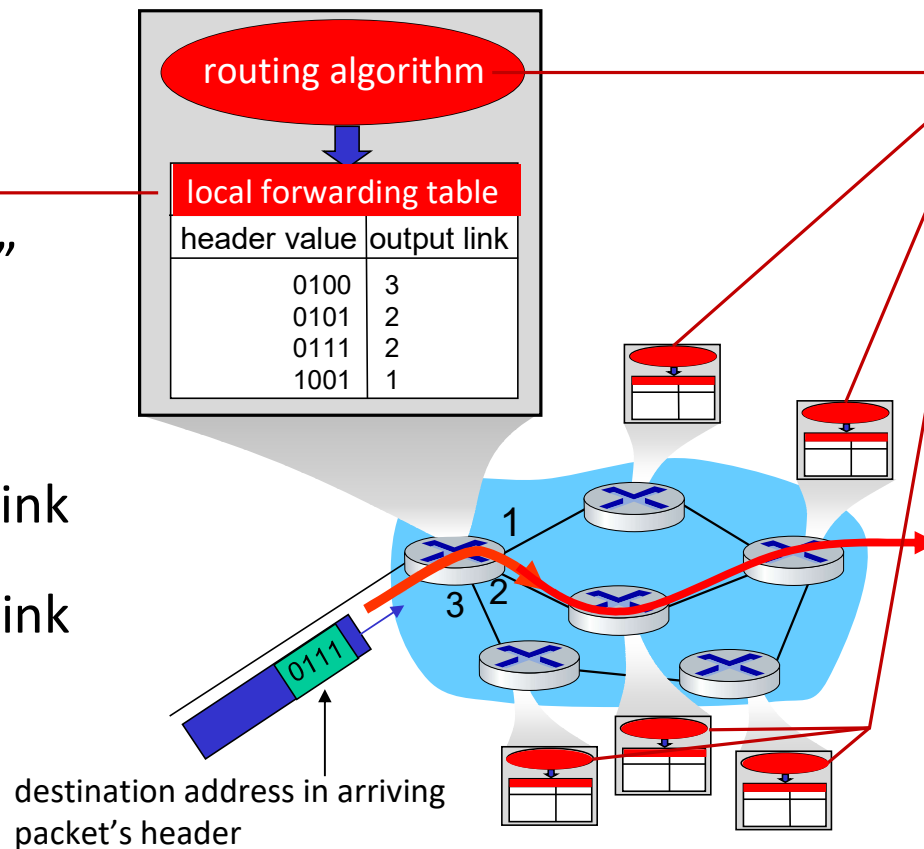
- mesh of interconnected routers
- **packet-switching**: hosts break application-layer messages into *packets*
 - network **forwards** packets from one router to the next, across links on path from **source to destination**



Two key network-core functions

Forwarding:

- aka “switching”
- *local* action: move arriving packets from router’s input link to appropriate router output link



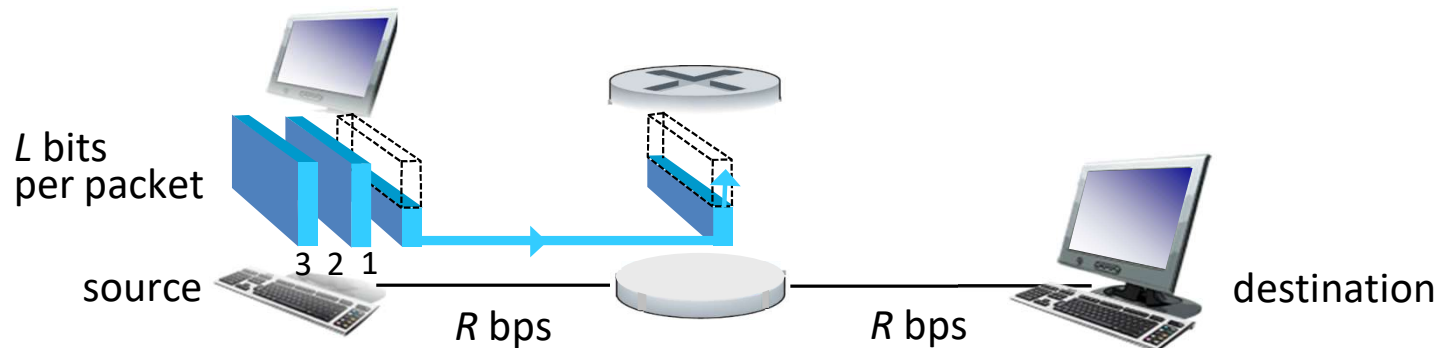
Routing:

- *global* action: determine source-destination paths taken by packets
- routing algorithms





Packet-switching: store-and-forward

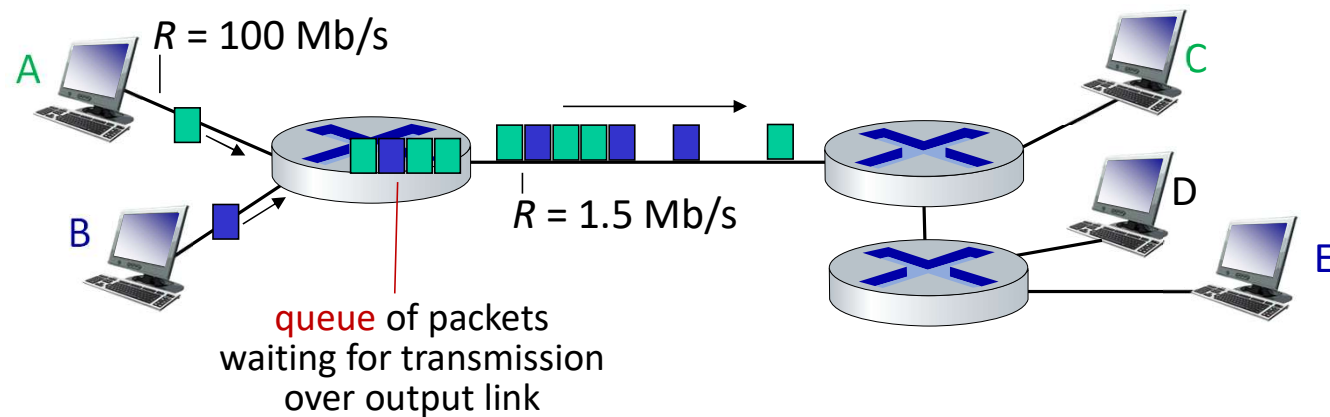


- **packet transmission delay:** takes L/R seconds to transmit (push out) L -bit packet into link at R bps
- **store and forward:** entire packet must arrive at router before it can be transmitted on next link

One-hop numerical example:

- $L = 10$ Kbits
- $R = 100$ Mbps
- one-hop transmission delay = 0.1 msec

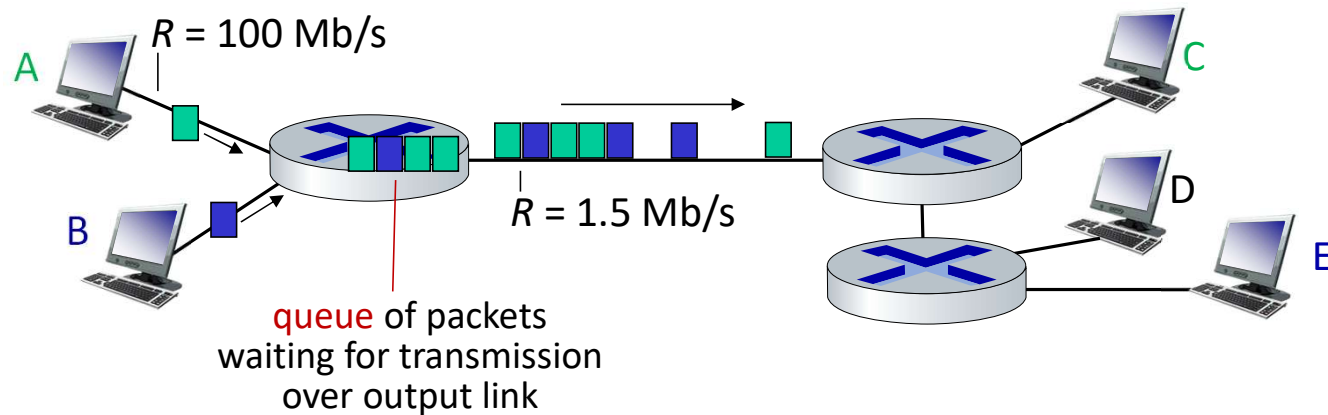
Packet-switching: queueing



Queueing occurs when work arrives faster than it can be serviced:



Packet-switching: queueing



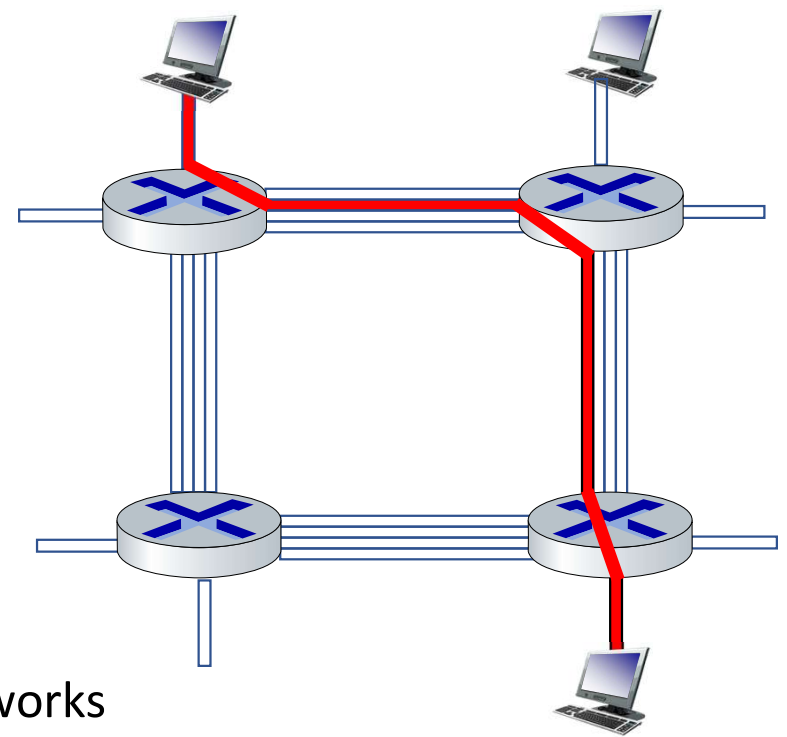
Packet queuing and loss: if arrival rate (in bps) to link exceeds transmission rate (bps) of link for some period of time:

- packets will queue, waiting to be transmitted on output link
- packets can be dropped (lost) if memory (buffer) in router fills up

Alternative to packet switching: circuit switching

end-end resources allocated to,
reserved for “call” between source
and destination

- in diagram, each link has four circuits.
 - call gets 2nd circuit in top link and 1st circuit in right link.
- dedicated resources: no sharing
 - circuit-like (guaranteed) performance
- circuit segment idle if not used by call (no sharing)
- commonly used in traditional telephone networks

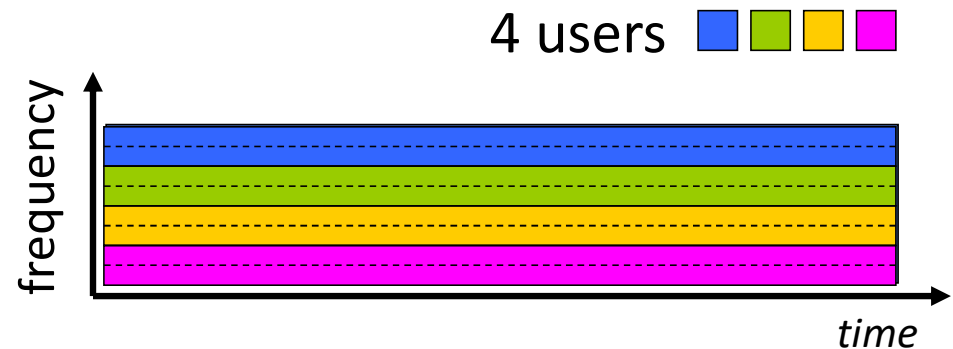


* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive

Circuit switching: FDM and TDM

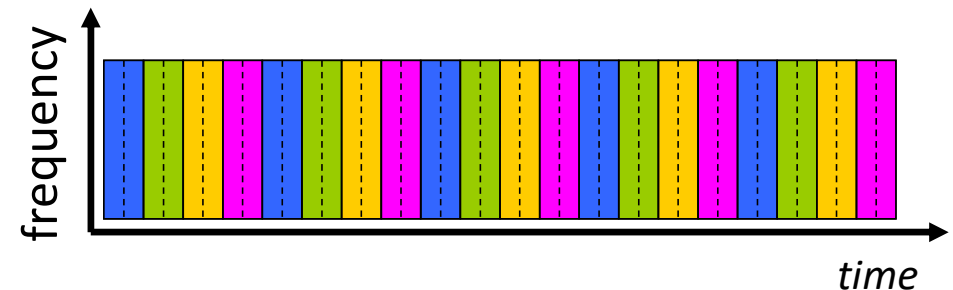
Frequency Division Multiplexing (FDM)

- optical, electromagnetic frequencies divided into (narrow) frequency bands
- each call allocated its own band, can transmit at max rate of that narrow band



Time Division Multiplexing (TDM)

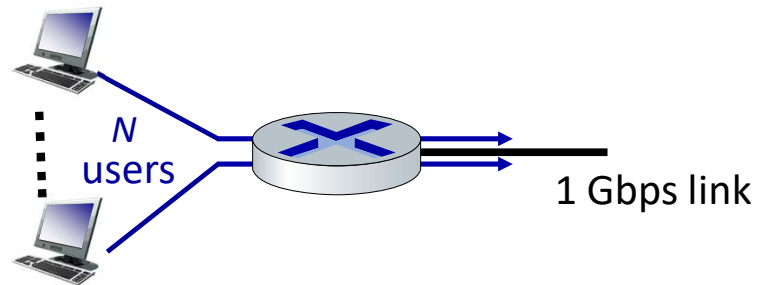
- time divided into slots
- each call allocated periodic slot(s), can transmit at maximum rate of (wider) frequency band (only) during its time slot(s)



Packet switching versus circuit switching

example:

- 1 Gb/s link
- each user:
 - 100 Mb/s when “active”
 - active 10% of time



Q: how many users can use this network under circuit-switching and packet switching?

- **circuit-switching:** 10 users
- **packet switching:** with 35 users, probability > 10 active at same time is less than .0004 *

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive

Packet switching versus circuit switching

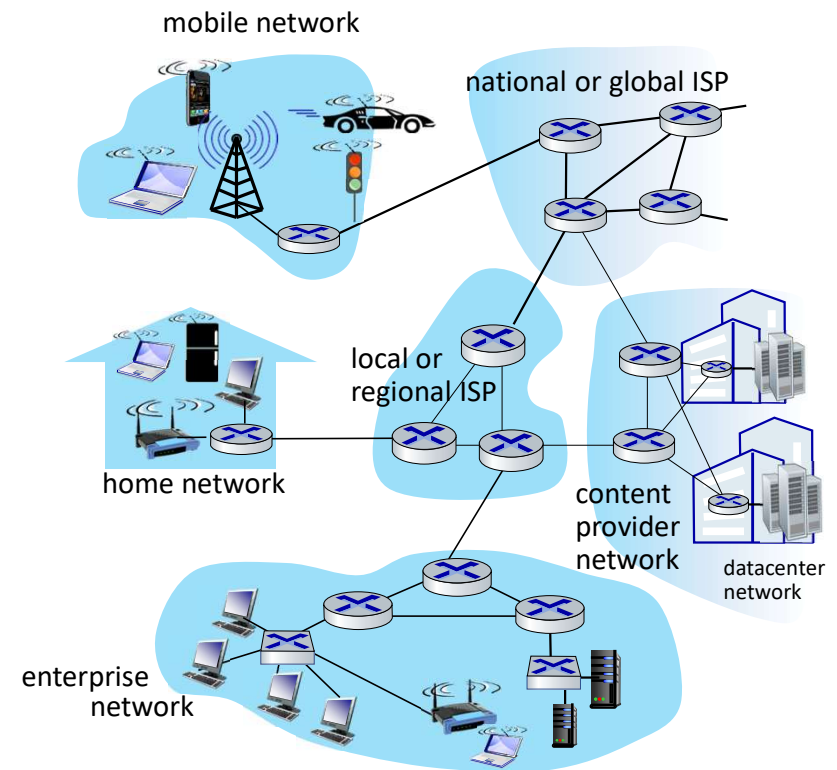
Is packet switching a “slam dunk winner”?

- great for “bursty” data – sometimes has data to send, but at other times not
 - resource sharing
 - simpler, no call setup
- **excessive congestion possible:** packet delay and loss due to buffer overflow
 - protocols needed for reliable data transfer, congestion control
- **Q: How to provide circuit-like behavior with packet-switching?**
 - “It’s complicated.” We’ll study various techniques that try to make packet switching as “circuit-like” as possible.

Q: human analogies of reserved resources (circuit switching) versus on-demand allocation (packet switching)?

Internet structure: a “network of networks”

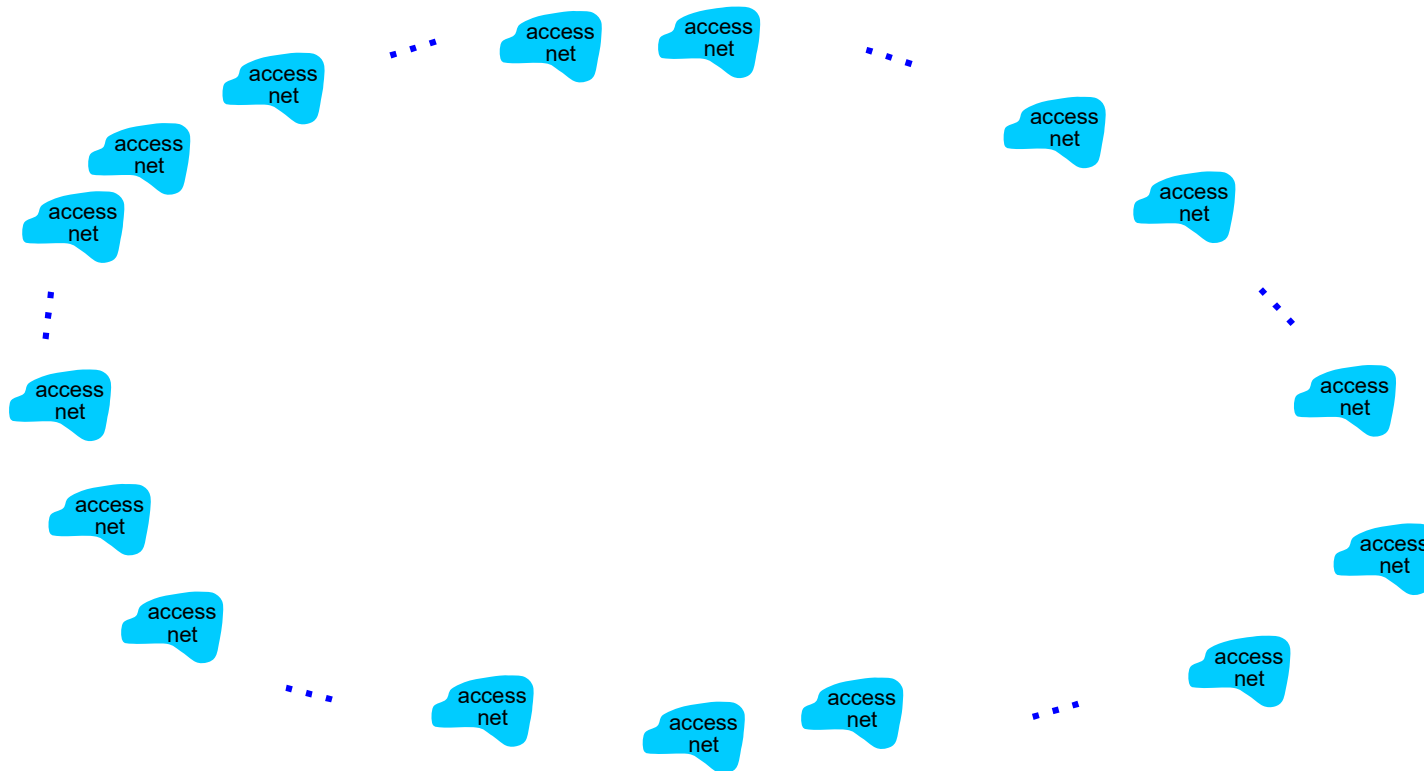
- hosts connect to Internet via **access** Internet Service Providers (ISPs)
- access ISPs in turn must be interconnected
 - so that *any* two hosts (*anywhere!*) can send packets to each other
- resulting network of networks is very complex
 - evolution driven by **economics**, **national policies**



Let's take a stepwise approach to describe current Internet structure

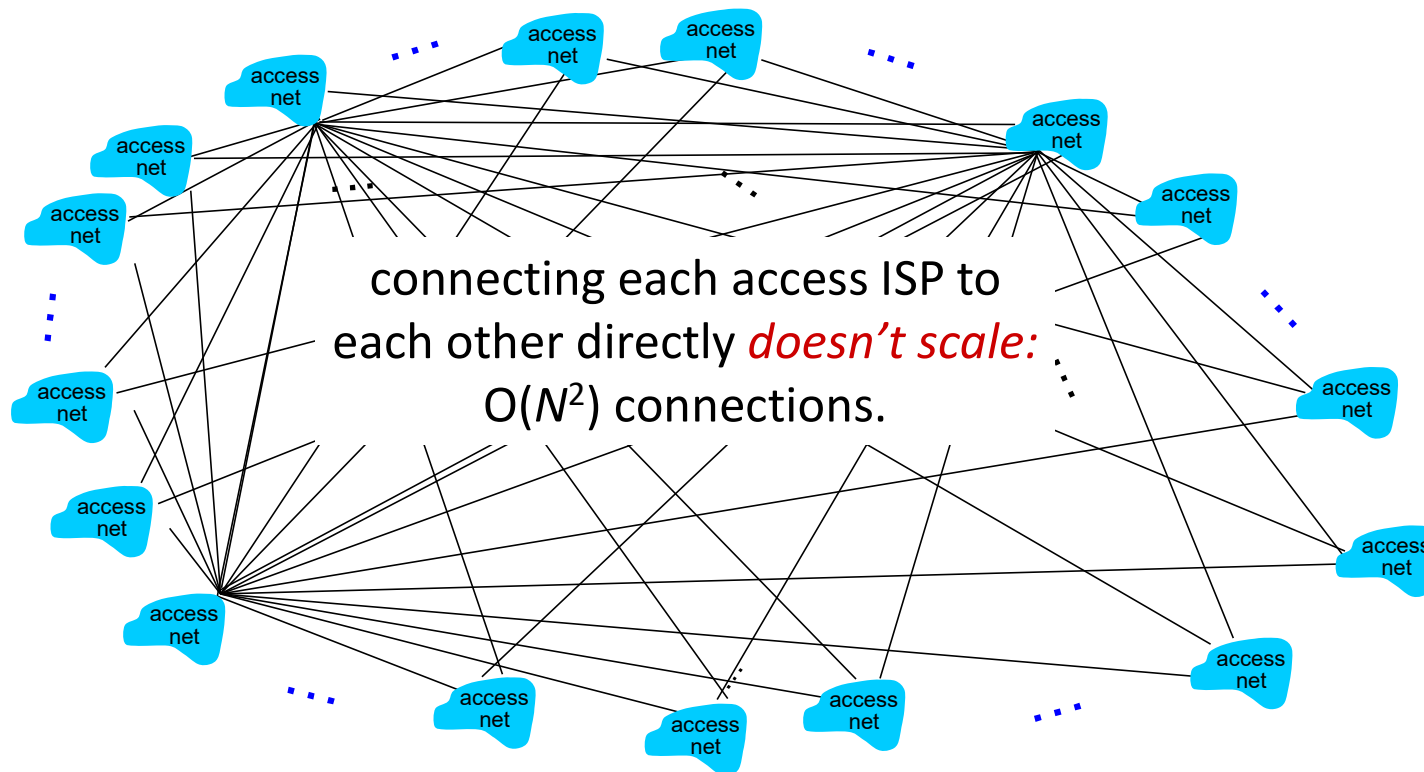
Internet structure: a “network of networks”

Question: given *millions* of access ISPs, how to connect them together?



Internet structure: a “network of networks”

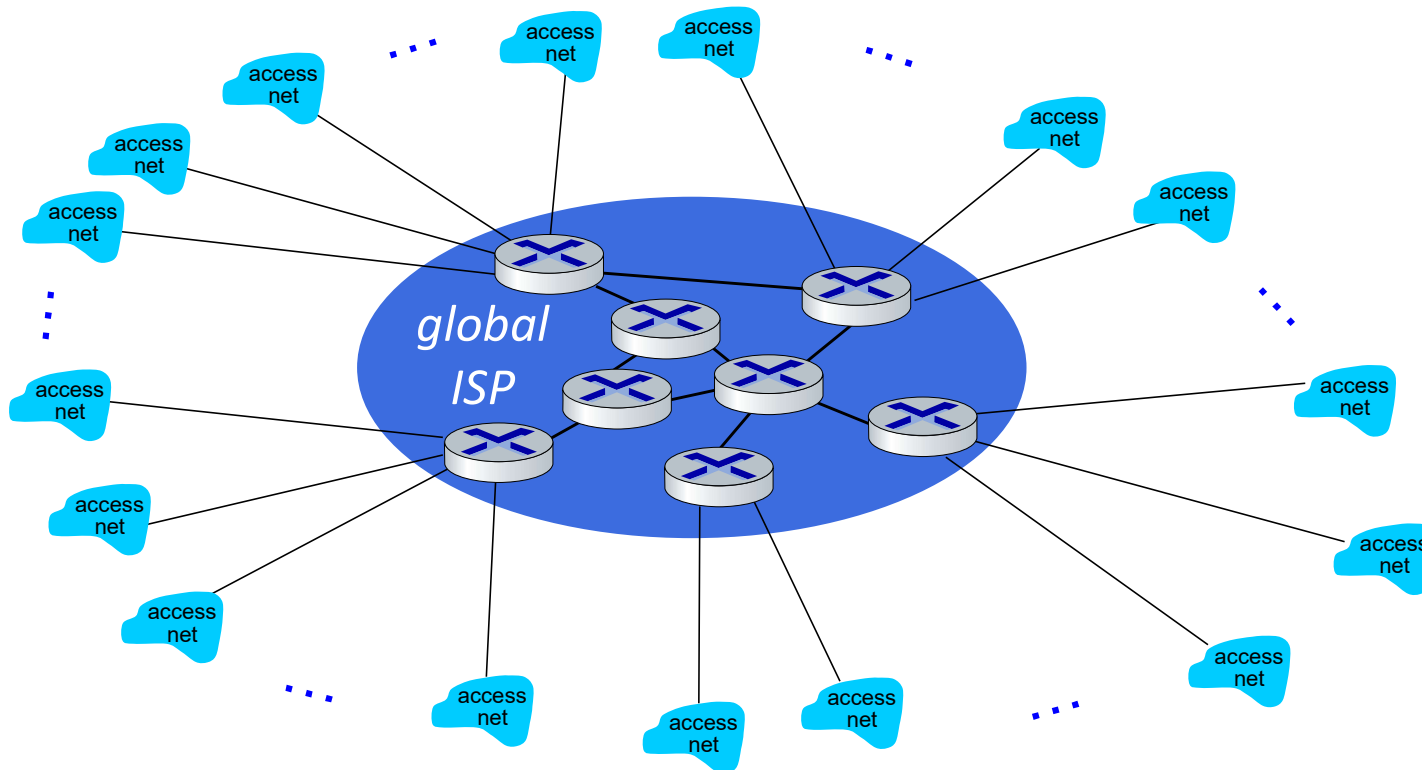
Question: given *millions* of access ISPs, how to connect them together?



Internet structure: a “network of networks”

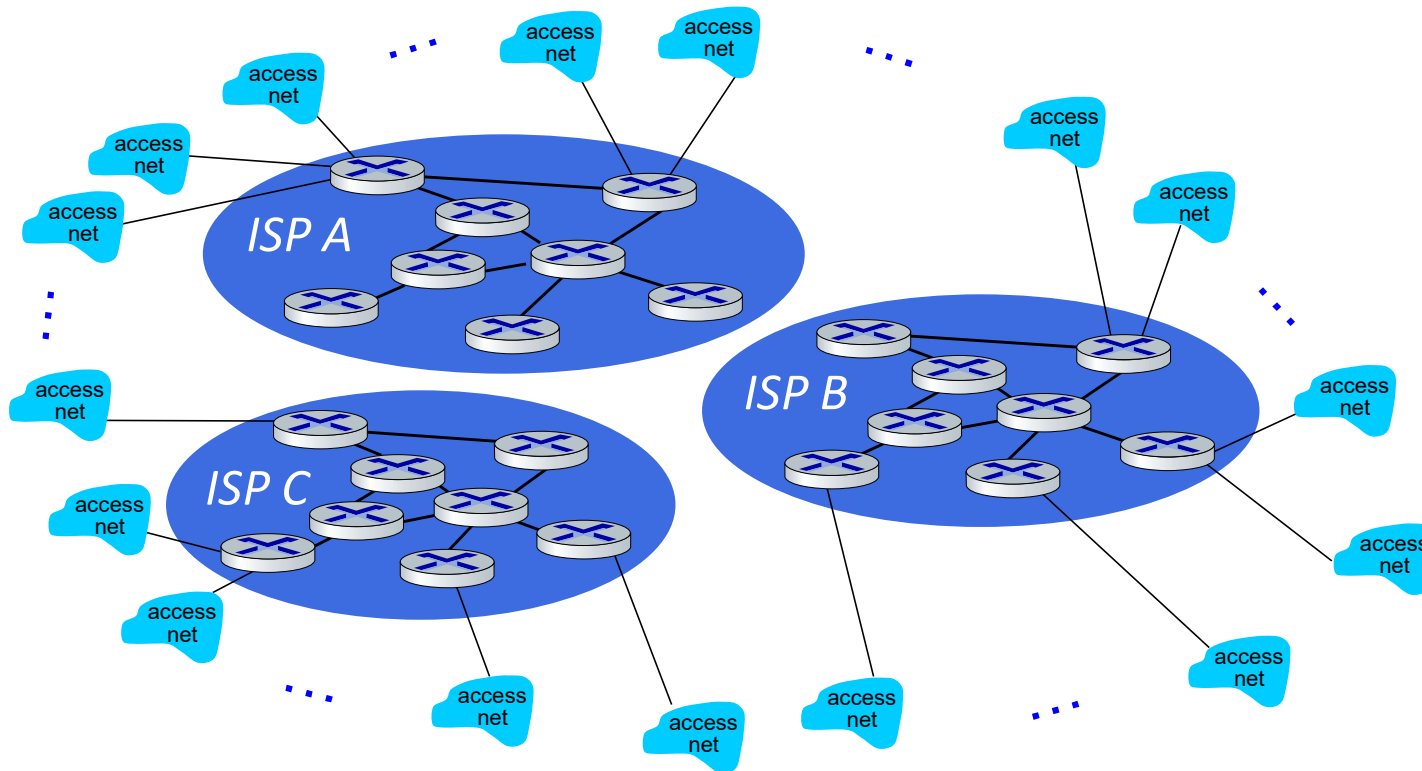
Option: connect each access ISP to one global transit ISP?

Customer and *provider* ISPs have economic agreement.



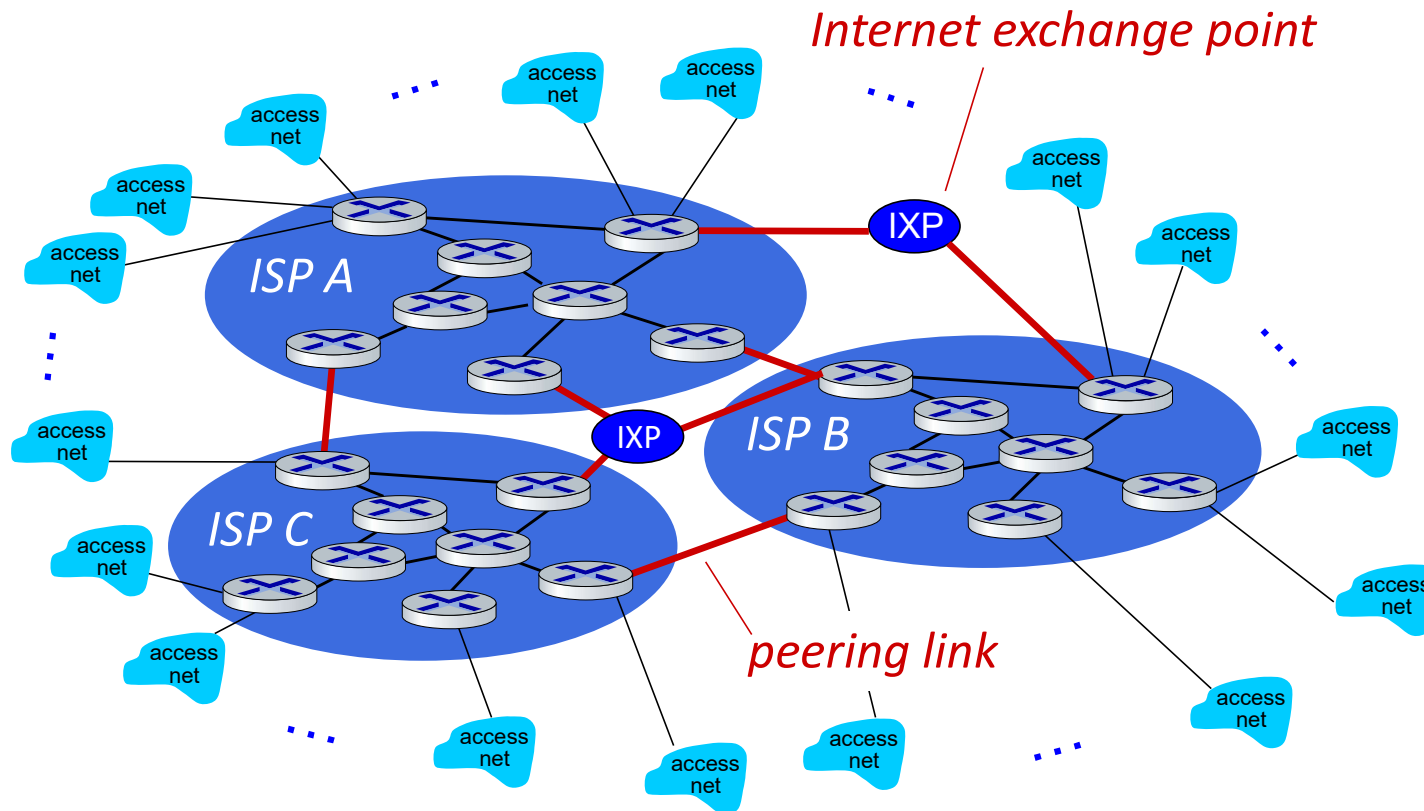
Internet structure: a “network of networks”

But if one global ISP is viable business, there will be competitors



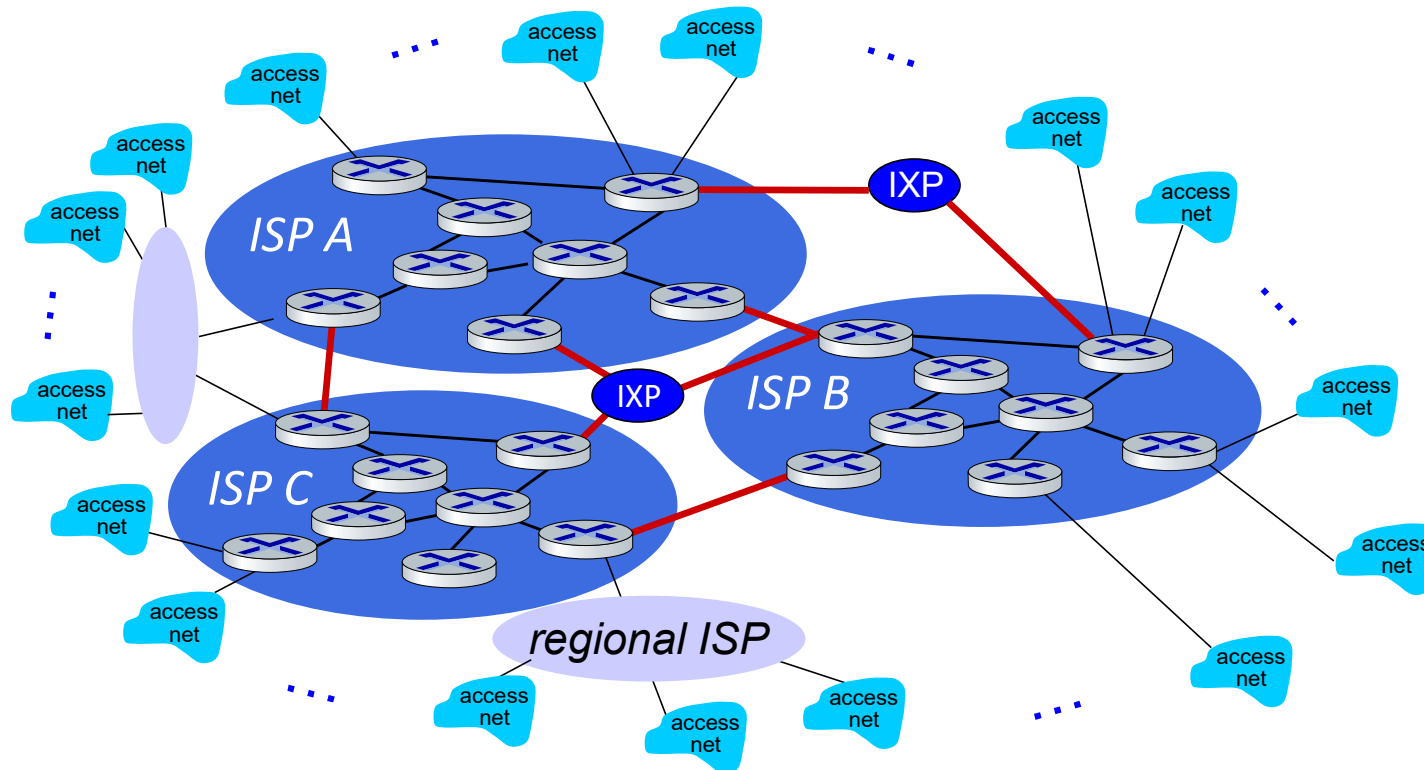
Internet structure: a “network of networks”

But if one global ISP is viable business, there will be competitors who will want to be connected



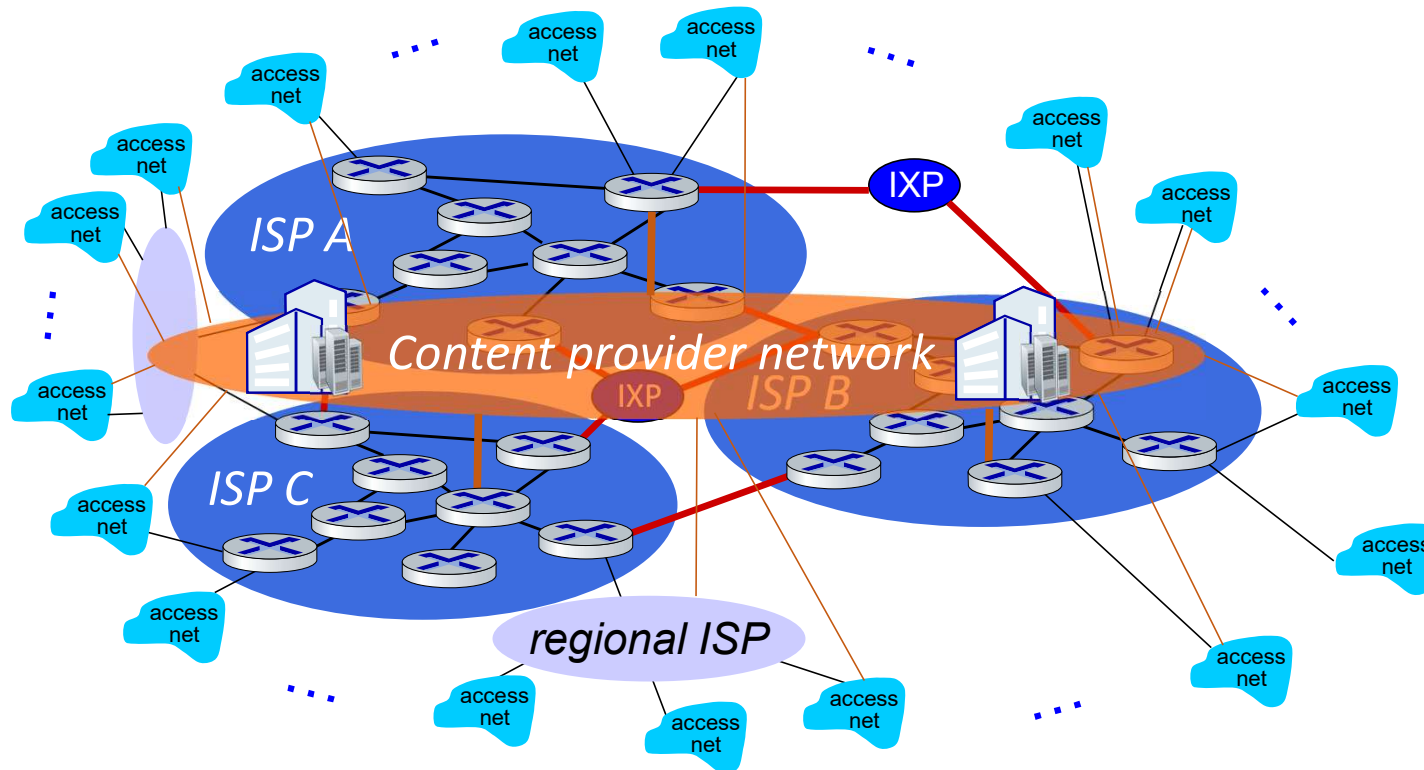
Internet structure: a “network of networks”

... and regional networks may arise to connect access nets to ISPs

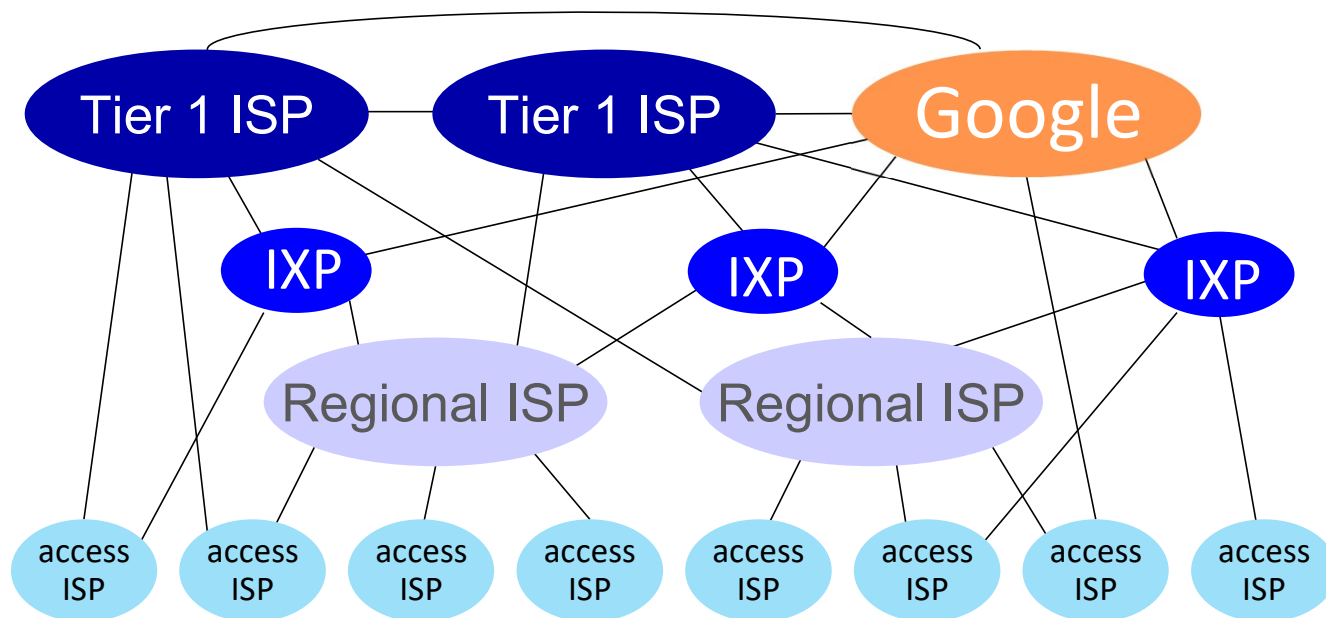


Internet structure: a “network of networks”

... and content provider networks (e.g., Google, Microsoft, Akamai) may run their own network, to bring services, content close to end users



Internet structure: a “network of networks”



At “center”: small # of well-connected large networks

- **“tier-1” commercial ISPs** (e.g., Level 3, Sprint, AT&T, NTT), national & international coverage
- **content provider networks** (e.g., Google, Facebook): private network that connects its data centers to Internet, often bypassing tier-1, regional ISPs

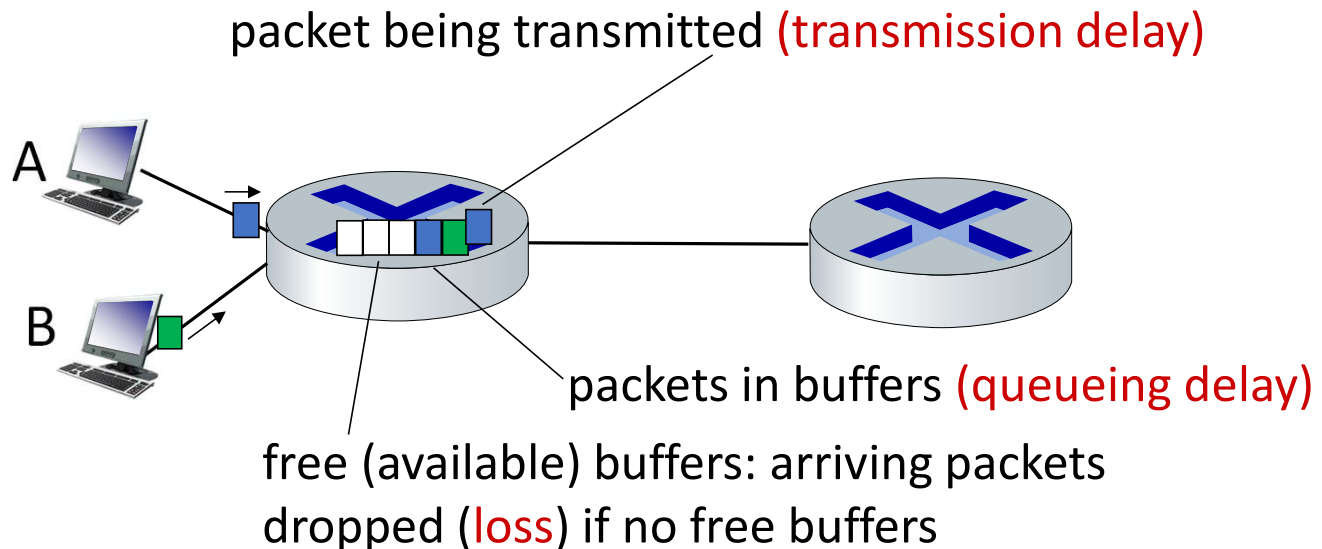
Lecture 1: roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- **Performance: loss, delay, throughput**
- Security
- Protocol layers, service models

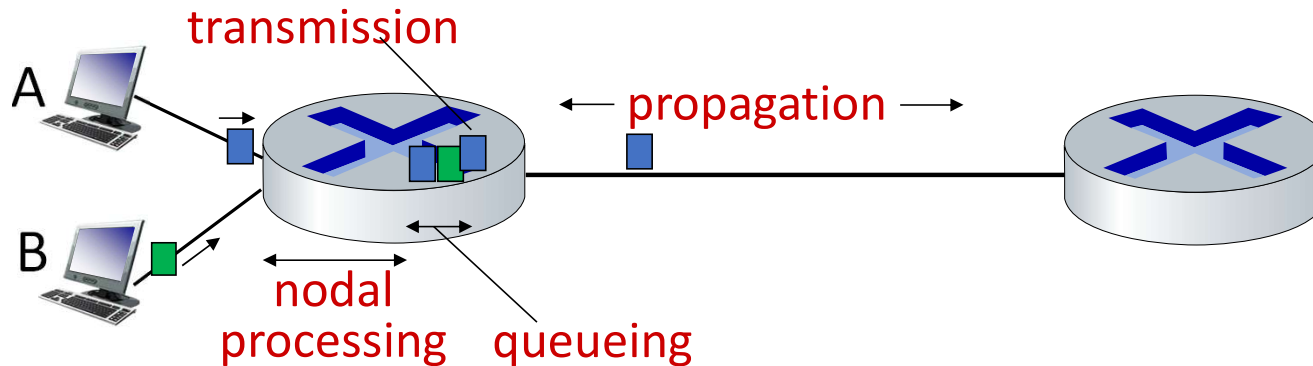


How do packet delay and loss occur?

- packets *queue* in router buffers, waiting for turn for transmission
 - queue length grows when arrival rate to link (temporarily) exceeds output link capacity
- packet *loss* occurs when memory to hold queued packets fills up



Packet delay: four sources



$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

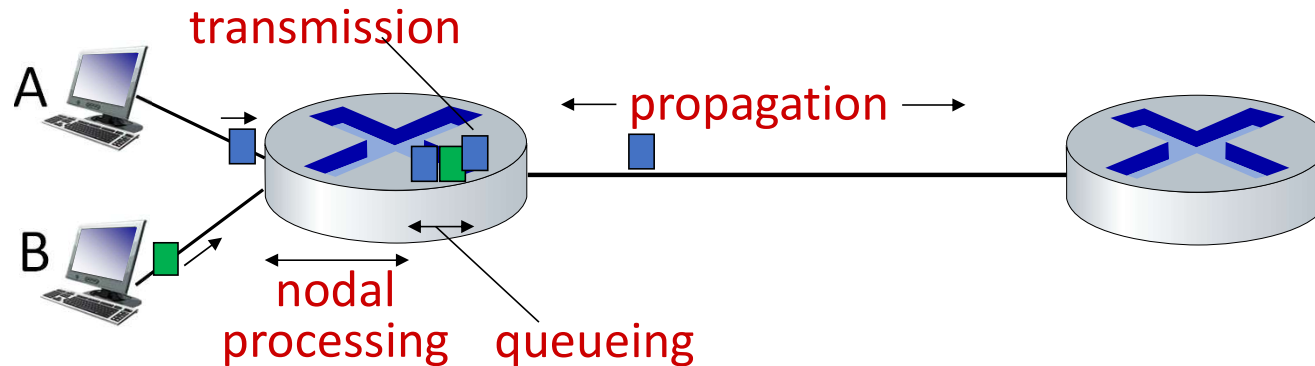
d_{proc} : nodal processing

- check bit errors
- determine output link
- typically < microsecs

d_{queue} : queueing delay

- time waiting at output link for transmission
- depends on congestion level of router

Packet delay: four sources



$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$

d_{trans} : transmission delay:

- L : packet length (bits)
- R : link transmission rate (bps)

▪ $d_{\text{trans}} = L/R$

d_{prop} : propagation delay:

- d : length of physical link
- s : propagation speed ($\sim 2 \times 10^8$ m/sec)

▪ $d_{\text{prop}} = d/s$

d_{trans} and d_{prop}
very different

NETWORKING DELAYS (summary)

▪ **Transmission Delay**

- The time it takes to transmit a packet through a certain medium (time it takes to put all bits on the cable or transmit them in the air)

▪ **Propagation Delay**

- The time it takes a signal to travel across a communication link.

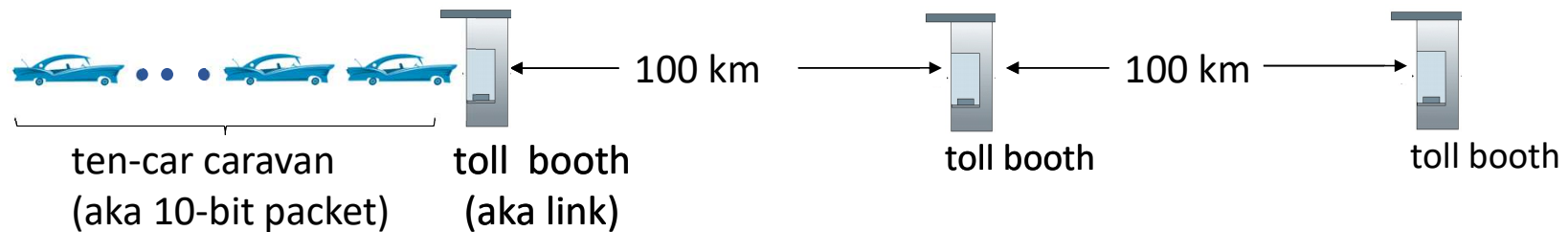
▪ **Processing Delay**

- The time needed by devices such as switches and routers to read the headers of incoming packets and choose the outgoing interface

▪ **Queuing Delay**

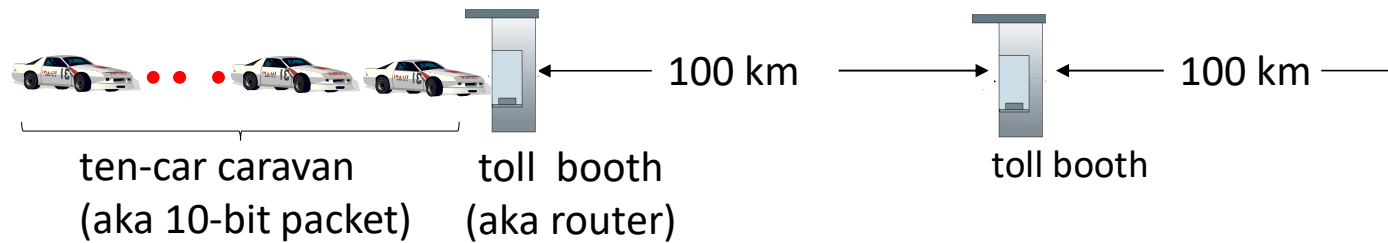
- occurs in packet-switched networks. Each router/switch enqueues incoming packets. If queue already contains packets, the new packet may need to wait while the device forwards/routes packets that arrived earlier

Caravan analogy



- car $\sim$ bit; caravan $\sim$ packet; toll service $\sim$ link transmission
- toll booth takes 12 sec to service car (bit transmission time)
- “propagate” at 100 km/hr
- **Q: How long until caravan is lined up before 2nd toll booth?**
- time to “push” entire caravan through toll booth onto highway = $12 \times 10 = 120$ sec
- time for last car to propagate from 1st to 2nd toll booth: $100\text{km}/(100\text{km/hr}) = 1$ hr
- **A: 62 minutes**

Caravan analogy



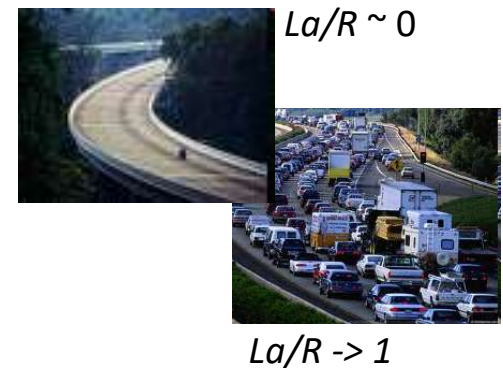
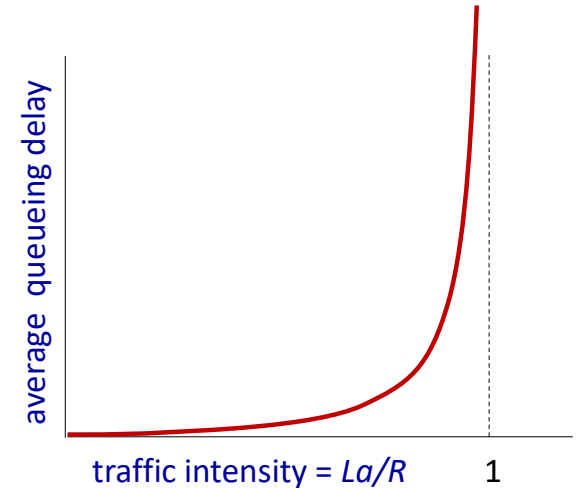
- suppose cars now “propagate” at 1000 km/hr
- and suppose toll booth now takes one min to service a car
- **Q: Will cars arrive to 2nd booth before all cars serviced at first booth?**
A: Yes! after 7 min, first car arrives at second booth; three cars still at first booth

Packet queueing delay (revisited)

- a : average packet arrival rate
- L : packet length (bits)
- R : link bandwidth (bit transmission rate)

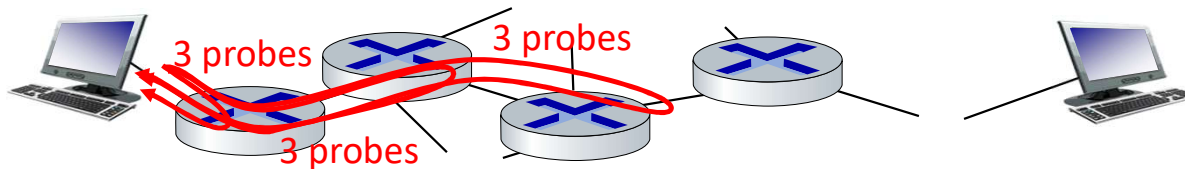
$$\frac{L \cdot a}{R} : \frac{\text{arrival rate of bits}}{\text{service rate of bits}} \quad \text{“traffic intensity”}$$

- $La/R \sim 0$: avg. queueing delay small
- $La/R \rightarrow 1$: avg. queueing delay large
- $La/R > 1$: more “work” arriving is more than can be serviced - average delay infinite!



“Real” Internet delays and routes

- what do “real” Internet delay & loss look like?
- **traceroute** program: provides delay measurement from source to router along end-end Internet path towards destination. For all i :
 - sends three packets that will reach router i on path towards destination (with time-to-live field value of i)
 - router i will return packets to sender
 - sender measures time interval between transmission and reply



Real Internet delays and routes

traceroute: gaia.cs.umass.edu to www.eurecom.fr

3 delay measurements from
gaia.cs.umass.edu to cs-gw.cs.umass.edu

3 delay measurements
to border1-rt-fa5-1-0.gw.umass.edu

trans-oceanic link

looks like delays
decrease! Why?

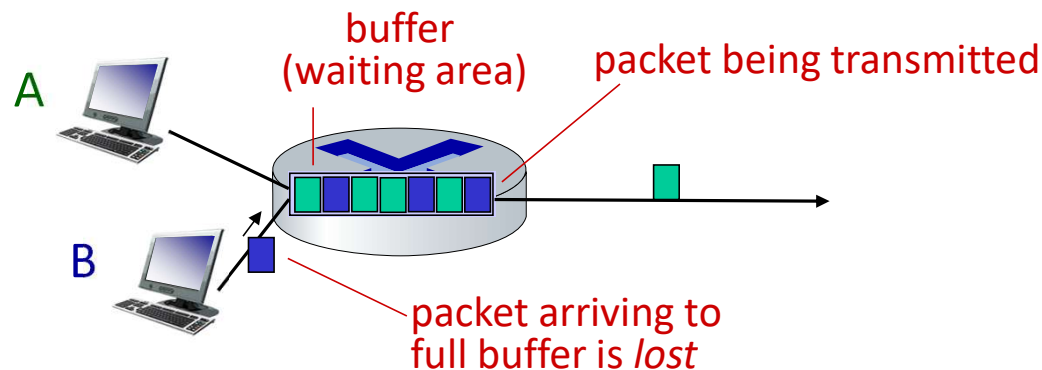
* means no response (probe lost, router not replying)

```
1 cs-gw (128.119.240.254) 1 ms 1 ms 2 ms
2 border1-rt-fa5-1-0.gw.umass.edu (128.119.3.145) 1 ms 1 ms 2 ms
3 cht-vbns.gw.umass.edu (128.119.3.130) 6 ms 5 ms 5 ms
4 jn1-at1-0-0-19.wor.vbns.net (204.147.132.129) 16 ms 11 ms 13 ms
5 jn1-so7-0-0-0.wae.vbns.net (204.147.136.136) 21 ms 18 ms 18 ms
6 abilene-vbns.abilene.ucaid.edu (198.32.11.9) 22 ms 18 ms 22 ms
7 nycm-wash.abilene.ucaid.edu (198.32.8.46) 22 ms 22 ms 22 ms
8 62.40.103.253 (62.40.103.253) 104 ms 109 ms 106 ms
9 de2-1.de1.de.geant.net (62.40.96.129) 109 ms 102 ms 104 ms
10 de.fr1.fr.geant.net (62.40.96.50) 113 ms 121 ms 114 ms
11 renater-gw.fr1.fr.geant.net (62.40.103.54) 112 ms 114 ms 112 ms
12 nio-n2.cssi.renater.fr (193.51.206.13) 111 ms 114 ms 116 ms
13 nice.cssi.renater.fr (195.220.98.102) 123 ms 125 ms 124 ms
14 r3t2-nice.cssi.renater.fr (195.220.98.110) 126 ms 126 ms 124 ms
15 eurecom-valbonne.r3t2.ft.net (193.48.50.54) 135 ms 128 ms 133 ms
16 194.214.211.25 (194.214.211.25) 126 ms 128 ms 126 ms
17 * * *
18 * * *
19 fantasia.eurecom.fr (193.55.113.142) 132 ms 128 ms 136 ms
```

* Do some traceroutes from exotic countries at www.traceroute.org

Packet loss

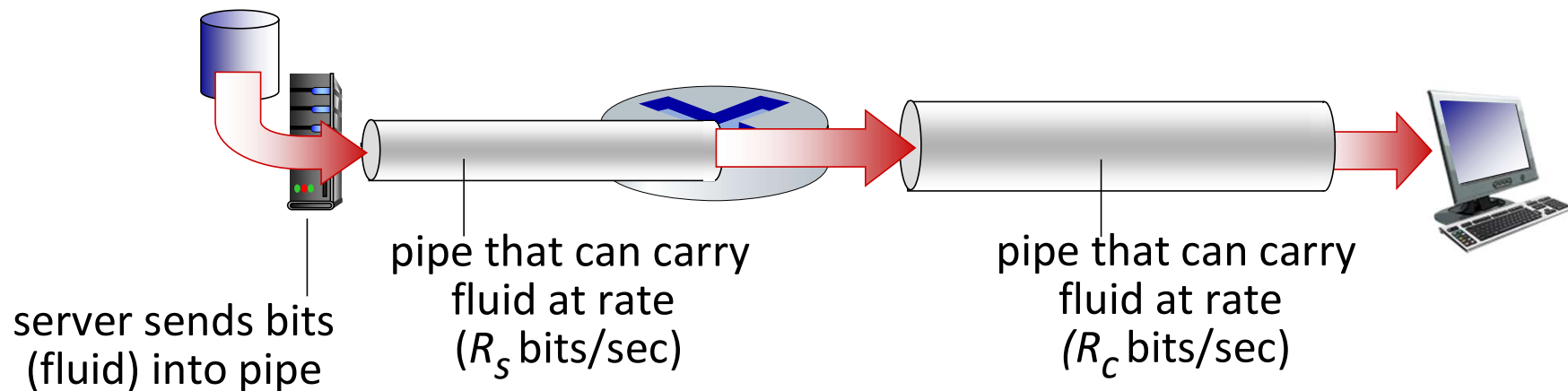
- queue (aka buffer) preceding link in buffer has finite capacity
- packet arriving to full queue dropped (aka lost)
- lost packet may be retransmitted by previous node, by source end system, or not at all



* Check out the Java applet for an interactive animation (on publisher's website) of queuing and loss

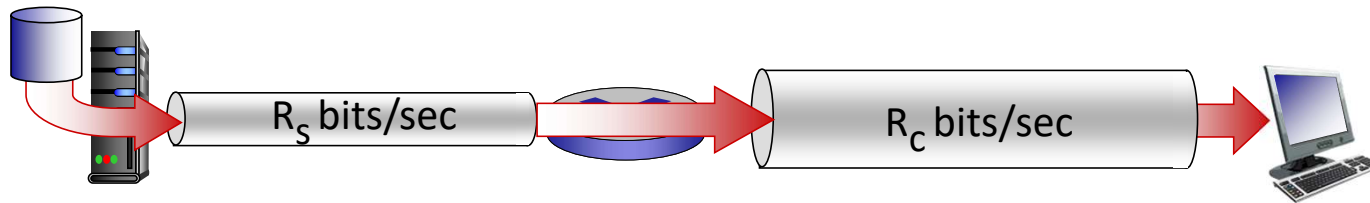
Throughput

- *throughput*: rate (bits/time unit) at which bits are being sent from sender to receiver
 - *instantaneous*: rate at given point in time
 - *average*: rate over longer period of time

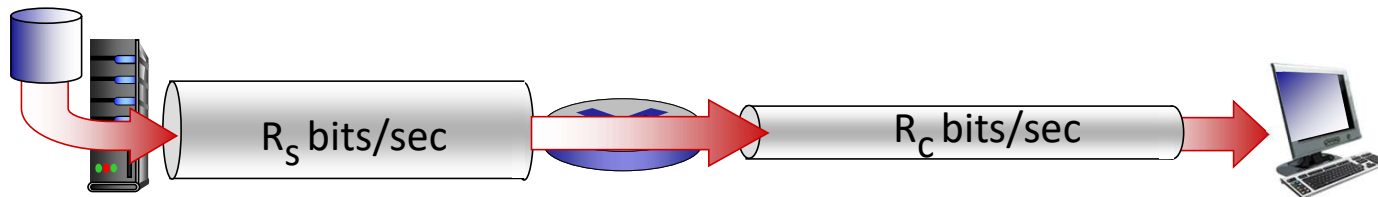


Throughput

$R_s < R_c$ What is average end-end throughput?



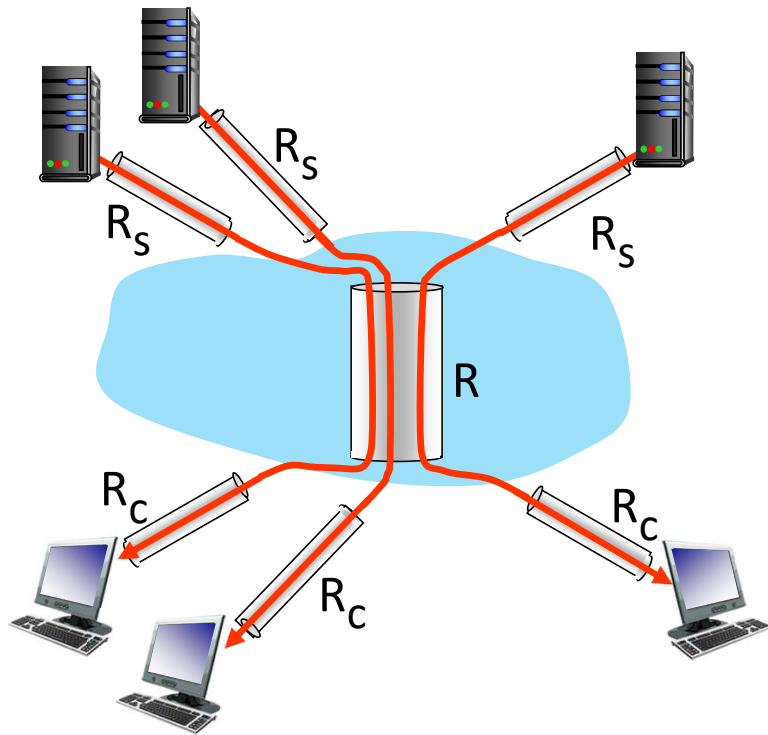
$R_s > R_c$ What is average end-end throughput?



bottleneck link

link on end-end path that constrains end-end throughput

Throughput: network scenario



10 connections (fairly) share
backbone bottleneck link R bits/sec

- per-connection end-end throughput:
 $\min(R_c, R_s, R/10)$
- in practice: R_c or R_s is often bottleneck

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/

Lecture 1: roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- **Security**
- Protocol layers, service models



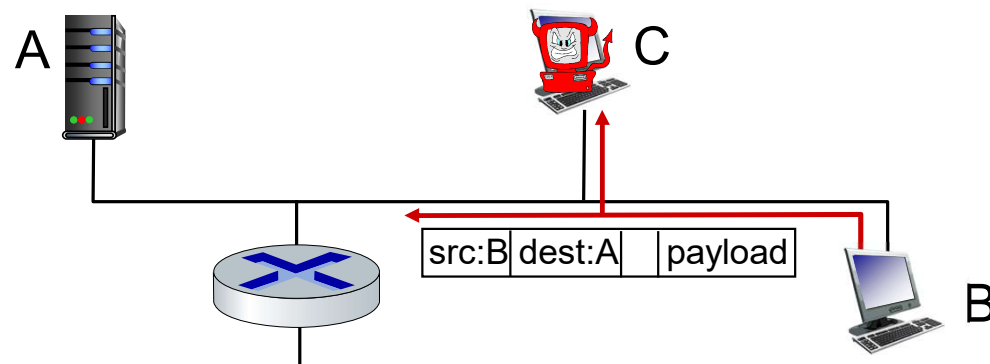
Network security

- Internet not originally designed with (much) security in mind
 - *original vision*: “a group of mutually trusting users attached to a transparent network” 😊
 - Internet protocol designers playing “catch-up”
 - security considerations in all layers!
- We now need to think about:
 - how bad guys can attack computer networks
 - how we can defend networks against attacks
 - how to design architectures that are immune to attacks

Bad guys: packet interception

packet “sniffing”:

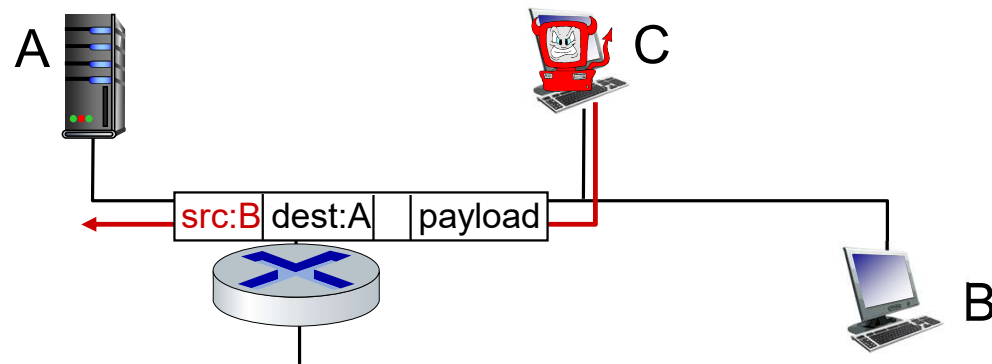
- broadcast media (shared Ethernet, wireless)
- promiscuous network interface reads/records all packets (e.g., including passwords!) passing by



Wireshark software used for our end-of-chapter labs is a (free) packet-sniffer

Bad guys: fake identity

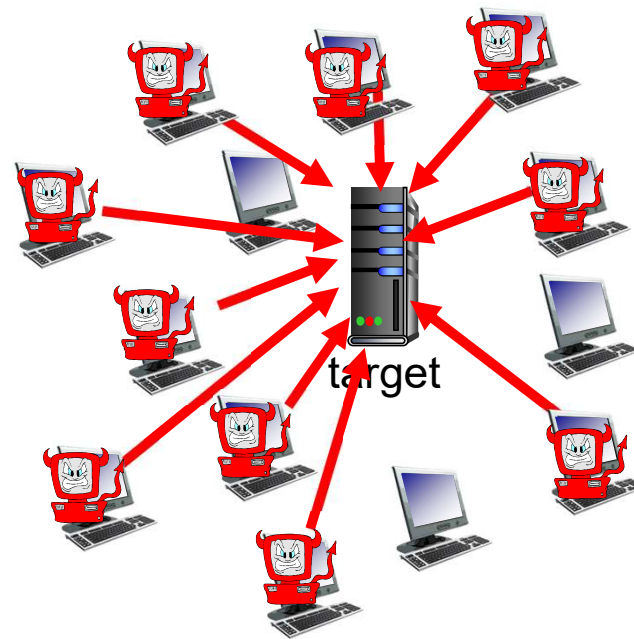
IP spoofing: injection of packet with false source address



Bad guys: denial of service

Denial of Service (DoS): attackers make resources (server, bandwidth) unavailable to legitimate traffic by overwhelming resource with bogus traffic

1. select target
2. break into hosts around the network (see botnet)
3. send packets to target from compromised hosts



Lines of defense:

- **authentication:** proving you are who you say you are
 - cellular networks provides hardware identity via SIM card; no such hardware assist in traditional Internet
- **confidentiality:** via encryption
- **integrity checks:** digital signatures prevent/detect tampering
- **access restrictions:** password-protected VPNs
- **firewalls:** specialized “middleboxes” in access and core networks:
 - off-by-default: filter incoming packets to restrict senders, receivers, applications
 - detecting/reacting to DOS attacks

... lots more on security (in 3rd year course)

Lecture 1: roadmap

- What *is* the Internet?
- What *is* a protocol?
- Network edge: hosts, access network, physical media
- Network core: packet/circuit switching, internet structure
- Performance: loss, delay, throughput
- Security
- Protocol layers, service models



Protocol “layers” and reference models

Networks are complex,
with many “pieces”:

- hosts
- routers
- links of various media
- applications
- protocols
- hardware, software

Question: is there any
hope of *organizing*
structure of network?

- and/or our *discussion*
of networks?

Example: organization of air travel



————— *end-to-end transfer of person plus baggage* —————>

ticket (purchase)

baggage (check)

gates (load)

runway takeoff

airplane routing

ticket (complain)

baggage (claim)

gates (unload)

runway landing

airplane routing

airplane routing

How would you *define/discuss* the *system* of airline travel?

- a series of steps, involving many services

Example: organization of air travel



layers: each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

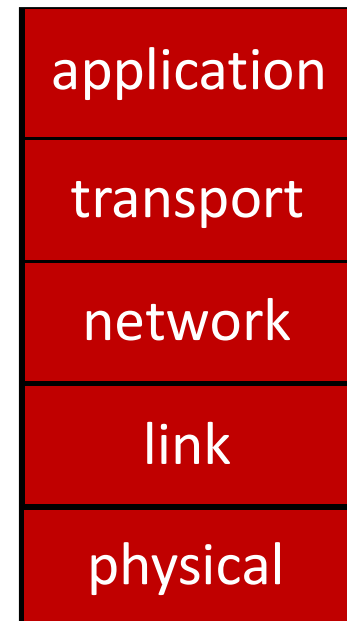
Why layering?

Approach to designing/discussing complex systems:

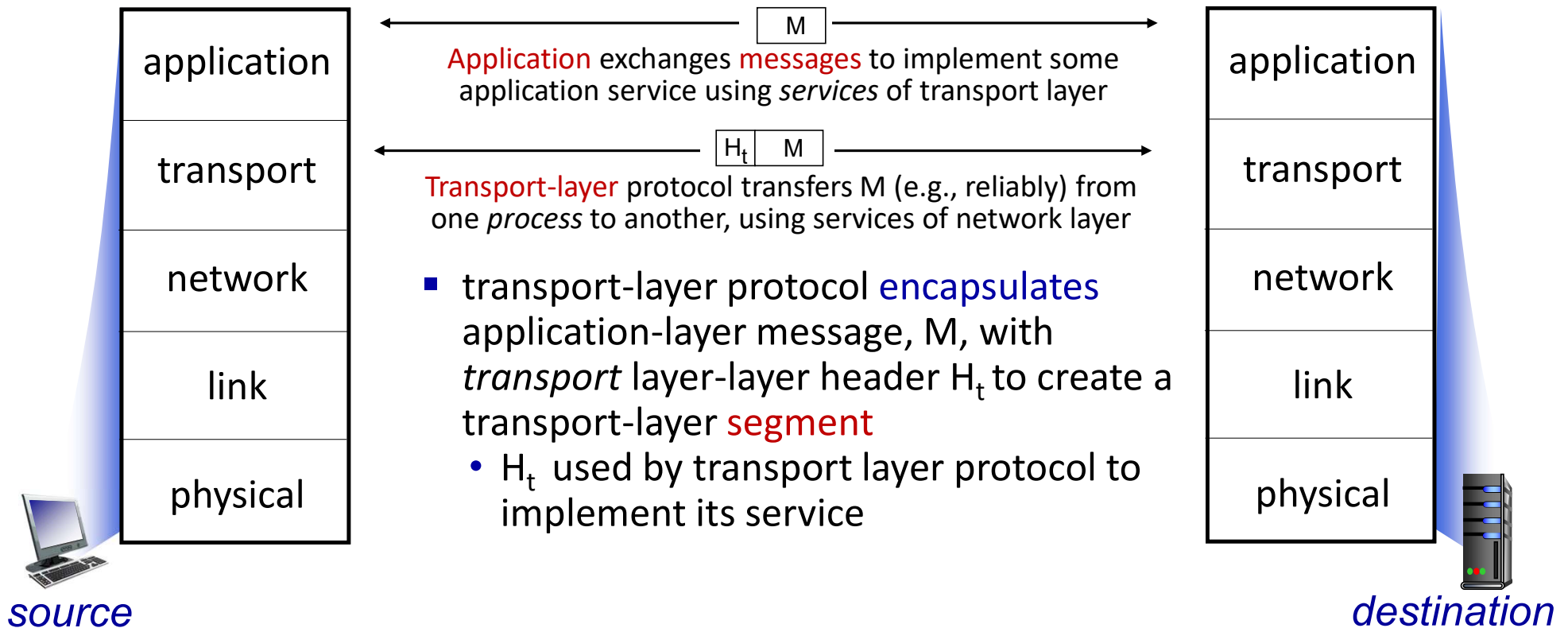
- explicit structure allows identification, relationship of system's pieces
 - layered *reference model* for discussion
- modularization eases maintenance, updating of system
 - change in layer's service *implementation*: transparent to rest of system
 - e.g., change in gate procedure doesn't affect rest of system

Layered Internet protocol stack

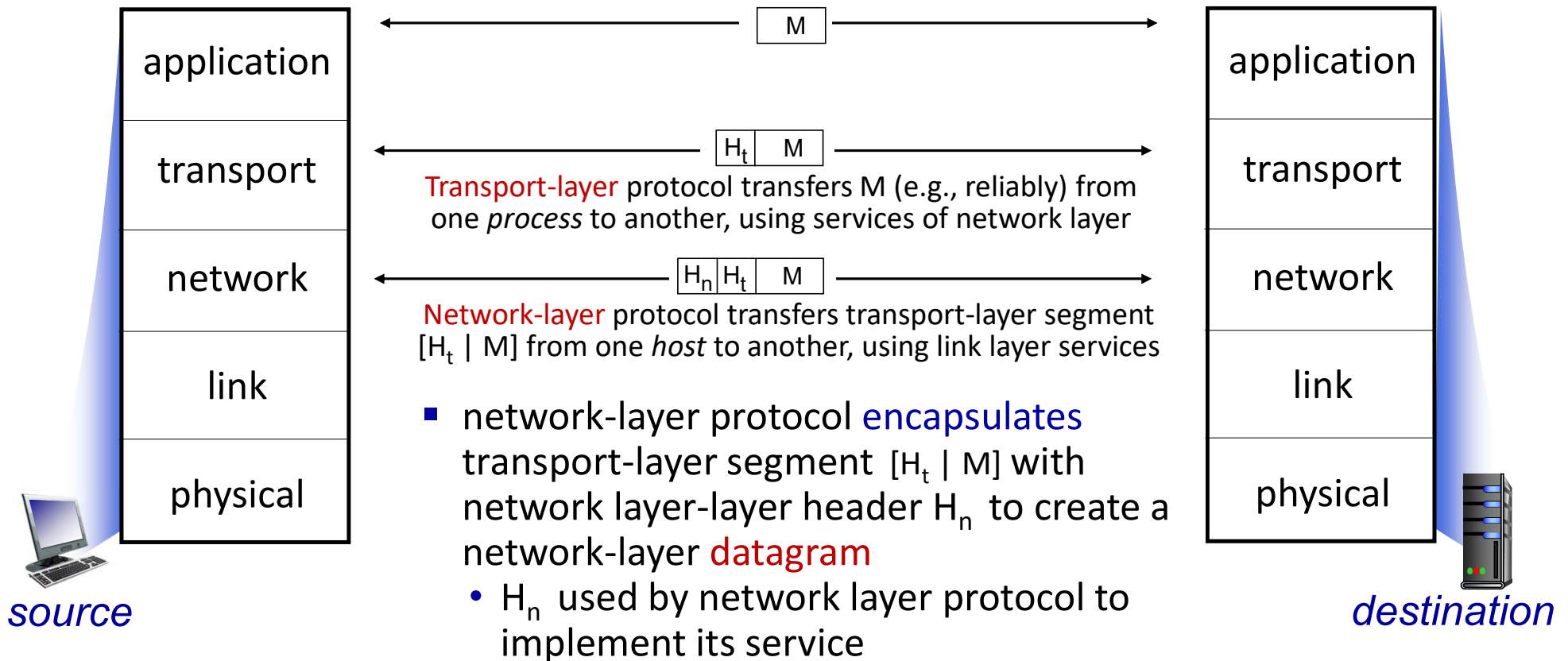
- *application*: supporting network applications
 - HTTP, IMAP, SMTP, DNS
- *transport*: process-process data transfer
 - TCP, UDP
- *network*: routing of datagrams from source to destination
 - IP, routing protocols
- *link*: data transfer between neighboring network elements
 - Ethernet, 802.11 (WiFi), PPP
- *physical*: bits “on the wire”



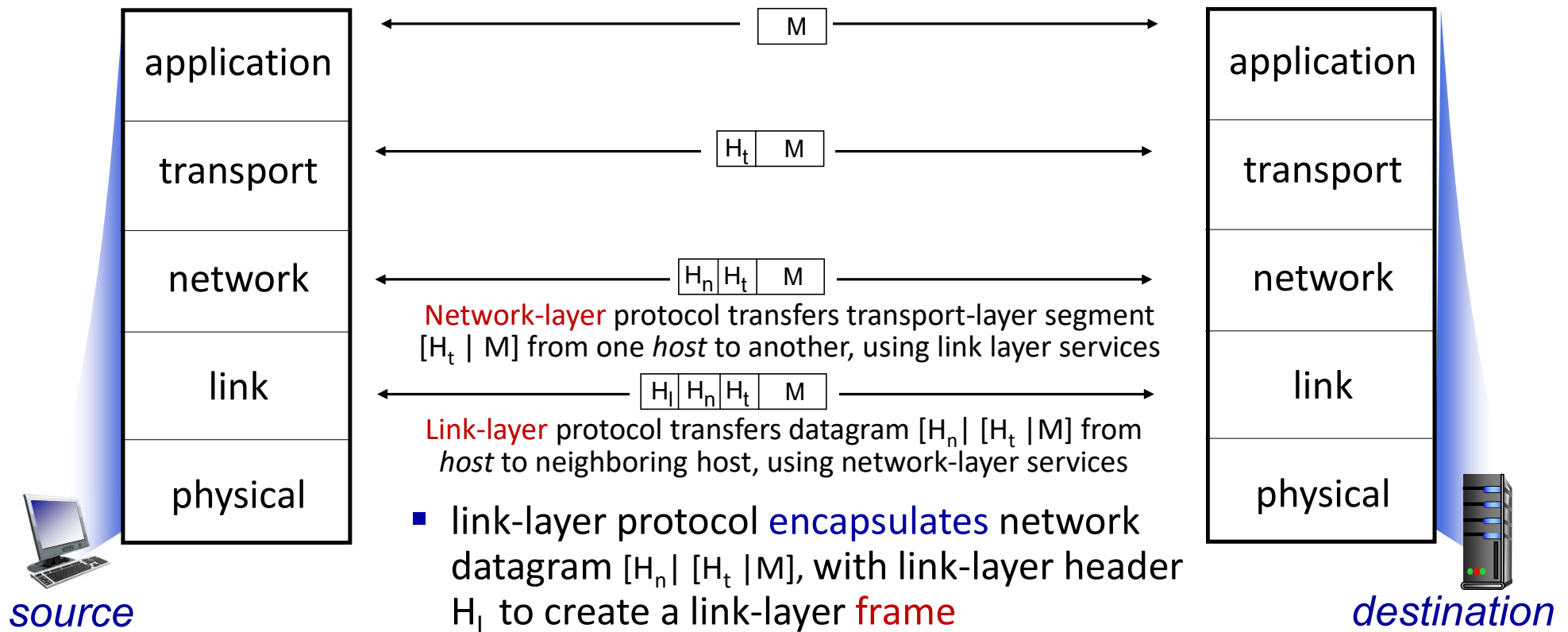
Services, Layering and Encapsulation



Services, Layering and Encapsulation

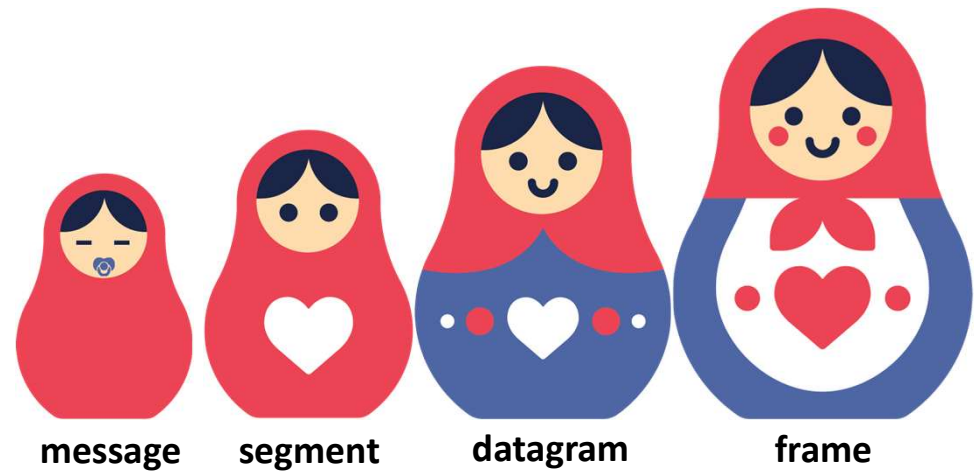
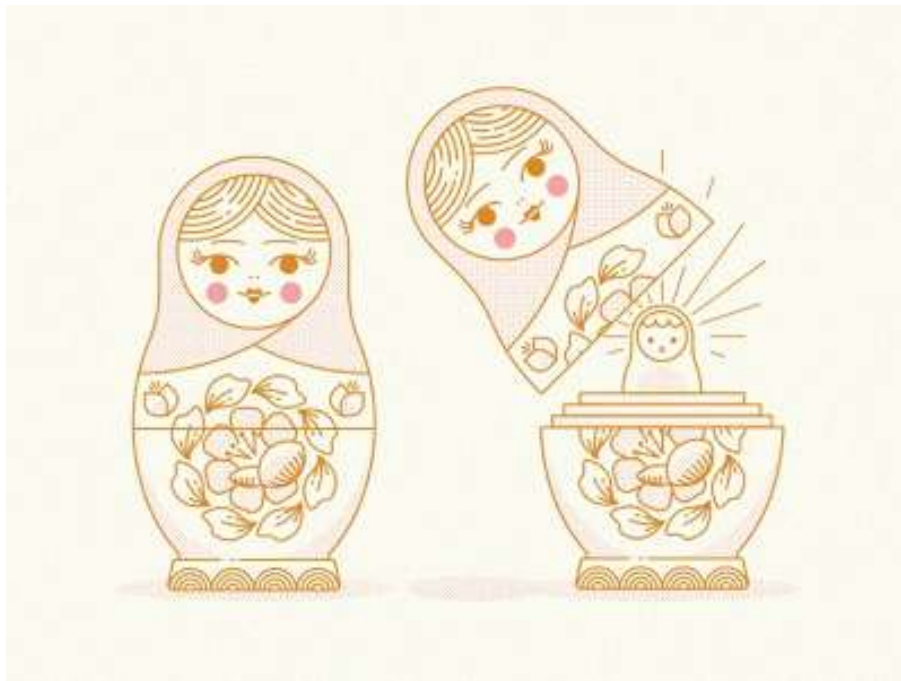


Services, Layering and Encapsulation

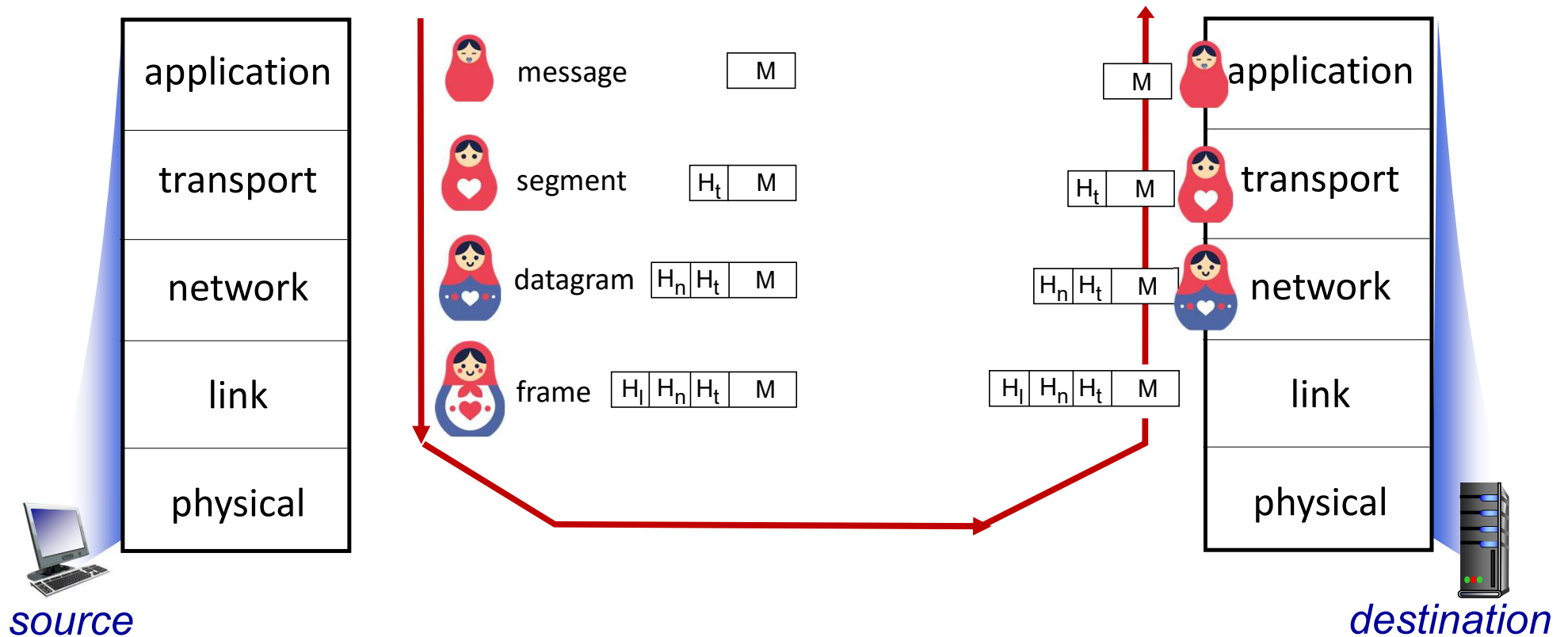


Encapsulation

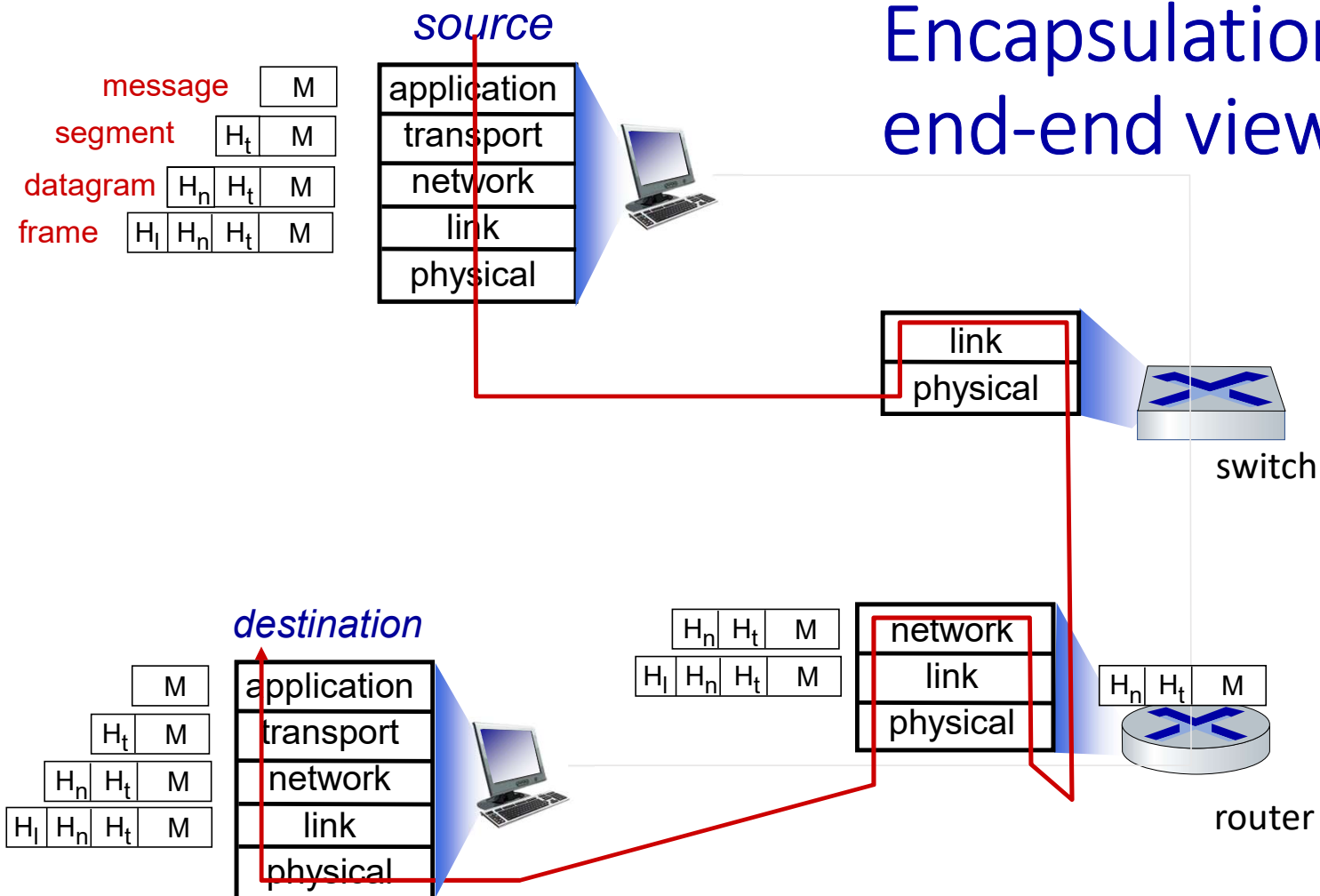
Matryoshka dolls (stacking dolls)



Services, Layering and Encapsulation



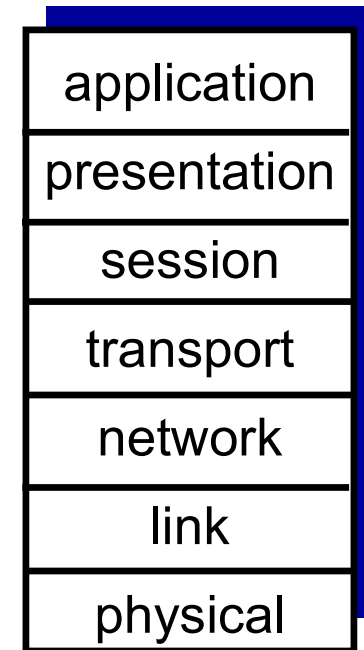
Encapsulation: an end-end view



ISO/OSI reference model

Two layers not found in Internet protocol stack!

- *presentation*: allow applications to interpret meaning of data, e.g., encryption, compression, machine-specific conventions
- *session*: synchronization, checkpointing, recovery of data exchange
- Internet stack “missing” these layers!
 - these services, *if needed*, must be implemented in application
 - needed?



The seven layer OSI/ISO reference model

Lecture 1: summary

We've covered a "ton" of material!

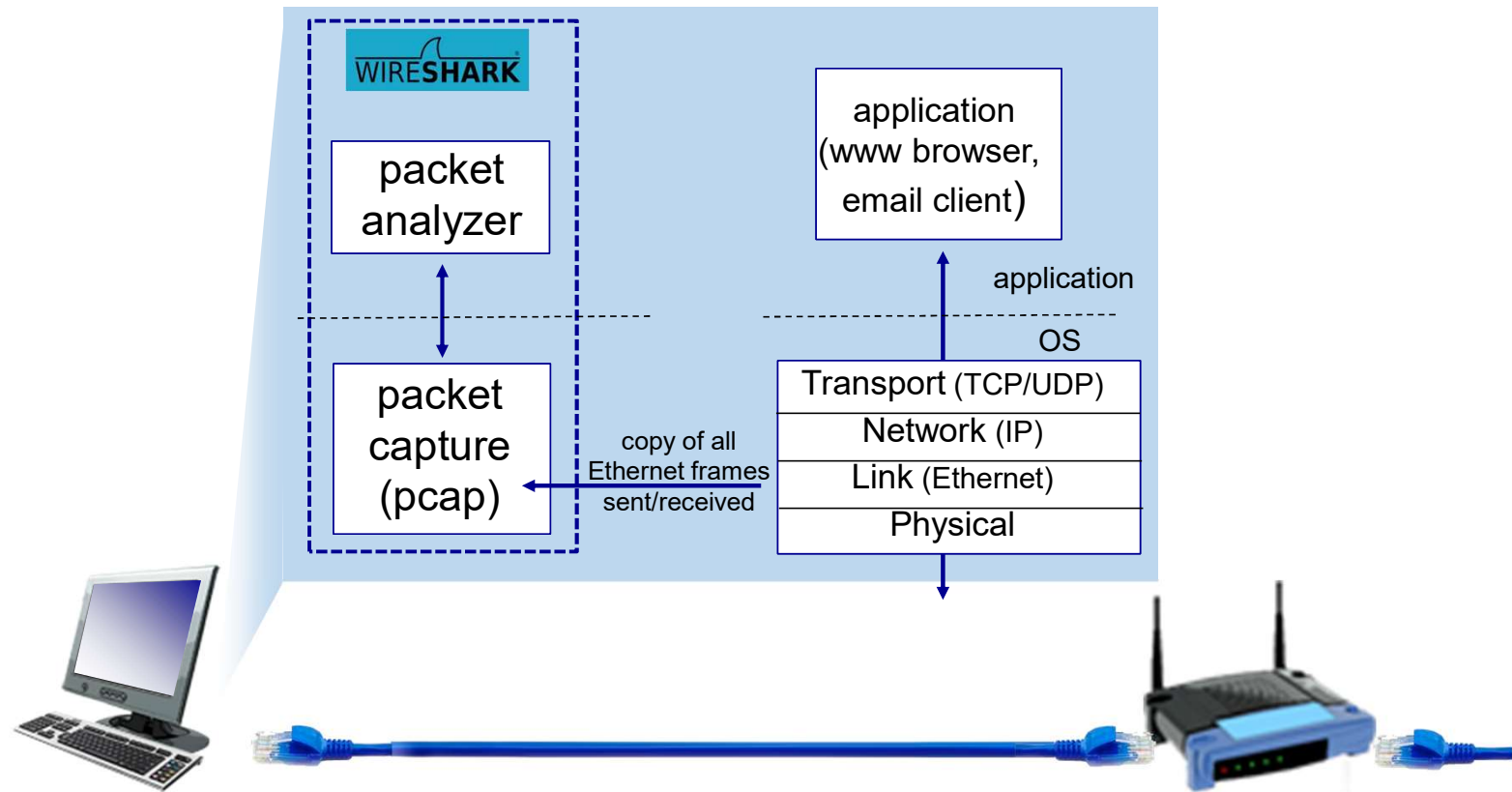
- Internet overview
- what's a protocol?
- network edge, access network, core
 - packet-switching versus circuit-switching
 - Internet structure
- performance: loss, delay, throughput
- layering, service models
- security

You now have:

- context, overview, vocabulary, "feel" of networking
- more depth, detail, *and fun* to follow!

Additional Chapter 1 slides

Wireshark





NETWORK ARCHITECTURES

Application Layer

G. Kalfas

Notes on the slides

- The material (slides) of this course is a mixture of the following:
 1. slides provided freely by J.F Kurose and K.W. Ross accompanying the book “Computer Networking: A top-down approach” (this book is mentioned in the references of the Syllabus - slides can be found here: https://gaia.cs.umass.edu/kurose_ross/ppt.php)
 2. slides provided by CITY college and
 3. my own slides

Application layer: overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System DNS



Application layer: overview

Our goals:

- conceptual *and* implementation aspects of application-layer protocols
 - transport-layer service models
 - client-server paradigm
 - peer-to-peer paradigm
- learn about protocols by examining popular application-layer protocols and infrastructure
 - HTTP
 - SMTP, IMAP
 - DNS

Some network apps

- social networking
 - Web
 - text messaging
 - e-mail
 - multi-user network games
 - streaming stored video (YouTube, Hulu, Netflix)
 - P2P file sharing
 - voice over IP (e.g., MS Teams)
 - real-time video conferencing (e.g., Zoom)
 - Internet search
 - remote login
 - ...
- Q: *your* favorites?

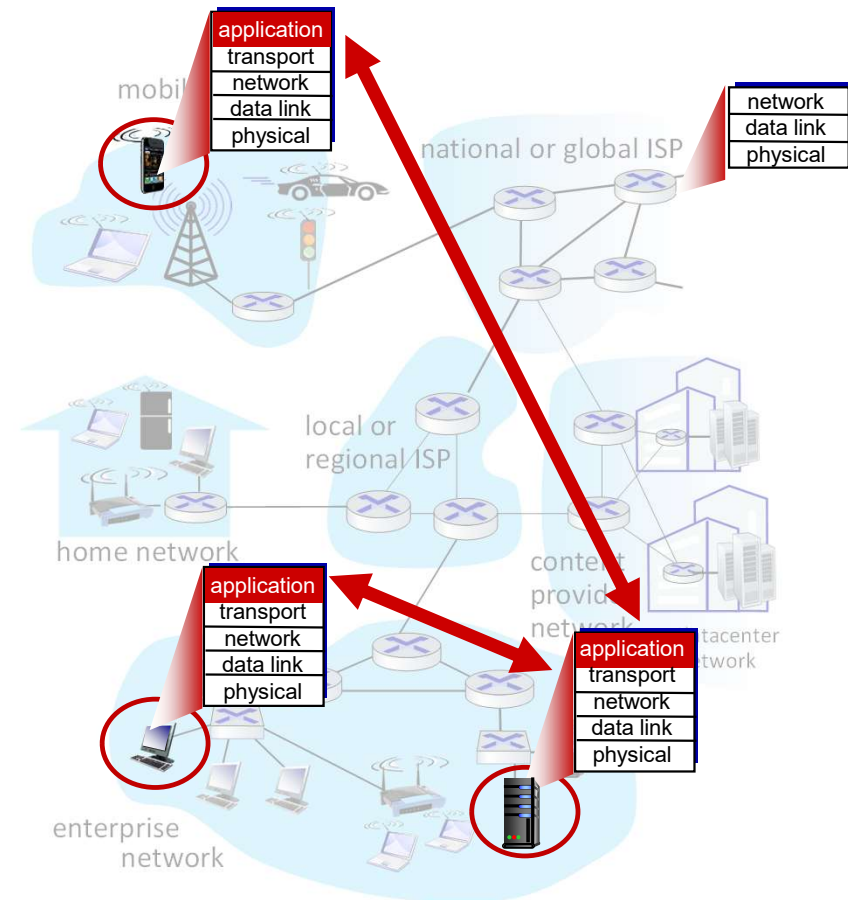
Creating a network app

write programs that:

- run on (different) end systems
- communicate over network
- e.g., web server software communicates with browser software

no need to write software for network-core devices

- network-core devices do not run user applications
- applications on end systems allows for rapid app development, propagation



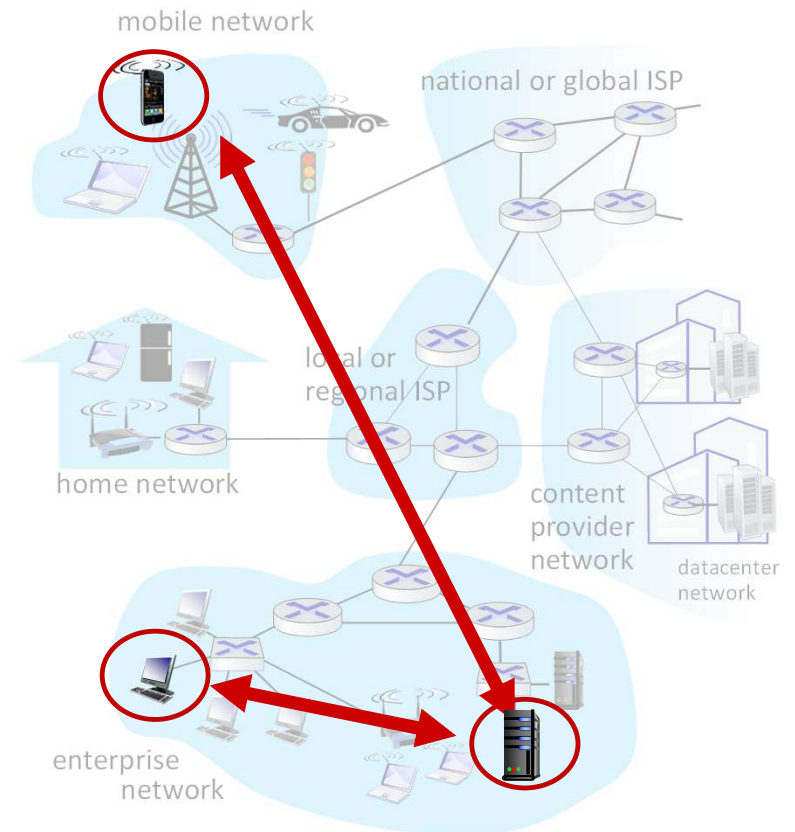
Client-server paradigm

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

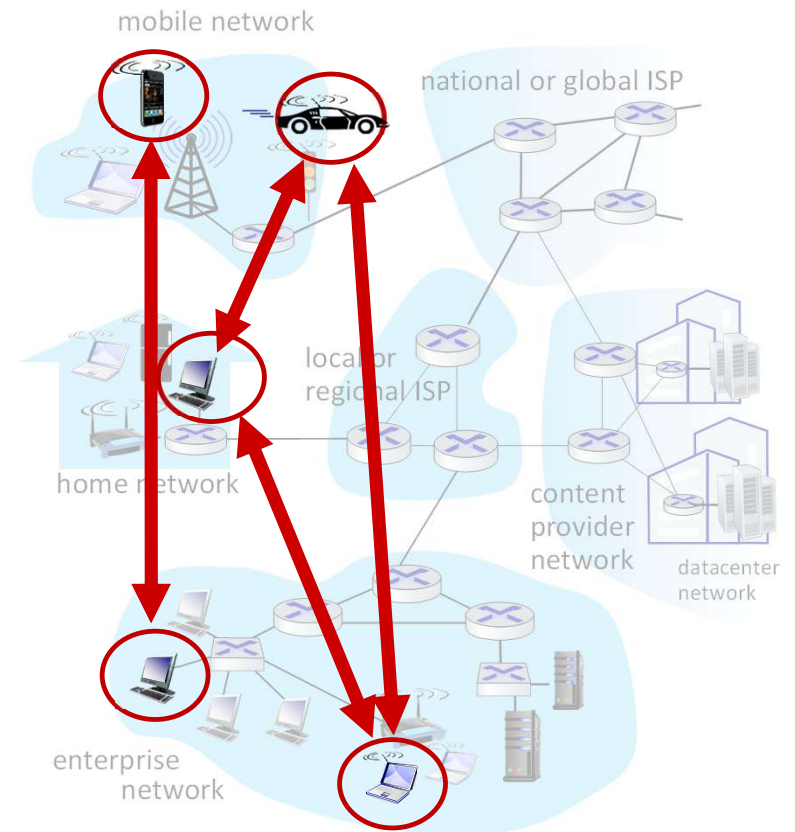
clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP



Peer-peer architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, as well as new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- example: P2P file sharing [BitTorrent]



Processes communicating

process: program running within a host

- within same host, two processes communicate using **inter-process communication** (defined by OS)
- processes in different hosts communicate by exchanging **messages**

clients, servers

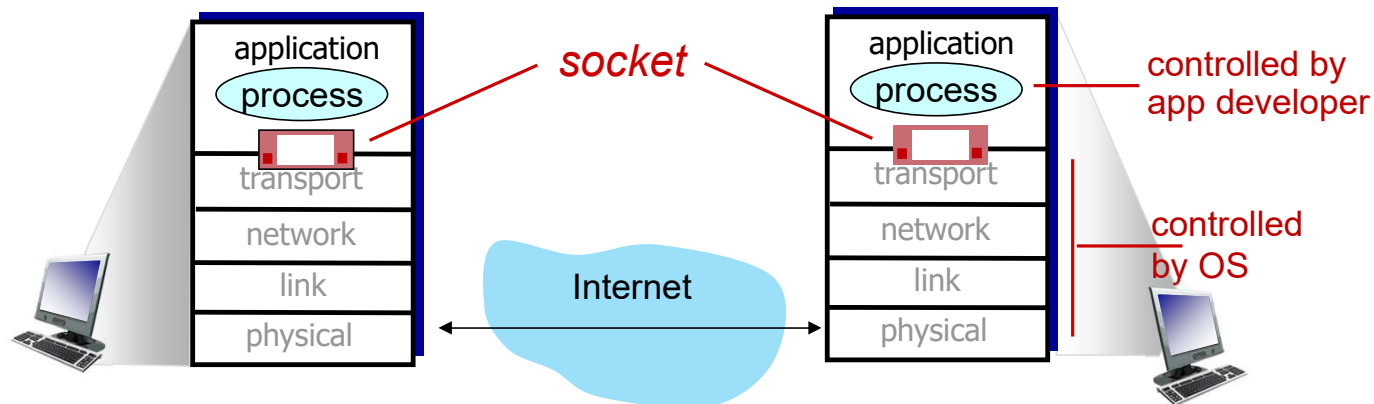
client process: process that initiates communication

server process: process that waits to be contacted

- note: applications with P2P architectures have client processes & server processes

Sockets

- process sends/receives messages to/from its **socket**
- socket analogous to door
 - sending process shoves message out door
 - sending process relies on transport infrastructure on other side of door to deliver message to socket at receiving process
 - two sockets involved: one on each side



Addressing processes

- to receive messages, process must have *identifier*
- host device has unique 32-bit IP address
- Q: does IP address of host on which process runs suffice for identifying the process?
 - A: no, *many* processes can be running on same host
- *identifier* includes both **IP address** and **port numbers** associated with process on host.
- example port numbers:
 - HTTP server: 80
 - mail server: 25
- to send HTTP message to gaia.cs.umass.edu web server:
 - **IP address:** 128.119.245.12
 - **port number:** 80
- more shortly...

An application-layer protocol defines:

- The following is true for all protocols!

- types of messages exchanged,

- e.g., request, response

- message syntax:

- what fields in messages & how fields are delineated

- message semantics

- meaning of information in fields

- rules for when and how processes send & respond to messages

open protocols:

- defined in RFCs, everyone has access to protocol definition
- allows for interoperability
- e.g., HTTP, SMTP

proprietary protocols:

- e.g., Skype, Zoom

What transport service does an app need?

data integrity

- some apps (e.g., file transfer, web transactions) require 100% reliable data transfer
- other apps (e.g., audio) can tolerate some loss

timing

- some apps (e.g., Internet telephony, interactive games) require low delay to be “effective”

throughput

- some apps (e.g., multimedia) require minimum amount of throughput to be “effective”
- other apps (“elastic apps”) make use of whatever throughput they get

security

- encryption, data integrity, ...

Transport service requirements: common apps

application	data loss	throughput	time sensitive?
file transfer/download	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5Kbps-1Mbps video:10Kbps-5Mbps	yes, 10's msec
streaming audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	Kbps+	yes, 10's msec
text messaging	no loss	elastic	yes and no

Internet transport protocols services

TCP service:

- *reliable transport* between sending and receiving process
- *flow control*: sender won't overwhelm receiver
- *congestion control*: throttle sender when network overloaded
- *connection-oriented*: setup required between client and server processes
- *does not provide*: timing, minimum throughput guarantee, security

UDP service:

- *unreliable data transfer* between sending and receiving process
- *does not provide*: reliability, flow control, congestion control, timing, throughput guarantee, security, or connection setup.

Q: why bother? *Why* is there a UDP?

Internet applications, and transport protocols

application	application layer protocol	transport protocol
file transfer/download	FTP [RFC 959]	TCP
e-mail	SMTP [RFC 5321]	TCP
Web documents	HTTP [RFC 7230, 9110]	TCP
Internet telephony	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP or UDP
streaming audio/video	HTTP [RFC 7230], DASH	TCP
interactive games	WOW, FPS (proprietary)	UDP or TCP

Securing TCP

Vanilla TCP & UDP sockets:

- no encryption
- cleartext passwords sent into socket traverse Internet in cleartext (!)

Transport Layer Security (TLS)

- provides encrypted TCP connections
- data integrity
- end-point authentication

TLS implemented in application layer

- apps use TLS libraries, that use TCP in turn
- cleartext sent into “socket” traverse Internet *encrypted*
- more: Security lecture in Network Computing course (3rd year)

Application layer: overview

- Principles of network applications
- **Web and HTTP**
- E-mail, SMTP, IMAP
- The Domain Name System DNS



Web and HTTP

First, a quick review...

- web page consists of *objects*, each of which can be stored on different Web servers
- object can be HTML file, JPEG image, Java applet, audio file,...
- web page consists of *base HTML-file* which includes *several referenced objects, each* addressable by a *Universal Resource Locator (URL)*, e.g.,

`www.someschool.edu/someDept/pic.gif`

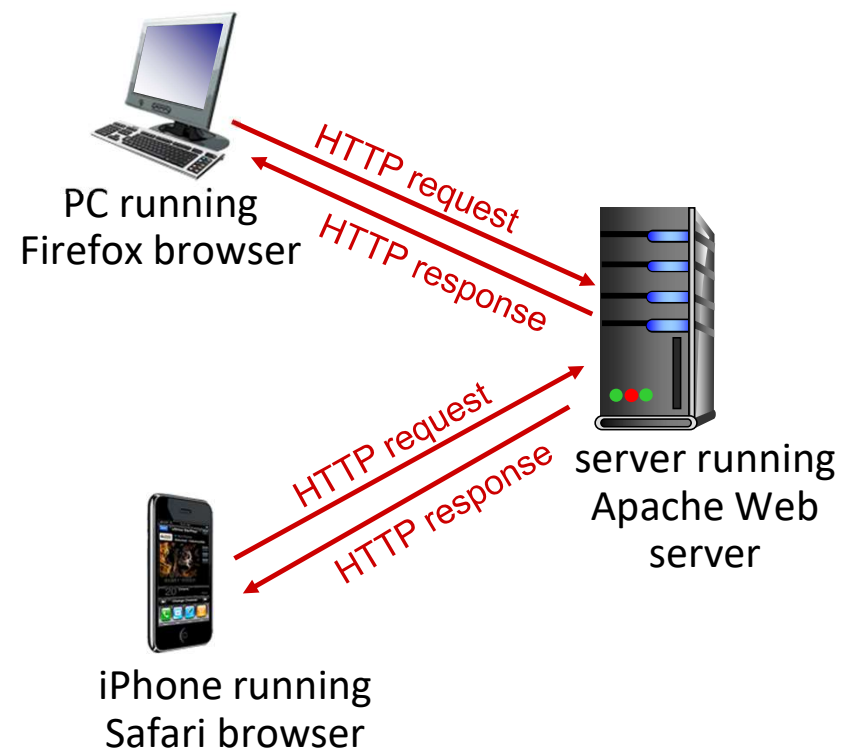
host name

path name

HTTP overview

HTTP: hypertext transfer protocol

- Web's application-layer protocol
- client/server model:
 - *client*: browser that requests, receives, (using HTTP protocol) and “displays” Web objects
 - *server*: Web server sends (using HTTP protocol) objects in response to requests



HTTP overview (continued)

HTTP uses TCP:

- client initiates TCP connection (creates socket) to server, port 80
- server accepts TCP connection from client
- HTTP messages (application-layer protocol messages) exchanged between browser (HTTP client) and Web server (HTTP server)
- TCP connection closed

HTTP is “stateless”

- server maintains *no* information about past client requests

aside
protocols that maintain
“state” are complex!

- past history (state) must be maintained
- if server/client crashes, their views of “state” may be inconsistent, must be reconciled

HTTP connections: two types

Non-persistent HTTP

1. TCP connection opened
2. at most one object sent over TCP connection
3. TCP connection closed

downloading multiple objects required multiple connections

Persistent HTTP

- TCP connection opened to a server
- multiple objects can be sent over *single* TCP connection between client, and that server
- TCP connection closed

Non-persistent HTTP/1.0: example

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



time
↓

1a. HTTP client initiates TCP connection to HTTP server (process) at `www.someSchool.edu` on port 80



1b. HTTP server at host `www.someSchool.edu` waiting for TCP connection at port 80 “accepts” connection, notifying client

2. HTTP client sends HTTP *request message* (containing URL) into TCP connection socket. Message indicates that client wants object `someDepartment/home.index`

3. HTTP server receives request message, forms *response message* containing requested object, and sends message into its socket

Non-persistent HTTP/1.0: example (cont.)

User enters URL: `www.someSchool.edu/someDepartment/home.index`
(containing text, references to 10 jpeg images)



4. HTTP server closes TCP connection.

5. HTTP client receives response message containing html file, displays html. Parsing html file, finds 10 referenced jpeg objects

6. Steps 1-5 repeated for each of 10 jpeg objects

time

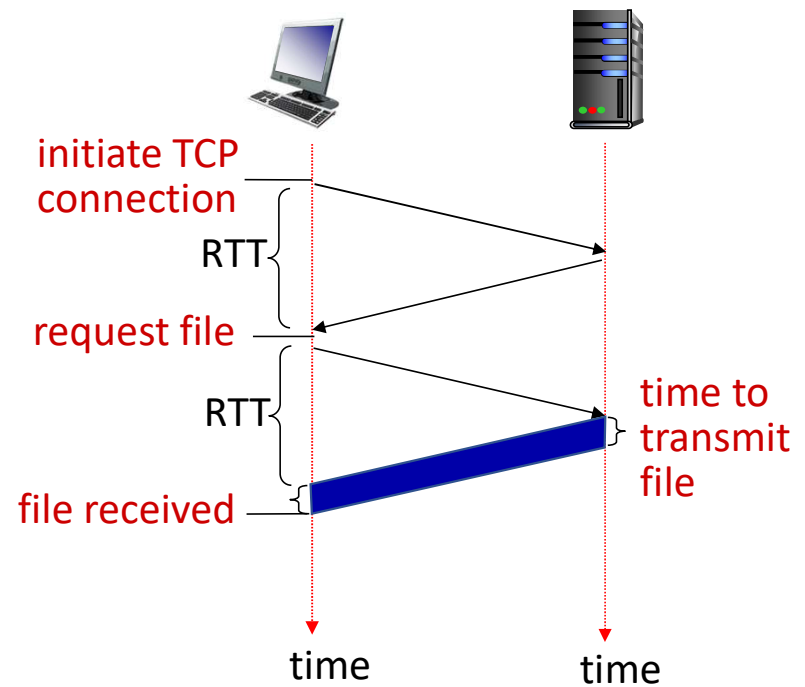


Non-persistent HTTP: Round Trip Time (RTT)

RTT (definition): time for a small packet to travel from client to server and back

HTTP response time (per object):

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- object/file transmission time



Non-persistent HTTP response time = $2RTT + \text{file transmission time}$

Persistent HTTP (HTTP 1.1)

Non-persistent HTTP issues:

- requires 2 RTTs per object
- OS overhead for *each* TCP connection
- browsers often open multiple parallel TCP connections to fetch referenced objects in parallel

Persistent HTTP (HTTP1.1):

- server leaves connection open after sending response
- subsequent HTTP messages between same client/server sent over open connection
- client sends requests as soon as it encounters a referenced object
- as little as one RTT for all the referenced objects (cutting response time in half)

HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
 - ASCII (human-readable format)

request line (GET, POST,
HEAD commands)

→ GET /index.html HTTP/1.1\r\n

carriage return character

line-feed character

HTTP request message

- two types of HTTP messages: *request, response*
- **HTTP request message:**
 - ASCII (human-readable format)

request line (GET, POST, HEAD commands) → GET /index.html HTTP/1.1\r\n

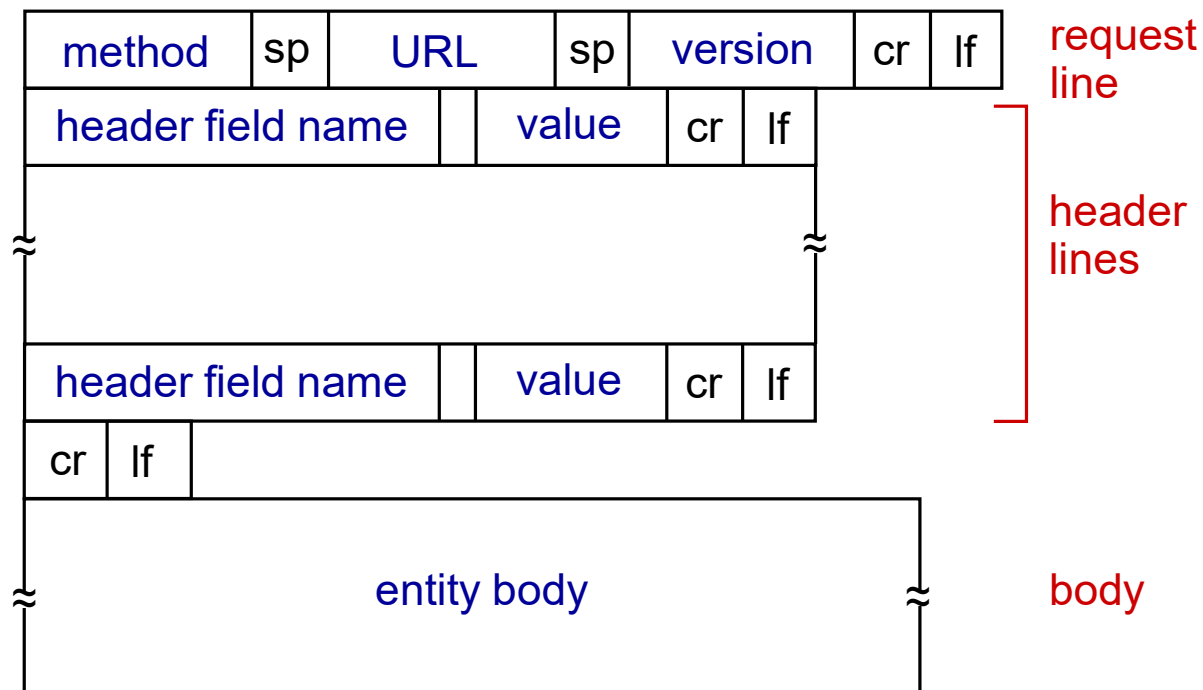
header lines → Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n

carriage return, line feed at start of line indicates end of header lines →

carriage return character
line-feed character

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP request message: general format



Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

HTTP request message

- two types of HTTP messages: *request, response*
- HTTP request message:
 - ASCII (human-readable format)

request line (GET, POST,
HEAD commands)

→ GET /index.html HTTP/1.1\r\n

carriage return character
line-feed character

HTTP request message

- two types of HTTP messages: *request, response*
- **HTTP request message:**
 - ASCII (human-readable format)

request line (GET, POST, HEAD commands) →

header lines {

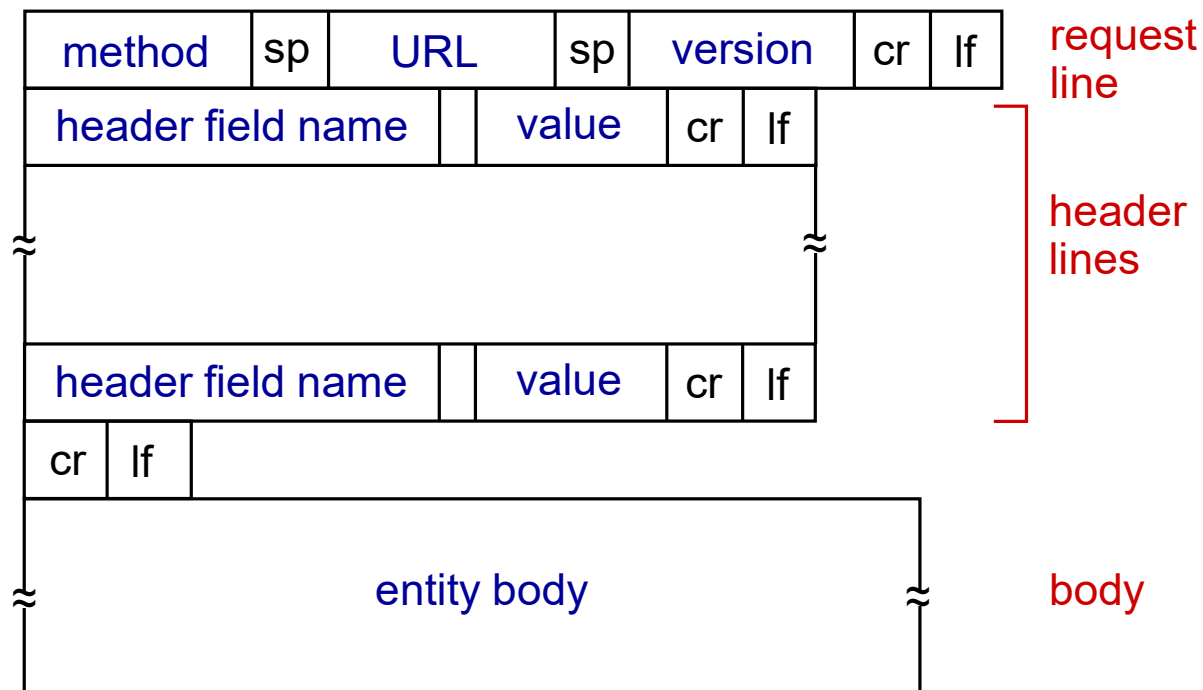
```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

carriage return, line feed
at start of line indicates
end of header lines →

carriage return character
line-feed character

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP request message: general format



Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

Other HTTP request messages

POST method:

- web page often includes form input
- user input sent from client to server in entity body of HTTP POST request message

GET method (for sending data to server):

- include user data in URL field of HTTP GET request message (following a '?'):

`www.somesite.com/animalsearch?monkeys&banana`

HEAD method:

- requests headers (only) that would be returned *if* specified URL were requested with an HTTP GET method.

PUT method:

- uploads new file (object) to server
- completely replaces file that exists at specified URL with content in entity body of POST HTTP request message

HTTP response message

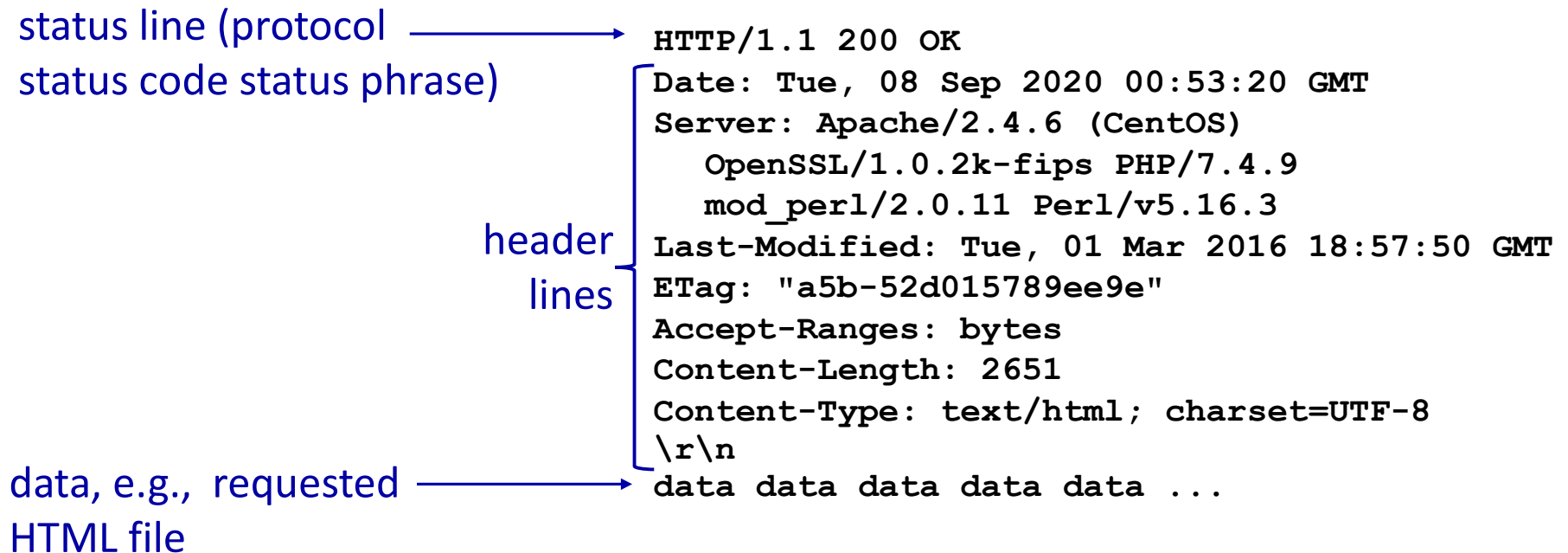
status line (protocol  status code status phrase) **HTTP/1.1 200 OK**

HTTP response message

status line (protocol status code status phrase) → HTTP/1.1 200 OK

header lines {
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
OpenSSL/1.0.2k-fips PHP/7.4.9
mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n

HTTP response message



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

HTTP response status codes

- status code appears in 1st line in server-to-client response message.
- some sample codes:

200 OK

- request succeeded, requested object later in this message

301 Moved Permanently

- requested object moved, new location specified later in this message (in Location: field)

400 Bad Request

- request msg not understood by server

404 Not Found

- requested document not found on this server

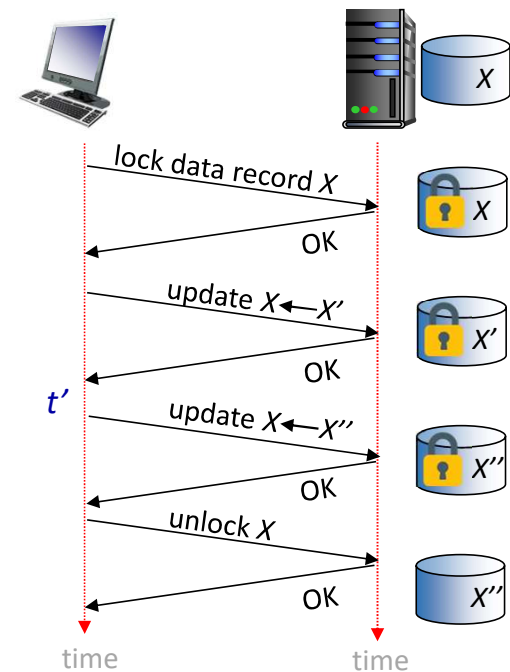
505 HTTP Version Not Supported

Maintaining user/server state: cookies

Recall: HTTP GET/response interaction is *stateless*

- no notion of multi-step exchanges of HTTP messages to complete a Web “transaction”
 - no need for client/server to track “state” of multi-step exchange
 - all HTTP requests are independent of each other
 - no need for client/server to “recover” from a partially-completed-but-never-completely-completed transaction

a **stateful protocol**: client makes two changes to X, or none at all



Q: what happens if network connection or client crashes at t' ?

Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

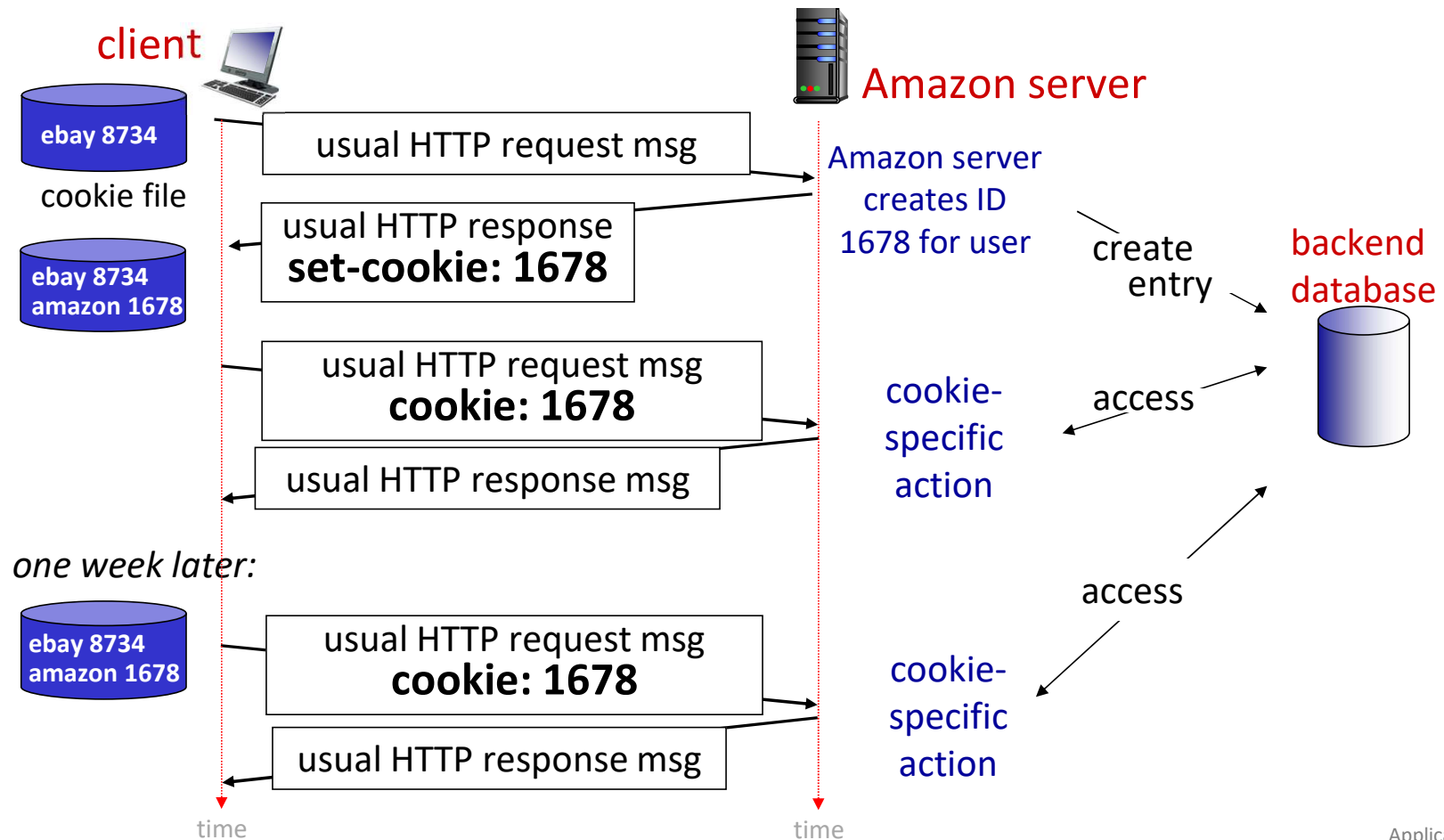
four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID (aka “cookie”)
 - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining user/server state: cookies



HTTP cookies: comments

What cookies can be used for:

- authorization
- shopping carts
- recommendations
- user session state (Web e-mail)

Challenge: How to keep state?

- *at protocol endpoints:* maintain state at sender/receiver over multiple transactions
- *in messages:* cookies in HTTP messages carry state

cookies and privacy: aside

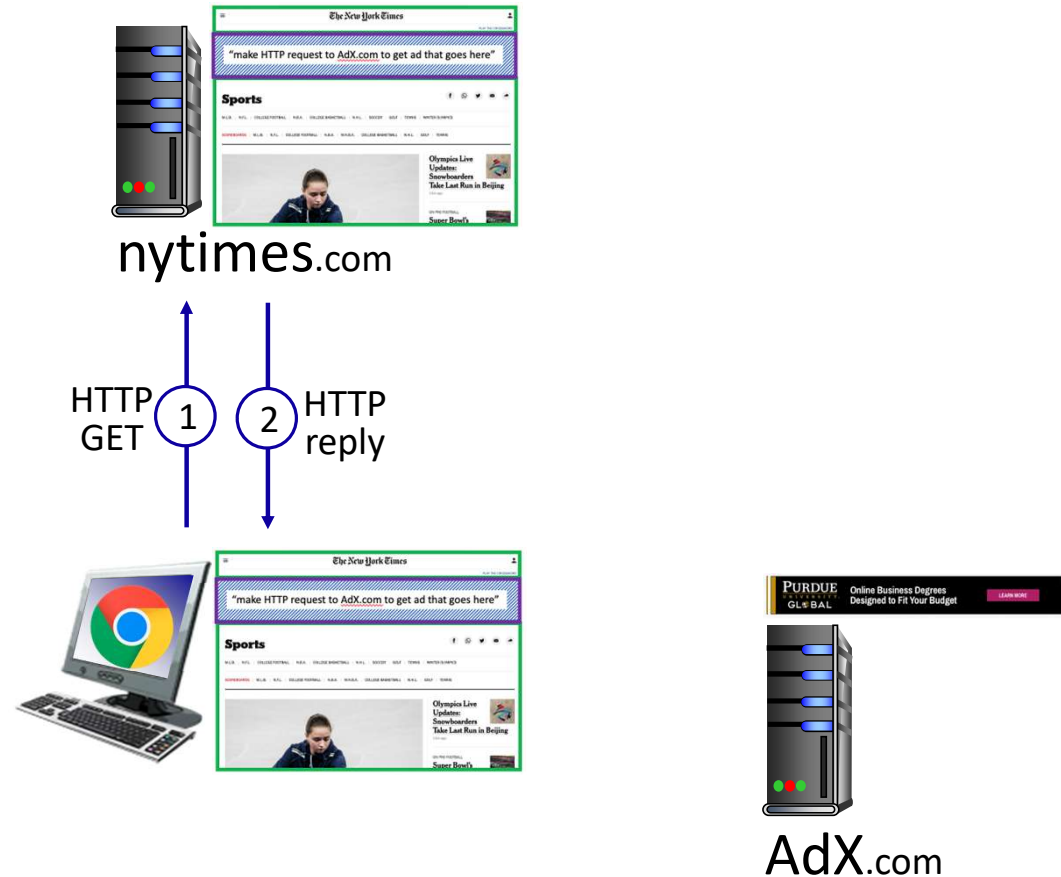
- cookies permit sites to *learn* a lot about you on their site.
- third party persistent cookies (tracking cookies) allow common identity (cookie value) to be tracked across multiple web sites

Example: displaying a NY Times web page



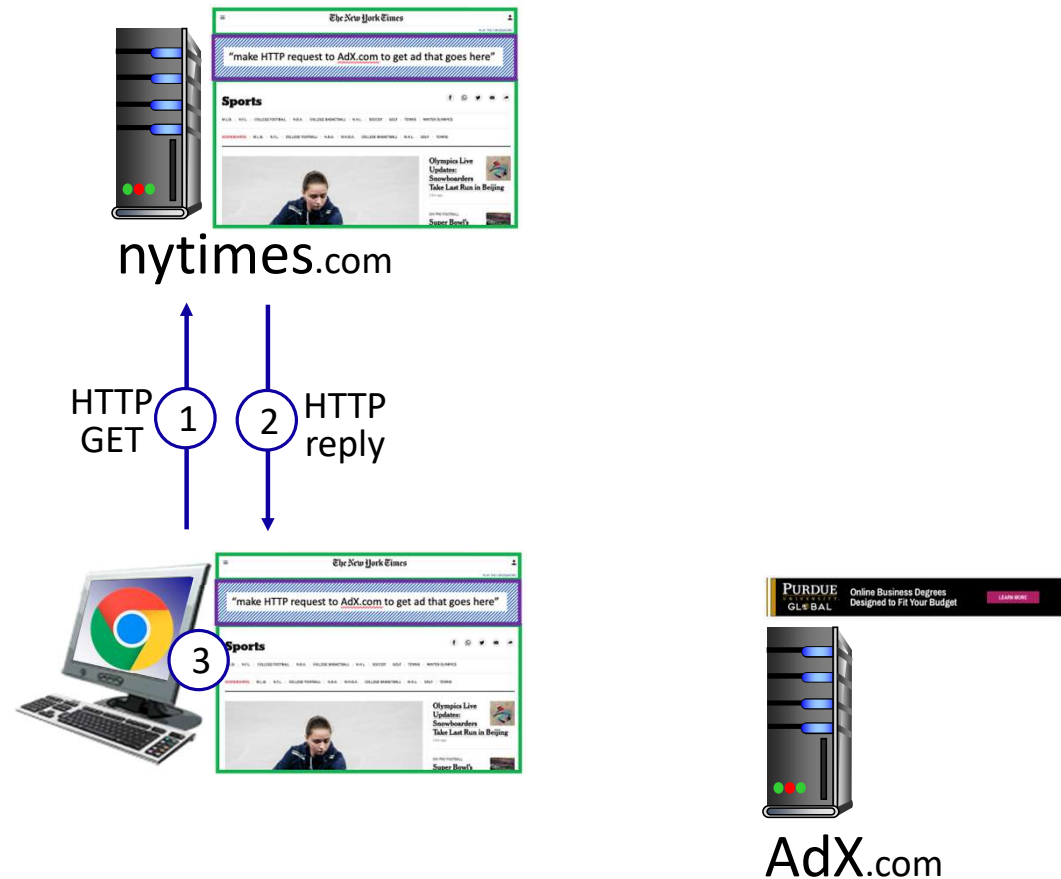
Example: displaying a NY Times web page

- 1 GET base html file
- 2 from nytimes.com



Example: displaying a NY Times web page

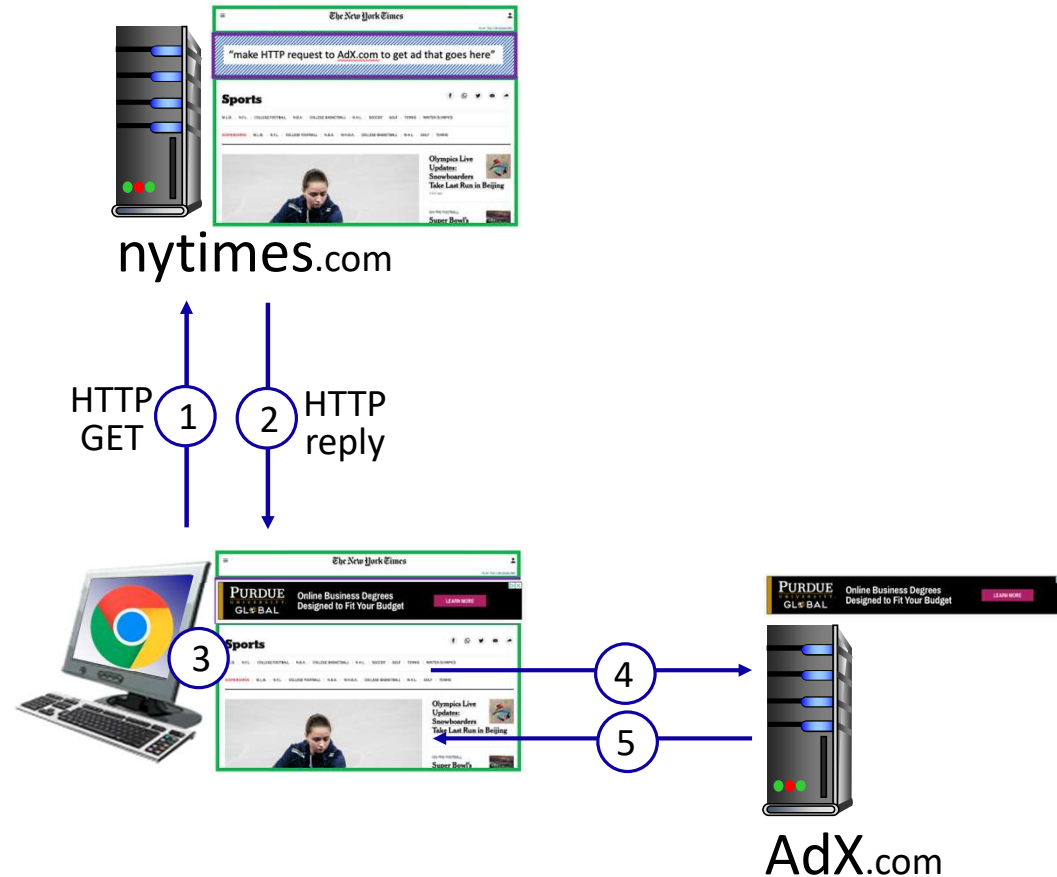
- 1 GET base html file
- 2 from nytimes.com



Example: displaying a NY Times web page

1 2 GET base html file
from nytimes.com

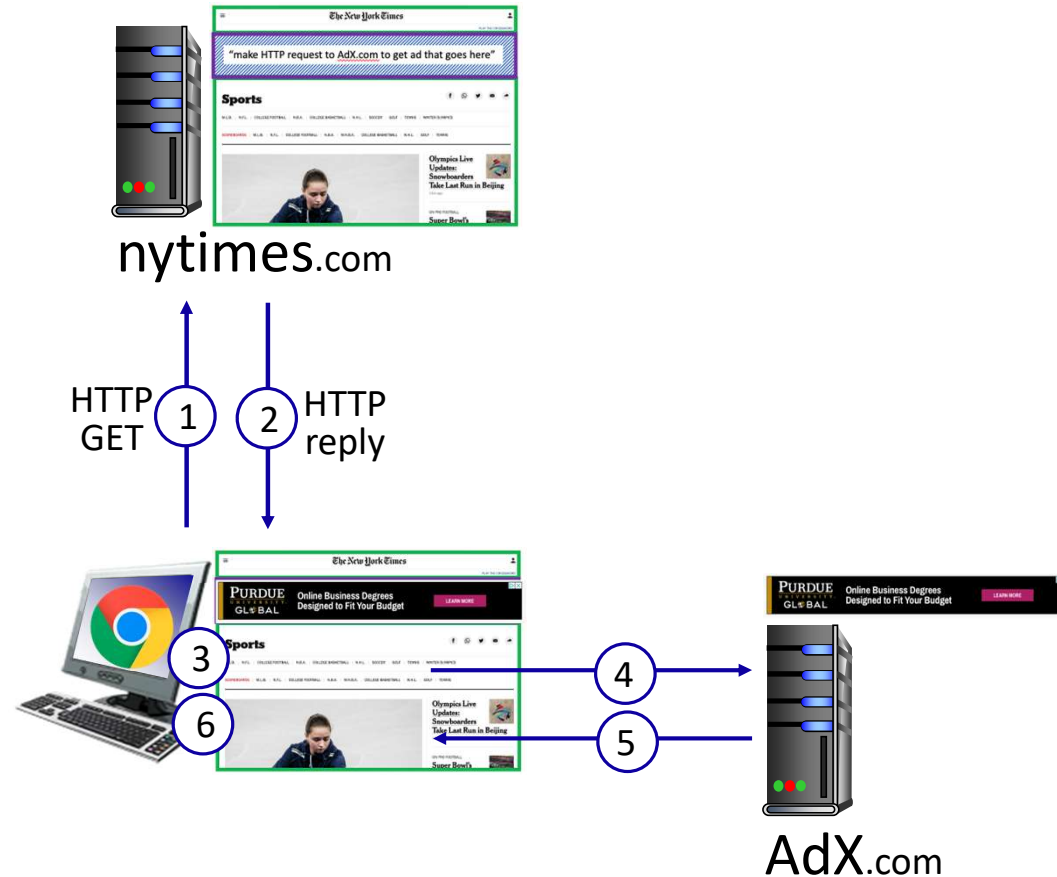
4 5 fetch ad from
AdX.com



Example: displaying a NY Times web page

1 2 GET base html file
from nytimes.com

4 5 fetch ad from
AdX.com

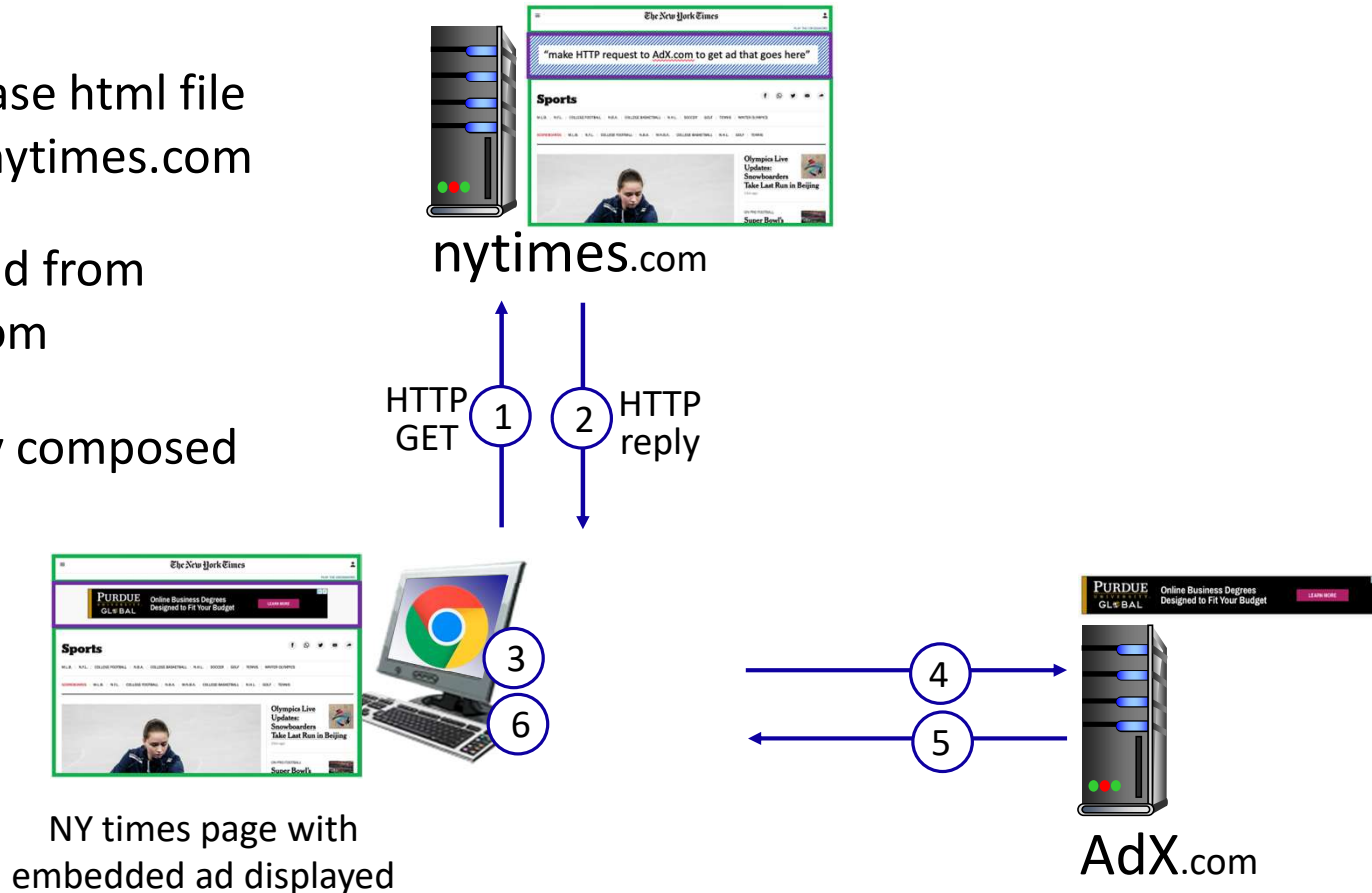


Example: displaying a NY Times web page

1 2 GET base html file
from nytimes.com

4 5 fetch ad from
AdX.com

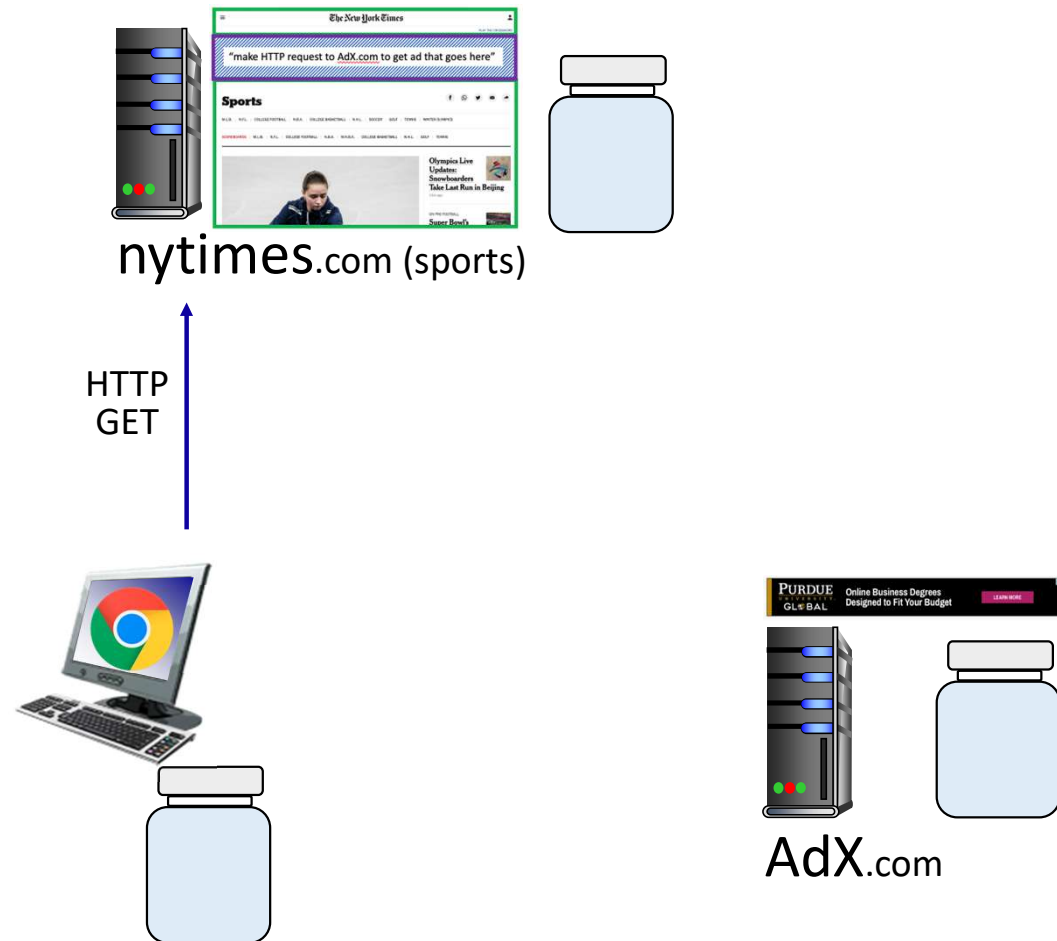
7 display composed
page



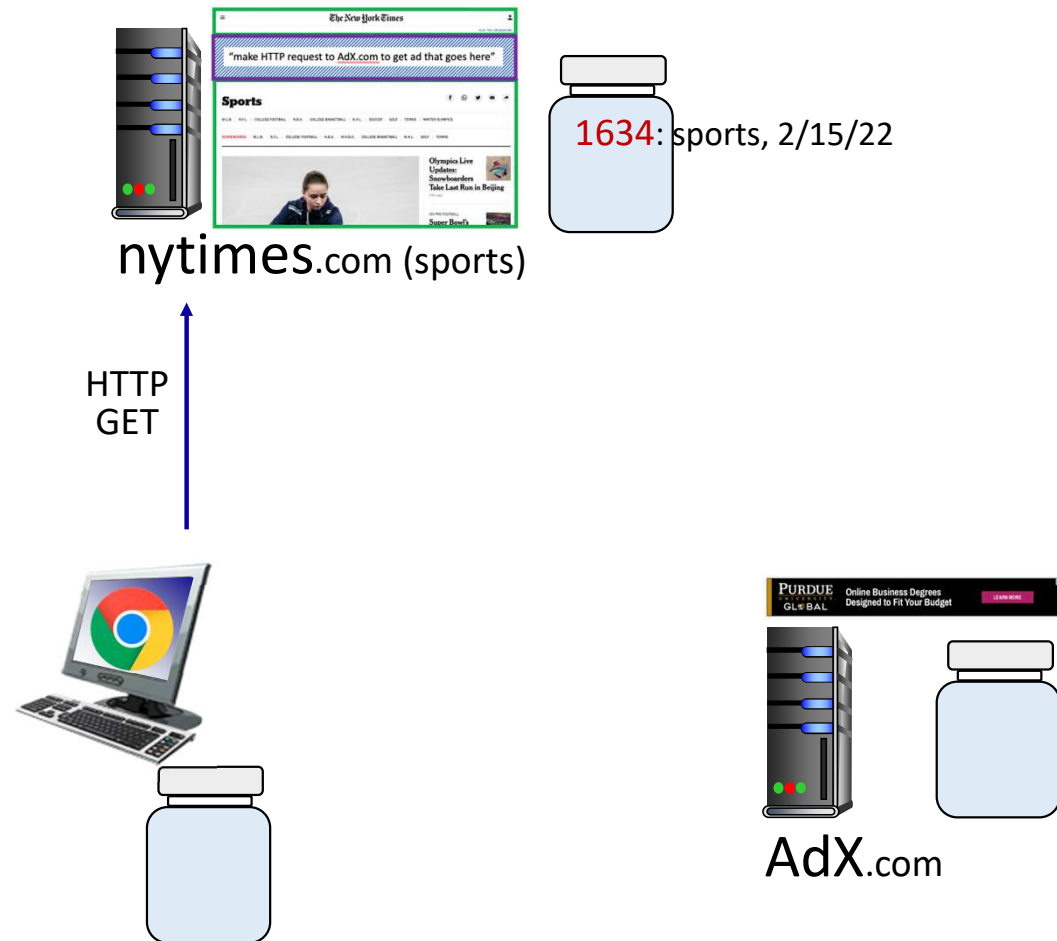
Cookies: tracking a user's browsing behavior



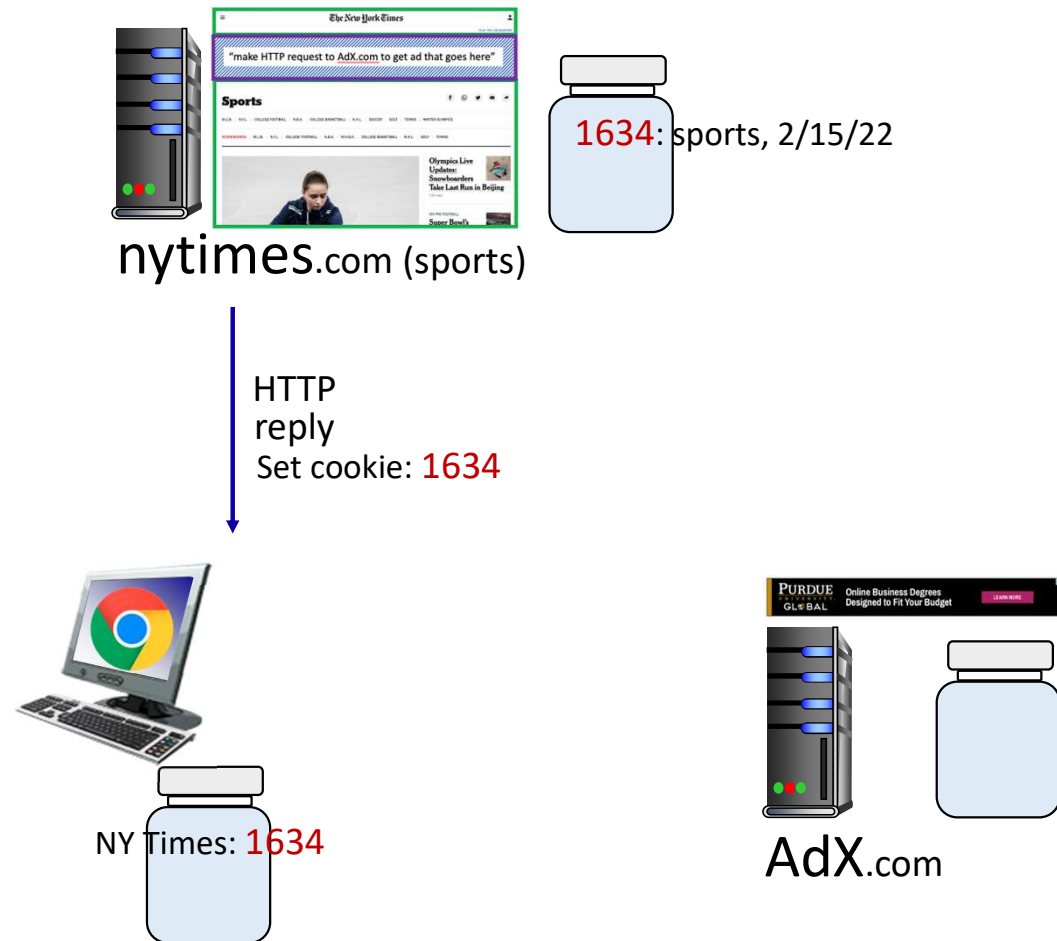
Cookies: tracking a user's browsing behavior



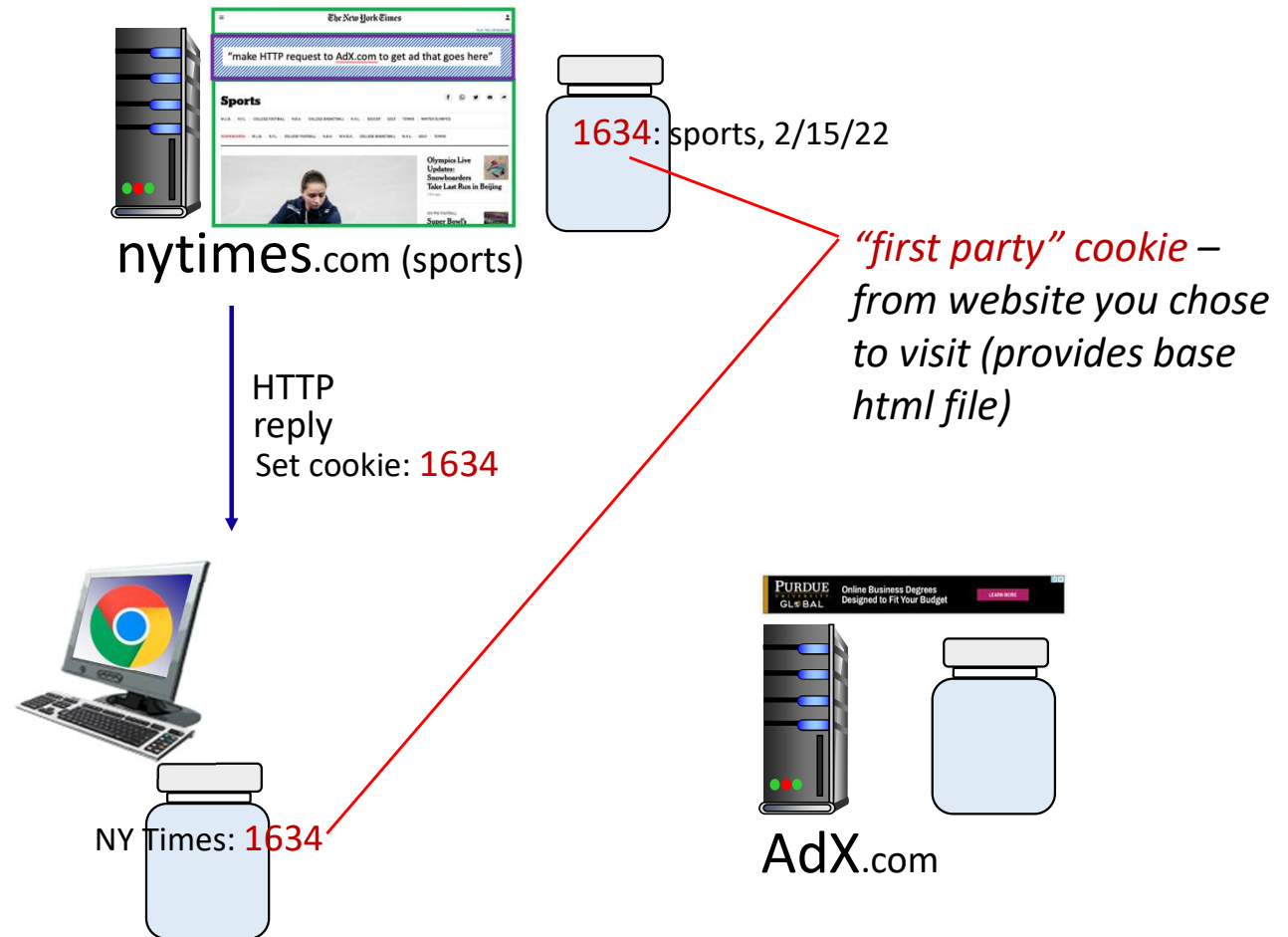
Cookies: tracking a user's browsing behavior



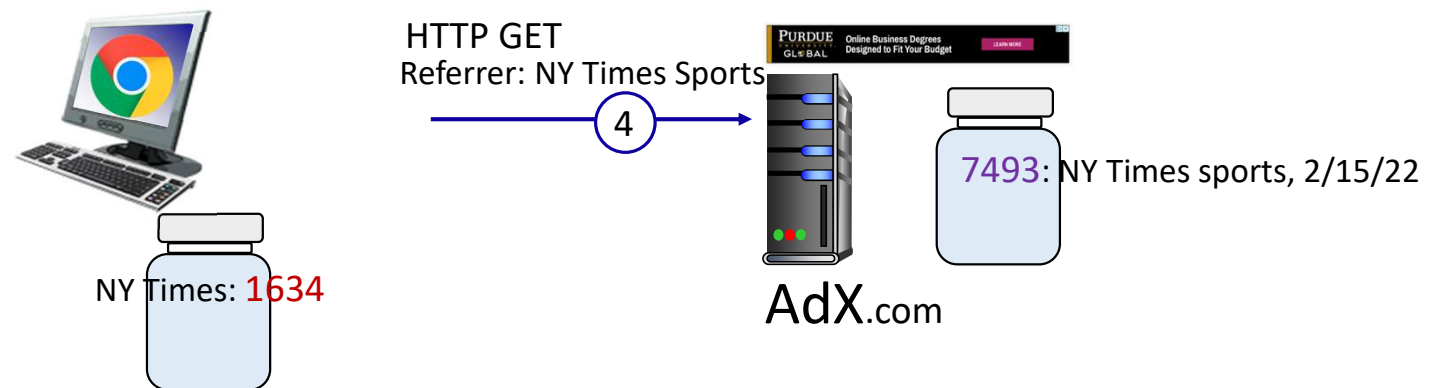
Cookies: tracking a user's browsing behavior



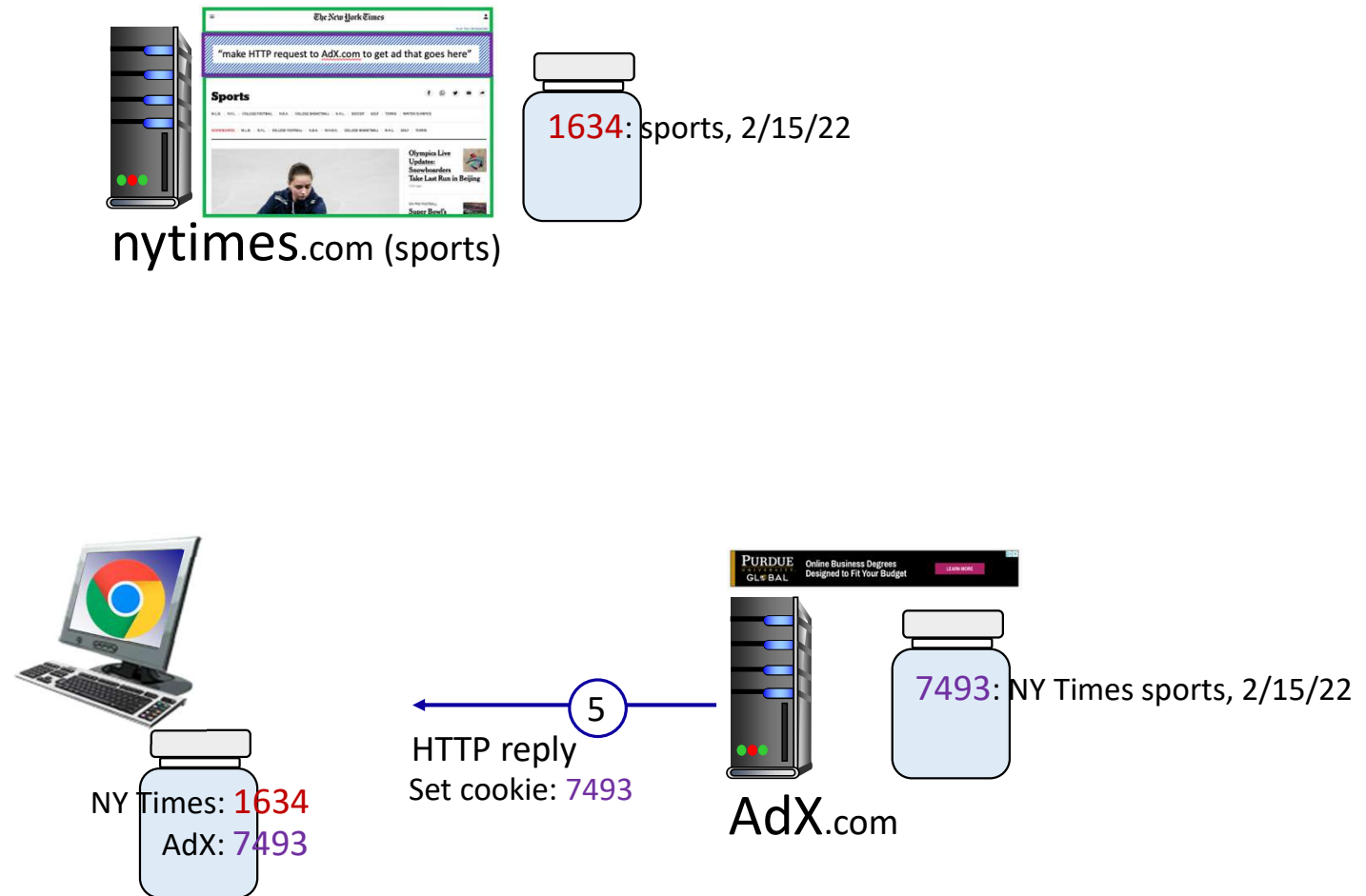
Cookies: tracking a user's browsing behavior



Cookies: tracking a user's browsing behavior



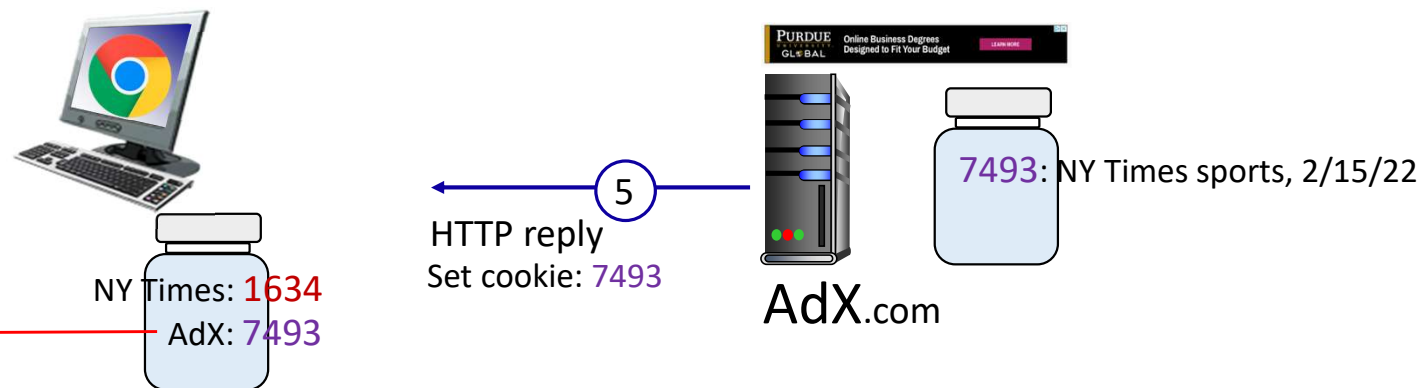
Cookies: tracking a user's browsing behavior



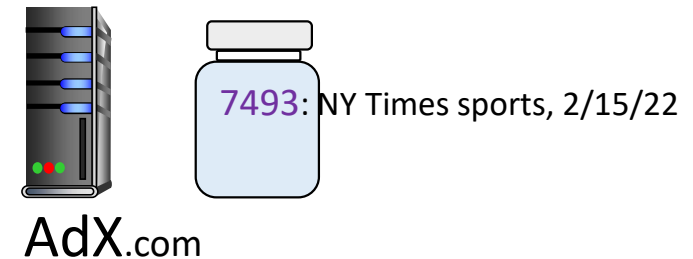
Cookies: tracking a user's browsing behavior



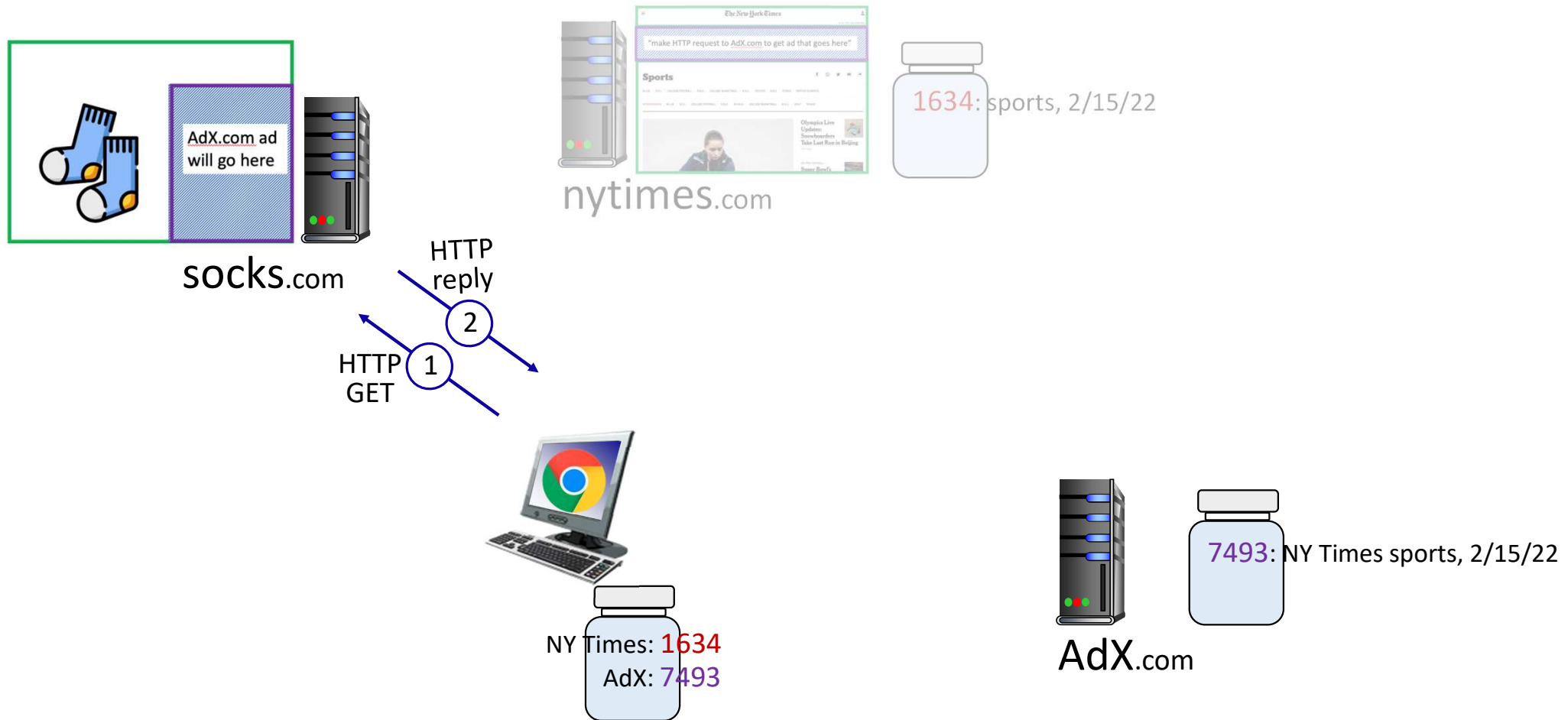
*"third party" cookie –
from website you did not
choose to visit*



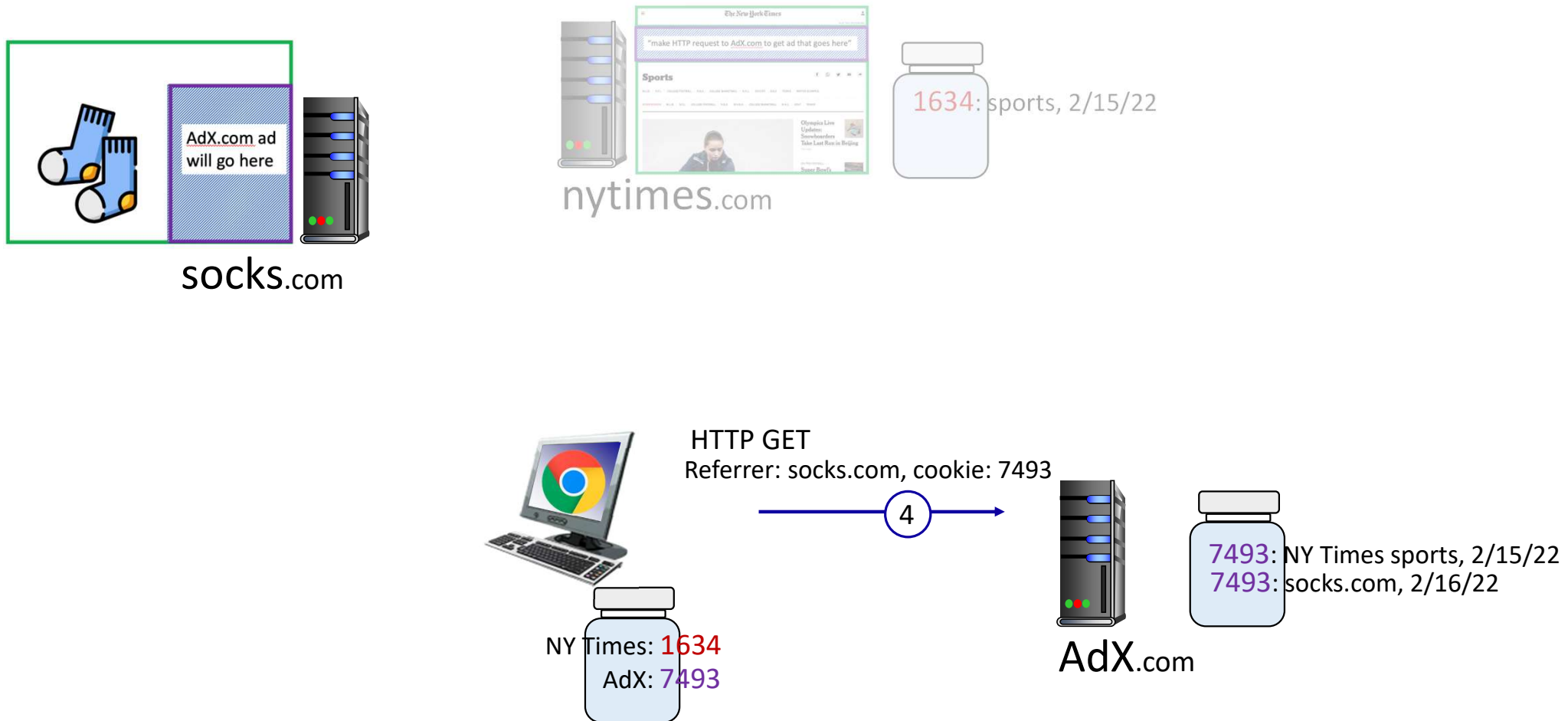
Cookies: tracking a user's browsing behavior



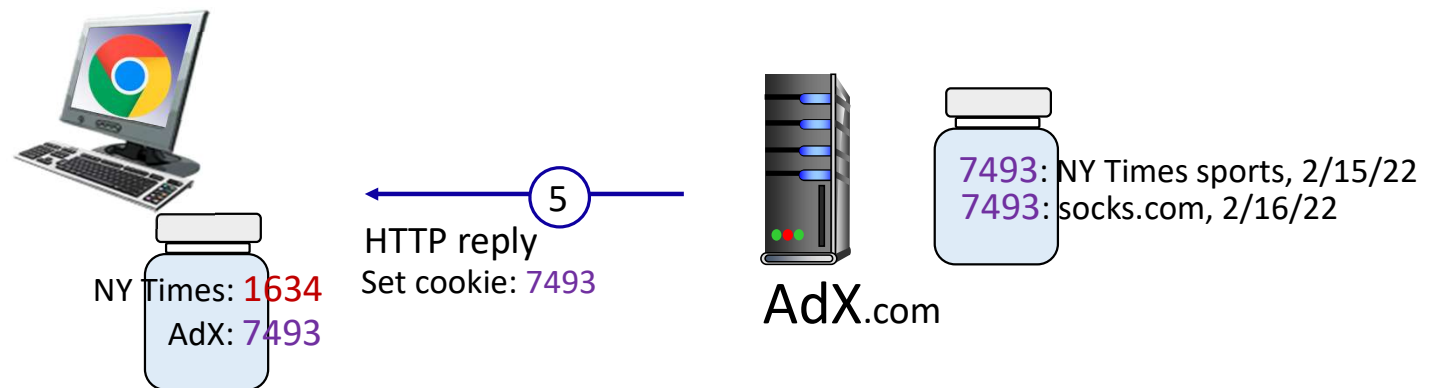
Cookies: tracking a user's browsing behavior



Cookies: tracking a user's browsing behavior



Cookies: tracking a user's browsing behavior

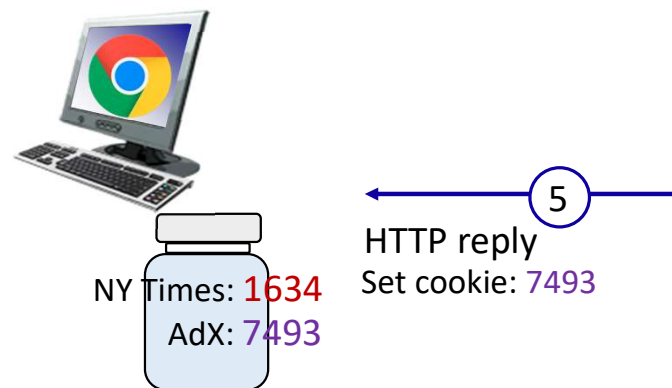


Cookies: tracking a user's browsing behavior



AdX:

- *tracks my web browsing* over sites with AdX ads
- can return targeted ads based on browsing history



5
HTTP reply
Set cookie: 7493



Cookies: tracking a user's browsing behavior

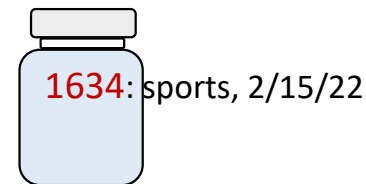


AdX:

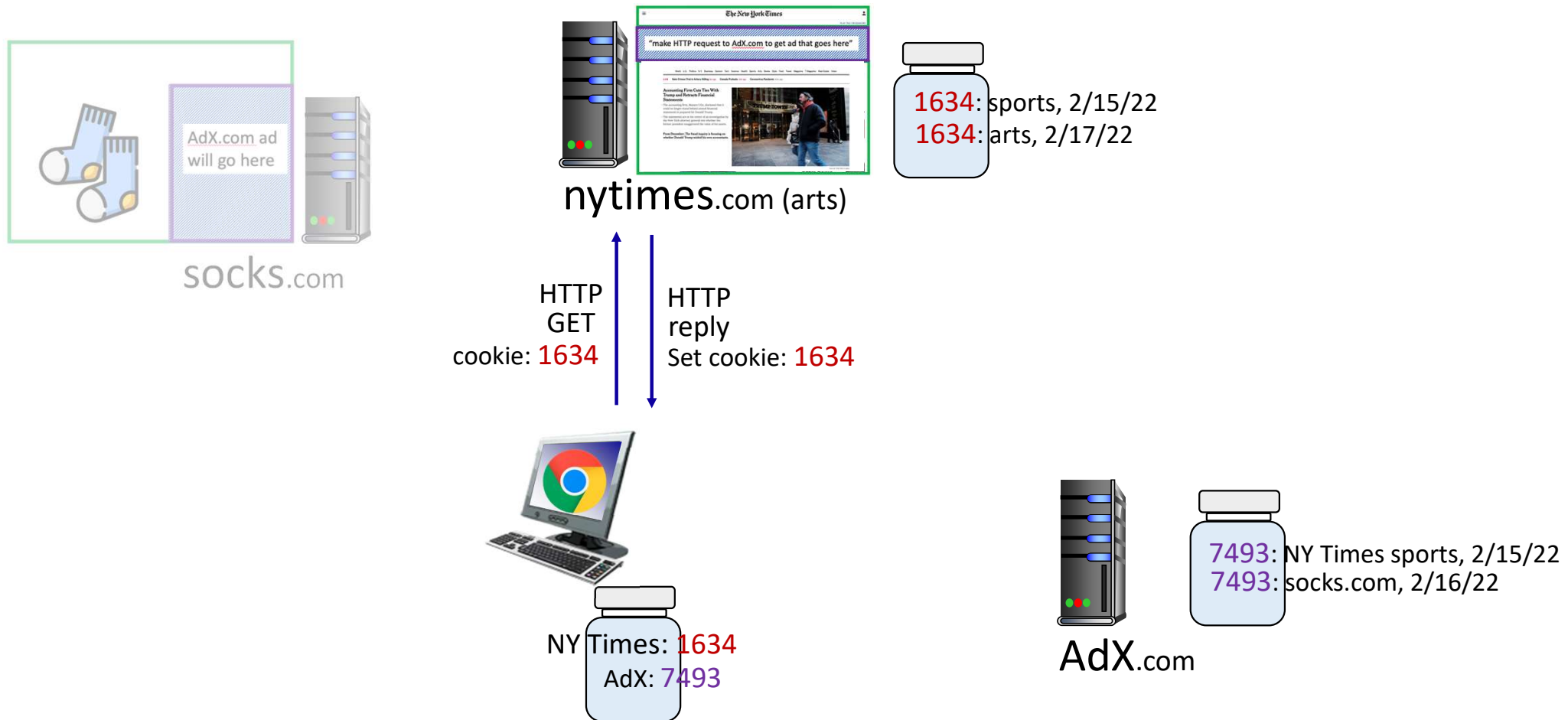
- *tracks my web browsing* over sites with AdX ads
- can return targeted ads based on browsing history



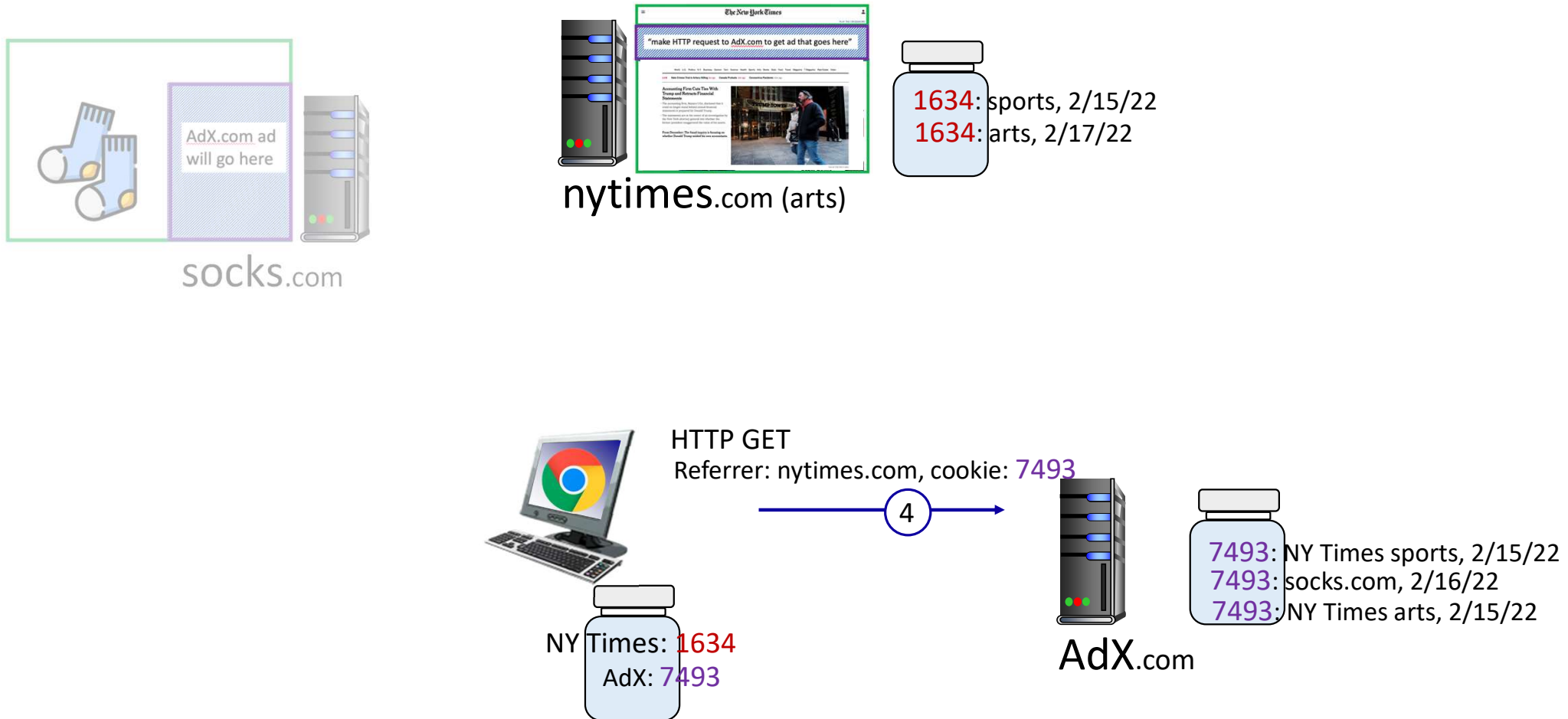
Cookies: tracking a user's browsing behavior (one day later)



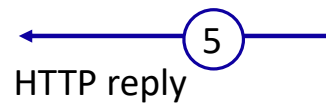
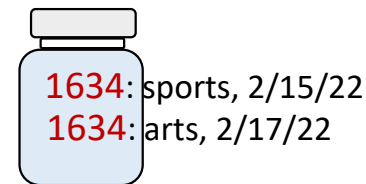
Cookies: tracking a user's browsing behavior (one day later)



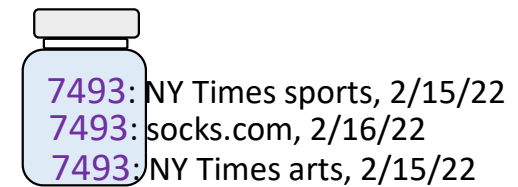
Cookies: tracking a user's browsing behavior (one day later)



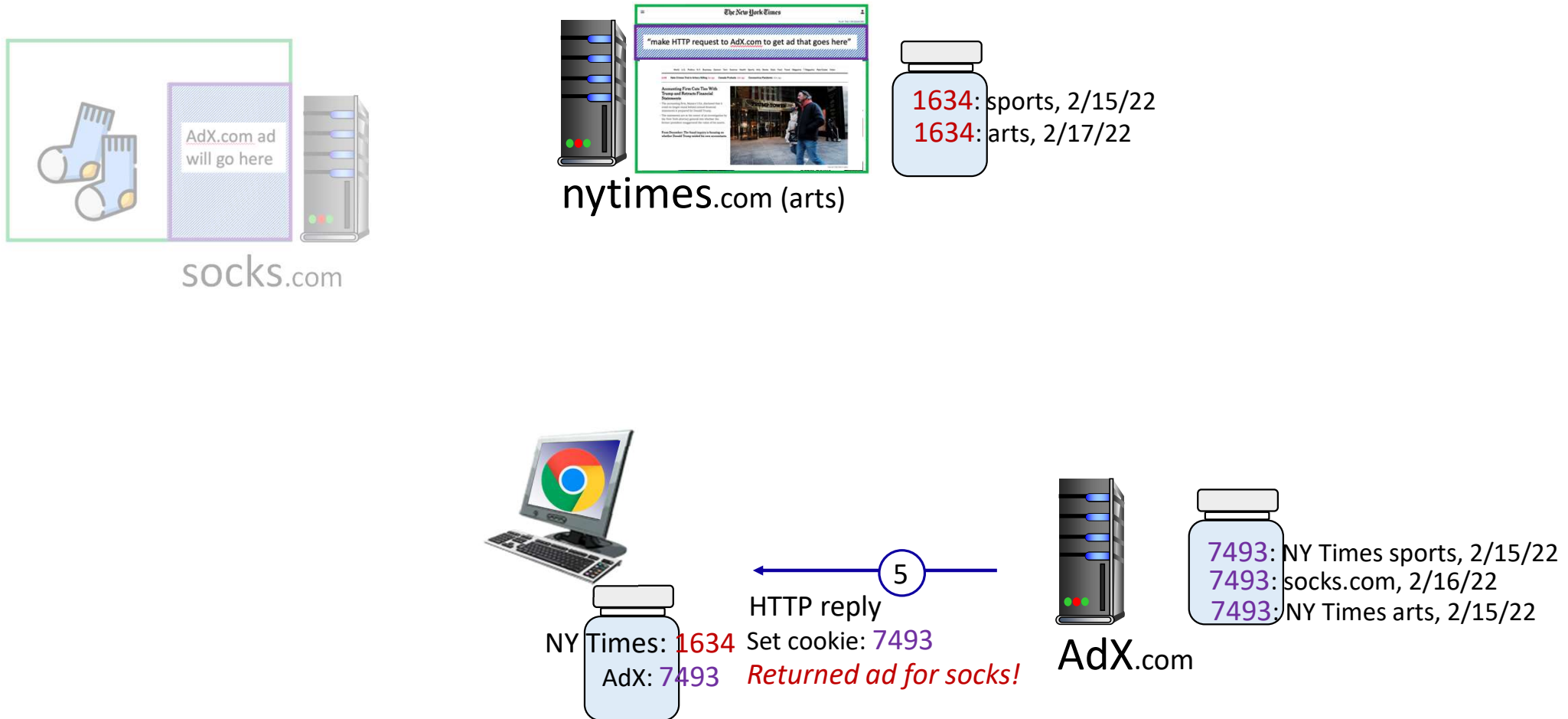
Cookies: tracking a user's browsing behavior (one day later)



AdX.com



Cookies: tracking a user's browsing behavior (one day later)



Cookies: tracking a user's browsing behavior

Cookies can be used to:

- track user behavior on a given website (**first party cookies**)
- track user behavior across multiple websites (**third party cookies**) without user ever choosing to visit tracker site (!)
- tracking may be *invisible* to user:
 - rather than displayed ad triggering HTTP GET to tracker, could be an invisible link

third party tracking via cookies:

- disabled by default in Safari
- mostly blocked in Firefox
- being phased out in Chrome (not yet fully disabled)

GDPR (EU General Data Protection Regulation) and cookies

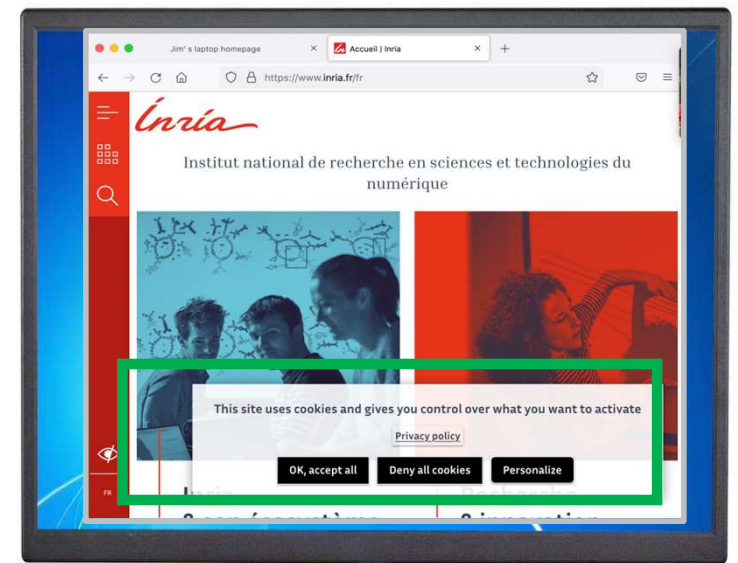
“Natural persons may be associated with online identifiers [...] such as internet protocol addresses, cookie identifiers or other identifiers [...].

This may leave traces which, in particular when combined with unique identifiers and other information received by the servers, may be used to create profiles of the natural persons and identify them.”

GDPR, recital 30 (May 2018)



when cookies can identify an individual, cookies are considered personal data, subject to GDPR personal data regulations

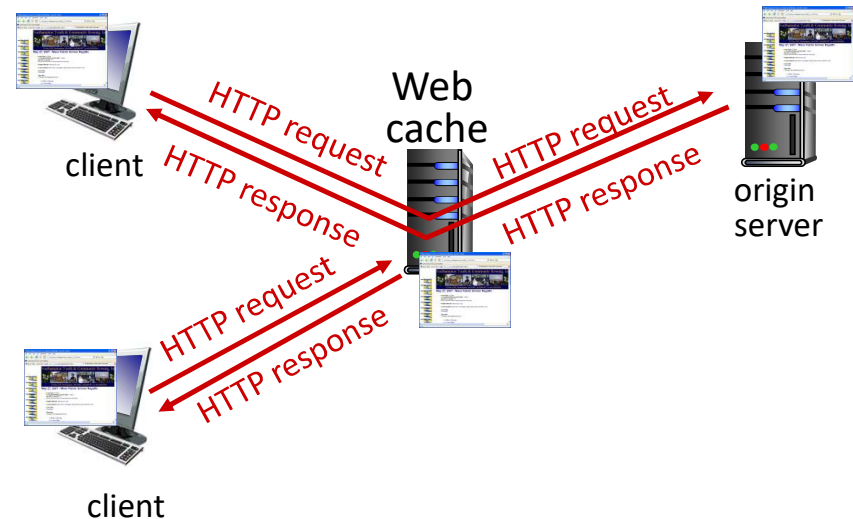


User has explicit control over whether or not cookies are allowed

Web caches

Goal: satisfy client requests without involving origin server

- user configures browser to point to a (local) *Web cache*
- browser sends all HTTP requests to cache
 - *if* object in cache: cache returns object to client
 - *else* cache requests object from origin server, caches received object, then returns object to client



Web caches (aka proxy servers)

- Web cache acts as both client and server
 - server for original requesting client
 - client to origin server
- server tells cache about object's allowable caching in response header:

```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

Why Web caching?

- reduce response time for client request
 - cache is closer to client
- reduce traffic on an institution's access link
- Internet is dense with caches
 - enables “poor” content providers to more effectively deliver content

Caching example

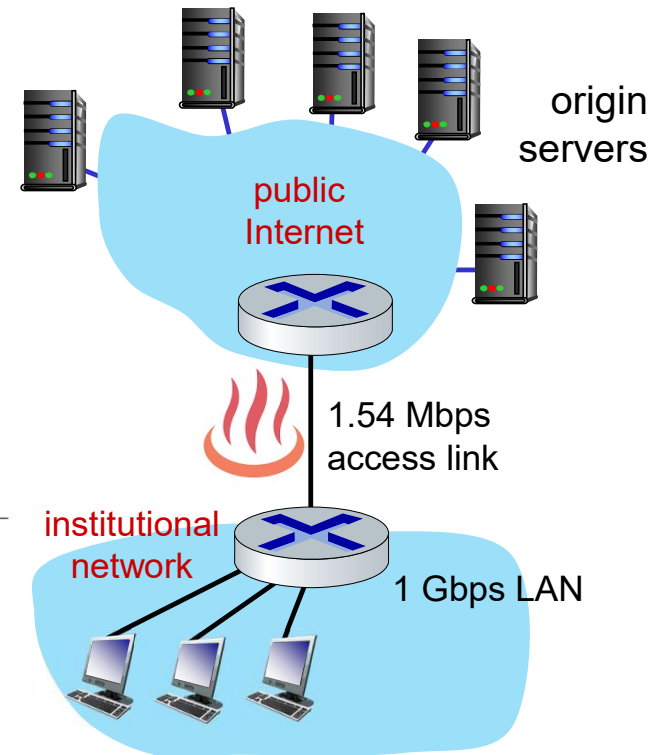
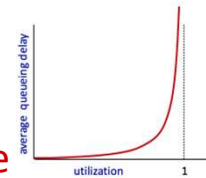
Scenario:

- access link rate: 1.54 Mbps
- RTT from ISP router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = .97
- LAN utilization: .0015
- end-end delay = Internet delay + access link delay + LAN delay
= 2 sec + minutes + usecs

problem: large queueing delays at high utilization!



Option 1: buy a faster access link

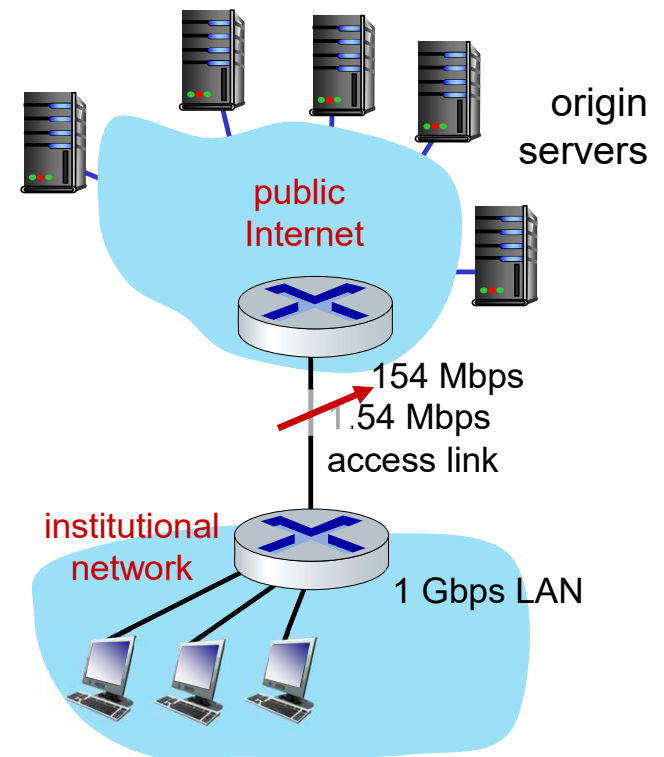
Scenario:

- access link rate: ~~1.54~~ 154 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Performance:

- access link utilization = ~~.97~~ .0097
- LAN utilization: .0015
- end-end delay = Internet delay +
access link delay + LAN delay
= 2 sec + ~~minutes~~ + usecs

Cost: faster access link (expensive!) ~~msecs~~



Option 2: install a web cache

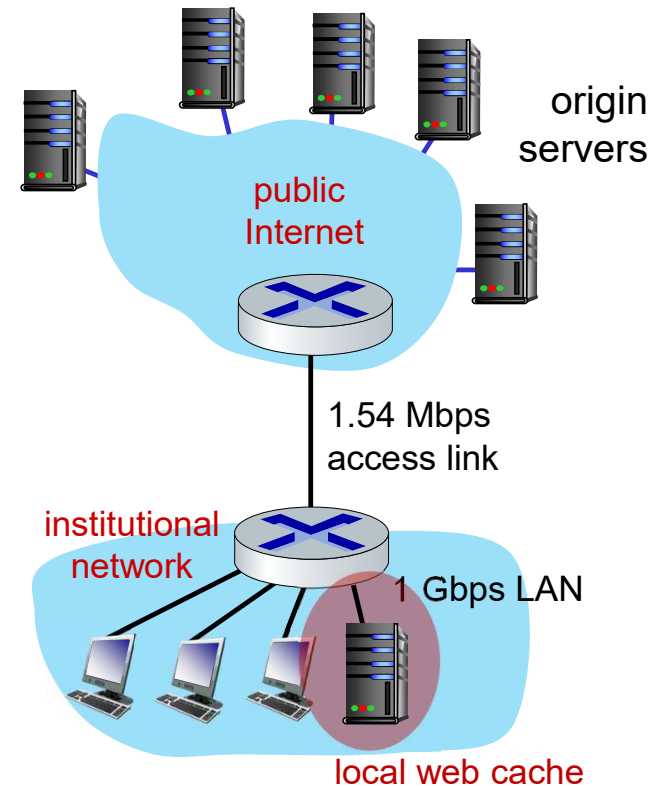
Scenario:

- access link rate: 1.54 Mbps
- RTT from institutional router to server: 2 sec
- web object size: 100K bits
- average request rate from browsers to origin servers: 15/sec
 - avg data rate to browsers: 1.50 Mbps

Cost: web cache (cheap!)

Performance:

- LAN utilization: .?
 - access link utilization = ?
 - average end-end delay = ?
- How to compute link utilization, delay?*

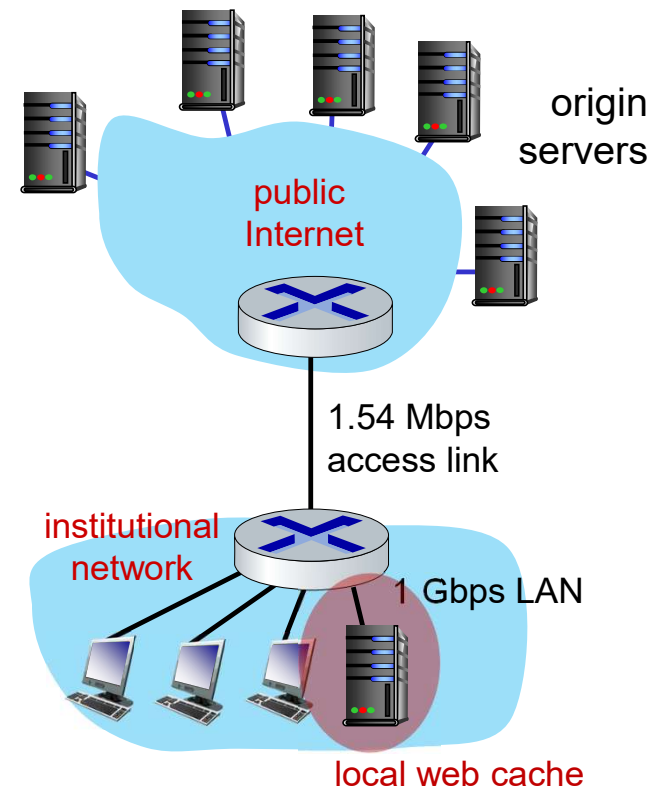


Calculating access link utilization, end-end delay with cache:

suppose cache hit rate is 0.4:

- 40% requests served by cache, with low (msec) delay
- 60% requests satisfied at origin
 - rate to browsers over access link
 $= 0.6 * 1.50 \text{ Mbps} = .9 \text{ Mbps}$
 - access link utilization $= 0.9/1.54 = .58$ means low (msec) queueing delay at access link
- average end-end delay:
 - $= 0.6 * (\text{delay from origin servers})$
 - $+ 0.4 * (\text{delay when satisfied at cache})$
 - $= 0.6 (2.01) + 0.4 (\sim \text{msecs}) = \sim 1.2 \text{ secs}$

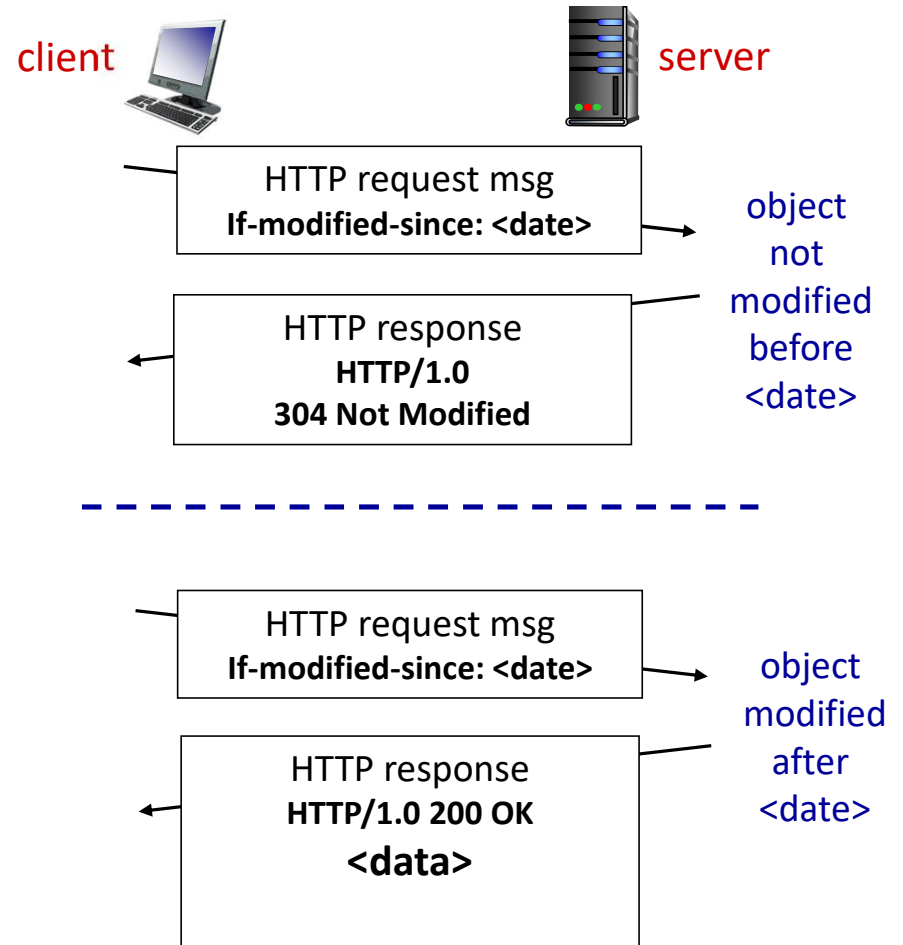
lower average end-end delay than with 154 Mbps link (and cheaper too!)



Browser caching: Conditional GET

Goal: don't send object if browser has up-to-date cached version

- no object transmission delay (or use of network resources)
- **client:** specify date of browser-cached copy in HTTP request
If-modified-since: <date>
- **server:** response contains no object if browser-cached copy is up-to-date:
HTTP/1.0 304 Not Modified



HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP1.1: introduced multiple, pipelined GETs over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

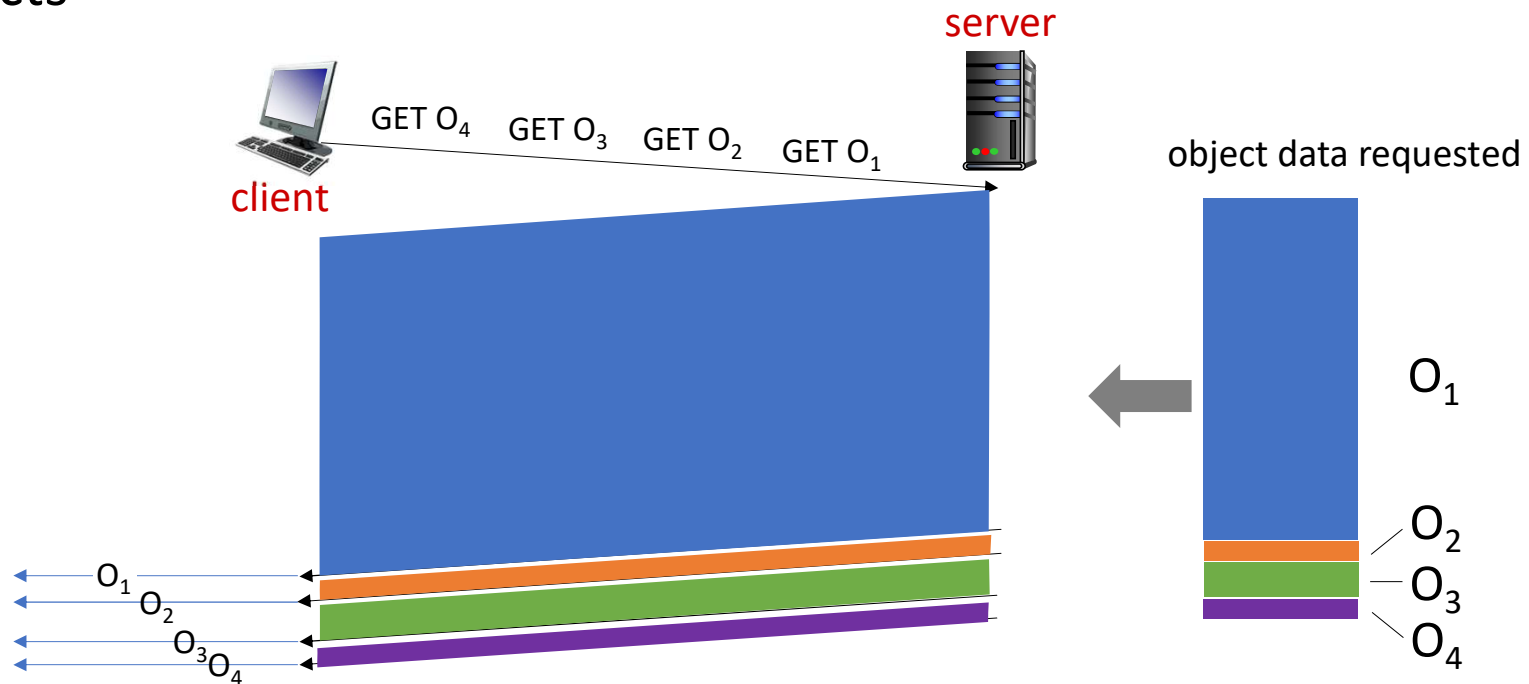
Key goal: decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

HTTP/2: mitigating HOL blocking

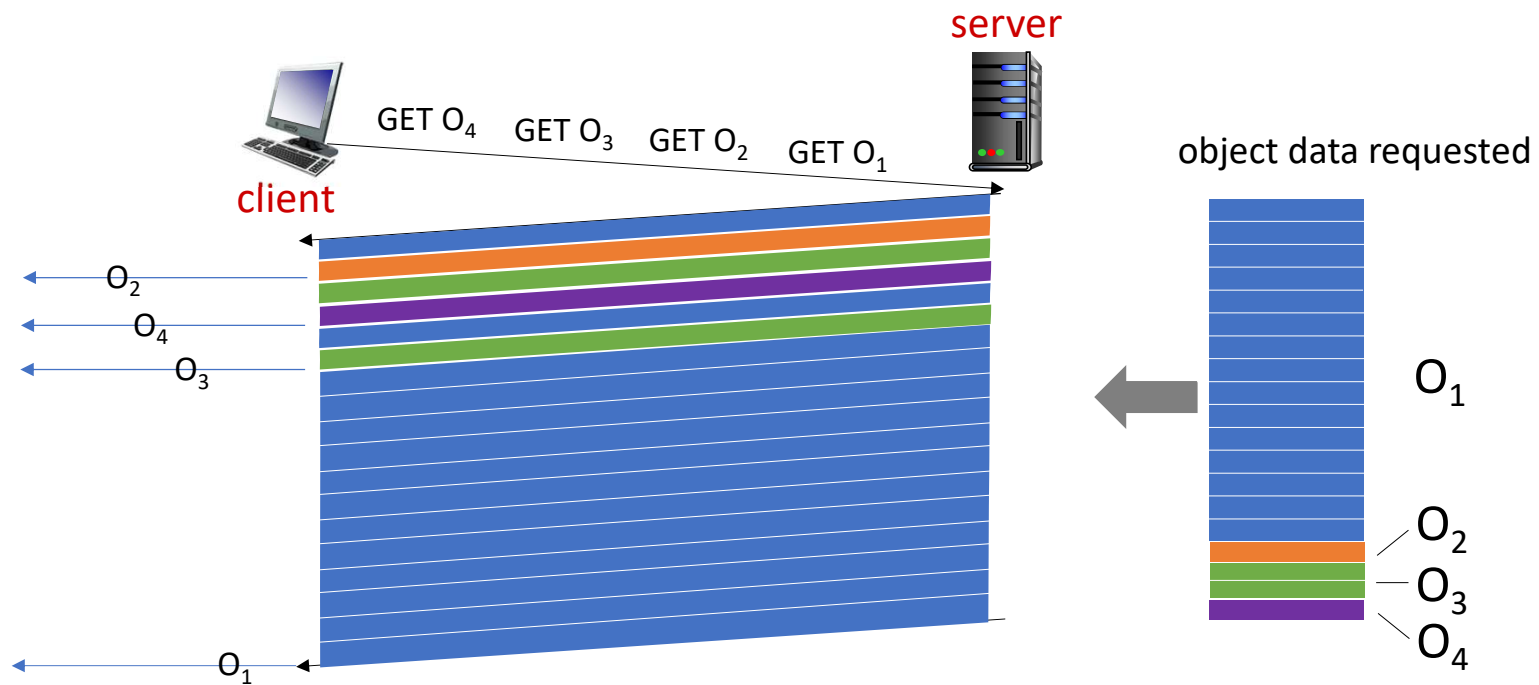
HTTP 1.1: client requests 1 large object (e.g., video file) and 3 smaller objects



objects delivered in order requested: O_2 , O_3 , O_4 wait behind O_1

HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



O₂, O₃, O₄ delivered quickly, O₁ slightly delayed

HTTP/2 to HTTP/3

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
 - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- **HTTP/3**: adds security, per object error- and congestion-control (more pipelining) over UDP
 - more on HTTP/3 in transport layer

Application layer: overview

- Principles of network applications
- Web and HTTP
- **E-mail, SMTP, IMAP**
- The Domain Name System
DNS



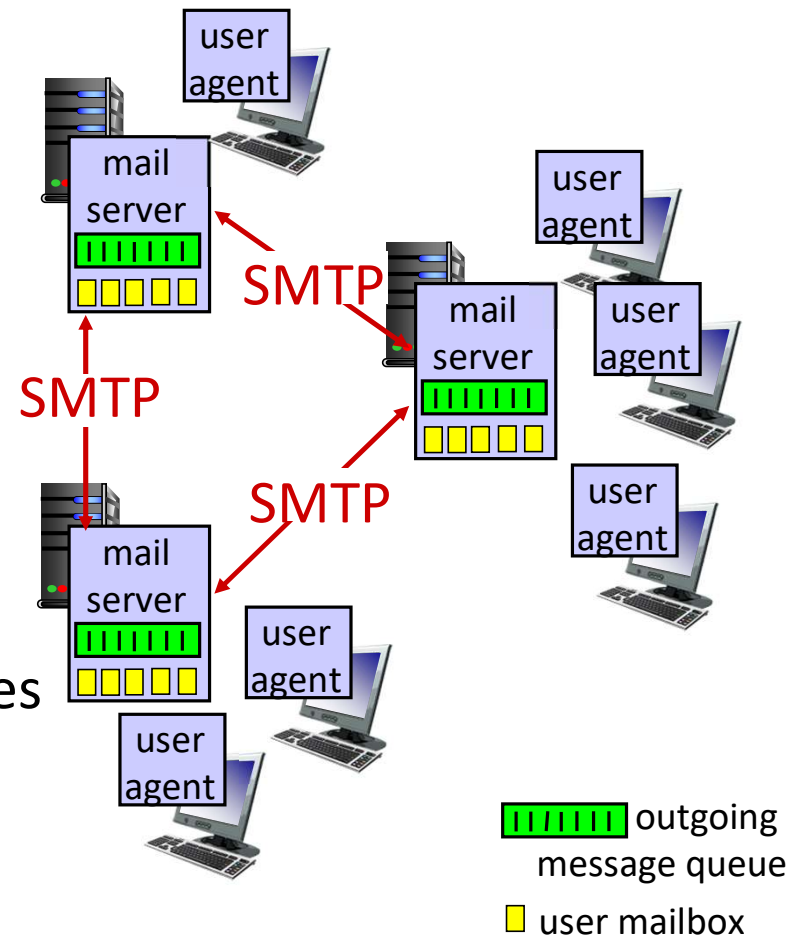
E-mail

Three major components:

- user agents
- mail servers
- simple mail transfer protocol: SMTP

User Agent

- a.k.a. “mail reader”
- composing, editing, reading mail messages
- e.g., Outlook, iPhone mail client
- outgoing, incoming messages stored on server



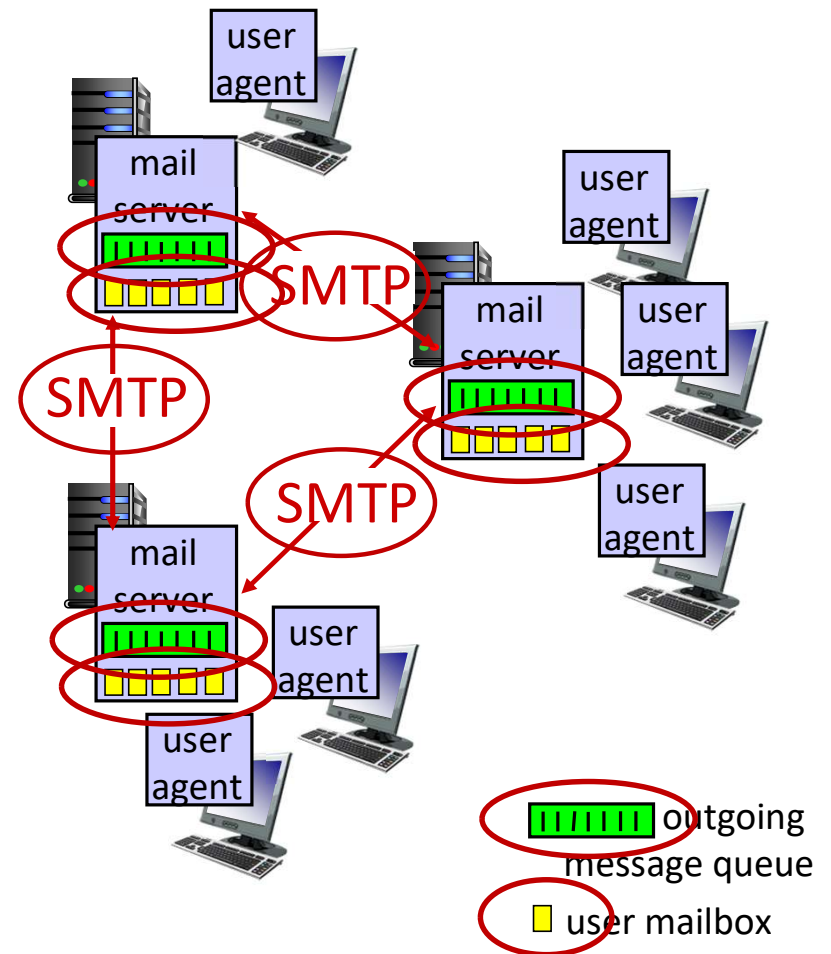
E-mail: mail servers

mail servers:

- *mailbox* contains incoming messages for user
- *message queue* of outgoing (to be sent) mail messages

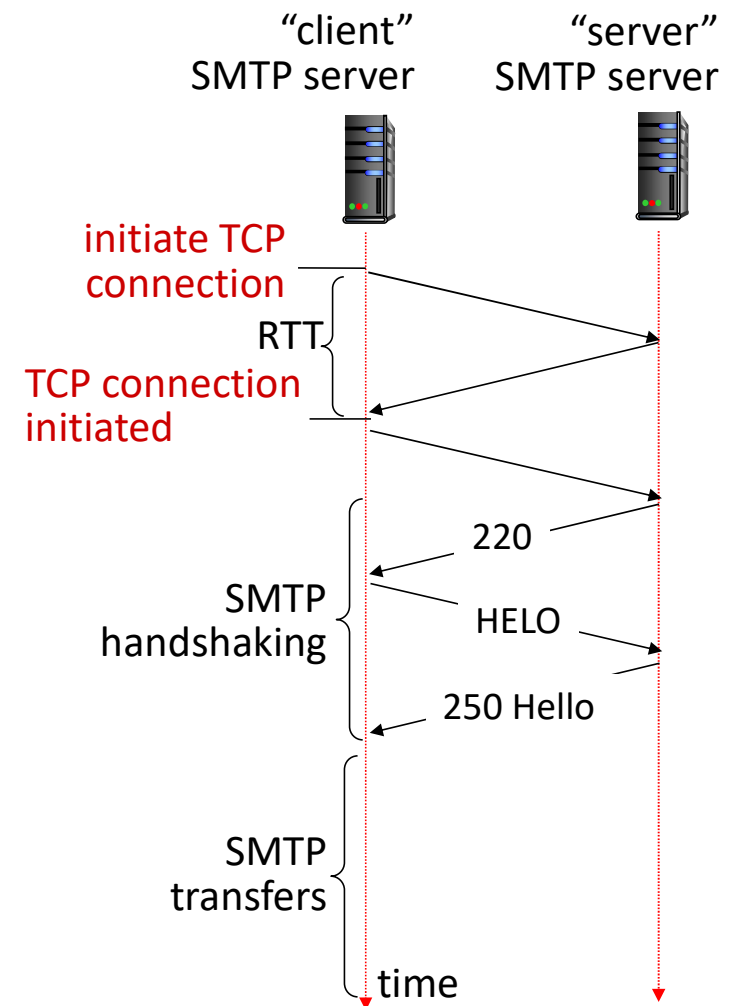
SMTP protocol between mail servers to send email messages

- **client**: sending mail server
- **“server”**: receiving mail server



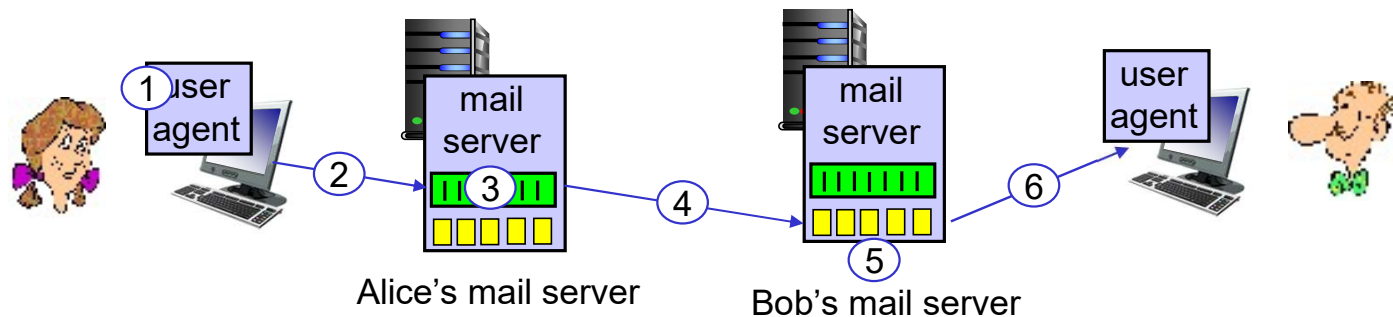
SMTP RFC (5321)

- uses TCP to reliably transfer email message from client (mail server initiating connection) to server, port 25
 - direct transfer: sending server (acting like client) to receiving server
- three phases of transfer
 - SMTP handshaking (greeting)
 - SMTP transfer of messages
 - SMTP closure
- command/response interaction (like HTTP)
 - **commands**: ASCII text
 - **response**: status code and phrase



Scenario: Alice sends e-mail to Bob

- 1) Alice uses UA to compose e-mail message "to" bob@some school.edu
- 2) Alice's UA sends message to her mail server using SMTP; message placed in message queue
- 3) client side of SMTP at mail server opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



Sample SMTP interaction

S: 220 hamburger.edu

Sample SMTP interaction

S: 220 hamburger.edu

C: HELO crepes.fr

S: 250 Hello crepes.fr, pleased to meet you

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
```

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
```

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
```

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
```

Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP: observations

comparison with HTTP:

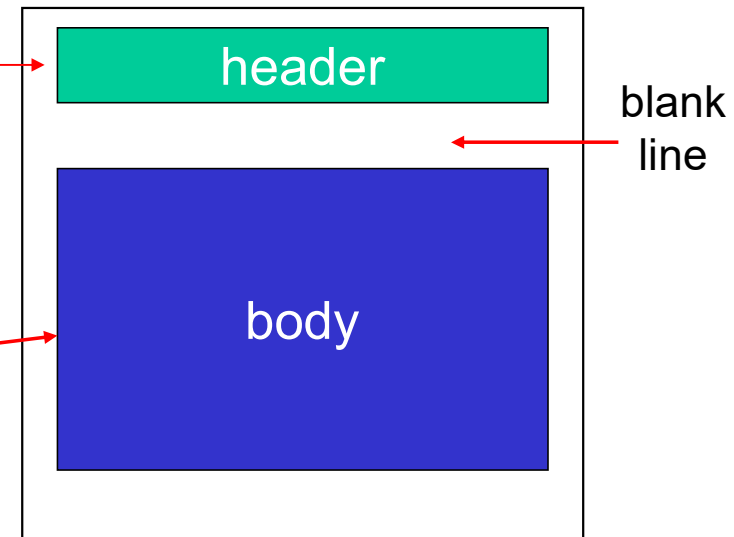
- HTTP: client pull
- SMTP: client push
- both have ASCII command/response interaction, status codes
- HTTP: each object encapsulated in its own response message
- SMTP: multiple objects sent in multipart message
- SMTP uses persistent connections
- SMTP requires message (header & body) to be in 7-bit ASCII
- SMTP server uses CRLF.CRLF to determine end of message

Mail message format

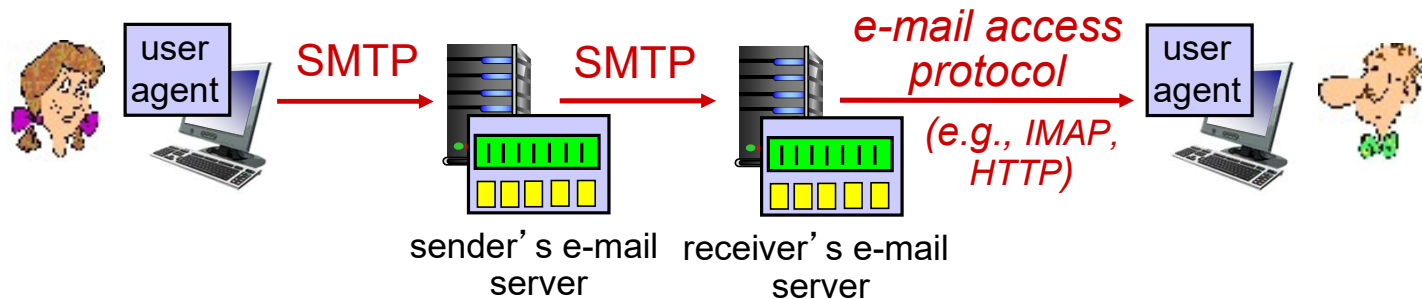
SMTP: protocol for exchanging e-mail messages, defined in RFC 5321 (like RFC 7231 defines HTTP)

RFC 2822 defines *syntax* for e-mail message itself (like HTML defines syntax for web documents)

- header lines, e.g.,
 - To:
 - From:
 - Subject:these lines, within the body of the email message area different from SMTP MAIL FROM:, RCPT TO: commands!
- Body: the “message” , ASCII characters only



Retrieving email: mail access protocols



- **SMTP**: delivery/storage of e-mail messages to receiver's server
- mail access protocol: retrieval from server
 - **IMAP**: Internet Mail Access Protocol [RFC 3501]: messages stored on server, IMAP provides retrieval, deletion, folders of stored messages on server
- **HTTP**: gmail, Hotmail, Yahoo!Mail, etc. provides web-based interface on top of SMTP (to send), IMAP (or POP) to retrieve e-mail messages

Application Layer: Overview

- Principles of network applications
- Web and HTTP
- E-mail, SMTP, IMAP
- The Domain Name System
DNS



DNS: Domain Name System

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g.,
www.facebook.com- used by humans

Q: how to map between IP address and name, and vice versa ?

Domain Name System (DNS):

- *distributed database* implemented in hierarchy of many *name servers*
- *application-layer protocol*: hosts, DNS servers communicate to *resolve* names (address/name translation)
 - *note*: core Internet function, implemented as application-layer protocol
 - complexity at network’s “edge”

DNS: services, structure

DNS services:

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- host with a complicated hostname can have one or more alias names.
Ex.: *relay1.west-coast.enterprise.com* can have alias *enterprise.com*. *Full hostname is called **canonical hostname***. Aliases are typically more mnemonic than canonical hostnames.
- DNS can be invoked by an application to obtain the canonical hostname for a supplied alias hostname as well as the IP address of the host.

DNS: services, structure

DNS services:

- hostname-to-IP-address translation
- host aliasing
 - canonical, alias names
- mail server aliasing (same reasons to have it as host aliasing)
- load distribution
 - replicated Web servers: many IP addresses correspond to one name

Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries/day
- Akamai DNS servers alone: 2.2T DNS queries/day

Thinking about the DNS

humongous distributed database:

- ~ billion records, each simple

handles many *trillions* of queries/day:

- *many* more reads than writes
- *performance matters*: almost every Internet transaction interacts with DNS - msec count!

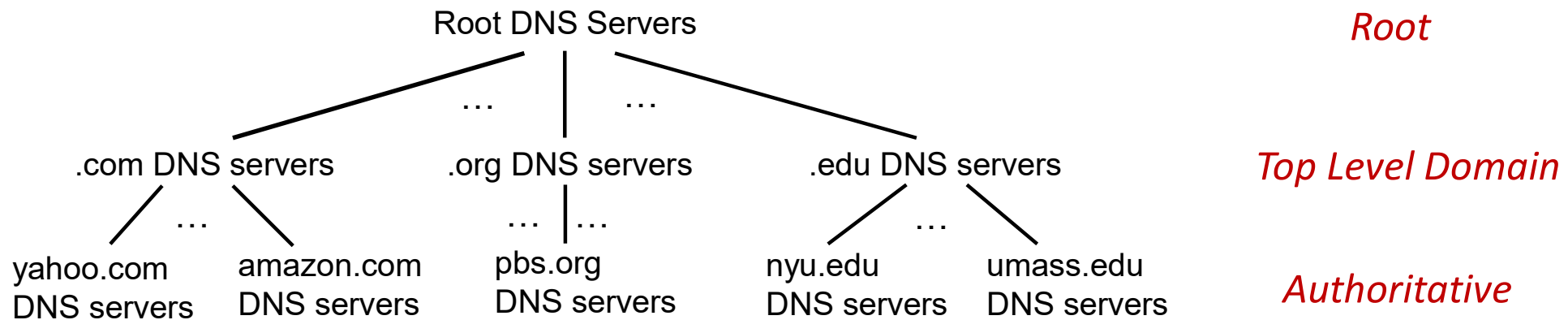
organizationally, physically decentralized:

- millions of different organizations responsible for their records

“bulletproof”: reliability, security



DNS: a distributed, hierarchical database

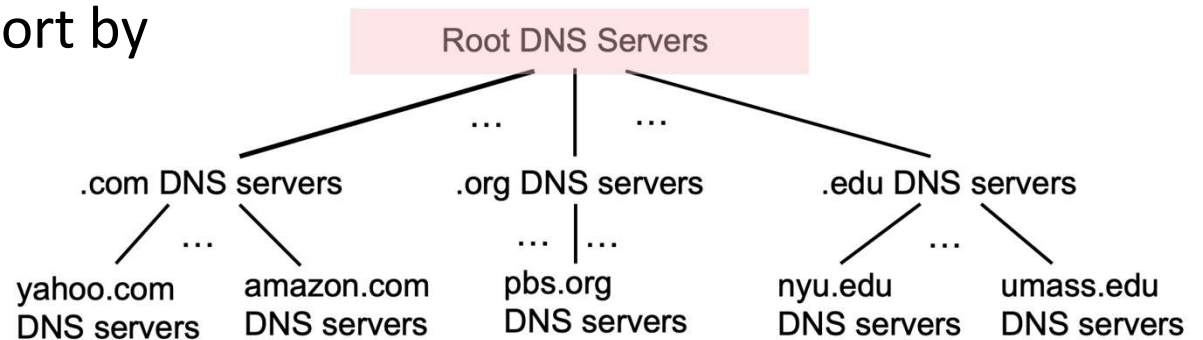


Client wants IP address for www.amazon.com; 1st approximation:

- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: root name servers

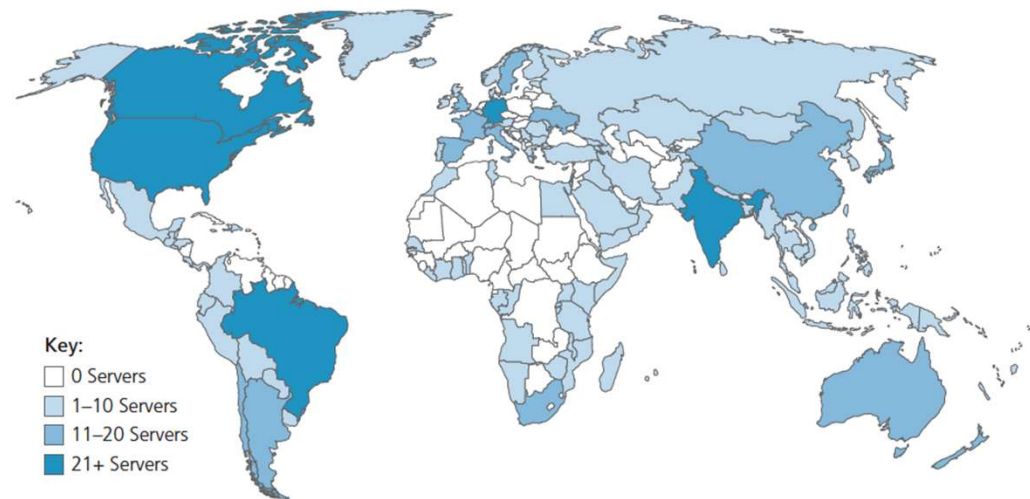
- official, contact-of-last-resort by name servers that can not resolve name



DNS: root name servers

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
 - Internet couldn't function without it!
 - DNSSEC – provides security (authentication, message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

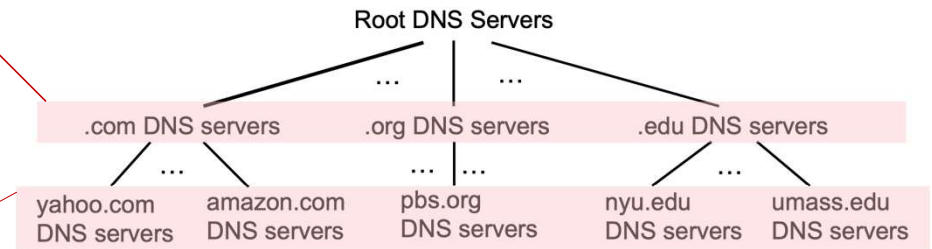
13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)



Top-Level Domain, and authoritative servers

Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD



authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider

Local DNS name servers

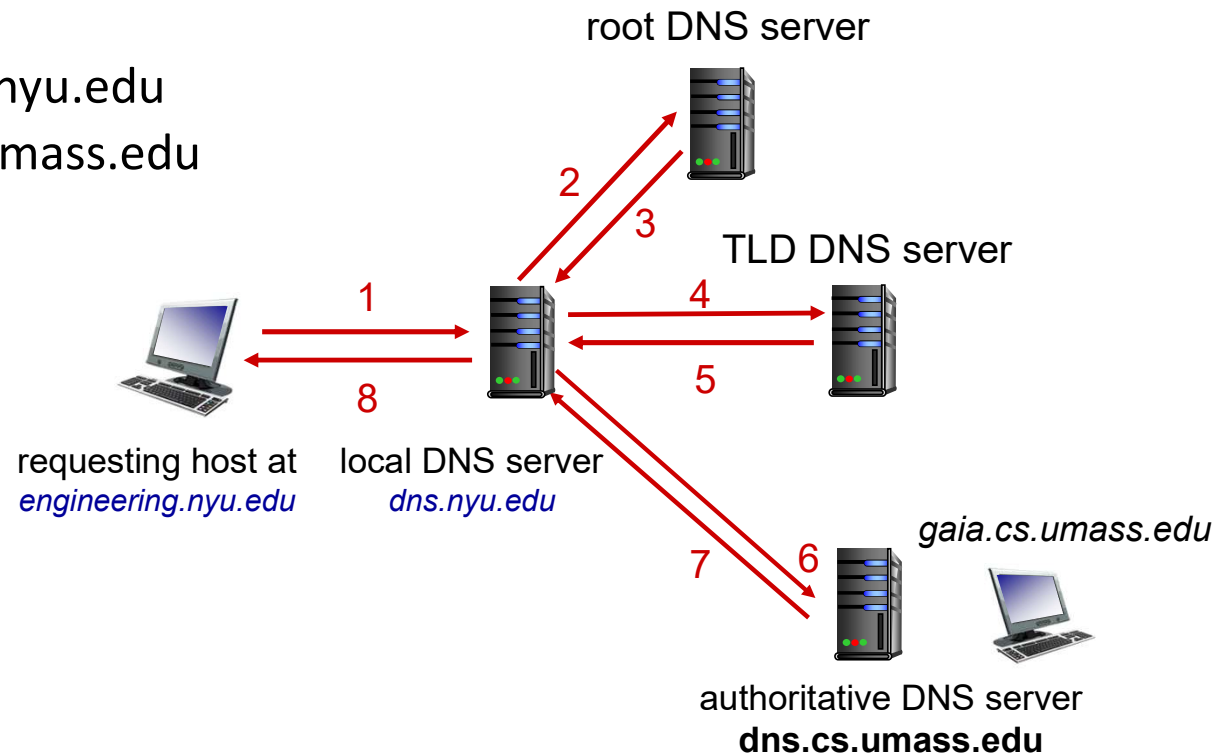
- when host makes DNS query, it is sent to its *local* DNS server
 - Local DNS server returns reply, answering:
 - from its local cache of recent name-to-address translation pairs (possibly out of date!)
 - forwarding request into DNS hierarchy for resolution
 - each ISP has local DNS name server; to find yours:
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- local DNS server doesn't strictly belong to hierarchy

DNS name resolution: iterated query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”

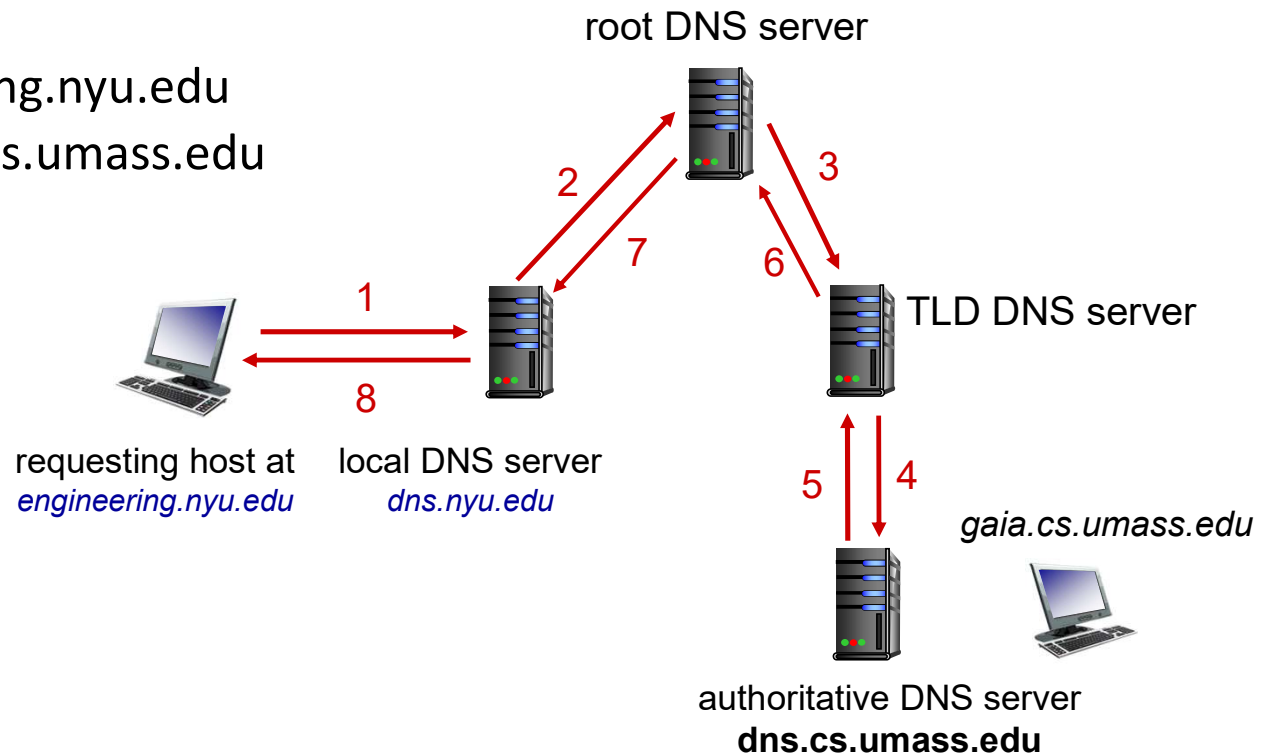


DNS name resolution: recursive query

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



Caching DNS Information

- once (any) name server learns mapping, it *caches* mapping, and *immediately* returns a cached mapping in response to a query
 - caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
- cached entries may be *out-of-date*
 - if named host changes IP address, may not be known Internet-wide until all TTLs expire!
 - *best-effort name-to-address translation!*

DNS records

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address
- Ex.: (relay1.bar.foo.com, 145.37.93.126, A)

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain
- Ex.: (foo.com, dns.foo.com, NS)

type=CNAME

- name is alias name for some “canonical” (the real) name
- value is canonical name
- Ex.: (foo.com, relay1.bar.foo.com, CNAME)

type=MX

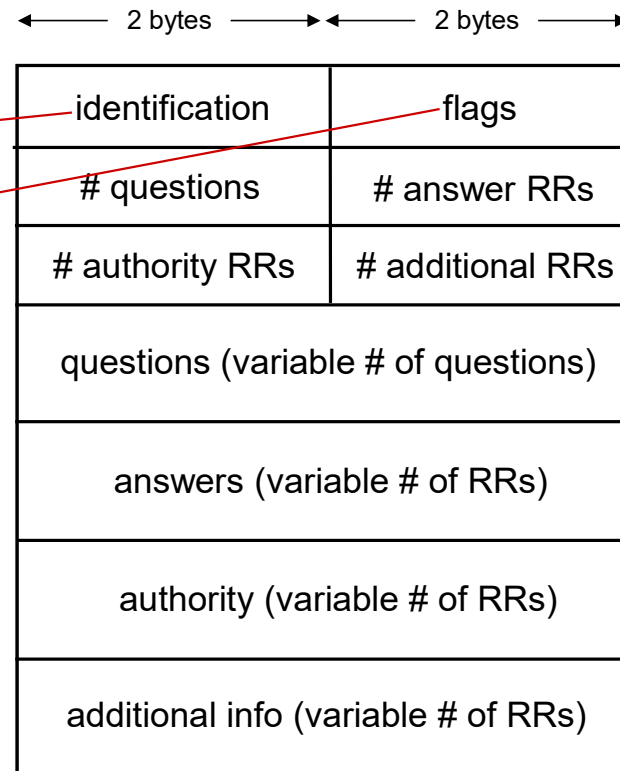
- value is name of SMTP mail server associated with name
- Ex.: (foo.com, relay1.bar.foo.com, CNAME)

DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

message header:

- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply
 - recursion desired
 - recursion available
 - reply is authoritative



DNS protocol messages

DNS *query* and *reply* messages, both have same *format*:

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

name, type fields for a query

RRs in response to query

records for authoritative servers

additional “helpful” info that may
be used

Getting your info into the DNS

example: new startup “Network Utopia”

- register name networkutopia.com at *DNS registrar* (e.g., Network Solutions)
 - provide names, IP addresses of authoritative name server (primary and secondary)
 - registrar inserts NS, A RRs into .com TLD server:
`(networkutopia.com, dns1.networkutopia.com, NS)`
`(dns1.networkutopia.com, 212.212.212.1, A)`
- create authoritative server locally with IP address 212.212.212.1
 - type A record for www.networkutopia.com
 - type MX record for networkutopia.com

DNS security

DDoS attacks

- bombard root servers with traffic
 - not successful to date
 - traffic filtering
 - local DNS servers cache IPs of TLD servers, allowing root server bypass
- bombard TLD servers
 - potentially more dangerous

Spoofing attacks

- intercept DNS queries, returning bogus replies
 - DNS cache poisoning
 - RFC 4033: DNSSEC authentication services

Lecture 2: Summary

our study of network application layer is now complete!

- application architectures
 - client-server
 - P2P
- application service requirements:
 - reliability, bandwidth, delay
- Internet transport service model
 - connection-oriented, reliable: TCP
 - unreliable, datagrams: UDP
- specific protocols:
 - HTTP
 - SMTP, IMAP
 - DNS

Lecture 2: Summary

Most importantly: learned about *protocols*!

- typical request/reply message exchange:
 - client requests info or service
 - server responds with data, status code
- message formats:
 - *headers*: fields giving info about data
 - *data*: info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer
- “complexity at network edge”



COMPUTER NETWORKS

Network topologies and signal principles

Lecturer: G. Kalfas

Notes on the slides

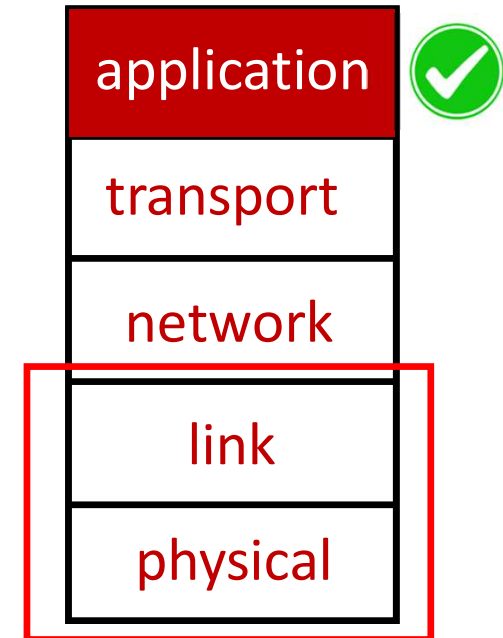
- The material (slides) of this course is a mixture of the following:
 1. slides provided freely by J.F Kurose and K.W. Ross accompanying the book “Computer Networking: A top-down approach” (this book is mentioned in the references of the Syllabus - slides can be found here:
https://gaia.cs.umass.edu/kurose_ross/ppt.php)
 2. slides provided by CITY college and
 3. my own slides

Issues Covered

- Types of Connections
- Network Categories and Topologies
- Signals
- Bandwidth
- Modulation

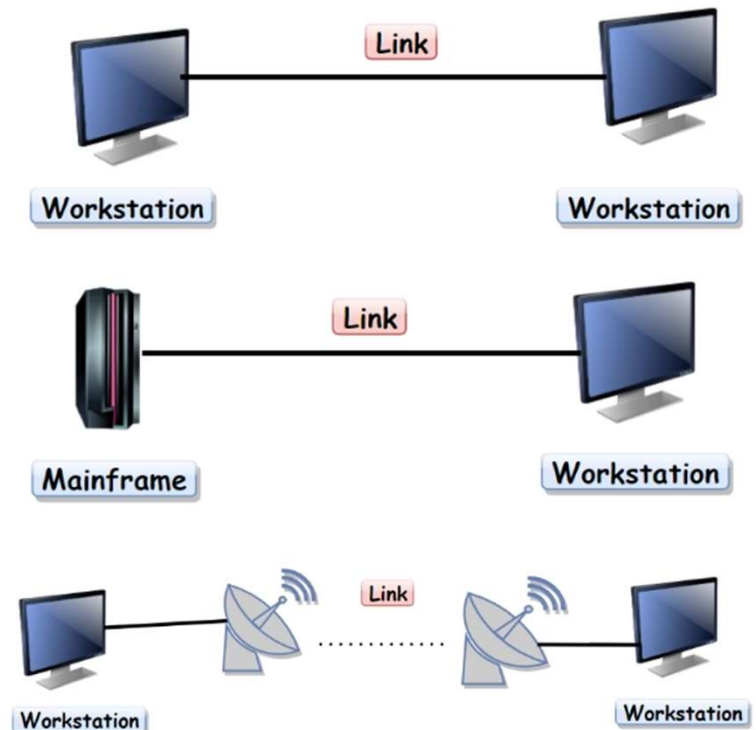
Story so far...

- We introduced the basic concepts of networking
- We discussed about layers
- We analyzed Application Layer
- Now we dive into the lower layers to introduce the Physical aspects of topologies and signals



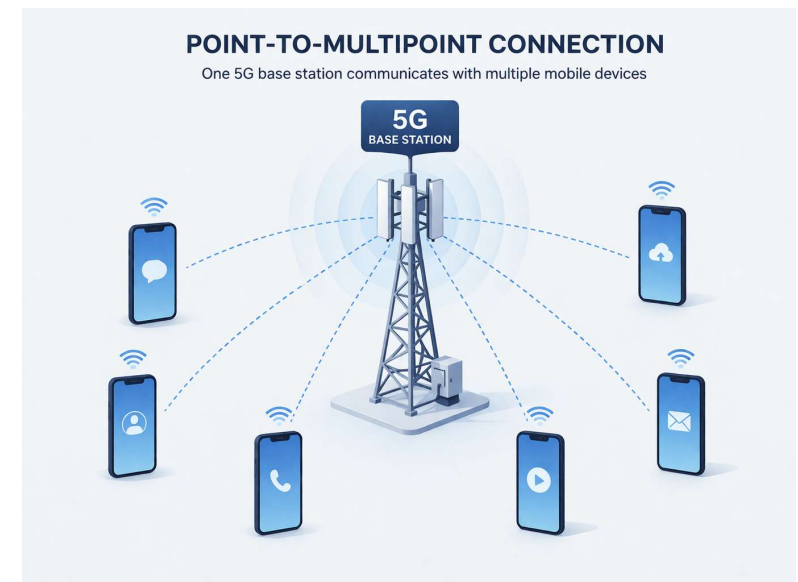
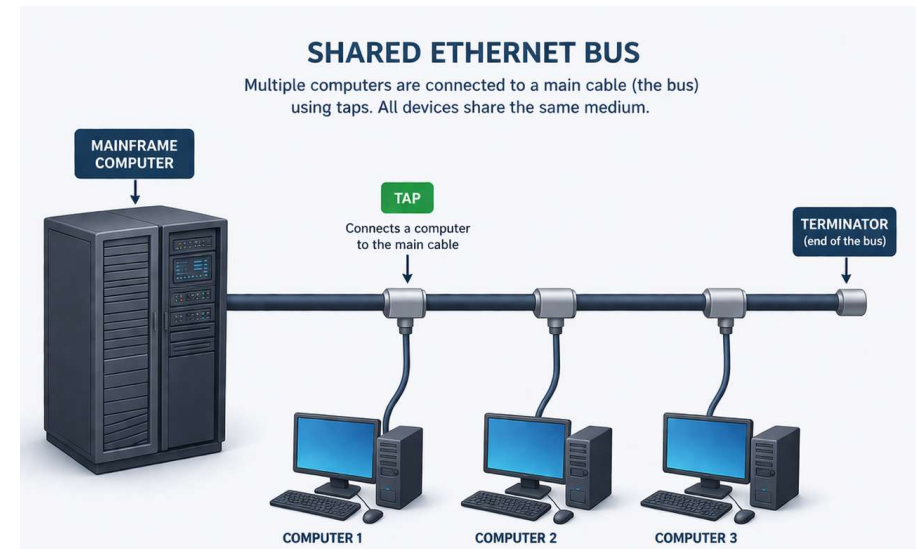
TYPES OF CONNECTIONS

- **Point-to-Point Connection:** A dedicated link connects exactly two devices. The full capacity of the link is available to those two devices.
- Link can be either wired or wireless.

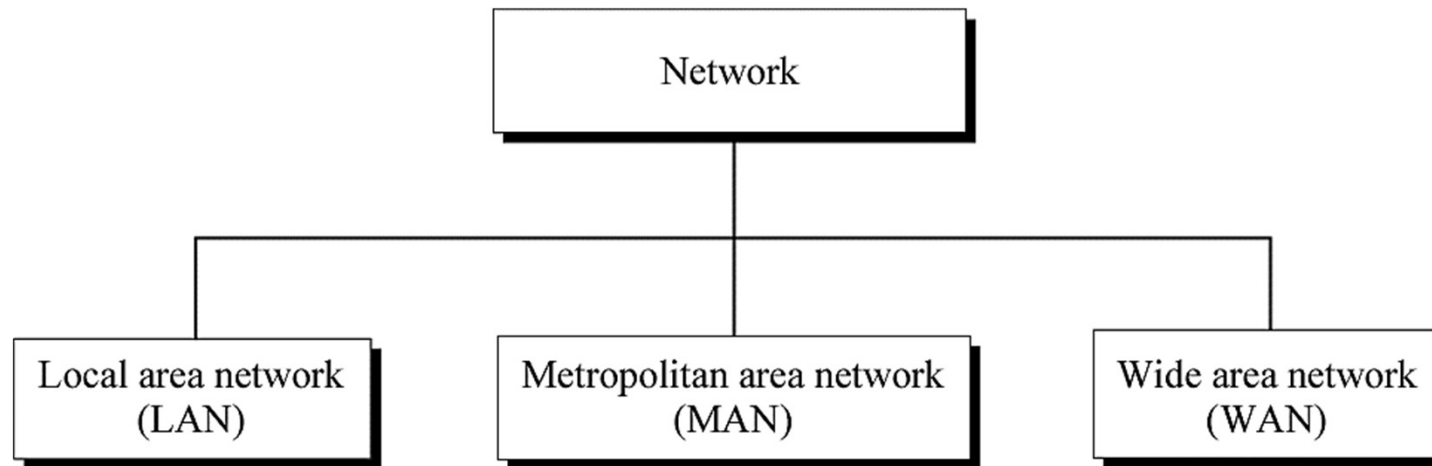


TYPES OF CONNECTIONS

- **Point-to-Multipoint Connection:**
Multiple devices share a single communication link. The link capacity is shared among the connected devices, either over time or by dividing the medium (such as frequency multiplexing).
- Shared link can be wired or wireless.



CATEGORIES OF NETWORKS



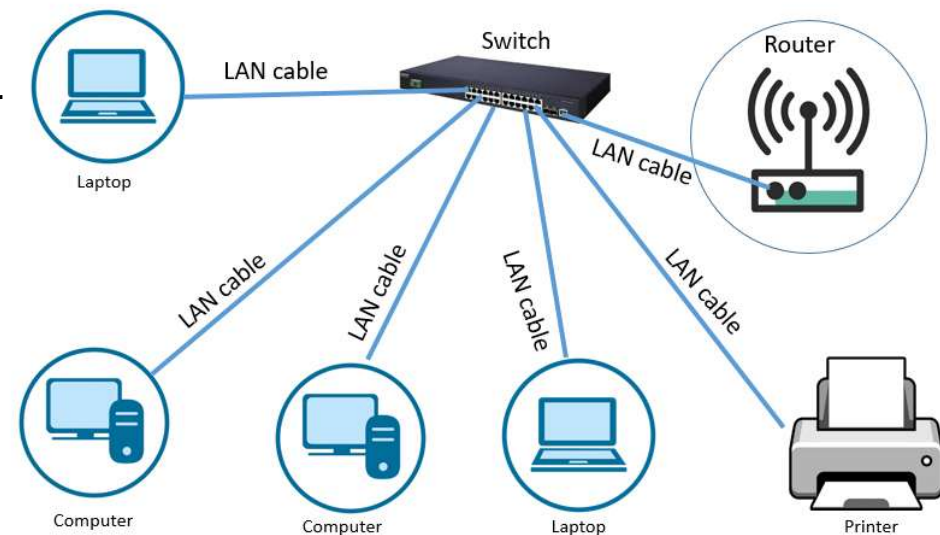
- Networks are commonly classified by the geographic area they cover, the number of users or devices and the way they are managed or administered
- The three common categories are: LAN, MAN, and WAN.

LOCAL AREA NETWORKS

- A LAN connects devices within a limited area such as a room, home, office, building, or campus.
- LANs are usually managed by one person or one organization.
- Modern LANs commonly use Ethernet switches and Wi-Fi access points

Examples:

- a school computer lab
- a home Wi-Fi network
- an office network connecting PCs, printers, and servers



METROPOLITAN AREA NETWORK

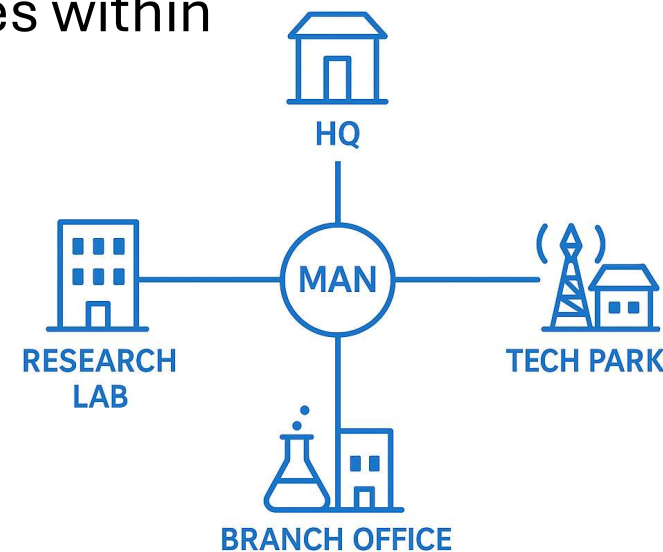
- A **MAN** connects multiple LANs across a city or metropolitan region. It is larger than a LAN but smaller than a WAN.

Typical examples:

- a university network connecting several campuses within one city
- a municipal government network
- a citywide fiber or metro Ethernet network

Common technologies:

- fiber optic links
- metropolitan Ethernet
- high-capacity wireless links in some cases



WIDE AREA NETWORK

- A **WAN** connects LANs or other networks across large geographic areas, such as regions, countries, or continents.
- A WAN is often described as a collection of LANs or other networks that communicate with one another — essentially a **network of networks**.
- WANs usually have **higher latency**, greater operational complexity, and often higher costs.

Examples

- a company connecting branch offices in different countries
- cloud-connected enterprise sites
- the Internet

Common WAN technologies

- leased lines
- broadband Internet
- fiber
- cellular networks such as **4G/5G**
- satellite



LAN vs MAN vs WAN

Type	Typical area	Example	Common technologies	Usually managed by
LAN	room, home, office, building, campus	school lab	Ethernet, Wi-Fi	one person or one organization
MAN	city / metropolitan area	city university network	fiber, metro Ethernet	large organization / service provider
WAN	country / continent / global	multinational company network	broadband, 4G/5G, satellite	telecom/provider + organization

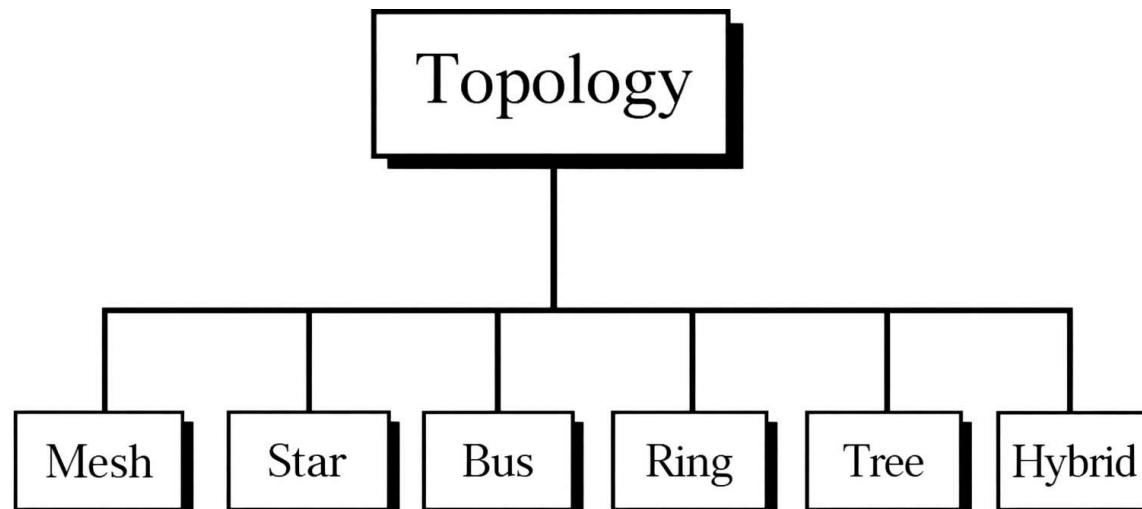
QUIZ: Identify the network type

1. Three campuses connected across one city → **LAN/MAN/WAN? → MAN**
2. A home with laptops, phones, and a Wi-Fi router → **LAN/MAN/WAN? → LAN**
3. Company offices in Greece, UK, and the US → **LAN/MAN/WAN? → WAN**

PHYSICAL TOPOLOGY

- **Physical topology** is the physical arrangement of devices and communication links in a network. It shows how devices are connected to each other.
- It is related to network design, but it is **not the main basis** for classifying a network as LAN, MAN, or WAN.

The Six Basic Topologies Are:



MESH TOPOLOGY

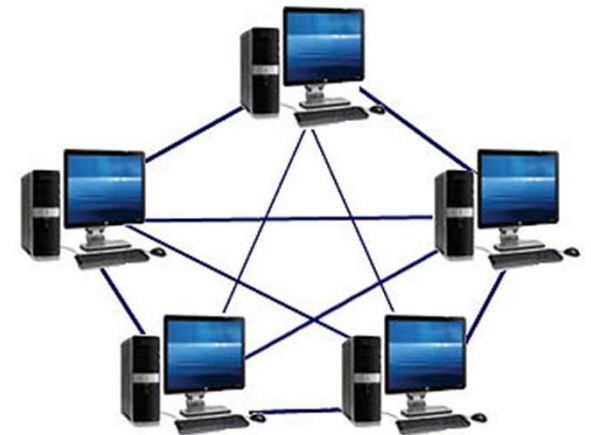
In a **mesh topology**, devices are connected with many direct links. In a **full mesh**, every device has a dedicated point-to-point link to every other device.

Advantages:

- High reliability
- Strong fault tolerance
- Easy fault identification and isolation
- Better traffic isolation through dedicated links

Disadvantages:

- Requires a large amount of cabling
- Needs many input/output ports
- Expensive to install and maintain



BUS TOPOLOGY

In a **bus topology**, all devices share a single main cable called a **backbone**. Data travels along the backbone, and all connected devices receive the signal.

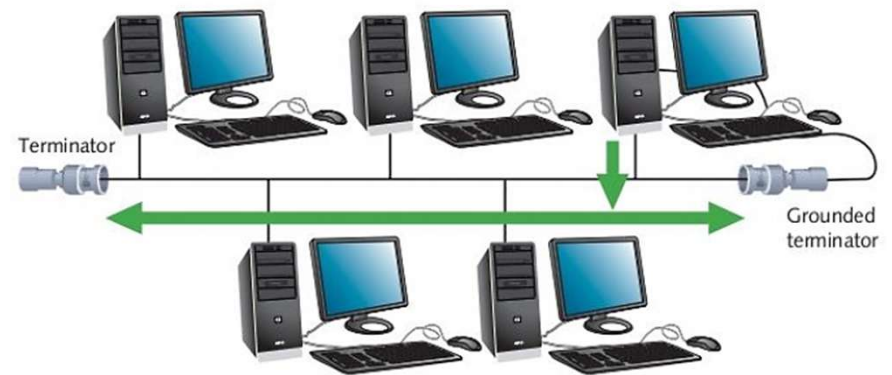
Advantages:

- Simple design
- Uses less cabling
- Inexpensive for small networks

Disadvantages:

- Difficult fault isolation
- Performance decreases as traffic increases
- If the backbone cable fails, the whole network may be disrupted

Deprecated as a wired technology



RING TOPOLOGY

In a **ring topology**, each device is connected to two neighboring devices, forming a closed loop. Data passes from device to device around the ring until it reaches the destination.

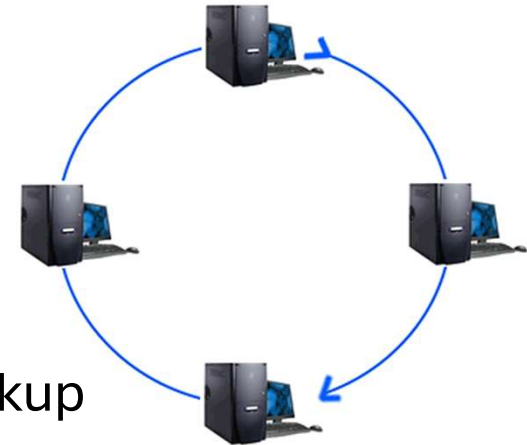
Advantages:

- Data follows a fixed and predictable path around the ring
- Easier fault detection than in a bus topology
- No central device is required

Disadvantages:

- A break in the ring can disrupt communication unless backup mechanisms are used
- Adding or removing devices can be more difficult than in a star topology

Less common in modern LAN design, sometimes used in MANs/WANs



STAR TOPOLOGY

In a **star topology**, each device is connected to a central device, usually a **switch**.

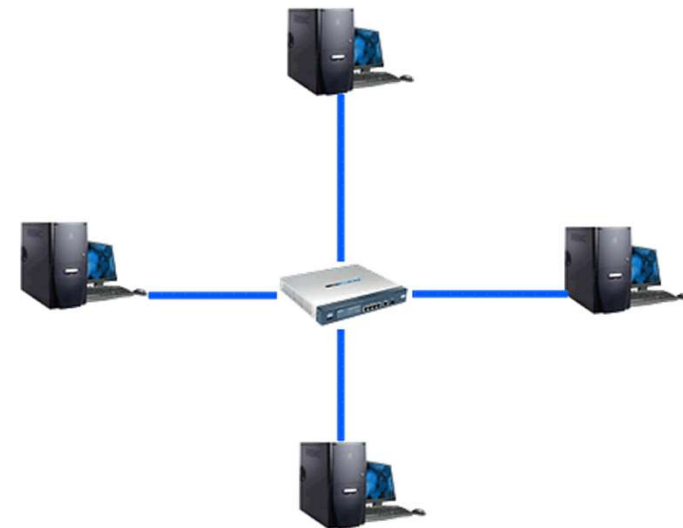
Advantages:

- Easy to install and expand
- Easy fault identification
- Failure of one cable usually affects only one device
- Good performance and manageability

Disadvantages:

- If the central switch fails, communication for the connected devices is disrupted
- Requires more cabling than a bus topology

Modern wired LANs are usually built as **switched star topologies**



TREE TOPOLOGY

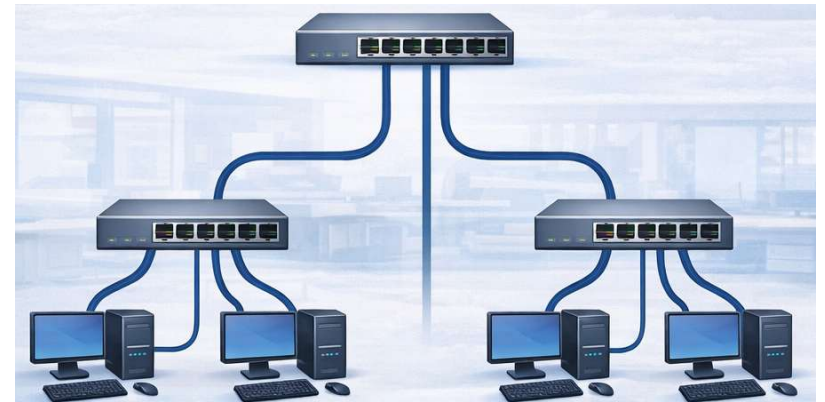
A **tree topology** is a hierarchical network structure in which devices are arranged in levels, forming a branching pattern. A central device connects to lower-level devices, which then connect to additional devices or end systems. Tree topology is commonly used in **large buildings, campuses, and enterprise networks** where devices are organized in hierarchical layers.

Advantages:

- Scalable for large networks
- Easy to expand by adding new branches
- Supports structured network management

Disadvantages:

- Failure of a higher-level device can affect many lower-level devices
- More cabling is required than in simpler topologies
- Configuration and maintenance can be more complex



HYBRID TOPOLOGY

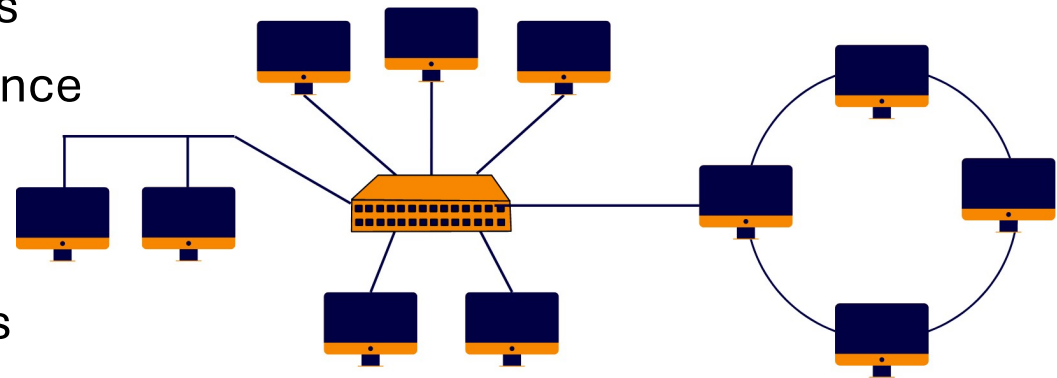
A **hybrid topology** is a network structure that combines two or more different topologies in the same network. It is used when different parts of a network have different requirements for performance, cost, scalability, or reliability.

Advantages:

- Flexible and adaptable
- Can be designed to meet specific needs
- Can improve scalability and fault tolerance

Disadvantages:

- More complex to design and manage
- More expensive than simpler topologies
- Troubleshooting can be more difficult



SIGNALS

Data transmission requires:

- encoding bits as physical energy
- transmitting that energy through a medium
- decoding the received energy back into data

The transmitted energy may take different forms, such as:

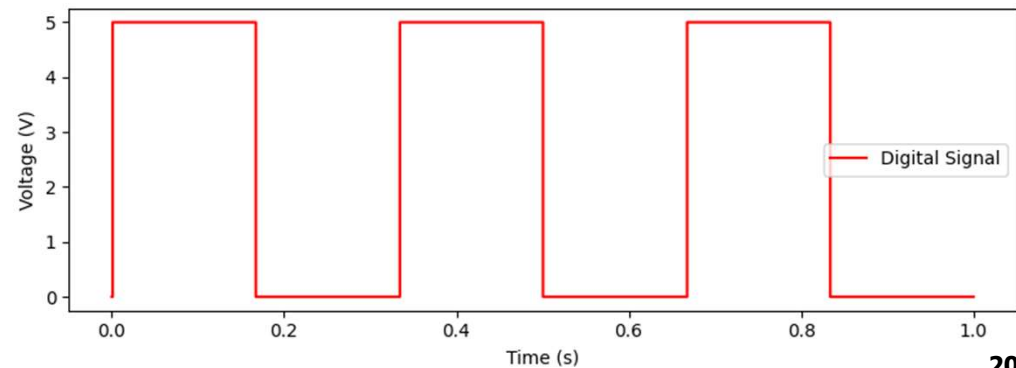
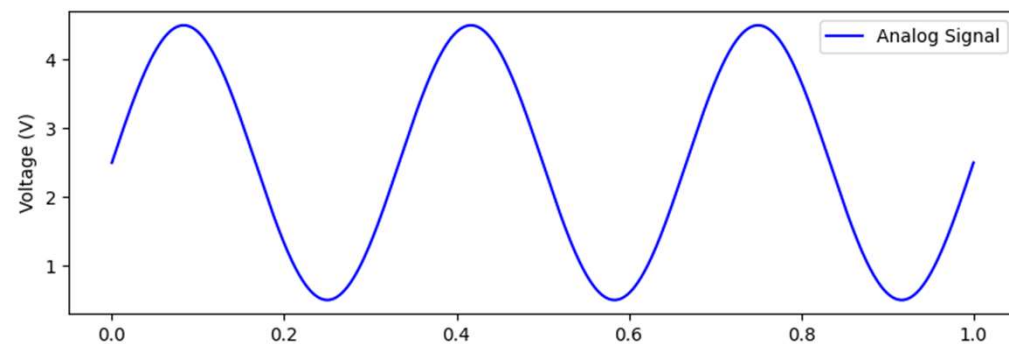
- electric voltage or current
- radio waves
- infrared
- light

Information such as data, voice, images, and video is transmitted using electromagnetic signals



ANALOG and DIGITAL SIGNALS

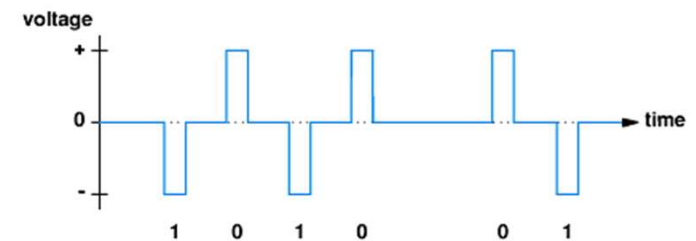
- Information to be transmitted must be represented by signals.
- An **analog signal** varies continuously over time and can take an infinite number of values within a range.
- A **digital signal** uses a finite set of discrete levels, (for instance two levels for binary communication).
- In practice, digital signals do **not** change instantaneously; real hardware always requires a small but finite transition time.



USING ELECTRIC CURRENT TO SEND BITS

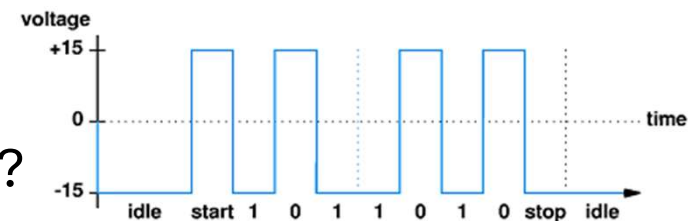
A simple digital link can represent bits using different voltage levels. Example:

- one voltage level represents binary 1
- another voltage level represents binary 0



Important questions:

- How long does each bit last?
- When does the next bit begin?
- How do sender and receiver stay synchronized?
- How can more data be transmitted in the same time?
- Answer: Handled by Physical and Data Link layer standards/protocols



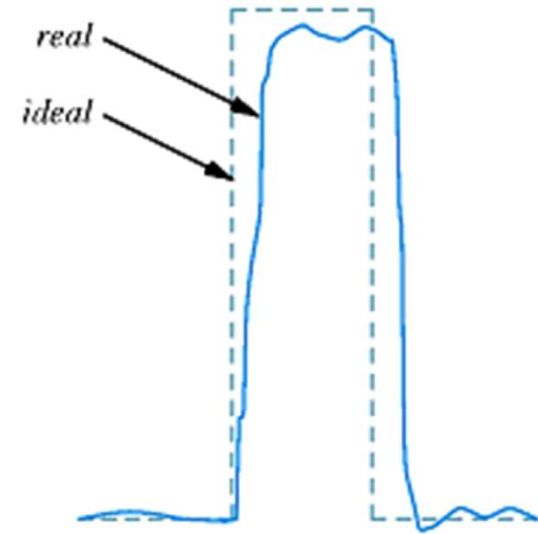
RS-232

LIMITATIONS OF HARDWARE

No electronic device can produce a perfect waveform or change voltage levels instantaneously! Because of this, communication standards must define:

- acceptable signal levels
- timing tolerances
- transmitter accuracy
- receiver tolerance to distortion and noise

These practical limitations affect the maximum data rate and transmission distance.



ANALOG VS DIGITAL COMMUNICATION

Analog communication

- information is represented by continuously varying signals
- examples: traditional **AM/FM radio, analog voice transmission**

Digital communication

- information is represented by discrete symbols, usually binary bits
- examples: **Ethernet, optical fiber communication, 5G/WiFi**

Digital data may be transmitted:

- directly as digital pulses
- or by changing the properties of an analog carrier (more info next slide)
- **In both cases, the communication is digital because the transmitted information is digital.**

SIGNALS ACROSS LONG DISTANCES

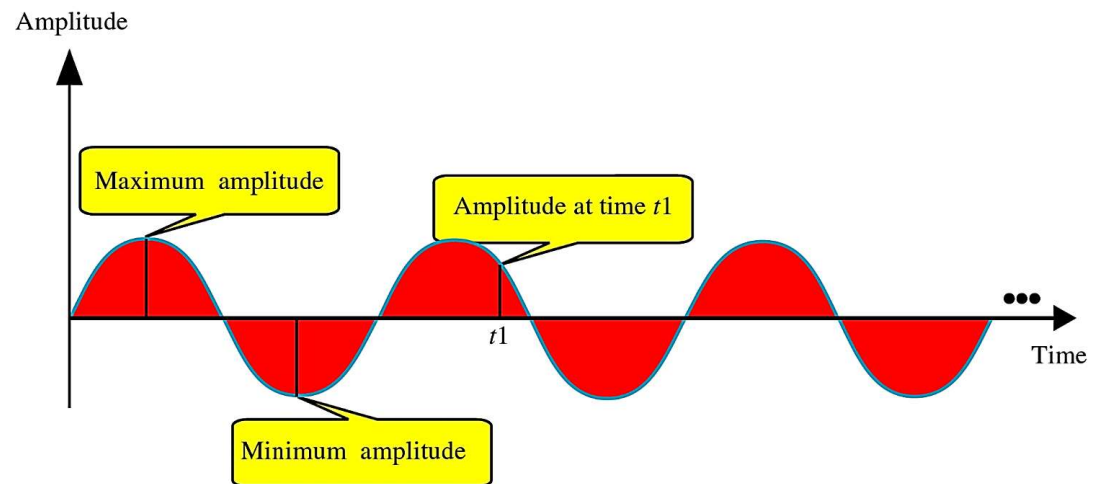
- Many communication systems, especially wireless and many long-distance links, use a continuous oscillating signal called a *carrier*
- Information can be conveyed by changing one or more properties of the carrier:
 - **Amplitude**
 - **Frequency**
 - **Phase**
- This process is called *modulation*.

AMPLITUDE OF A SIGNAL

- **Amplitude** is the strength or magnitude of a signal at a given point.
- It is measured as the vertical distance from the signal waveform to the reference axis.

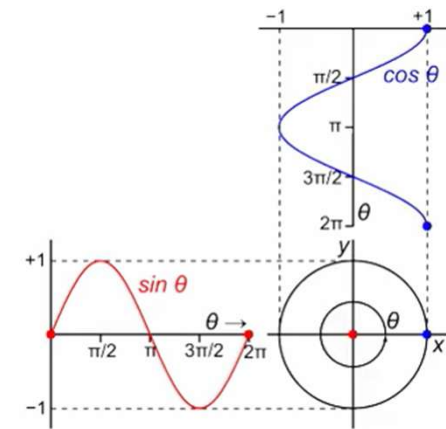
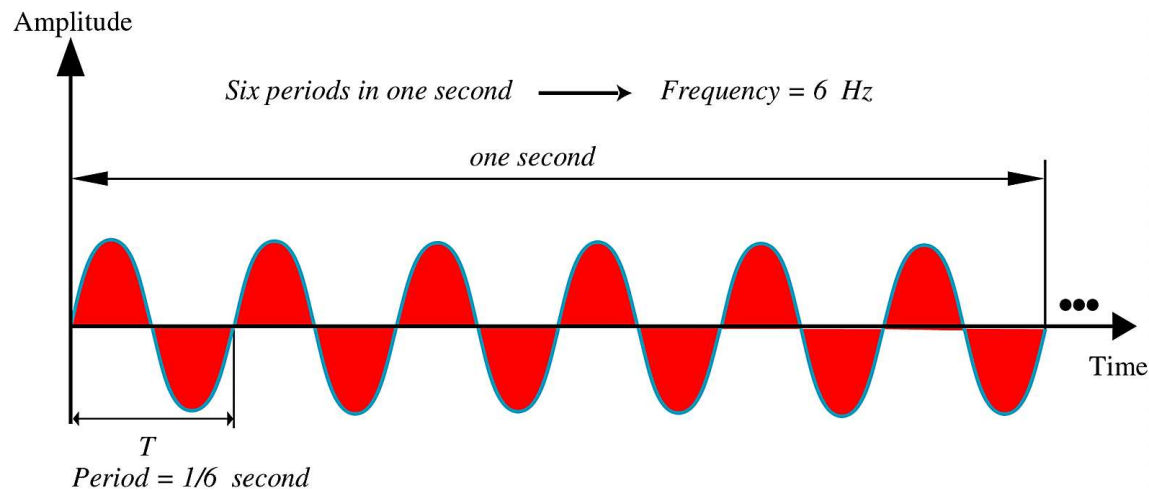
Amplitude may be expressed in:

- Volts: voltage
- Amperes: current
- Watts: power



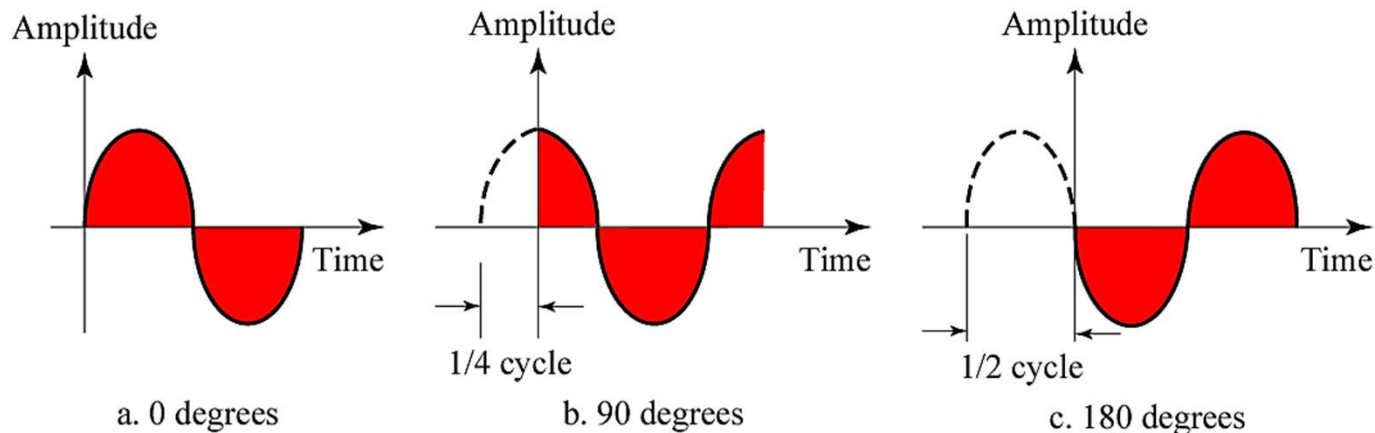
FREQUENCY OF A SIGNAL

- **Period** is the time required for a signal to complete one cycle.
- **Frequency** is the number of cycles completed in one second.
- Frequency is measured in **hertz (Hz)**.
- Relationship:
- Frequency = $1 / \text{Period}$



PHASE OF A SIGNAL

- **Phase** describes the relative position of a waveform within its cycle.
- It indicates how far a signal is shifted compared with a reference wave.
- Phase is measured in **degrees** or **radians**.

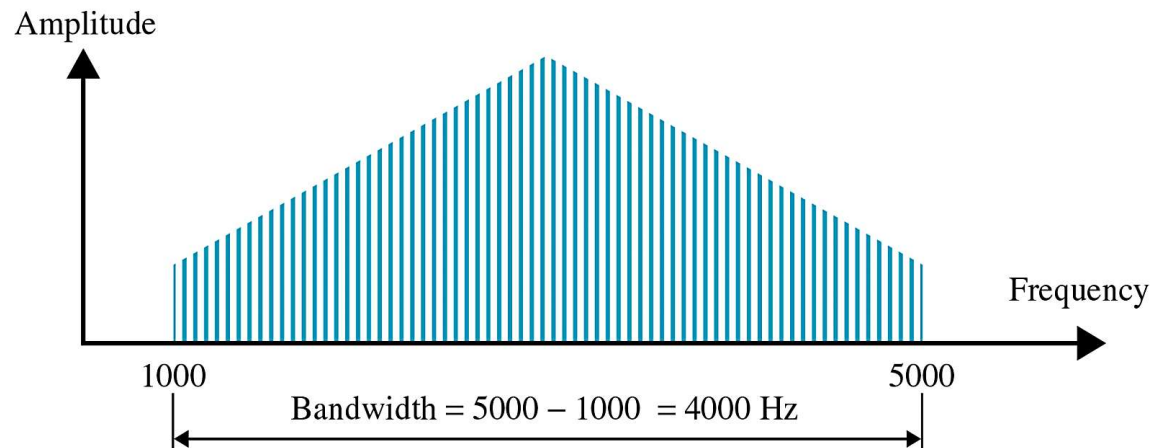


BANDWIDTH

- In **analog systems**, bandwidth is the range of frequencies a signal or channel occupies:

Bandwidth = highest frequency - lowest frequency

- It is measured in **hertz (Hz)**.
- In **digital communications**, people often informally use “bandwidth” to mean **data rate**, but the more precise term is **bit rate**, measured in **bits per second (bit/s or bps)**.



BIT RATE and BAUD RATE

- **Bit rate**: the number of bits transmitted per second (unit: bits per second)
- **Baud Rate**: the number of signal units or symbols (pulses) transmitted per second (unit: symbols per second)
- A **symbol** is one distinct signal state sent during one signaling interval. For example, a symbol may be represented by a specific voltage level, frequency, phase, or amplitude.
- A single symbol may represent one or more bits.

$$\text{Bit Rate} = \text{Baud rate} * \text{Bits per symbol}$$

$$\text{Baud Rate} = \text{Bit rate} / \text{Bits per symbol}$$

- If each symbol carries 1 bit, then bit rate equals baud rate. If each symbol carries **more than 1 bit**, then bit rate is greater than baud rate
- Analogy: baud analogous to car, bit to passenger

CHANNEL CAPACITY

- **Channel capacity** is the maximum rate at which data can be transmitted reliably over a communication channel under given conditions.
- It depends on factors such as:

- Bandwidth
- Noise
- acceptable error rate
- signal power
- transmission delay and distortion



These factors limit the amount of information a channel can carry

DATA ENCODING

How can analog carrier signals represent binary data?

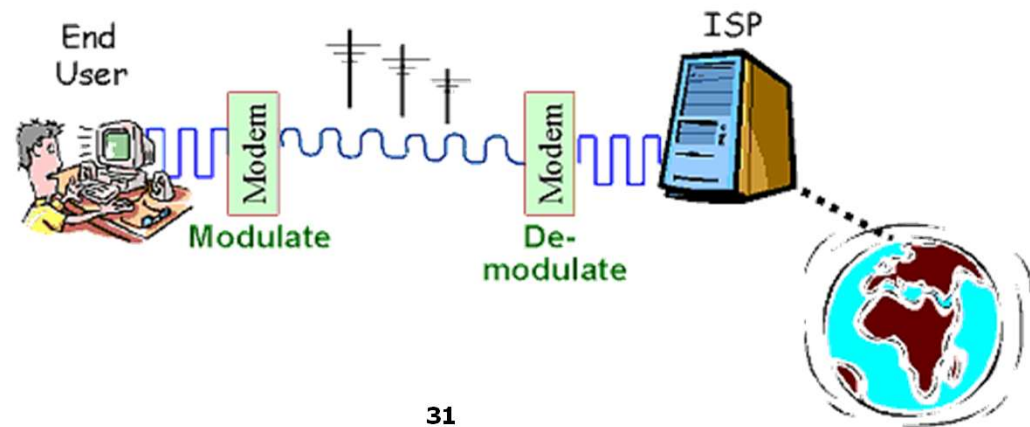
By changing one of the carrier's properties:

- amplitude
- frequency
- phase

This process is called **modulation**.

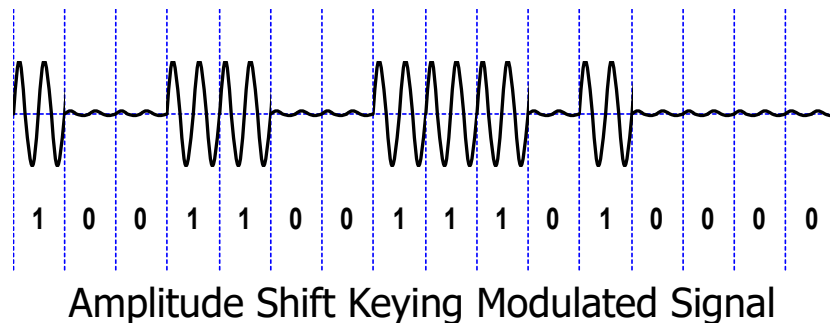
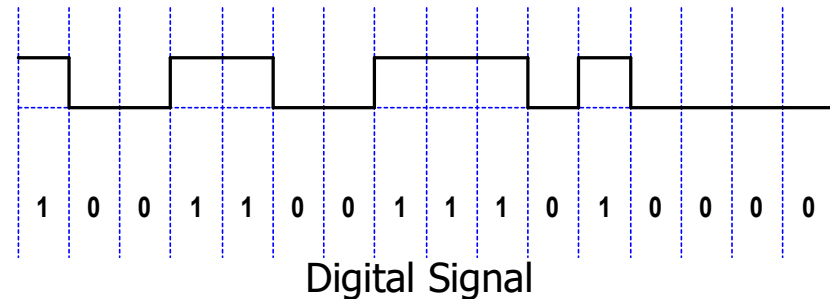
For digital data, the corresponding techniques are commonly called:

- **ASK**: Amplitude Shift Keying
- **FSK**: Frequency Shift Keying
- **PSK**: Phase Shift Keying



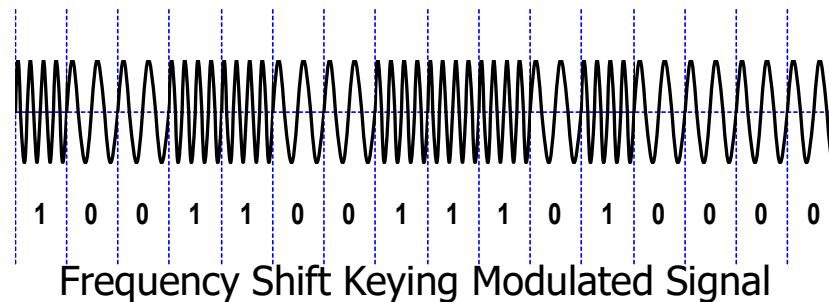
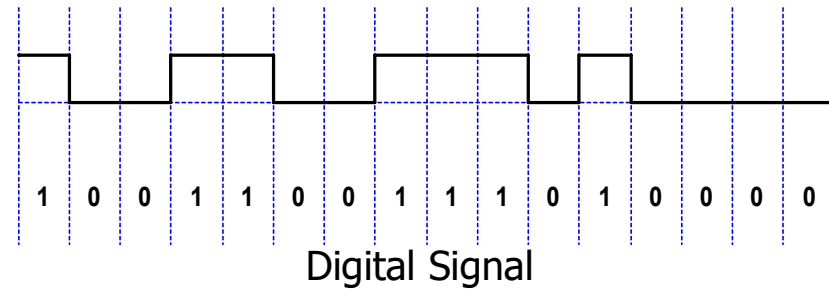
AMPLITUDE SHIFT KEYING

- The **amplitude** of the carrier is changed to represent digital symbols.
- Frequency and phase remain constant.



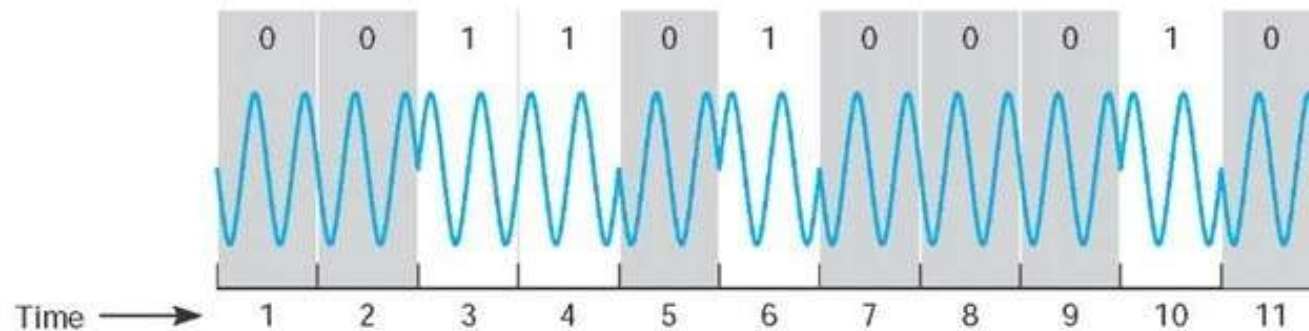
FREQUENCY SHIFT KEYING

- The **frequency** of the carrier is changed to represent digital symbols
- Amplitude and phase remain constant.

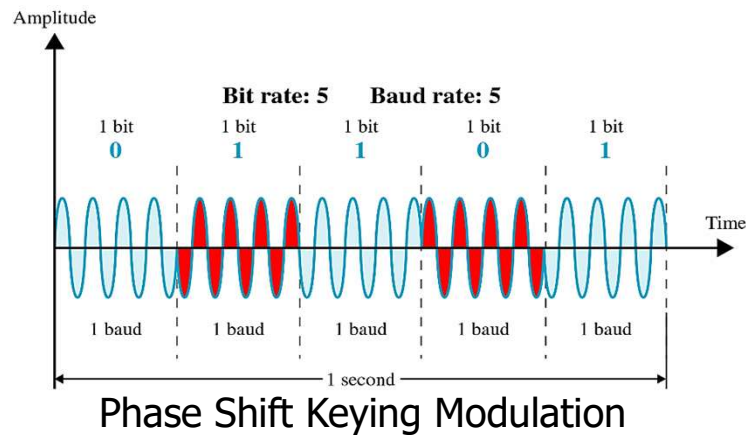
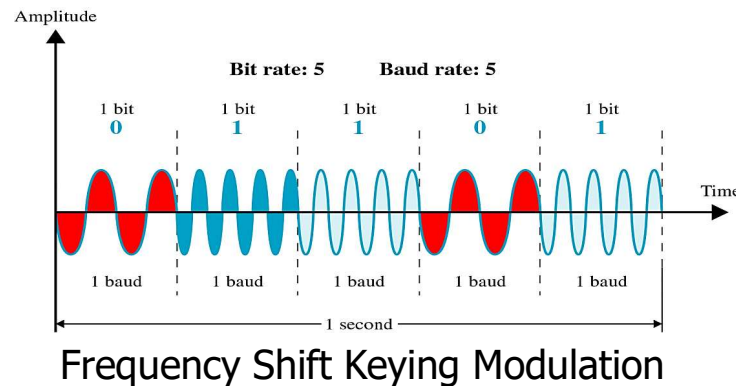
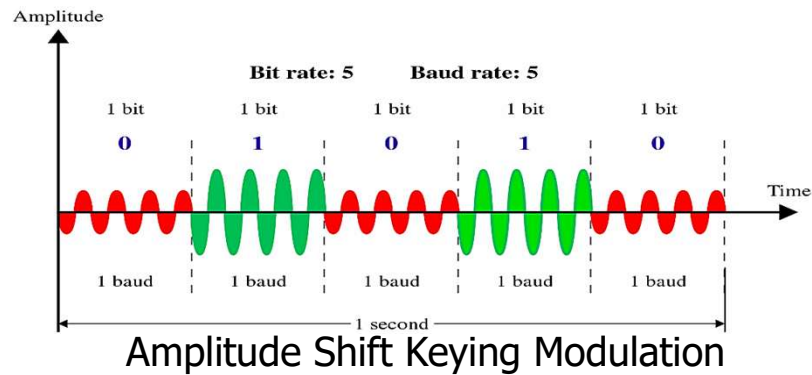


PHASE SHIFT KEYING

- The **phase** of the carrier signal is changed to represent digital symbols.
- Amplitude and frequency remain constant.



ASK, FSK, AND PSK



Notice here that each signal has two distinct symbols:

- ASK: Low Amplitude, High Amplitude
- FSK: Low Frequency, High Frequency
- PSK: 0 degrees, 180 degrees

Since we have **only two signal states** (e.g. HIGH/LOW) each symbol can carry how many bits?

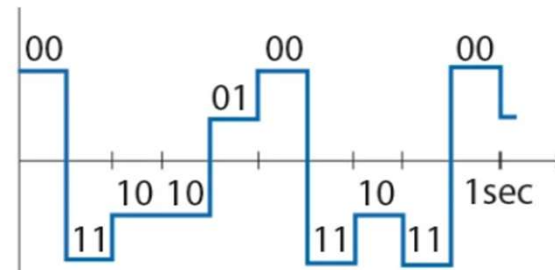
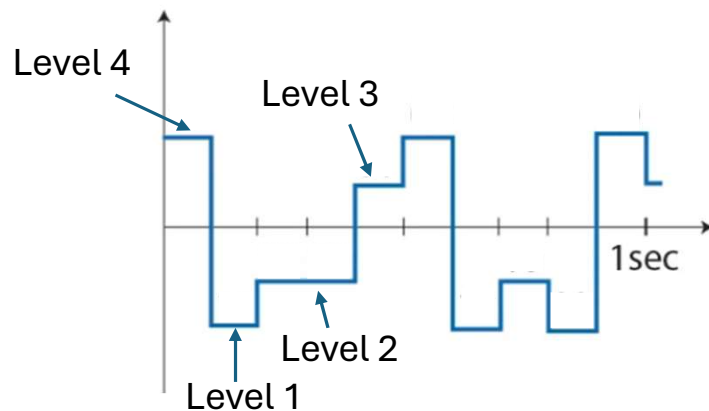
Answer: 1 bit

Multi-bit symbols

Now imagine our Hardware is capable of producing **more than two distinct symbols**. For instance let's assume that our hardware produces digital pulses with 4 available amplitude levels like in the image show below.

Question: How many bits can now each symbol carry? **Answer:** 2 bits!

In general: If signal has M distinct symbols, each symbol transmits $\log_2 M$ bits



What is the **baud rate** and **bit rate** of this signal?

Answer: Baud rate: 10 sym/s, Bit rate: 20 bps!

Synchronization

- In digital communications, the receiver must know when each bit begins and ends.
- If the sender and receiver do not stay synchronized, the receiver may sample bits at the wrong time and decode incorrect data.
- Accurate timing is essential for reliable digital communication.

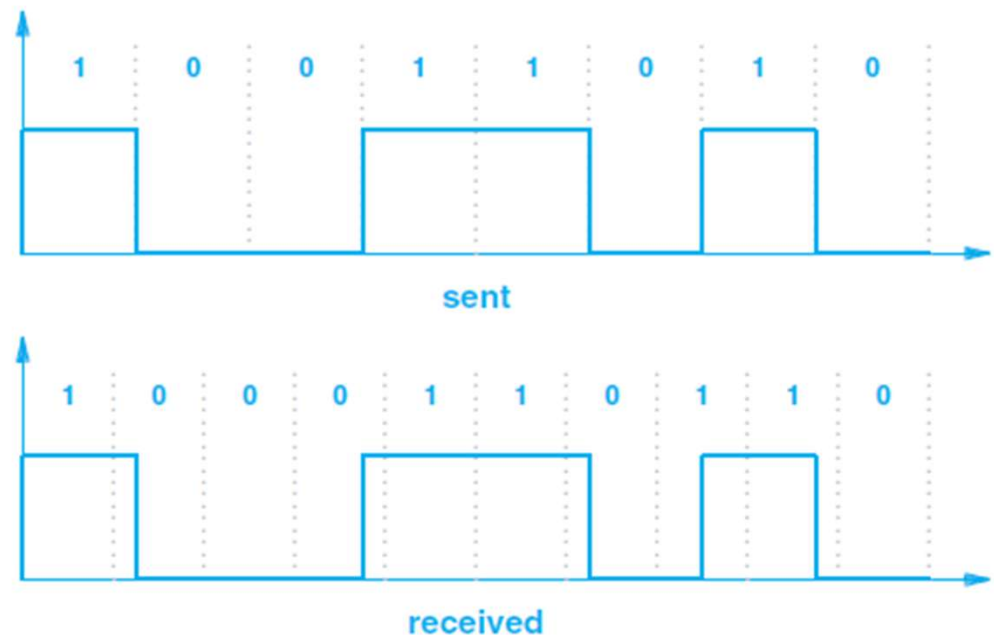


Illustration of a synchronization error in which the receiver allows slightly less time per bit than the sender (i.e. their clocks tick differently).

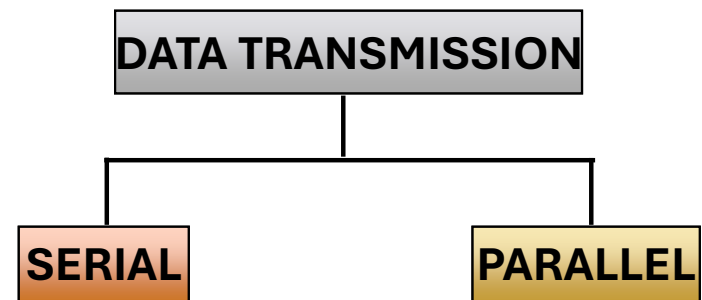
DIGITAL DATA TRANSMISSION

Digital data can be transmitted in two main ways:

- **parallel transmission**
- **serial transmission**

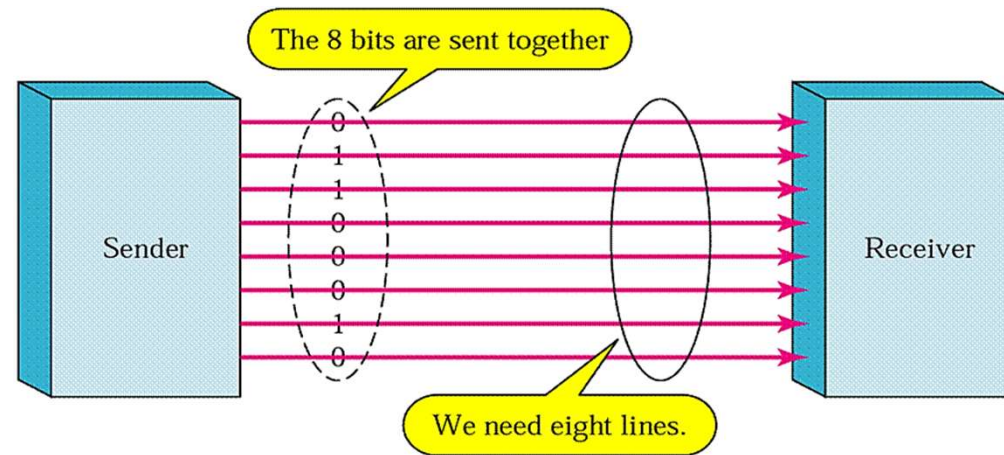
The choice depends on factors such as:

- distance
- cost
- speed
- hardware complexity



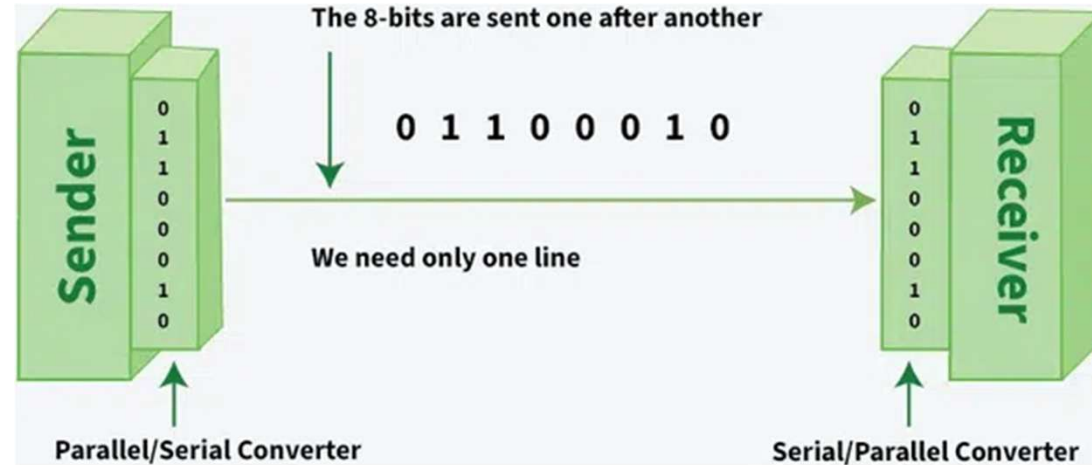
PARALLEL TRANSMISSION

- In **parallel transmission**, multiple bits are sent simultaneously.
- To send **N** bits at the same time, the system requires **N** separate signal lines.
- **Advantages:**
 - high speed
- **Disadvantages:**
 - higher cost
 - more wiring
 - greater synchronization problems over longer distances
- Parallel transmission is mainly used over short distances inside devices or systems.



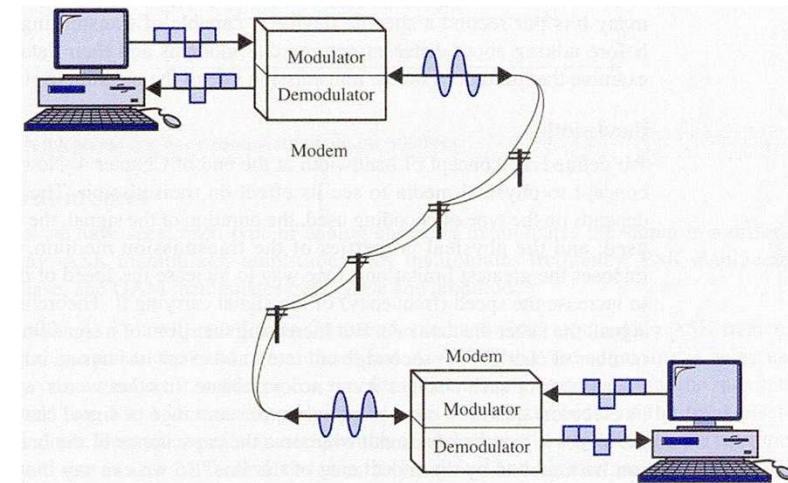
SERIAL TRANSMISSION

- In **serial transmission**, bits are sent one after another over a single communication channel.
- **Advantages:**
 - lower cost
 - less wiring
 - more reliable over longer distances
- **Disadvantages:**
 - fewer bits sent simultaneously
- Serial transmission is the dominant method in modern communication networks.



MODEMS

- A **modem** is a **modulator/demodulator**.
- It converts:
 - digital data into a modulated signal for transmission
 - a received modulated signal back into digital data
- Modems were especially important for communication over **analog telephone networks** (for example, dial-up Internet).
- **More generally, modulation is widely used in many modern communication systems, including:**
 - wireless networks (Wi-Fi, 4G/5G)
 - broadband access systems (DSL, cable)
 - optical communication systems (e.g., GPON, Ethernet fiber links)
- **Note:** Optical transceivers (such as **SFP modules**) also perform modulation and demodulation of light signal



TRANSMISSION IMPAIRMENT

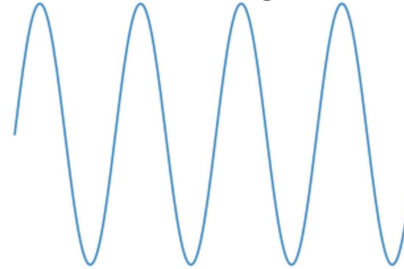
- During transmission, signals may be affected by impairments such as:

- noise
- attenuation
- distortion
- interference

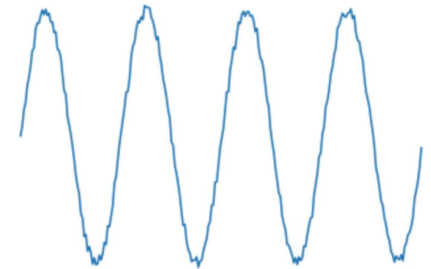
- These impairments can cause:

- lost data
- corrupted data
- additional unwanted signals
- decoding errors

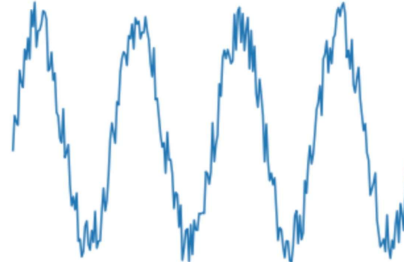
2-seconds 2 Hz signal



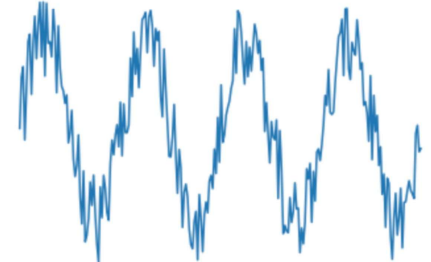
Added a little noise



Some more noise



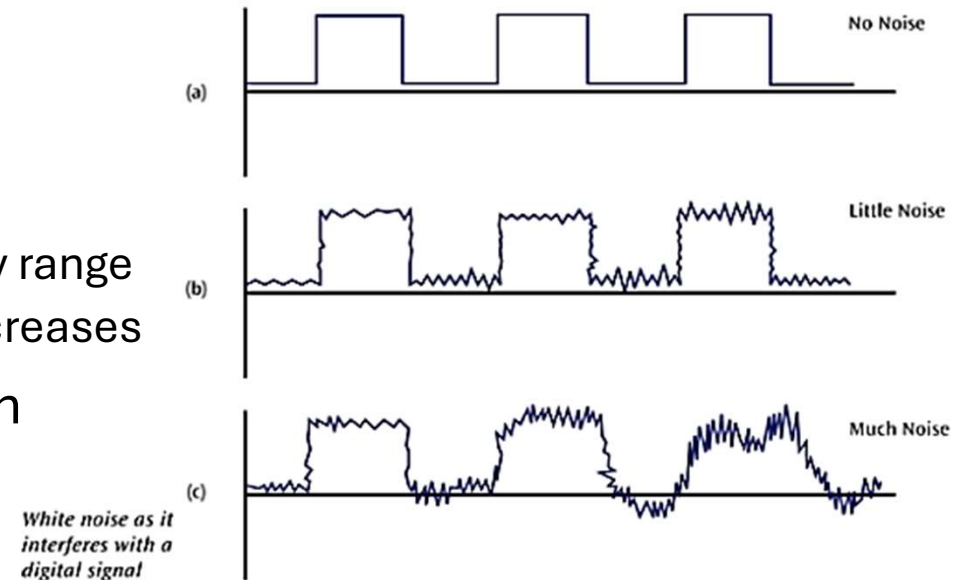
Even more noise



- Communication systems often include error detection and, in some cases, error correction mechanisms.

NOISE AND ERRORS - THERMAL

- **Thermal noise** is caused by the random motion of electrons in electronic components.
- It is also called **white noise**.
- Characteristics:
 - always present
 - approximately constant over a wide frequency range
 - becomes more harmful as signal strength decreases
- If thermal noise becomes too strong, it can disrupt communication.



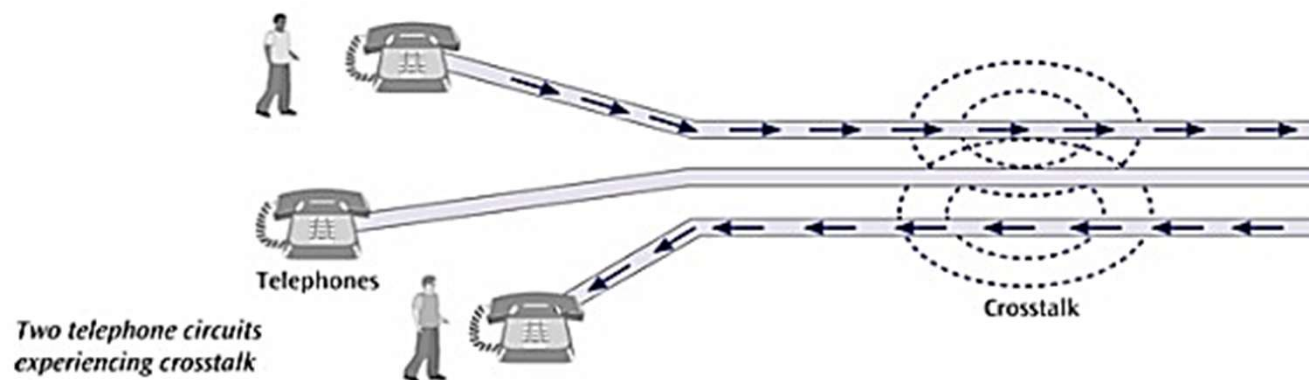
NOISE AND ERRORS - IMPULSE

- **Impulse noise** consists of sudden spikes of energy or voltage.
- Characteristics:
 - short duration
 - high intensity
 - highly disruptive
- Impulse noise can destroy one or more bits of information.
- At higher transmission speeds, a single impulse may affect more bits.



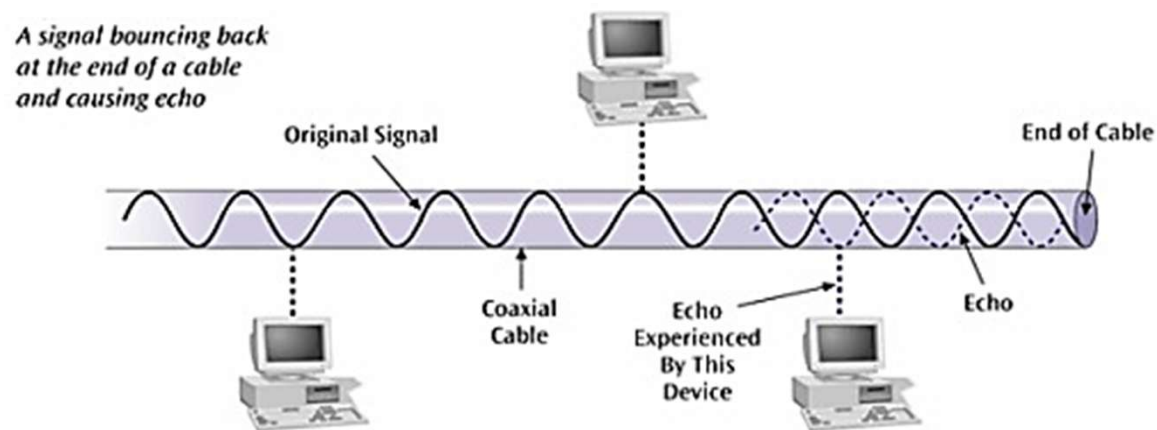
NOISE AND ERRORS - CROSSTALK

- **Crosstalk** is unwanted coupling between nearby communication channels or signal paths.
- It occurs when a signal in one circuit interferes with another circuit.
- Example:
 - hearing another conversation on a telephone line
- Crosstalk can be reduced by proper shielding, twisting, spacing, and circuit design.



NOISE AND ERRORS - ECHO

- **Echo** is the reflected return of a transmitted signal as it travels through a medium.
- If the reflection is strong enough, it can interfere with the original signal.
- Echo is often caused by imperfect line termination.





COMPUTER NETWORKS

Link Layer principles

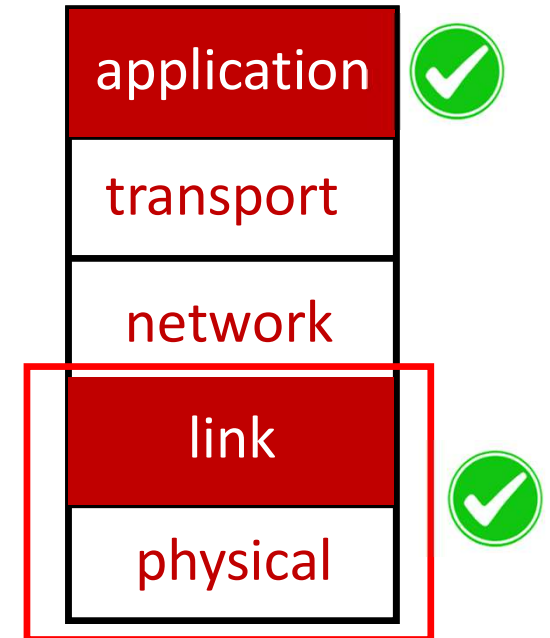
Lecturer: G. Kalfas

Notes on the slides

- The material (slides) of this course is a mixture of the following:
 1. slides provided freely by J.F Kurose and K.W. Ross accompanying the book “Computer Networking: A top-down approach” (this book is mentioned in the references of the Syllabus - slides can be found here: https://gaia.cs.umass.edu/kurose_ross/ppt.php)
 2. slides provided by CITY college and
 3. my own slides

Story so far...

- We introduced the basic concepts of networking
- We discussed about layers
- We analyzed Application Layer
- We saw the basics of signals, bandwidth, modulation and topologies
- Now we will take a deep dive into the link layer and its architecture!



Link layer: roadmap

- **introduction**
- multiple access protocols
- LANs
 - addressing, ARP
 - Ethernet
 - switches
- error detection, correction

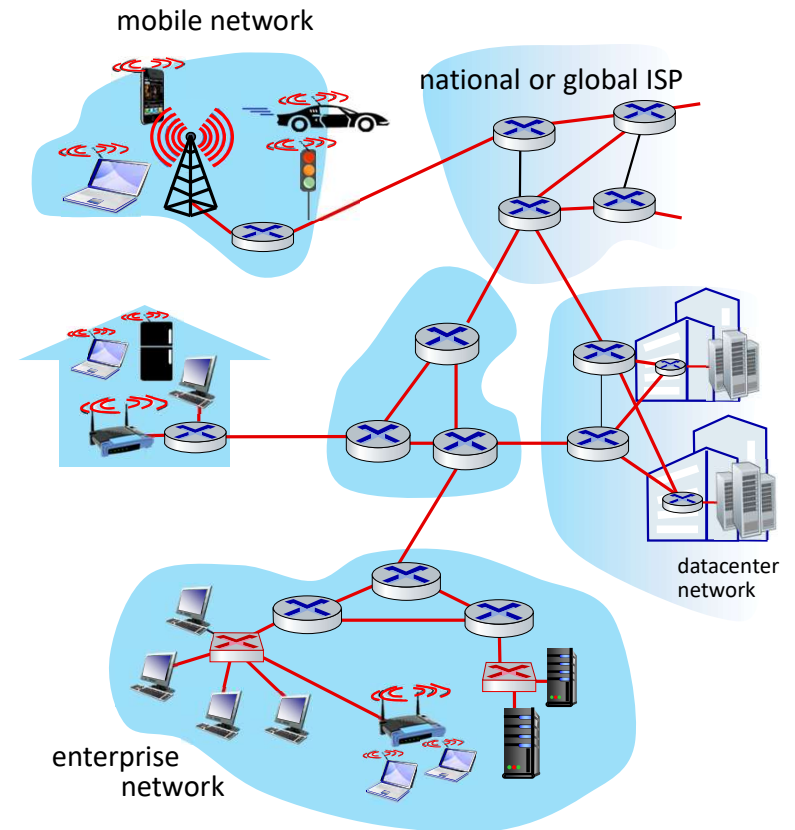


Link layer: introduction

terminology:

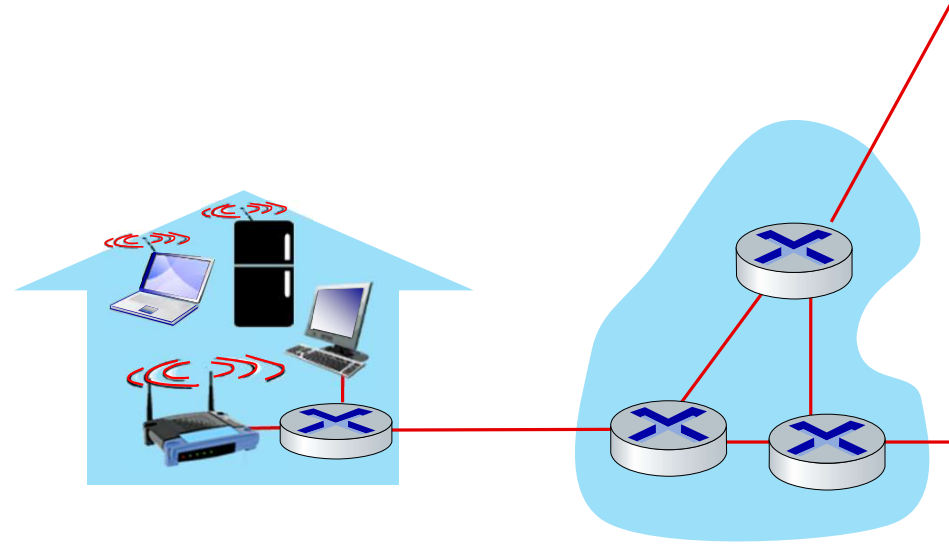
- hosts, routers: **nodes**
- communication channels that connect **adjacent** nodes along communication path: **links**
 - wired , wireless
 - LANs
- layer-2 packet: **frame**, encapsulates datagram (Layer 3 packet)

*link layer has responsibility of transferring datagram from one node to **physically adjacent** node over a link*

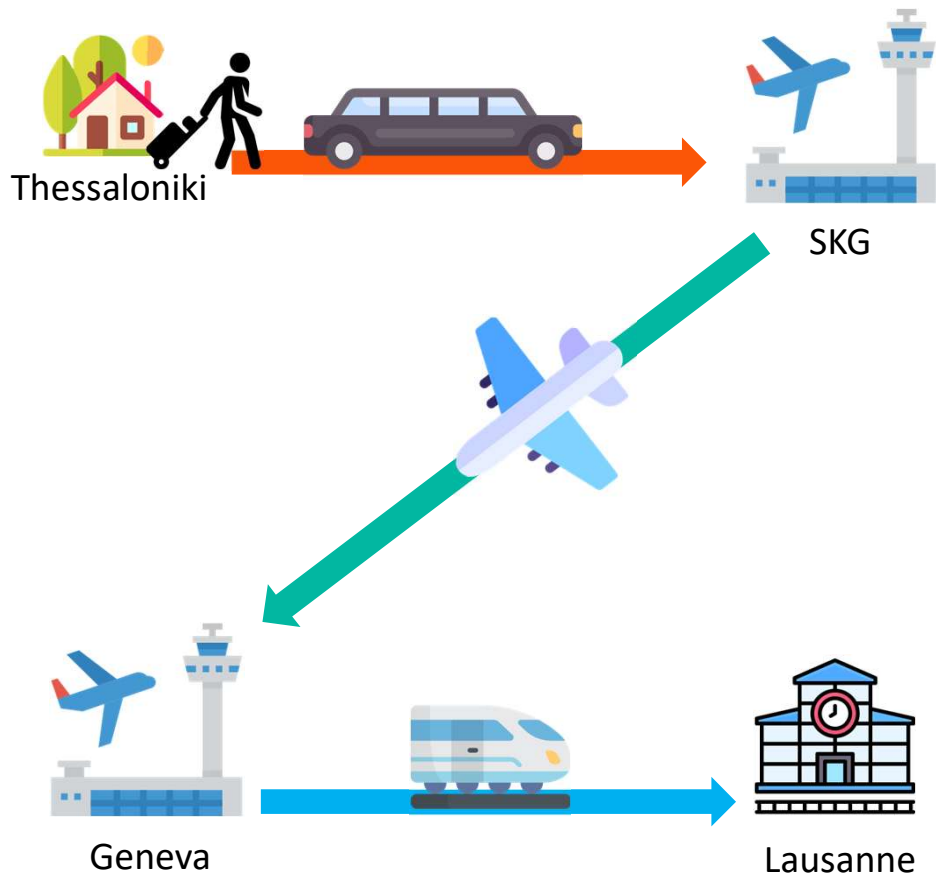


Link layer: context

- datagram transferred by **different link protocols** over different links:
 - e.g., WiFi on first link, Ethernet on next link
- each link protocol provides different services
 - e.g., **may or may not** provide reliable data transfer over link



Transportation analogy



transportation analogy:

- trip from Thessaloniki to Lausanne
 - car: Home to SKG airport
 - plane: SKG to Geneva airport
 - train: Geneva to Lausanne
- tourist = **datagram**
- transport segment = **communication link**
- transportation mode = **link-layer protocol**
- travel agent = **routing algorithm**

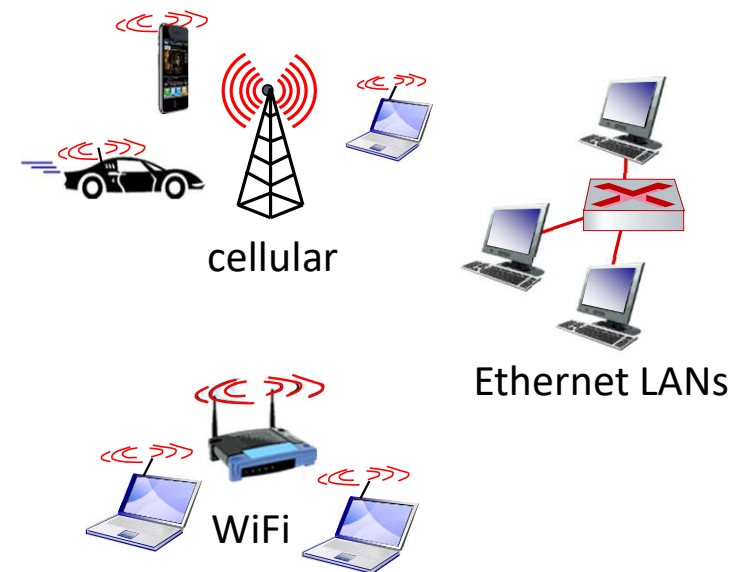
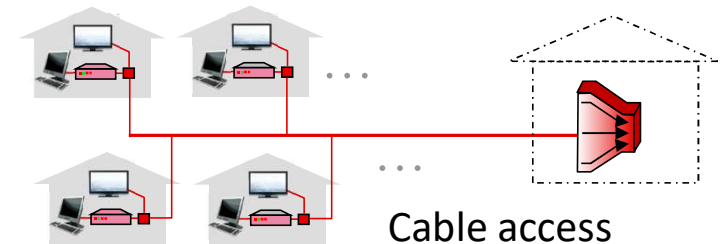
Link layer: services

■ framing, link access:

- encapsulate datagram into frame, adding header, trailer
- channel access if shared medium
- “Physical” addresses in frame headers identify source, destination

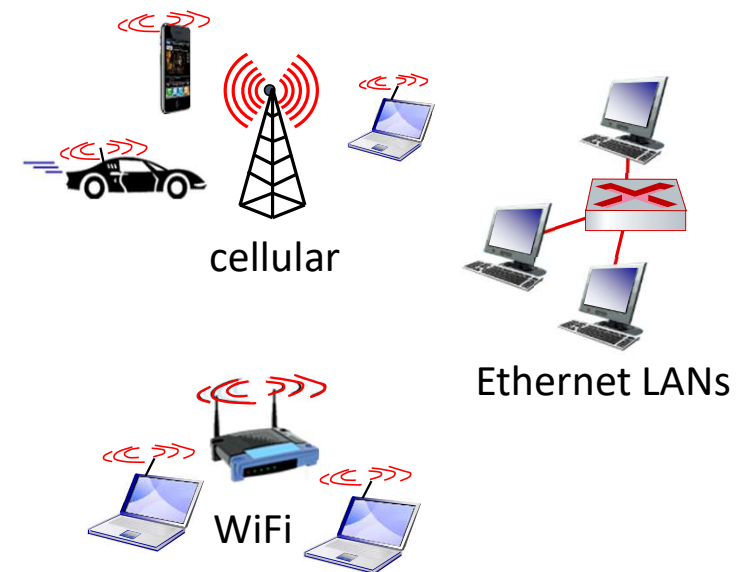
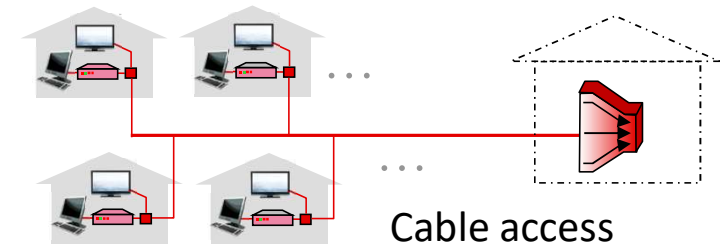
■ reliable delivery between adjacent nodes

- reliable data transfer means that if something goes wrong (e.g. packet not delivered), the protocol has some way to recover (e.g. lost packet is retransmitted by sender)
- seldom used on low bit-error links (e.g. wired)
- but used in wireless links due to high error rates



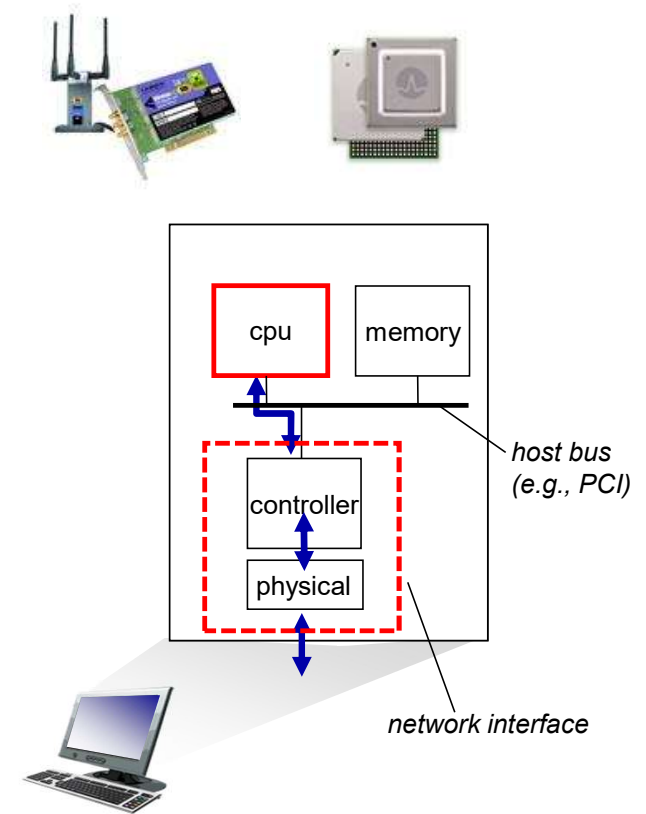
Link layer: services (more)

- **flow control:**
 - pacing between adjacent sending and receiving nodes
- **error detection:**
 - errors caused by signal attenuation, noise.
 - receiver detects errors, signals retransmission, or drops frame
- **error correction:**
 - receiver identifies *and corrects* bit error(s) without retransmission
- **half-duplex and full-duplex:**
 - with half duplex, nodes at both ends of link can transmit, but not at same time



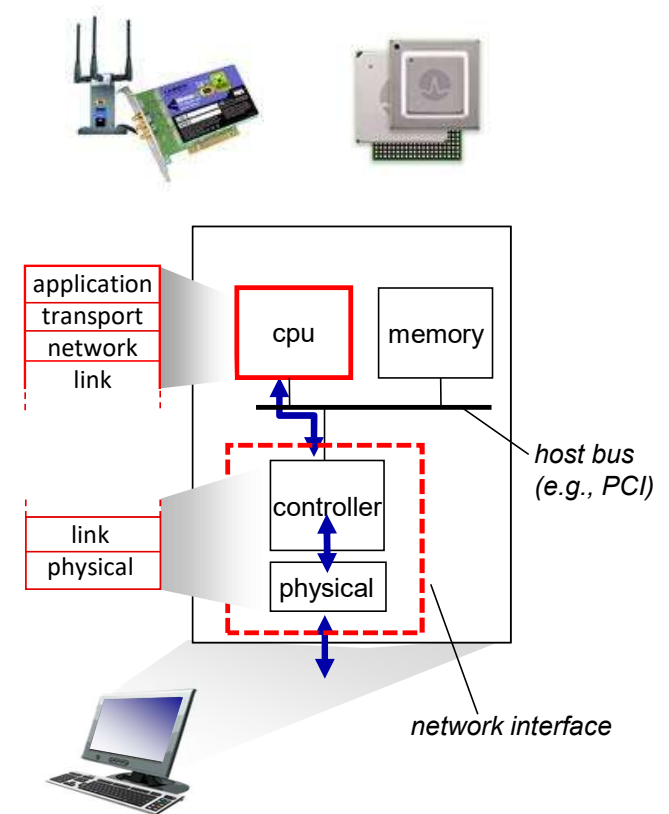
Host link-layer implementation

- in each-and-every host
- CPU can't process data at network speeds
- link layer implemented on special purpose hardware (either on-chip or in network interface card (NIC))
 - implements link, physical layer
- attaches into host's system buses
- NIC is built for one kind of physical/link network
 - e.g. Ethernet interface can't be used for WiFi

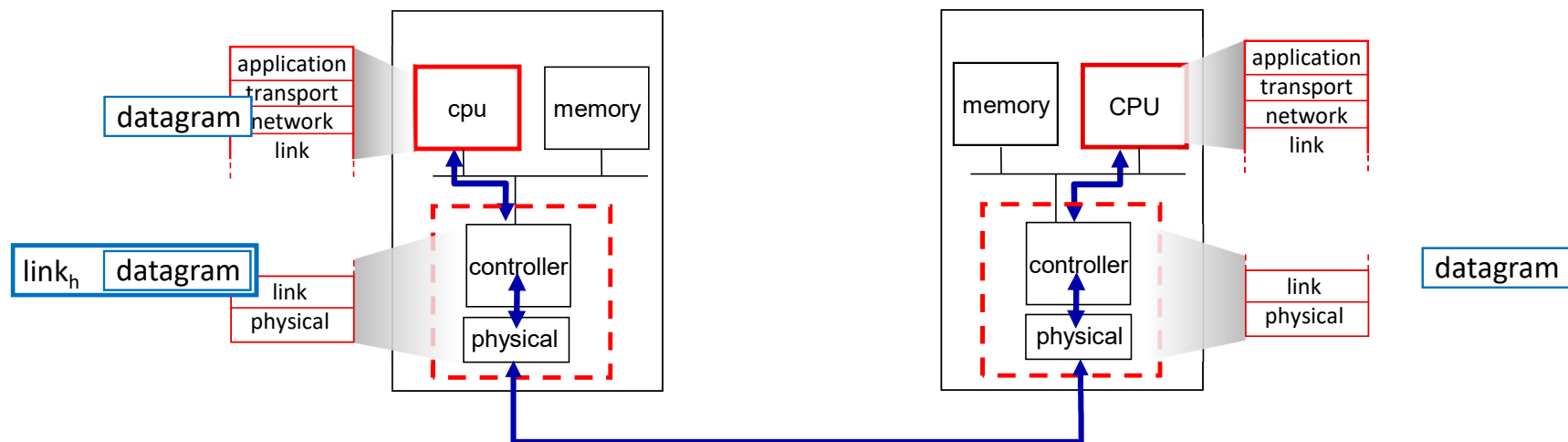


Host link-layer implementation

- Link layer is implemented as a combination of hardware, software, firmware
- Firmware is code that runs **inside the Network Interface Card (NIC)** on a small embedded processor.
 - Stored in **flash memory** on the NIC
 - Runs **below the operating system**
 - Controls **hardware behavior directly**
 - Can be updated, but not as easily as normal software
 - Handles **real-time operations**



Interfaces communicating



sending side:

- encapsulates datagram in frame
- adds error checking bits, reliable data transfer, flow control, etc.

receiving side:

- looks for errors, reliable data transfer, flow control, etc.
- extracts datagram, passes to upper layer at receiving side

Link layer, LANs: roadmap

- introduction
- **multiple access protocols**
- LANs
 - addressing, ARP
 - Ethernet
 - switches
- error detection, correction



Multiple access links, protocols

Reminder: two types of “links”

- point-to-point
 - point-to-point link between Ethernet switch, host
 - PPP for dial-up access
- **Point-to-multipoint (shared wire or wireless medium)**
 - Also called broadcast links
 - old-school Ethernet
 - upstream GPON/XGS-PON in Fiber-To-The-Home (FTTH) access networks
 - 802.11 wireless LAN (WiFi), 4G/5G, satellite



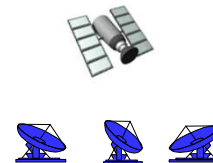
shared wire (e.g.,
cabled Ethernet)



shared radio: 4G/5G



shared radio: WiFi



shared radio: satellite



humans at a cocktail party
(shared air, acoustical)

Multiple access protocols

- single shared broadcast channel
- two or more simultaneous transmissions by nodes: interference
 - *collision* if node receives two or more signals at the same time

multiple access protocol

- distributed algorithm that determines how nodes share channel, i.e., determine when node can transmit
- communication about channel sharing must use channel itself!
 - no out-of-band channel for coordination

An ideal multiple access protocol

given: multiple access channel (MAC) of rate R bps

Desired Properties:

1. when one node wants to transmit, it can send at rate R .
2. when M nodes want to transmit, each can send at average rate R/M
3. fully decentralized:
 - no special node to coordinate transmissions (to avoid single points of failure and scalability problems when number of nodes too large)
 - no synchronization of clocks, slots (maintaining synch across multiple entities is still a challenging issue).
4. simple

MAC protocols: taxonomy

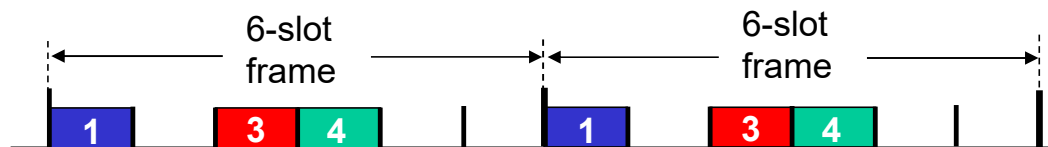
three broad classes:

- **channel partitioning**
 - divide channel into smaller “pieces” (time slots, frequency, code)
 - allocate piece to node for exclusive use
- **random access**
 - channel not divided, allow collisions
 - “recover” from collisions
- **“taking turns”**
 - nodes take turns, but nodes with more to send can take longer turns

Channel partitioning MAC protocols: TDMA

TDMA: time division multiple access

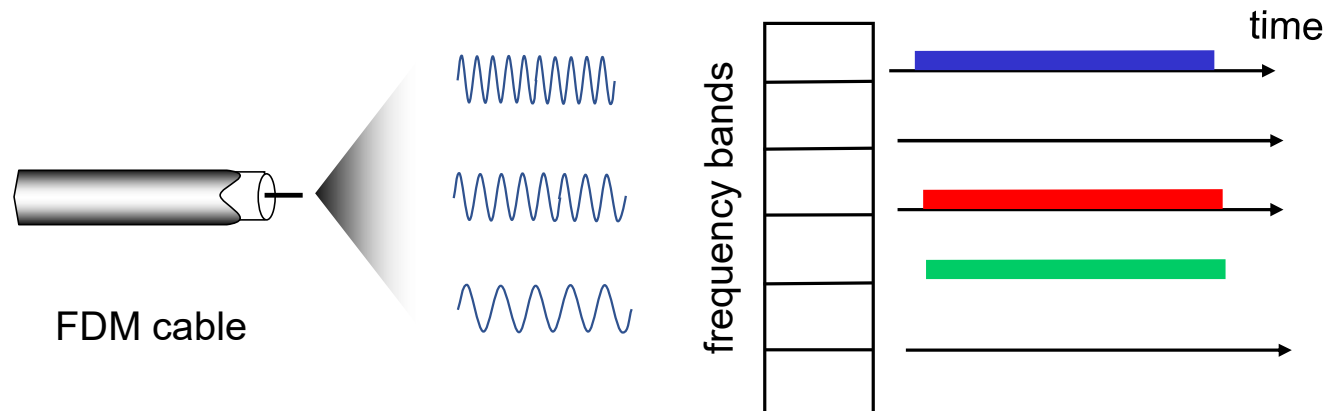
- access to channel in “rounds”
- each station gets fixed length slot (length = packet transmission time) in each round
- unused slots go idle
- example: 6-station LAN, 1,3,4 have packets to send, slots 2,5,6 idle



Channel partitioning MAC protocols: FDMA

FDMA: frequency division multiple access

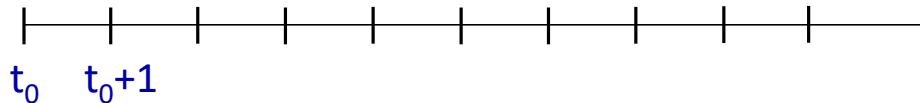
- channel spectrum divided into frequency bands
- each station assigned fixed frequency band
- unused transmission time in frequency bands go idle
- example: 6-station LAN, 1,3,4 have packet to send, frequency bands 2,5,6 idle



Random access protocols

- when node has packet to send
 - transmit at full channel data rate R
 - no *a priori* coordination among nodes
- two or more transmitting nodes:
“collision”
- **random access protocol** specifies:
 - how to detect collisions
 - how to recover from collisions (e.g., via delayed retransmissions)
- examples of random access MAC protocols:
 - ALOHA, slotted ALOHA
 - CSMA, CSMA/CD

Slotted ALOHA



assumptions:

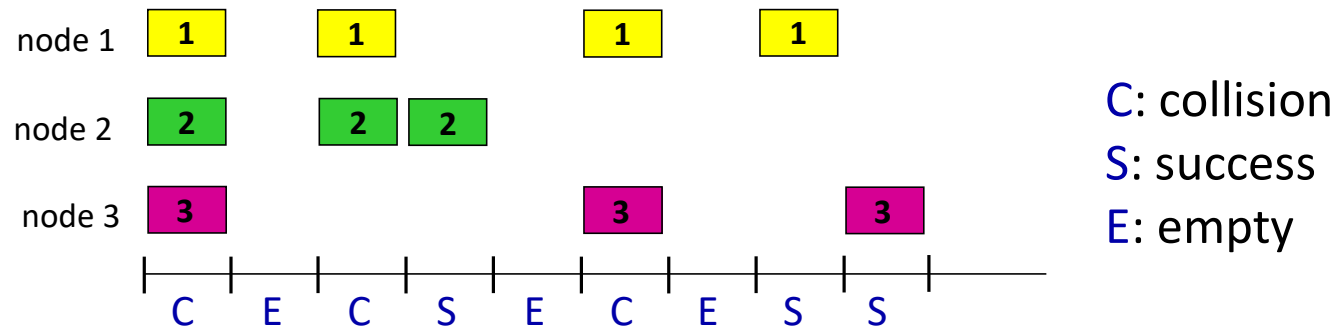
- all frames same size
- time divided into equal size slots (time to transmit 1 frame)
- nodes start to transmit only at slot beginning
- nodes are synchronized
- if 2 or more nodes transmit in slot, all nodes detect collision

operation:

- when node obtains fresh frame, transmits in next slot
 - *if no collision*: node can send new frame in next slot
 - *if collision*: node retransmits frame in each subsequent slot with probability p until success

randomization – why?

Slotted ALOHA



Pros:

- single active node can continuously transmit at full rate of channel
- highly decentralized: only slots in nodes need to be in sync
- simple

Cons:

- collisions, wasting slots
- idle slots
- nodes may be able to detect collision in less than time to transmit packet
- clock synchronization

Slotted ALOHA: efficiency

efficiency: long-run fraction of successful slots (many nodes, all with many frames to send)

- *suppose:* N nodes with many frames to send, each transmits in slot with probability p
 - prob that given node has success in a slot $= p(1-p)^{N-1}$
 - prob that *any* node has a success $= Np(1-p)^{N-1}$
 - max efficiency: find p^* that maximizes $Np(1-p)^{N-1}$
 - for many nodes, take limit of $Np^*(1-p^*)^{N-1}$ as N goes to infinity, gives:

max efficiency $= 1/e = .37$

- *at best:* channel used for useful transmissions 37% of time!



CSMA (carrier sense multiple access)

simple **CSMA**: listen before transmit:

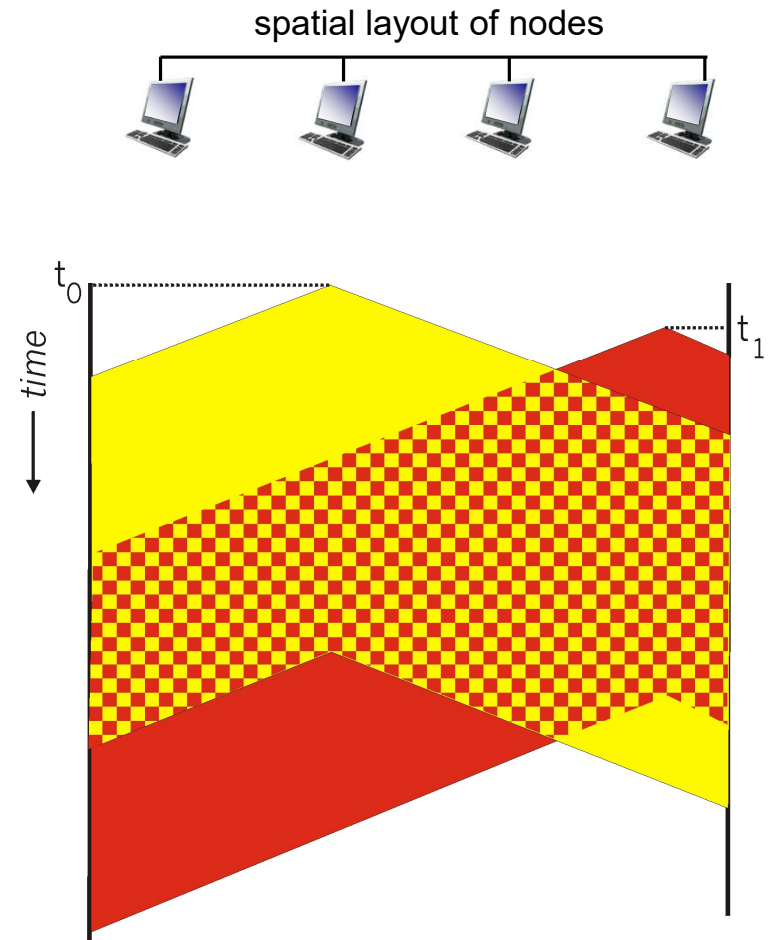
- if channel sensed idle: transmit entire frame
- if channel sensed busy: defer transmission
- human analogy: don't interrupt others!

CSMA/CD: CSMA with *collision detection*

- nodes can sense when collisions are happening
- collisions *detected* within short time
- colliding transmissions aborted, reducing channel wastage
- collision detection easy in wired, difficult with wireless
- human analogy: the polite conversationalist

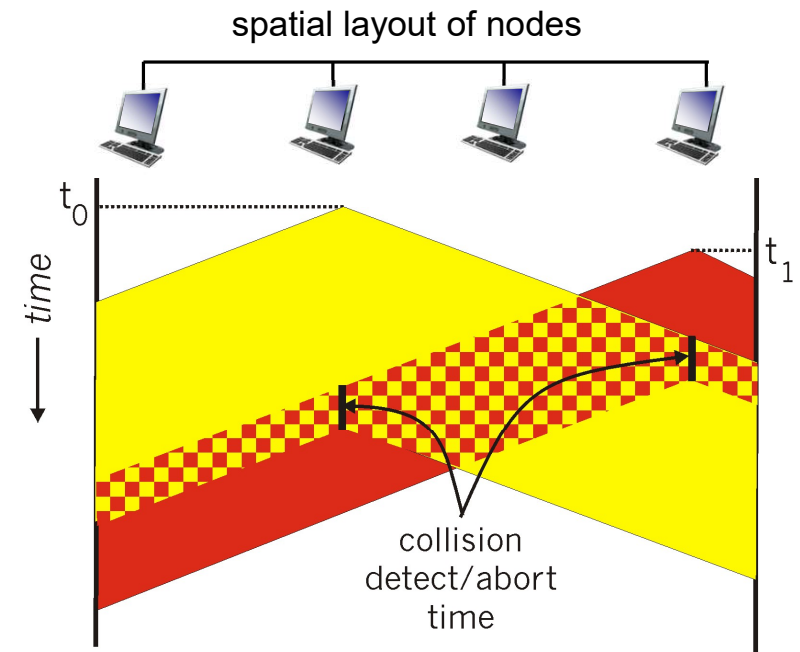
CSMA: collisions

- collisions can *still* occur with carrier sensing:
 - **propagation delay** means two nodes may not hear each other's just-started transmission
- **collision**: entire packet transmission time wasted
 - distance & propagation delay play role in determining collision probability



CSMA/CD:

- CSMA/CD reduces the amount of time wasted in collisions
 - transmission aborted on collision detection



Ethernet CSMA/CD algorithm

1. Ethernet receives datagram from network layer, creates frame
2. Ethernet senses channel:
 - if **idle**: start frame transmission.
 - if **busy**: wait until channel idle, then transmit
3. If entire frame transmitted without collision - done!
4. If another transmission detected while sending: abort, send jam signal
5. After aborting, enter *binary (exponential) backoff*:
 - after m th collision, node chooses K at random from $\{0, 1, 2, \dots, 2^m - 1\}$. Ethernet waits $K \times 512$ bit times, returns to Step 2
 - more collisions: longer backoff interval

“Taking turns” MAC protocols

channel partitioning MAC protocols:

- share channel *efficiently* and *fairly* at high load
- inefficient at low load: delay in channel access, $1/N$ bandwidth allocated even if only 1 active node!

random access MAC protocols

- efficient at low load: single node can fully utilize channel
- high load: collision overhead

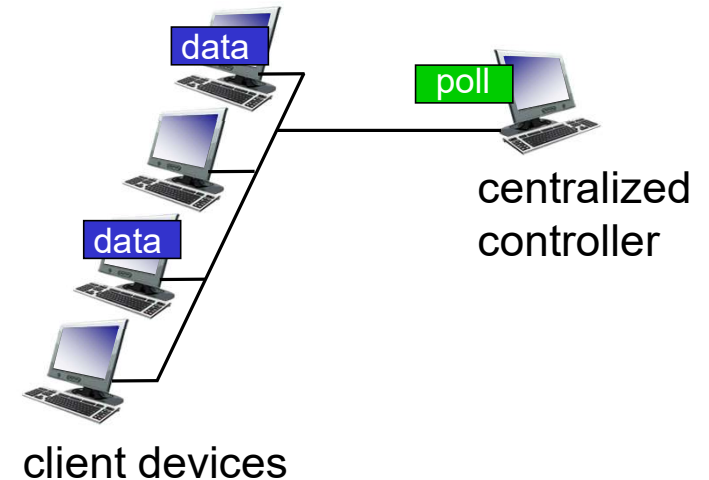
“taking turns” protocols

- look for best of both worlds!

“Taking turns” MAC protocols

polling:

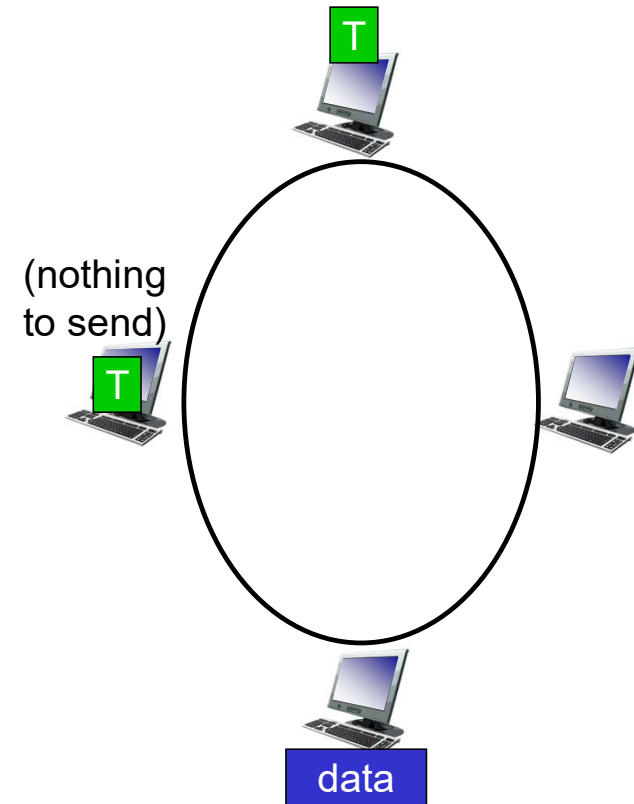
- centralized controller “invites” other nodes to transmit in turn
- typically used with “dumb” devices
- concerns:
 - polling overhead
 - latency
 - single point of failure (centralized controller)
- Bluetooth uses polling



“Taking turns” MAC protocols

token passing:

- control *token* (special reserved message) explicitly passed from one node to next, sequentially
 - Nodes can transmit ONLY while holding token
 - hardware ensures that exactly one token exists (if it is lost it is regenerated).
 - Intermediate stations forward frame. Destination makes a copy of frame and puts it back in the ring
 - Frame reaches sender to check for errors and releases token
- concerns:
 - token overhead/latency/single point of failure (token). Hardware must be designed to pass token even if attached PC is powered down



Summary of MAC protocols

- **channel partitioning**, by time, frequency or code
 - Time Division, Frequency Division
- **random access** (dynamic),
 - ALOHA, S-ALOHA, CSMA, CSMA/CD
 - carrier sensing: easy in some technologies (wire), hard in others (wireless)
 - CSMA/CD used in Ethernet
- **taking turns**
 - polling from central site, token passing
 - Bluetooth, Fiber Distributed Data Interconnect (FDDI), IBM token ring

Summary of MAC protocols

MAC protocol comparison table

Protocol	Collision Possible?	Efficiency	Used Today In
TDMA	NO	Medium	Cellular
FDMA	NO	Medium	Cellular
ALOHA	YES	Low	Historical
CSMA/CD	YES	High	Ethernet (legacy)

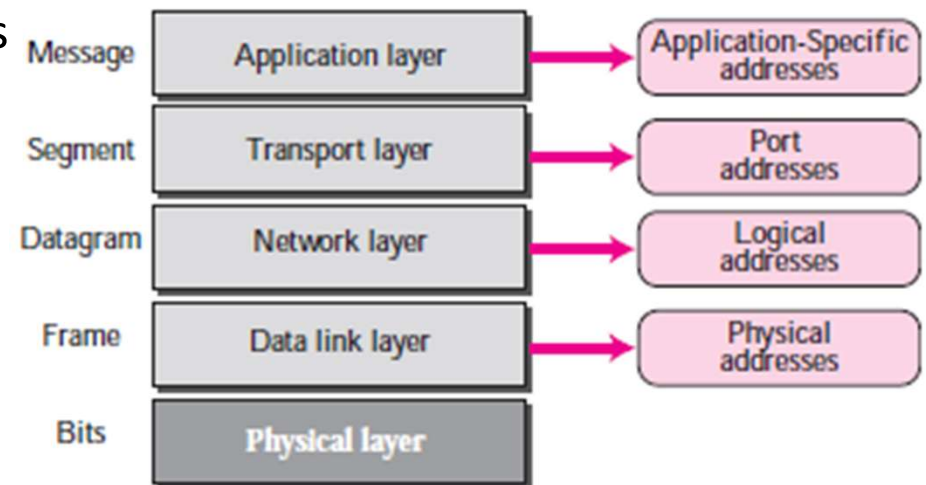
Link layer, LANs: roadmap

- introduction
- multiple access protocols
- LANs
 - addressing, ARP
 - Ethernet
 - switches
- error detection, correction



Address levels

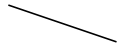
- Since each layer transmits packets from a sender to a receiver, each layer needs some type of address! PHY layer is an exception because it **does not use addresses** - it simply transmits raw bits over the medium. The medium itself “guides” the bits to the receiver.
- Each layer has its own address:
 - **Data Link:** physical address (Or MAC address address)
 - **Network:** logical address (Or IP address)
 - **Transport:** port address (Or port number)
 - **Application:** application-specific address



Physical (or MAC or Eth or LAN) addresses

- Address used in Layer 2
 - Function of address:
 - used “locally” to get frame from one interface to another physically-connected interface
 - 48-bit MAC address (for most LANs) burned in NIC ROM by manufacturer, also sometimes software settable (so it is tied up to specific NIC HW, hence the name “Physical”).
 - Like the Social Security Number, i.e. permanent
 - Switches use this address to forward frames to the correct port (more later...)
 - Must be unique on LAN
 - e.g.: 1A-2F-BB-76-09-AD
- hexadecimal (base 16) notation
(each “numeral” represents 4 bits)*

Logical (or IP) addresses

- Address used in Layer 3
- Function:
 - Designed for Communication over the Internet, i.e. a logical address uniquely defines a host connected to the Internet.
 - No two publicly reachable hosts on the Internet should have the same public IP address at the same time
- Independent of underlying Layer 2 technology. Depends on the network we are located (i.e. Nodes have different IP addresses at different LANs, hence the name “Logical”). So it is like the Postal address (i.e. temporary, depends on where you are at each moment)
- A logical address on the Internet is currently 32 bits long (IPv4 address) or 128 bits (IPv6 address).
- e.g. IPv4: 128.119.40.136
 *decimal notation, each of the 4 decimal numbers represents 8 bits*

Port Addresses

- Computers run **multiple processes** at the same time.
- The end objective of the Internet communication is a **process communicating** with another process.
- For the processes to receive data simultaneously, we need a **method to label** the different processes.
- In the 5-layer Internet model, the label assigned to a process is called a **port address or number**, and it is **16 bits** long
- e.g. Port 80 for HTTP
 - decimal notation, this number represents 16 bits (i.e. valid values are between 0 and 65535)*

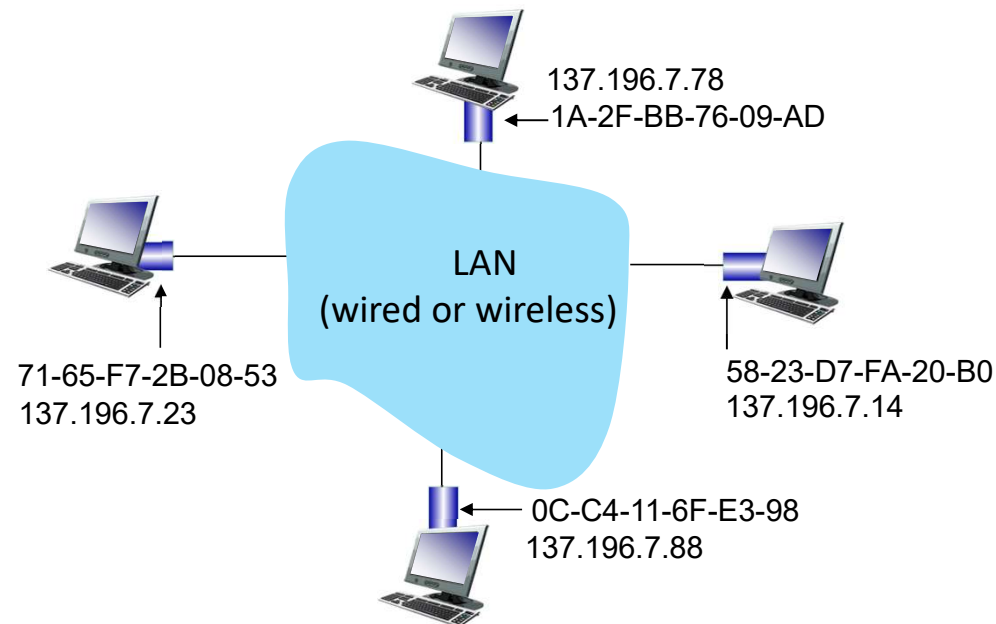
Application-Specific addresses

- Communicating applications need to be provided with the name of the receiving end of the service. Most of them have user-friendly addresses.
- Examples:
 - e-mail address: gkalfas@yorkeuropecampus.eu
 - Browsing the web (URL): <https://city.yorkeuropecampus.eu/>
 - WhatsApp/Viber etc: The telephone number of your friend.
 - Teams: The user name of the user you are trying to reach
- These addresses get changed to the corresponding port and logical addresses by the sending computer.

MAC/IP addresses

each interface on LAN

- has unique 48-bit **MAC** address
- has a locally unique 32-bit IP address



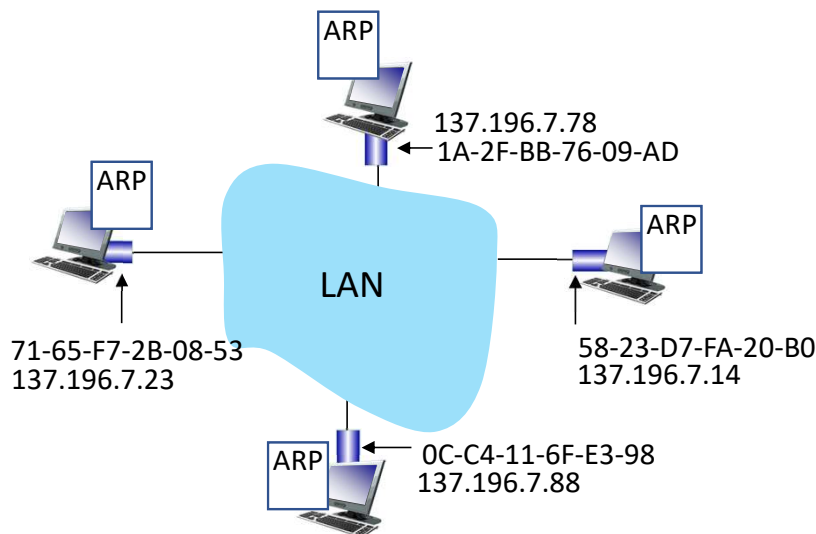
ARP: address resolution protocol

Question: how to determine interface's MAC address, knowing its IP address? (usually IP address is known to the network app users – how?)

Answer: With the use of the ARP protocol!

ARP table: each node (host, router) on LAN has table

- IP/MAC address mappings for some LAN nodes:
< IP address; MAC address; TTL >
- TTL (Time To Live): time after which address mapping will be forgotten (typically 20 min)



ARP protocol in action

example: A wants to send datagram to B

- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address

A broadcasts ARP query, containing B's IP addr

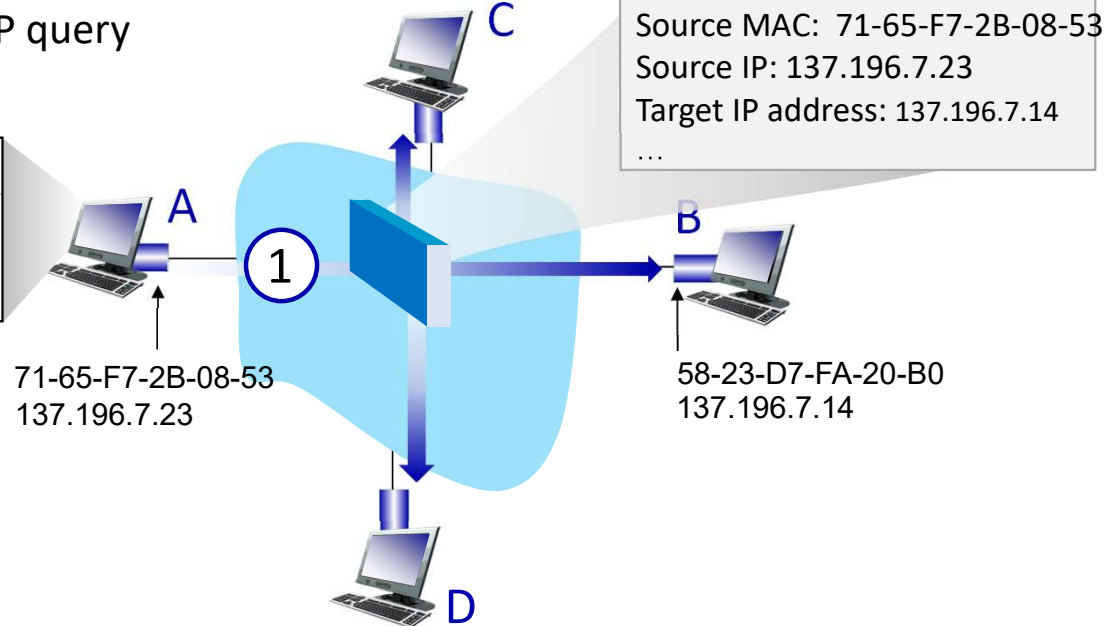
Special broadcast address, meaning "All nodes of LAN should accept this packet"

- ①
- destination MAC address = FF-FF-FF-FF-FF-FF
 - all nodes on LAN receive ARP query

Ethernet frame (sent to FF-FF-FF-FF-FF-FF)

ARP table in A

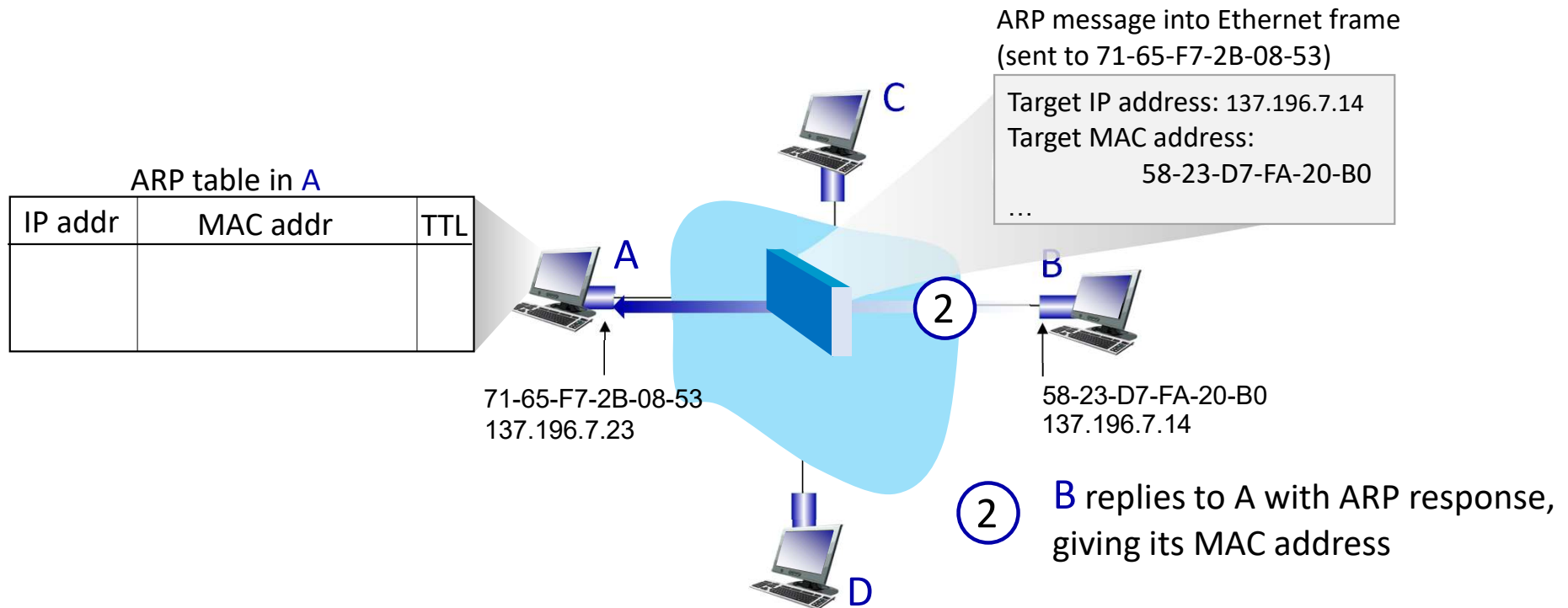
IP addr	MAC addr	TTL



ARP protocol in action

example: A wants to send datagram to B

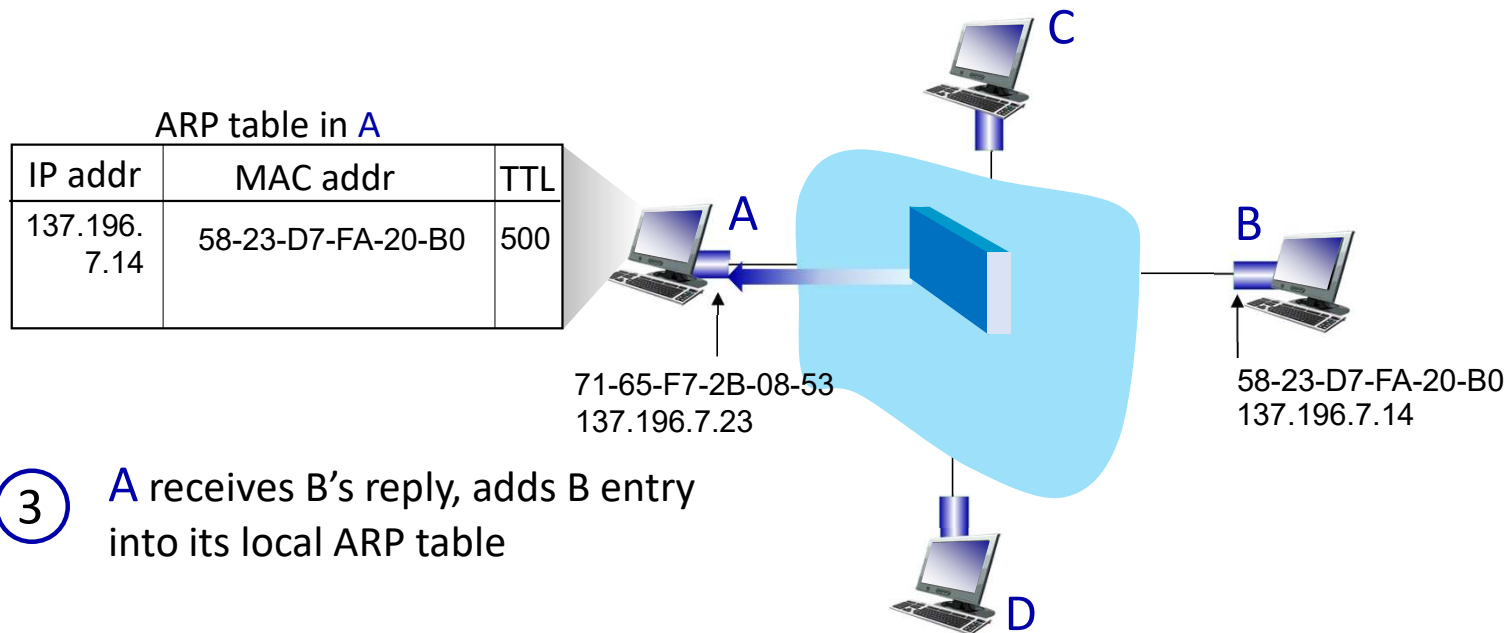
- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address



ARP protocol in action

example: A wants to send datagram to B

- B's MAC address not in A's ARP table, so A uses ARP to find B's MAC address



Link layer, LANs: roadmap

- introduction
- multiple access protocols
- **LANs**
 - addressing, ARP
 - **Ethernet**
 - switches
- error detection, correction



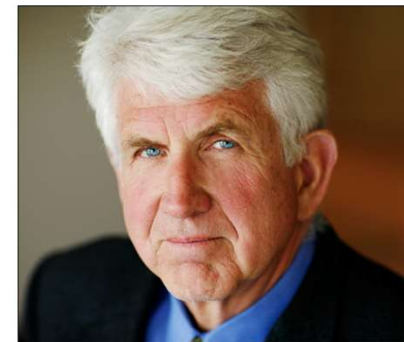
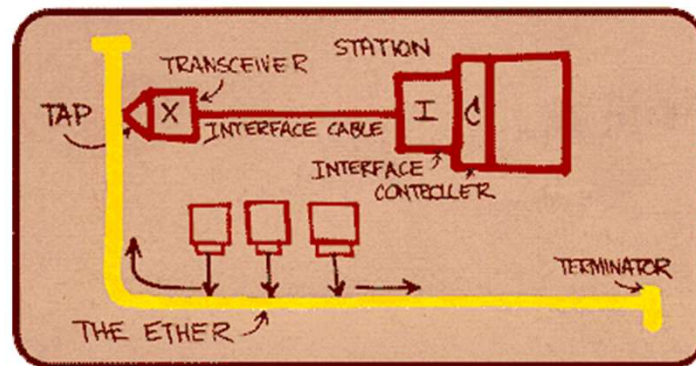
Ethernet

“dominant” wired LAN technology:

- first widely used LAN technology
- simpler, cheap
- kept up with speed race: 10 Mbps – 800 Gbps
- single chip, multiple speeds (e.g., Broadcom BCM5761)

Bob Metcalfe: Ethernet co-inventor,
2022 ACM Turing Award recipient

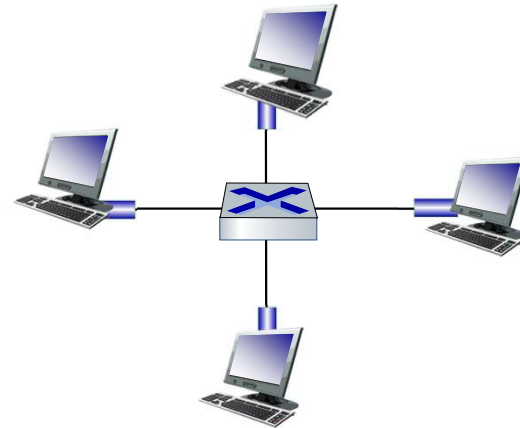
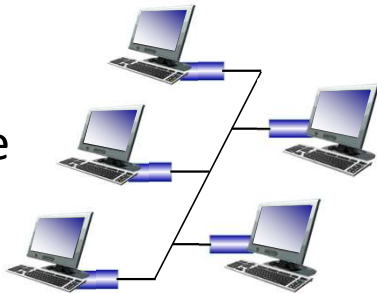
Metcalfe's Ethernet sketch



Ethernet: physical topology

- **bus:** popular through mid 90s
 - all nodes in same collision domain (can collide with each other)
- **switched:** prevails today
 - active link-layer 2 *switch* in center
 - each “spoke” runs a (separate) Ethernet protocol (nodes do not collide with each other)

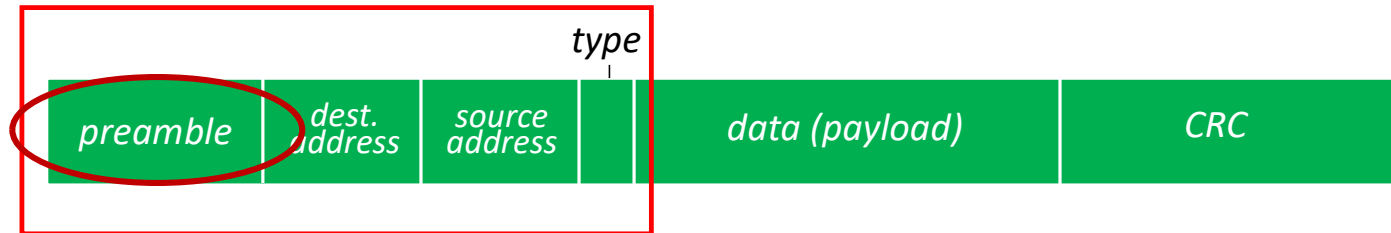
bus: coaxial cable



switched

Ethernet frame structure

sending interface encapsulates IP datagram (or other network layer protocol packet) in **Ethernet frame**

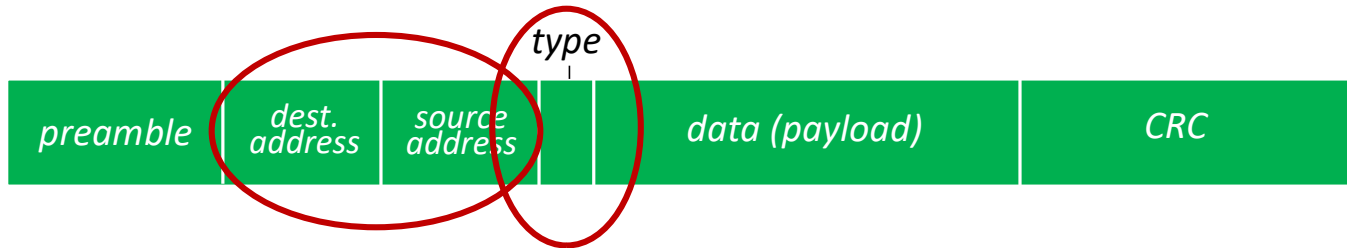


Packet Header contains the following information:

preamble:

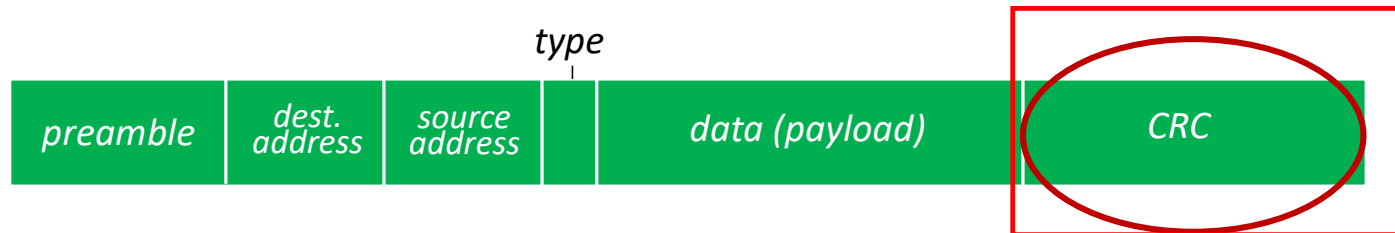
- used to synchronize receiver, sender clock rates
- 7 bytes of 10101010 followed by one byte of 10101011

Ethernet frame structure (more)



- **addresses:** 6 byte source, destination MAC addresses
 - if adapter receives frame with matching destination address, or with broadcast address (e.g., ARP packet), it passes data in frame to network layer protocol
 - otherwise, adapter discards frame
- **type:** indicates higher layer protocol
 - mostly IP but others possible, e.g., Novell IPX, AppleTalk
 - used to demultiplex up at receiver

Ethernet frame structure (more)



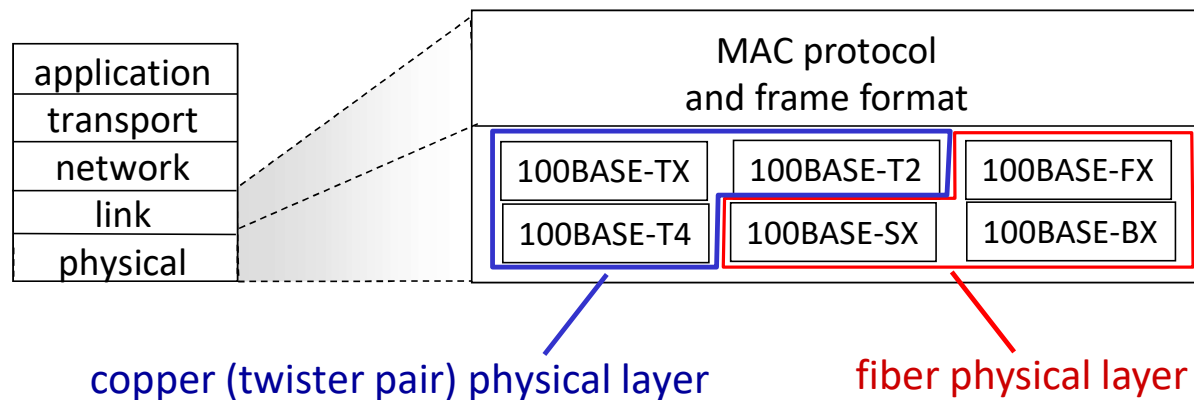
- **Trailers:** In networking (especially the OSI model and Internet model), all layers add headers, but **the Link Layer also adds a trailer!** This is the **only** layer that **commonly** uses trailers. A trailer in a packet typically carries error control information, mainly for error detection and/or error correction
- Example technologies that use trailers:
 - Ethernet, PPP (Point-to-Point Protocol)
- **CRC:** this is an error detection mechanism (cyclic redundancy check) performed at receiver (more on that later)
 - error detected: frame is dropped

Ethernet: unreliable, connectionless

- **connectionless**: no handshaking between sending and receiving NICs
- **unreliable**: receiving NIC doesn't acknowledge that it has received the packet (or not) to sending NIC
 - data in dropped frames recovered only if initial sender uses higher layer reliability mechanisms (e.g., TCP), otherwise dropped data lost
- Ethernet's MAC protocol: unslotted **CSMA/CD with binary backoff**

802.3 Ethernet standards: link & physical layers

- *many* different Ethernet standards
 - common MAC protocol and frame format
 - different speeds: 2 Mbps, ... 100 Mbps, 1Gbps, 10 Gbps, 40 Gbps, 80 Gbps
 - different physical layer media: fiber, cable



Link layer, LANs: roadmap

- introduction
- multiple access protocols
- **LANs**
 - addressing, ARP
 - Ethernet
 - **switches**
- error detection, correction



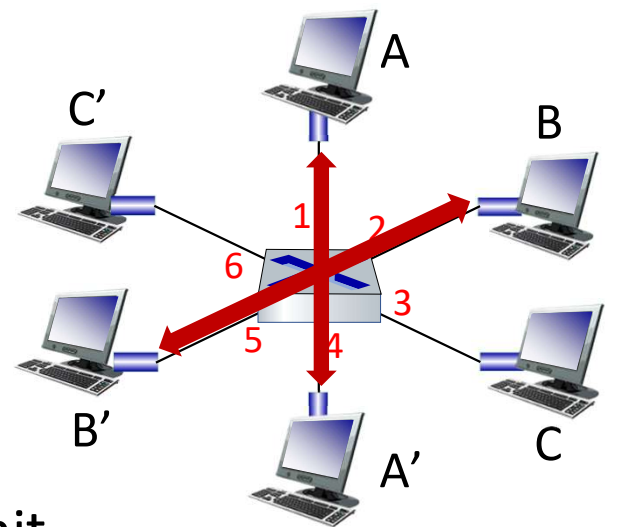
Ethernet switch

- Switch is a **link-layer** device (i.e. it implements Layers 1 and 2, its main job is to forward packets between adjacent nodes): takes an *active* role
 - store, forward Ethernet (or other type of) frames
 - examine incoming frame's MAC address, *selectively* forward (this is the *active* role part) frame to one-or-more outgoing links when frame is to be forwarded on segment, uses CSMA/CD to access segment*
- **transparent**: hosts *unaware* of presence of switches
- It is essentially the device that enables the star topology
- **plug-and-play, self-learning**
 - switches do not need to be configured

*Historically Ethernet used CSMA/CD. Modern switched Ethernet uses **full-duplex communication**, so collisions **do not** occur.

Switch: multiple simultaneous transmissions

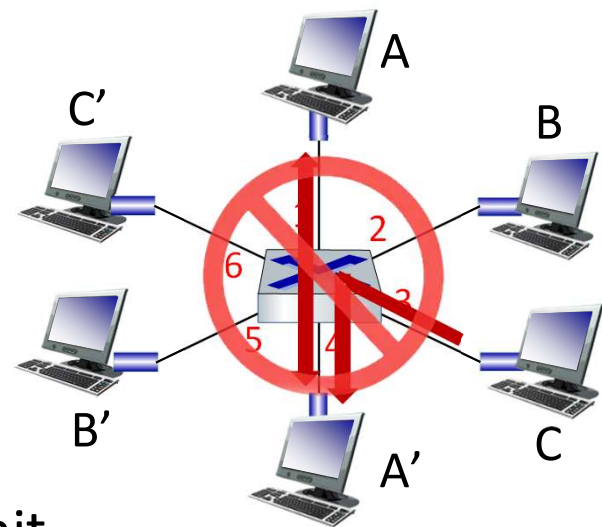
- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on *each* incoming link, so:
 - no collisions; full duplex
 - each link is its own collision domain
- **switching:** A-to-A' and B-to-B' can transmit simultaneously, without collisions



switch with six
interfaces (1,2,3,4,5,6)

Switch: multiple simultaneous transmissions

- hosts have dedicated, direct connection to switch
- switches buffer packets
- Ethernet protocol used on *each* incoming link, so:
 - no collisions; full duplex
 - each link is its own collision domain
- **switching:** A-to-A' and B-to-B' can transmit simultaneously, without collisions
 - but A-to-A' and C to A' **cannot** happen simultaneously



switch with six
interfaces (1,2,3,4,5,6)

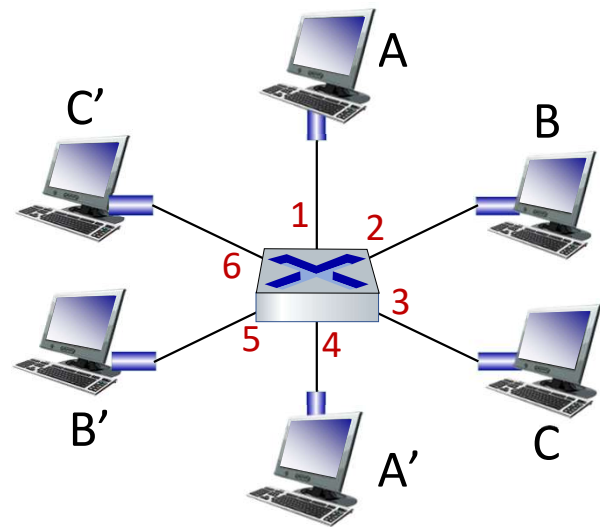
Switch forwarding table

Q: how does switch know A' reachable via interface 4, B' reachable via interface 5?

A: each switch has a **switch table**, each entry contains:

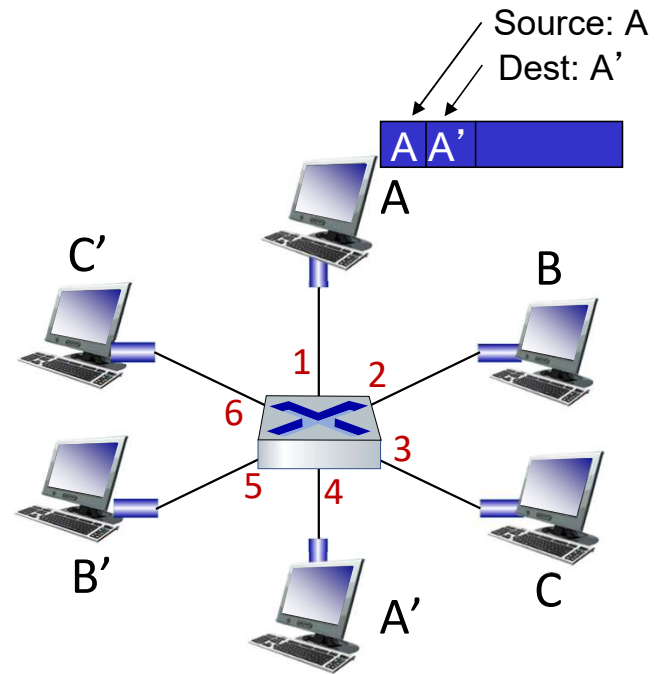
- (MAC address of host, interface to reach host, time stamp)

Q: how are entries created, maintained in switch table?



Switch: self-learning

- switch *learns* which hosts can be reached through which interfaces

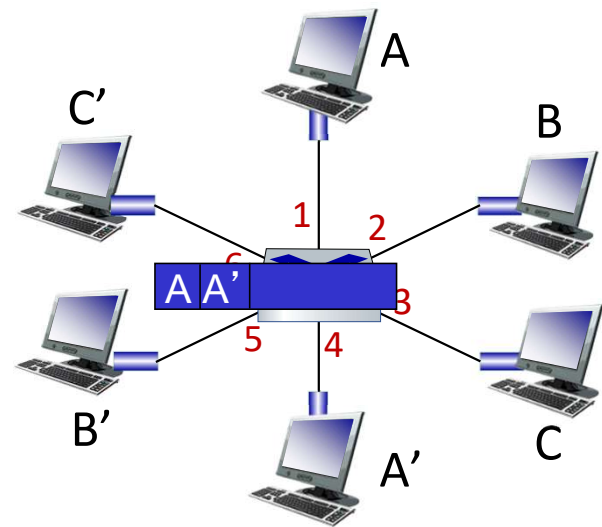


MAC addr	interface	TTL

*Switch table
(initially empty)*

Switch: self-learning

- switch *learns* which hosts can be reached through which interfaces
- when frame received, switch “learns” location of sender: incoming LAN segment

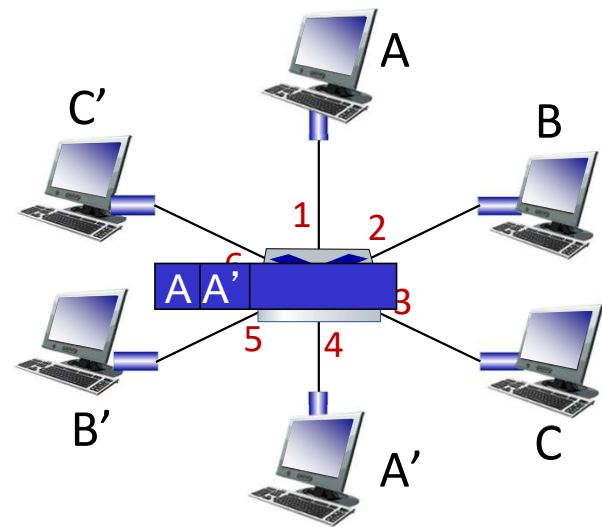


MAC addr	interface	TTL

*Switch table
(initially empty)*

Switch: self-learning

- switch *learns* which hosts can be reached through which interfaces
- when frame received, switch “learns” location of sender: incoming LAN segment
- records sender/location pair in switch table



MAC addr	interface	TTL
A	1	60

*Switch table
(initially empty)*

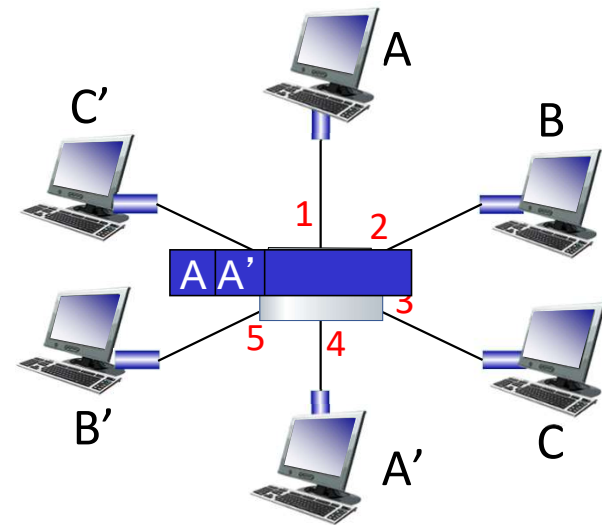
Switch: frame filtering/forwarding

when frame received at switch:

1. record incoming link, MAC address of sending host
2. index switch table using MAC destination address
3. **if** entry found for destination
 then {
 if destination on segment from which frame arrived
 then drop frame
 else forward frame on interface indicated by entry
 }
 else flood /* forward on all interfaces except arriving interface */

Self-learning, forwarding: example

- frame destination, A',
location unknown:

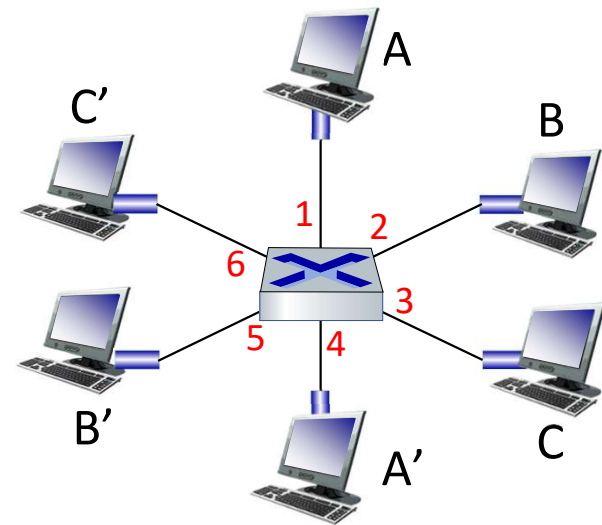


MAC addr	interface	TTL
A	1	60

*switch table
(initially empty)*

Self-learning, forwarding: example

- frame destination, A',
location unknown: **flood**

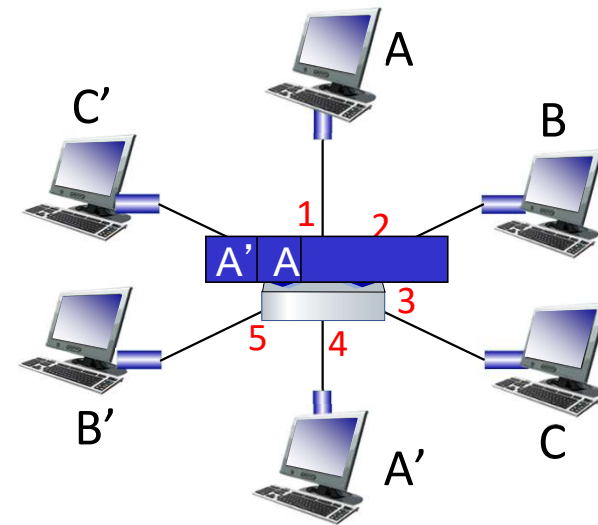


MAC addr	interface	TTL
A	1	60

*switch table
(initially empty)*

Self-learning, forwarding: example

- frame destination, A',
location unknown: **flood**
- destination A location
known:

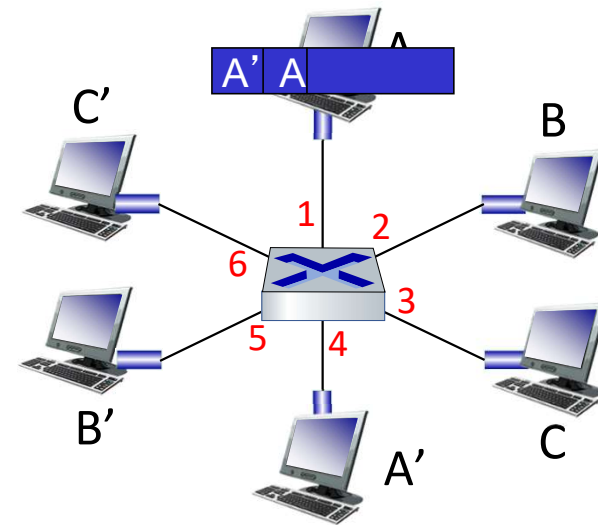


MAC addr	interface	TTL
A	1	60

*switch table
(initially empty)*

Self-learning, forwarding: example

- frame destination, A',
location unknown: **flood**
- destination A location
known: **selectively send**
on just one link



MAC addr	interface	TTL
A	1	60
A'	4	60

*switch table
(initially empty)*

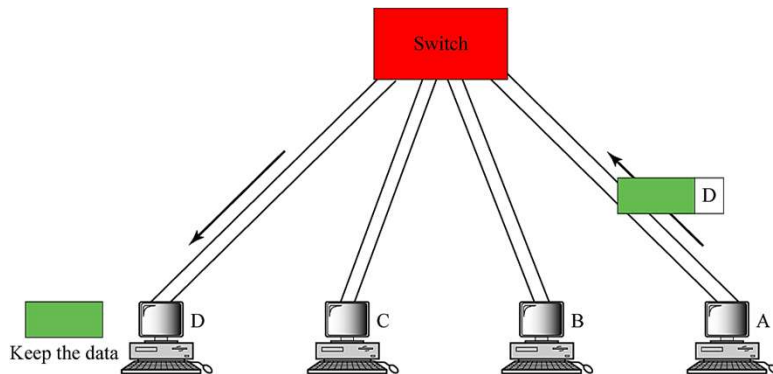
A trip to the past: Hubs

- Layer 1 (PHY) device
- Essentially repeat the received bits into other ports
 - When a bit comes into a hub interface, the hub simply broadcasts the bit on all the other interfaces
- No buffering of frames
- They were used prior the invention of switches as the star topology controller (now obsolete)



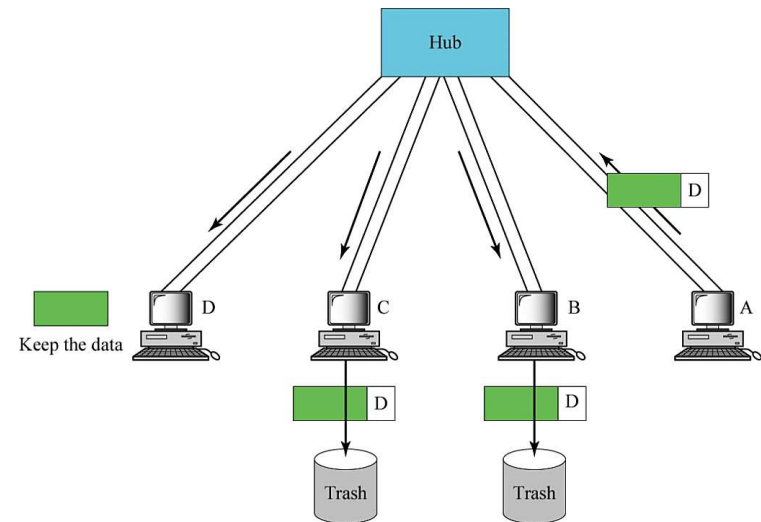
STAR topology controller (Switch vs. Hubs)

Using a switch in a Star Topology



- After the switch learns the position of the receiver, it **selectively sends frames only** on the link of the receiver

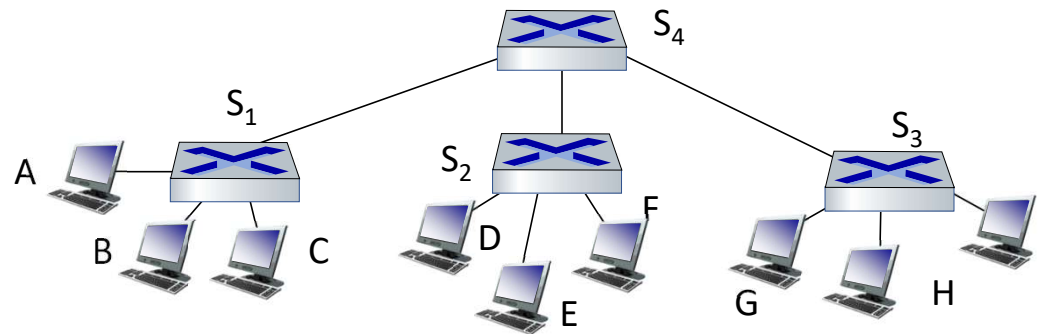
Using a hub in a Star Topology



- The hub forwards frames **indiscriminately** to all links (except incoming frame link)

Interconnecting switches

self-learning switches can be connected together:

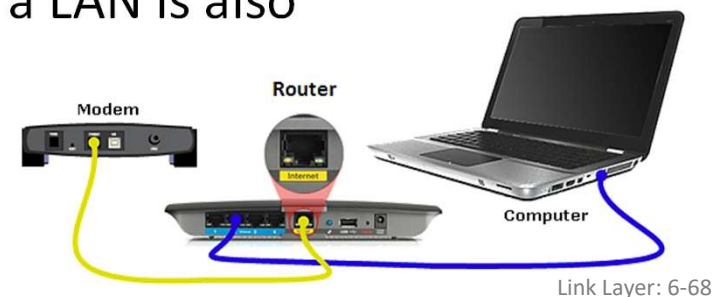


Q: sending from A to G - how does S₁ know to forward frame destined to G via S₄ and S₃?

- A: self learning! (works exactly the same as in single-switch case!)

Routers

- Layer 3 (Network) device (i.e. it implements Layers 1, 2 and 3).
- Its main job is to “route” data between different networks (for instance LANs), i.e. use routing algorithms to find an end-to-end path between sender and receiver devices.
- It connects different networks together. For instance our Home LAN to the WAN of our ISP.
- Since it is at the borders of LANs, usually it is NOT classified as a LAN device
- A router that forwards information in and out of a LAN is also called Gateway



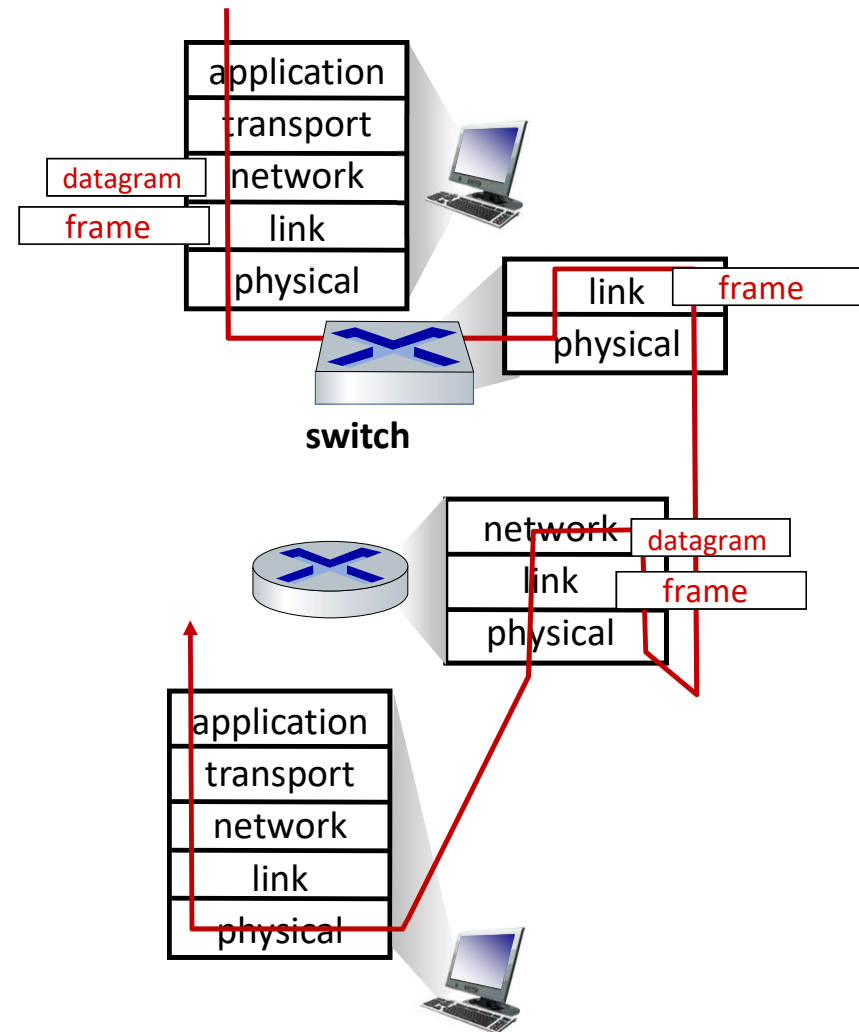
Switches vs. routers

both are store-and-forward:

- *routers*: network-layer devices (examine network-layer headers)
- *switches*: link-layer devices (examine link-layer headers)

both have forwarding tables:

- *routers*: compute tables using routing algorithms, IP addresses
- *switches*: learn forwarding table using flooding, learning, MAC addresses



Link layer, LANs: roadmap

- introduction
- multiple access protocols
- LANs
 - addressing, ARP
 - Ethernet
 - switches
- error detection, correction



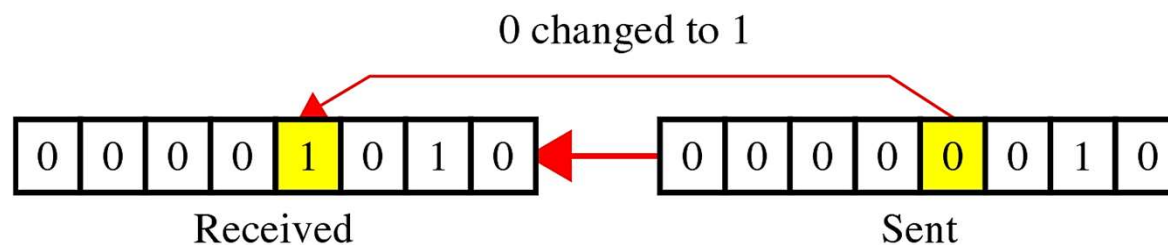
Transmission errors

- We have already mentioned that much of the complexity in computer networks is due to errors that can happen during transmission.
- It would be very efficient if the receiver could detect that there are errors in the received data and even possibly correct them
- There are two types of errors:
 - **Single-Bit Errors**
 - **Burst Errors**



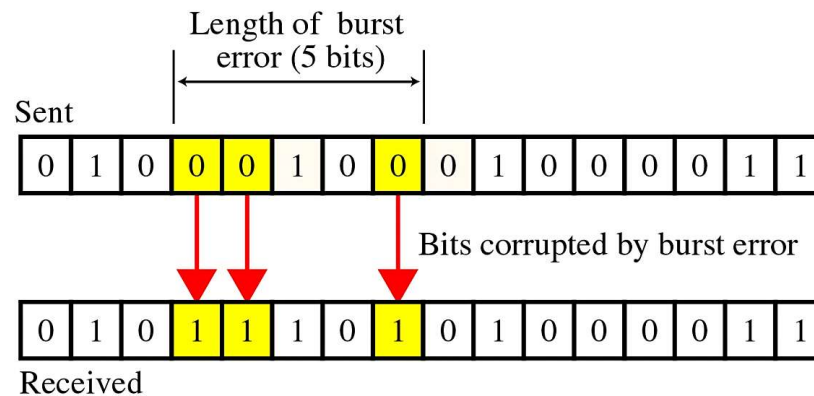
Single-bit errors

- One bit from the data is changed
- Highly unlikely in serial transmission due to duration of noise
 - In a 100Mbps link a bit lasts 10ns but noise can not be this short
- Can happen in parallel transmission if one of the multiple wires has noise



Burst Errors

- Two or more bits in the data are changed
- Bits don't have to be consecutive
- More likely to happen in serial transmission because duration of noise is longer than duration of transmitting bit, so it affects a lot of bits.



Error detection

- What could we do to help receiver detect errors?
 - We could send every data unit twice and do bit-by-bit comparison but this is slow and inefficient. However **concept of including extra information is good.**
- Including a shorter group of bits to assist in error detection is called **redundancy checking**.
- 3 types of redundancy checking:

Parity

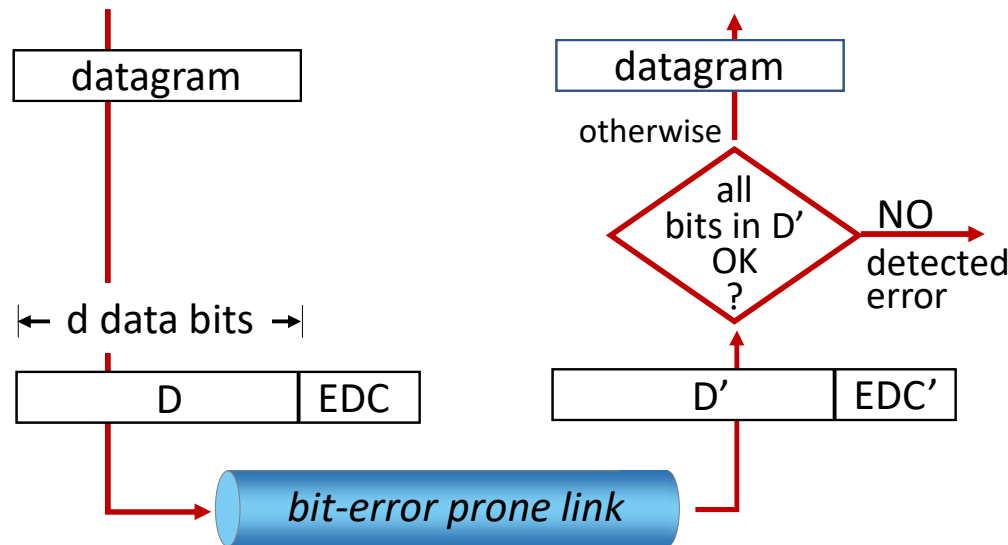
CHECKSUM

CRC

Error detection

EDC: error detection and correction (EDC) bits (e.g., redundancy)

D: data protected by error checking, may include header fields



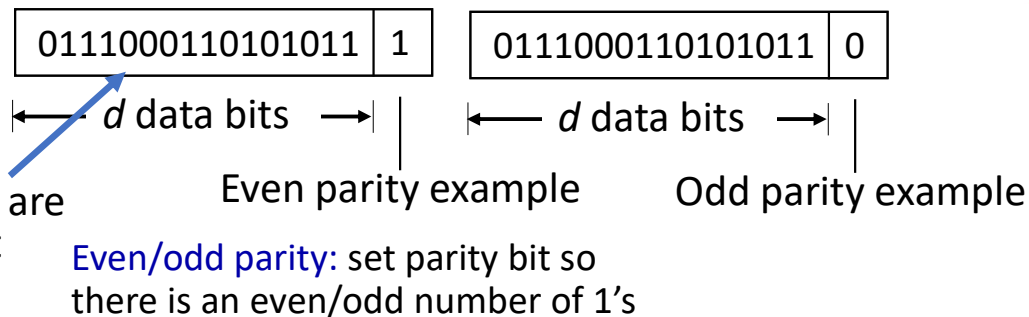
Error detection not 100% reliable!

- protocol may miss some errors, but rarely
- larger EDC field yields better detection and correction but **at the expense of transmitting more redundant data**

Parity checking

single bit parity:

- detect single bit errors



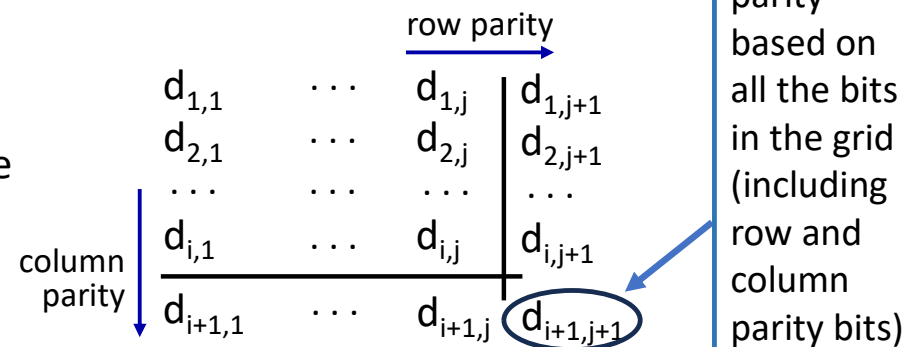
At receiver:

- compute parity of d received bits
- compare with received parity bit – if different then error detected



Can detect *and* correct errors (without retransmission!)

- two-dimensional parity: detect *and correct* single bit errors



Even parity example

1	0	1	0	1	1
1	1	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

Received: detected and correctable single-bit error:

1	0	1	0	1	1
1	0	1	1	0	0
0	1	1	1	0	1
0	0	1	0	1	0

Annotations: parity error (row 2), parity error (column 2).

Bit at 2nd row, 2nd column has flipped!

Internet checksum

Goal: detect errors (*i.e.*, flipped bits) in transmitted segment

sender:

- treat contents of packet as a sequence of n-bit integers
- **checksum:** addition of n-bit integers and perform 1's complement on the sum to produce the checksum
- checksum value appended at the packet

receiver:

- takes the entire received segment, including the checksum field
- adds all n-bit words together using 1's complement addition
- Then it checks the result:
 - If the result is all 1s → no error detected
 - Otherwise → error detected
- But maybe errors nonetheless? More later

Checksum exercise

SENDER

Bits to send:

10101001 00111001

10101001

00111001

11100010 SUM

00011101 CHECKSUM

Data send:

10101001 00111001 00011101

RECEIVER

Bits received:

10101001 00111001 00011101

Don't forget the
carry bit here
if one exists
(add it again)

10101001

00111001

00011101

SUM 11111111

Since sum is all 1s:

Data is intact and accepted

IF NOT all 1s → error detected

Checksum performance

- Detects all errors with odd number of bits changed
- Advantage is ease of computation (addition and complement are very fast)
- But can not detect all errors

Consider the following case where two bits are changed in same column

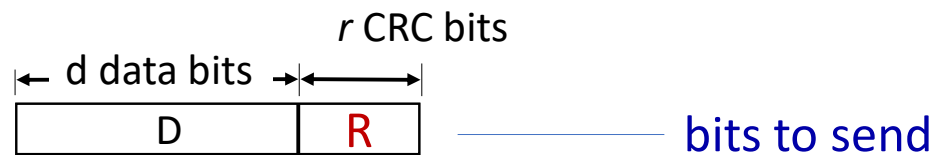
0001	
0010	
00 1 1	
00 0 1	
0111	SUM
1000	CHECKSUM

0001	
0010	
00 0 1	
00 1 1	
1000	
1111	SUM

Although there are errors, checksum fails to detect them!

Cyclic Redundancy Check (CRC)

- more powerful error-detection coding
- **D**: data bits (given, think of these as a binary number)
- **G**: bit pattern (generator), of $r+1$ bits (given, specified in CRC standard)



- sender:* compute r CRC bits, **R**, such that $\langle D, R \rangle$ *exactly* divisible by $G \pmod{2}$
- receiver knows G , divides $\langle D, R \rangle$ by G . If non-zero remainder: error detected!
 - can detect all burst errors less than $r+1$ bits. Burst error of length greater than $r + 1$ bits is detected with probability $1 - 0.5^r$. Can detect any odd number of bit errors.
 - widely used in practice (Ethernet, 802.11 WiFi)

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Cyclic Redundancy Check (CRC): example

Sender wants to compute R:

1st step add r zero bits (r = size of R) at the end of the message

If G has k bits, then R has $k - 1$

G here is 4 bits, therefore R size is 3 bits

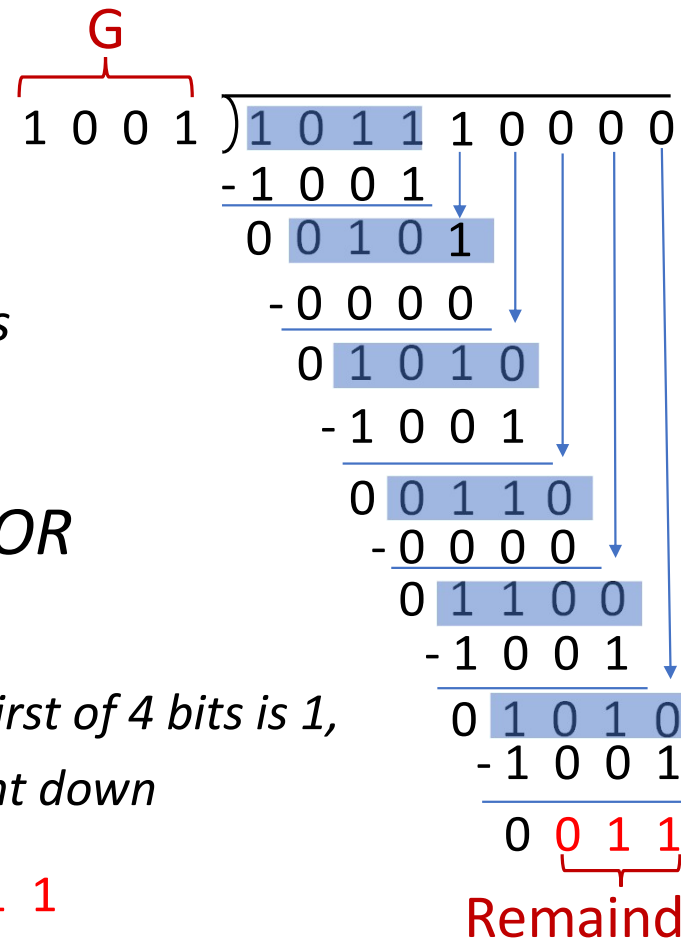
Subtract G from the first 4 bits of $\langle D, R \rangle$

*In mod-2 arithmetic
addition/subtraction is equal to XOR*

Bring down next bit, subtract using the next 4 bits

Two rules: if first of 4 bits is 0, subtract 0000, if first of 4 bits is 1, subtract G . Process ends when last bit is brought down

So final message to send: 1 0 1 1 1 0 0 1 1

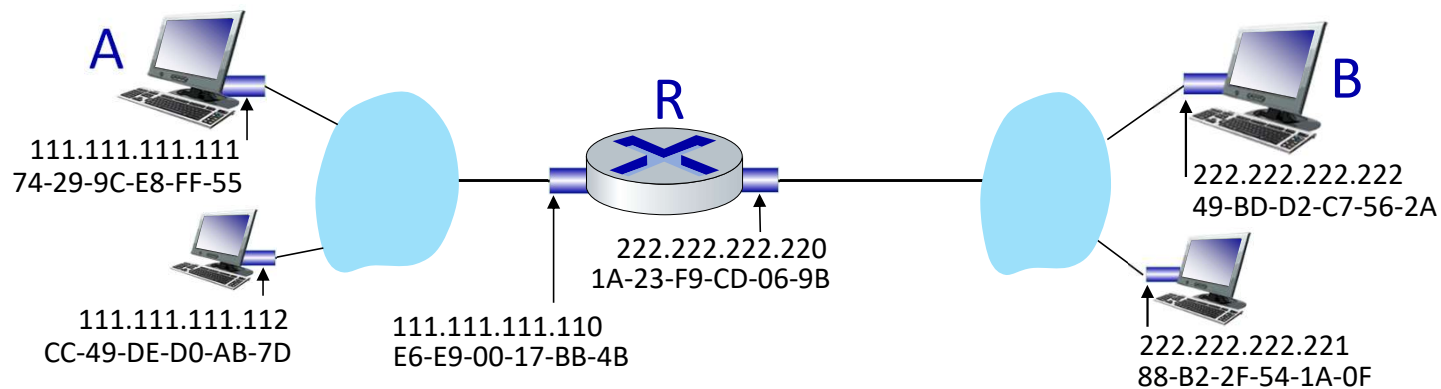


At home:
Do the same process for the receiver and see if you get zero remainder!

Routing to another LAN: addressing

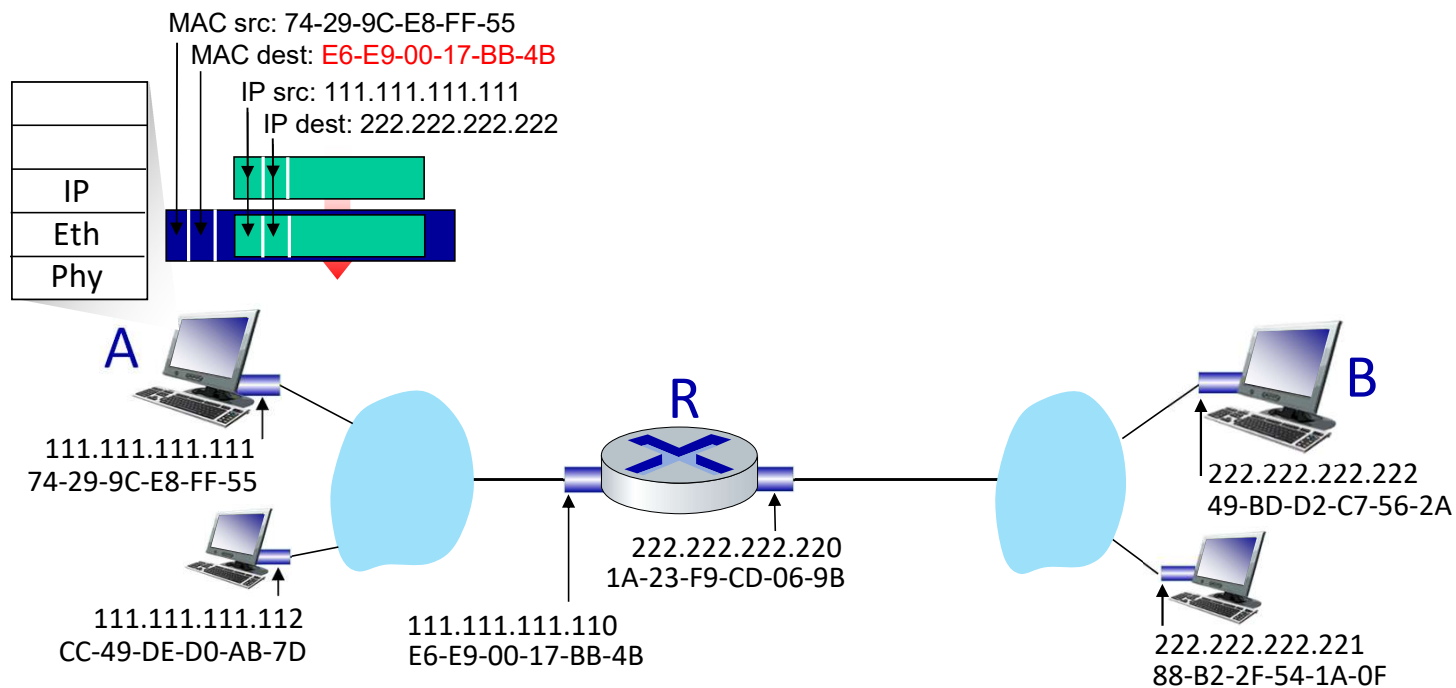
walkthrough: sending a datagram from *A* to *B* via *R*

- focus on addressing – at IP (datagram) and MAC layer (frame) levels
- assume that:
 - A knows B's IP address (how?)
 - A knows IP address of first hop router, R
 - A knows R's MAC address (how?)



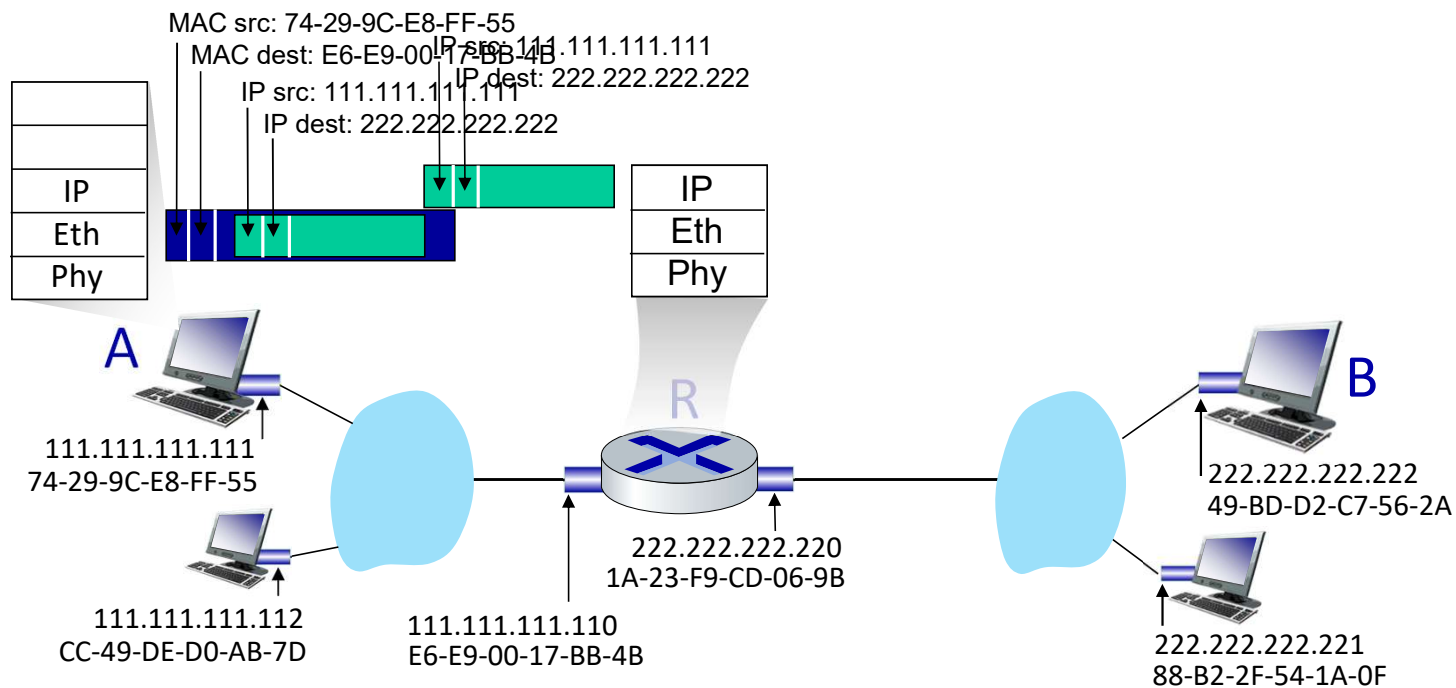
Routing to another LAN: addressing

- A creates IP datagram with IP source A, destination B
- A creates link-layer frame containing A-to-B IP datagram
 - R's MAC address is frame's destination



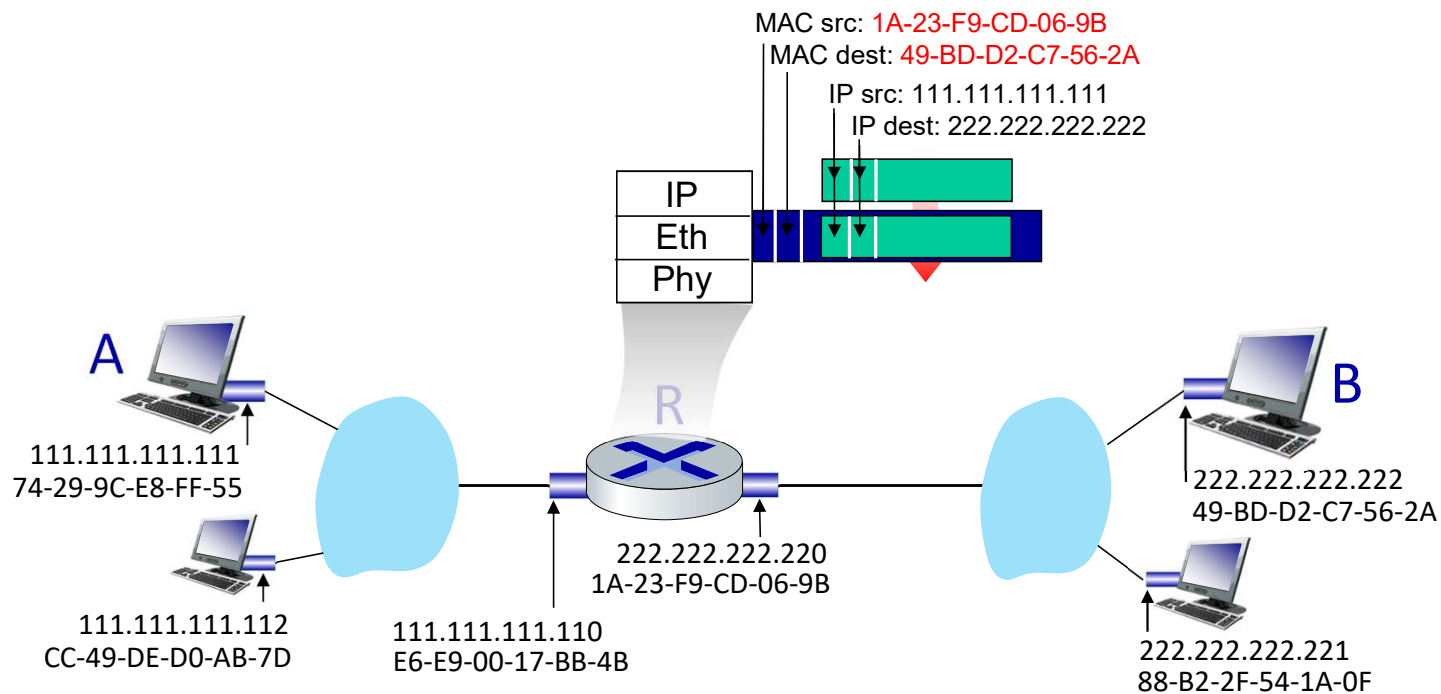
Routing to another LAN: addressing

- frame sent from A to R
- frame received at R, datagram removed, passed up to IP



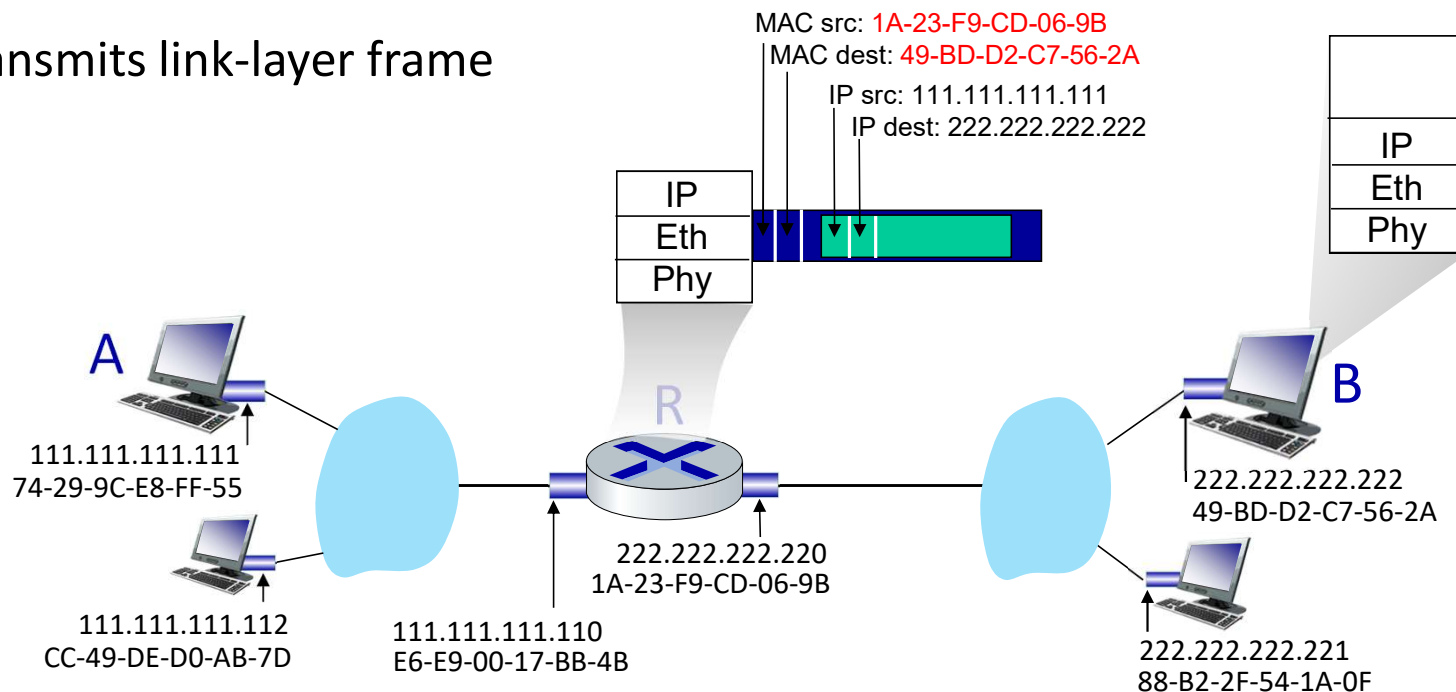
Routing to another LAN: addressing

- R determines outgoing interface, passes datagram with IP source A, destination B to link layer
- R creates link-layer frame containing A-to-B IP datagram. Frame destination address: B's MAC address



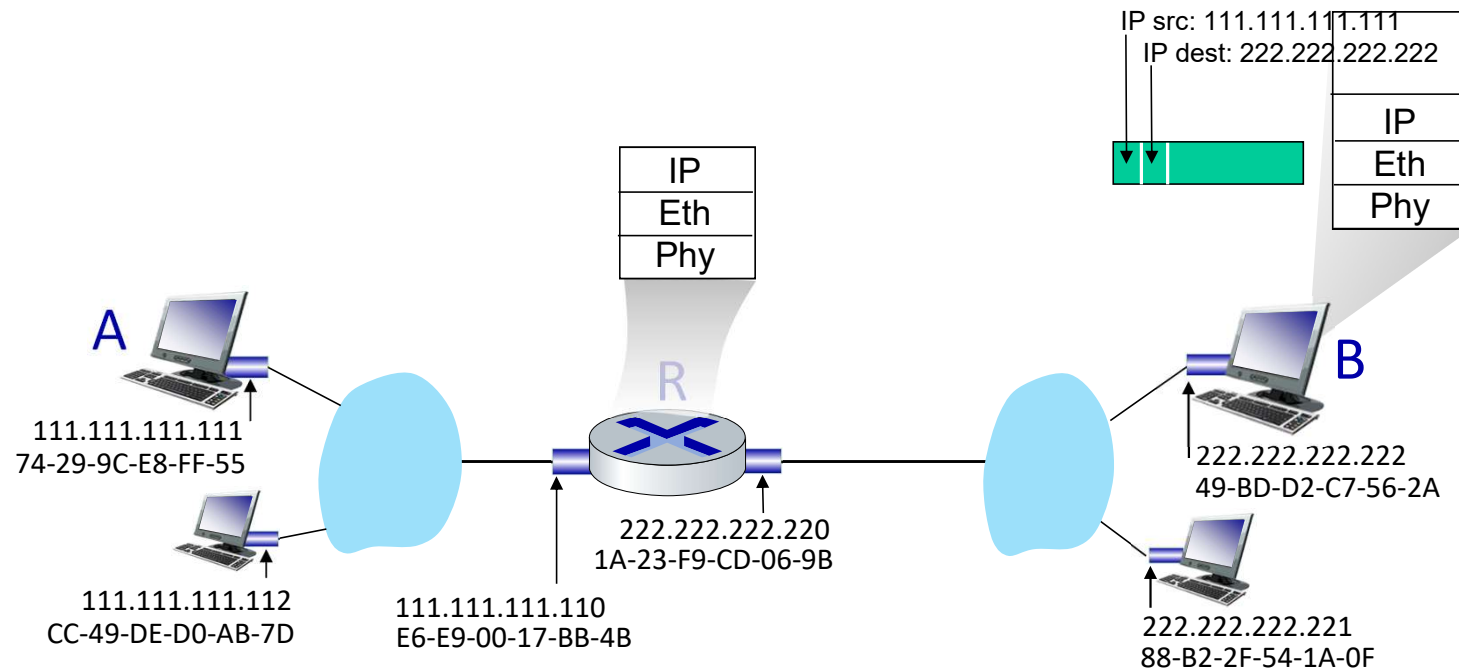
Routing to another LAN: addressing

- R determines outgoing interface, passes datagram with IP source A, destination B to link layer
- R creates link-layer frame containing A-to-B IP datagram. Frame destination address: B's MAC address
- transmits link-layer frame



Routing to another LAN: addressing

- B receives frame, extracts IP datagram destination B
- B passes datagram up protocol stack to IP



Lecture 4: Summary

- principles behind data link layer services:
 - sharing a broadcast channel: multiple access
 - link layer addressing
- implementation of various link layer technologies
 - Ethernet
 - switched LANS
- error detection, correction
- routing to another LAN